

全国高等教育自学考试指定教材
计算机及应用专业（专科）

操作系统概论

（2017 年版）

（含：操作系统概论自学考试大纲）

全国高等教育自学考试指导委员会 组编

主编 张琼声



机械工业出版社

本书分为六章，重点阐述操作系统内核设计和实现的原理，包括操作系统内核作为一种复杂软件涉及的硬件、数据结构、算法、策略。

本书第一章阐述操作系统的作用、功能、发展、特征、指令的执行过程。第二章阐述程序的并发执行、进程的描述与控制、进程同步、系统调用、中断、时钟机制、线程。第三章阐述进程调度算法及相关问题、死锁及解决死锁的方法。第四章阐述程序的链接与装入、连续内存管理和离散内存管理的实现原理、基于分页的虚拟内存管理的实现原理。第五章阐述文件的按名访问和文件存储的实现原理。第六章阐述设备管理软件的构成和各部分的实现原理、设备控制方式、磁盘调度等内容。附录给出了部分习题的参考答案。

本书是高等教育自学考试计算机信息管理专业（独立本科段）、计算机及应用专业（专科）的指定教材，也可作为高等院校本科及专科相关专业的教材及工程技术人员的参考资料。

图书在版编目（CIP）数据

操作系统概论/全国高等教育自学考试指导委员会
组编；张琼声主编. —北京：机械工业出版社，
2017.9
全国高等教育自学考试指定教材
ISBN 978-7-111-58185-7

I. ①操… II. ①全…②张… III. ①操作系统-高
等教育-自学考试-教材 IV. ①TP316

中国版本图书馆 CIP 数据核字（2017）第 242748 号

机械工业出版社（北京市百万庄大街 22 号 邮政编码 100037）
责任编辑：郝建伟
北京市鑫霸印务有限公司印刷
2017 年 10 月第 1 版·2018 年 2 月第 2 次印刷
184mm×260mm·13.25 印张·317 千字
标准书号：ISBN 978-7-111-58185-7
定价：29.00 元

本书如有质量问题，请与教材供应部门联系。

官方淘宝店 网址：<http://shop136348527.taobao.com>

组 编 前 言

21 世纪是一个变幻难测的世纪，是一个催人奋进的时代。科学技术飞速发展，知识更替日新月异。希望、困惑、机遇、挑战，随时随地都有可能出现在每一个社会成员的生活之中。抓住机遇，寻求发展，迎接挑战，适应变化的制胜法宝就是学习——依靠自己学习、终生学习。

作为我国高等教育组成部分的自学考试，其职责就是在高等教育这个水平上倡导自学、鼓励自学、帮助自学、推动自学，为每一个自学者铺就成才之路。组织编写供读者学习的教材就是履行这个职责的重要环节。毫无疑问，这种教材应当适合自学，应当有利于学习者掌握和了解新知识、新信息，有利于学习者增强创新意识，培养实践能力，形成自学能力，也有利于学习者学以致用，解决实际工作中所遇到的问题。具有如此特点的书，我们虽然沿用了“教材”这个概念，但它与那种仅供教师讲、学生听，教师不讲、学生不懂，以“教”为中心的教科书相比，已经在内容安排、编写体例、行文风格等方面都大不相同了。希望读者对此有所了解，以便从一开始就树立起依靠自己学习的坚定信念，不断探索适合自己的学习方法，充分利用自己已有的知识基础和实际工作经验，最大限度地发挥自己的潜能，以达到学习的目标。

欢迎读者提出意见和建议。

祝每一位读者自学成功。

全国高等教育自学考试指导委员会
2016 年 12 月

目 录

组编前言

出版前言

I. 课程性质与课程目标	3
II. 考核目标	4
III. 课程内容与考核要求	4

IV. 关于大纲的说明与考核实施要求	12
--------------------------	----

V. 题型举例	13
---------------	----

后记	15
----------	----

编者的话	18
第一章 操作系统简介	19
第一节 什么是操作系统	19
第二节 操作系统的发展	22
第三节 操作系统的特征	28
第四节 操作系统的功能	29
第五节 操作系统的体系结构	33
第六节 指令的执行	37
思考题与练习题	40
第二章 进程管理	42
第一节 进程的描述	42
第二节 进程的控制	51
第三节 操作系统内核	54
第四节 进程同步	64
第五节 进程通信	80
第六节 线程	81
思考题与练习题	87
第三章 进程调度与死锁	88
第一节 进程调度的功能与时机	88
第二节 进程调度算法	88
第三节 实时系统中的调度	94
第四节 进程切换	98
第五节 多处理器调度	98
第六节 死锁	101
思考题与练习题	111

第四章 内存管理	113
第一节 存储器的层次结构	113
第二节 程序的链接和装入	115
第三节 连续分配存储管理方式	118
第四节 基本分页存储管理方式	128
第五节 基于分页的虚拟存储系统	138
第六节 分段存储管理	150
第七节 Linux 的伙伴系统	155
思考题与练习题	155
第五章 文件系统	158
第一节 文件	158
第二节 目录	161
第三节 文件系统的实现	165
思考题与练习题	170
第六章 I/O 设备管理	172
第一节 I/O 系统的组成	172
第二节 I/O 控制方式	175
第三节 缓冲管理	178
第四节 设备分配	183
第五节 I/O 软件原理	188
第六节 磁盘管理	191
思考题与练习题	197
附录 部分习题参考答案	198
参考文献	202
后记	203

全国高等教育自学考试
计算机及应用专业（专科）

操作系统概论 自学考试大纲

（含考核目标）

全国高等教育自学考试指导委员会 制定

出版前言

为了适应社会主义现代化建设事业的需要,鼓励自学成才,我国在 20 世纪 80 年代初建立了高等教育自学考试制度。高等教育自学考试是个人自学、社会助学和国家考试相结合的一种高等教育形式。应考者通过规定的专业课程考试并经思想品德鉴定达到毕业要求的,可获得毕业证书;国家承认学历并按照规定享有与普通高等学校毕业生同等的有关待遇。经过 30 多年的发展,高等教育自学考试为国家培养造就了大批专门人才。

课程自学考试大纲是国家规范自学者学习范围、要求和考试标准的文件。它是按照专业考试计划的要求,具体指导个人自学、社会助学、国家考试、编写教材及自学辅导书的依据。

为更新教育观念,深化教学内容、考试制度、质量评价制度改革,更好地提高自学考试人才培养的质量,全国考委各专业委员会按照专业考试计划的要求,组织编写了课程自学考试大纲。

新编写的大纲,在层次上,专科参照一般普通高校专科或高职院校的水平,本科参照一般普通高校本科水平;在内容上,力图反映学科的发展变化以及自然科学和社会科学近年来研究的成果。

全国考委电子电工与信息类专业委员会参照普通高等学校操作系统概论类课程的教学基本要求,结合自学考试计算机及应用专业(专科)、计算机信息管理专业(独立本科段)的实际情况,组织制定的《操作系统概论自学考试大纲》,经教育部批准,现颁发施行。各地教育部门、考试机构应认真贯彻执行。

全国高等教育自学考试指导委员会

2017 年 7 月

I. 课程性质与课程目标

一、课程性质和特点

《操作系统概论》是高等教育自学考试计算机及应用专业（专科）、计算机信息管理专业（独立本科段）考试计划中规定必考的课程，是为满足信息管理领域、计算机技术相关领域的人才培养需求而设置的专业课。其特点是理论性和综合性强，要求学生有较好的数据结构、程序设计、计算机组成原理的基础知识。计算机操作系统是一种复杂的系统软件，其功能是管理计算机系统的软件和硬件资源，屏蔽硬件细节，为应用程序开发者提供方便的软件开发环境，为应用程序的运行提供平台。其发展主要受计算机体系结构变化和应用软件开发需要两种因素的影响。操作系统的具体实现，从策略、机制、算法到程序编码都因不同的产品而千差万别，操作系统概论需要从不同的实现中抽取共性的内容，把操作系统实现的基本原理以深入浅出的方式表达出来。该课程既需要把握现代操作系统的基本概念、原理，又需要适当地与实际系统的实现技术相结合，使学生能够学以致用。

二、课程目标

本课程的目标是使学生了解并掌握操作系统设计与实现涉及的基本概念、基本原理；熟悉操作系统的基本功能、软件结构、主要数据结构及算法、操作系统与硬件的接口、操作系统与其他软件的接口及关系；使学生了解操作系统在计算机系统中的地位与作用，培养学生的计算机系统观，加强学生的专业素质；使学生深入理解计算机系统的工作原理，为学生开发高效、稳定的应用软件以及可能从事的系统软件的开发工作打下基础。

通过本课程的学习，应达到的目标为：

- 1) 了解操作系统在计算机系统中的作用，现代操作系统的功能、发展、分类和特点。
- 2) 理解操作系统进程管理功能的实现原理。包括进程的概念、组织、控制；实现进程同步的必要性和实现技术；进程调度的功能、算法；死锁的概念、产生的原因和解决方法；进程间通信的方式。
- 3) 理解线程的概念，线程与进程的关系，线程实现的基本方式。
- 4) 理解内存管理实现的基本原理，包括连续内存管理及离散内存管理方式的实现，掌握不同存储管理方式的内存分配、地址映射、内存回收的原理。
- 5) 理解文件系统的功能及其实现的基本原理，掌握实现文件按名访问和外存储空间管理的主要数据结构和实现思路。
- 6) 理解操作系统设备管理模块的软件构成及各部分软件的功能。了解设备管理的对象、设备管理要解决的问题、设备管理软件与硬件的接口。理解实现设备管理的基本原理。

三、与相关课程的联系与区别

本课程是计算机相关专业及信息管理相关专业的重要专业课程，主要目的是使学生对计算机工作的原理有全面、深刻的理解，熟悉操作系统软件的功能和实现原理，为从事计算机相关领域的研究和开发工作打下坚实的理论基础。该课程的先修课是数据结构、程序设计语言、计算机组成原理、汇编语言。与其他课程相比，该课程内容涉及的知识面广、内容抽象、逻辑性强。

四、课程的重点和难点

本课程的重点包括：进程和线程的概念、进程的描述与组织、进程的控制、操作系统的时钟机制、系统调用、中断机制、进程的同步、进程调度、页式存储管理、段式存储管理、虚拟内存管理、文件的按名访问和文件的存储管理、设备 I/O 的控制方式、设备管理软件的构成、设备的独立性、设备驱动程序、I/O 工作的过程。

难点包括：操作系统与硬件的关系、操作系统与应用程序的关系以及对应用程序的影响、进程并发的实现、同步机制的实现和应用、进程与线程的关系、多级分页的地址变换、文件管理中的数据结构对实现文件系统功能的作用、系统调用的执行过程、操作系统的时钟机制。

II . 考核目标

本大纲在考核目标中，按照识记、领会、简单应用和综合应用四个层次规定其应达到的能力层次要求。四个能力层次是递升的关系，后者必须建立在前者的基础上。各能力层次的含义如下。

识记（I）：要求考生能够识别和记忆本课程中有关的概念性内容（如各种相关的术语、定义、特点、分类、组成、过程、功能、作用等），并能够根据考核的不同要求，做出正确的表述、选择和判断。

领会（II）：要求考生能够理解操作系统是如何工作的、与硬件有什么关系、与应用程序的开发有什么联系。操作系统功能的实现一般采用什么策略、数据结构、算法。

简单应用（III）：要求考生根据已知的基本概念、基本原理等基础知识，分析和解决问题，如银行家算法的应用、系统平均周转时间的计算。

综合应用（IV）：要求考生能够综合运用原理、方法、技术，分析或解决较为复杂的应用问题。如信号量机制的应用、进程调度算法的设计。

III . 课程内容与考核要求

第一章 操作系统简介

一、课程内容

- 1.1 什么是操作系统？
- 1.2 操作系统的发展。
- 1.3 操作系统的特征。
- 1.4 操作系统的功能。
- 1.5 操作系统的体系结构。
- 1.6 CPU 的逻辑结构与指令的执行。

二、学习目的与要求

本章的学习目的是要求考生从操作系统的作用、发展、功能、特点、体系结构等不同角度理解什么是操作系统。掌握现代操作系统的作用、功能、体系结构、分类、特点。

本章的重点是掌握现代操作系统的特点：并发、共享、虚拟、异步；操作系统的基本功能；操作系统在计算机系统中的地位和作用；程序自动执行的过程。

本章的难点是对并发、共享、虚拟和异步的理解；CPU 的逻辑结构与指令执行过程之间的关系。

三、考核内容与考核要求

1. 操作系统的作用。要求达到“识记”层次。
 - 1.1 操作系统是计算机用户与硬件的接口。
 - 1.2 操作系统是计算机系统资源的管理者。
2. 操作系统的发展。
 - 2.1 单道批处理系统及多道批处理系统的特点与缺点。要求达到“识记”层次。
 - 2.2 分时系统的特点与优点。要求达到“识记”层次。
 - 2.3 实时系统的特点与应用领域。要求达到“识记”层次。
 - 2.4 主流操作系统及其应用领域。要求达到“识记”层次。
 - 2.5 现代操作系统的特点：并发、共享、虚拟、异步的含义。要求达到“领会”层次。
 - 2.6 操作系统的功能。要求达到“识记”层次。
 - 2.7 了解单体结构模型、层次结构模型、微内核结构。要求达到“识记”层次。
3. CPU 的逻辑结构与指令的执行。
 - 3.1 CPU 的逻辑结构。要求达到“识记”层次。
 - 3.2 高级语言程序的机器级表示和指令的执行。要求达到“领会”层次。

第二章 进程管理

一、课程内容

- 2.1 进程的描述与组织。
- 2.2 进程的控制。
- 2.3 操作系统内核。
- 2.4 线程的描述与控制。
- 2.5 进程同步。
- 2.6 进程间通信。

二、学习目的与要求

本章的学习目的是要求考生理解、掌握现代操作系统中进程及其相关概念、进程管理的功能和基本实现原理。要求考生熟练掌握进程、进程控制块、进程状态、进程同步、线程、操作系统内核等基本概念。理解程序顺序执行和并发执行的特点；进程创建、撤销、唤醒、阻塞的基本步骤；进程状态的转换；进程同步机制的功能；临界资源、临界区等基本概念；整型信号量和记录型信号量机制；管程机制；生产者—消费者问题、读者—写者问题、哲学家进餐问题的模型及解决其同步问题的方案；操作系统内核的基本功能、中断、系统调用、操作系统时钟；进程间通信的实现方式。

本章的重点是理解进程的定义与特点、进程并发执行的特征、进程的描述与组织、进程控制的实现步骤、进程同步的实现与应用；操作系统内核实现的中断处理、系统调用、操作

系统时钟机制；线程的实现方式、线程与进程的关系；进程间通信的实现方式。

本章的难点是理解进程并发执行的特征、同步机制的应用；操作系统内核的基本功能对于进程管理的支持、重要性及其实现原理。

三、考核内容与考核要求

1. 程序的并发执行。

1.1 程序顺序执行的特点。要求达到“识记”层次。

1.2 程序并发执行的特点。要求达到“领会”层次。

2. 进程的描述，要求达到“识记”层次。

2.1 进程的定义与结构。

2.2 进程控制块的定义与作用。

2.3 进程的执行态、就绪态、阻塞态及状态转换的条件。

2.4 进程的组织。

3. 进程控制，要求达到“识记”层次。

3.1 进程的创建与撤销的条件。

3.2 进程创建与撤销的过程。

3.3 进程阻塞与唤醒的条件。

3.4 进程阻塞与唤醒的过程。

4. 操作系统内核。

4.1 什么是操作系统内核及操作系统内核的功能。要求达到“识记”层次。

4.2 中断的定义。要求达到“识记”层次。

4.3 中断的处理过程。要求达到“领会”层次。

4.4 系统态、用户态、系统调用的定义。要求达到“识记”层次。

4.5 系统调用的执行过程和实现机制。要求达到“领会”层次。

4.6 计算机系统时钟的重要性和时钟体系的构成。要求达到“识记”层次。

4.7 操作系统的时钟机制及其实现原理。要求达到“领会”层次。

5. 线程。

5.1 线程的定义、线程的特点、线程与进程的关系、线程与进程实现比较。要求达到“领会”层次。

5.2 线程的控制。要求达到“领会”层次。

6. 进程同步。

6.1 进程同步机制的任务；临界资源、临界区、进入区、退出区的概念；实现进程同步的基本原理；进程同步机制遵循的四条准则。要求达到“领会”层次。

6.2 整型信号量机制中信号量的类型、整型信号量的 wait 操作和 signal 操作、如何利用整型信号量机制实现进程的互斥与协调。要求达到“综合应用”层次。

6.3 记录型信号量机制中信号量的类型、记录型信号量的 wait 操作和 signal 操作、如何利用记录型信号量机制实现进程的互斥与协调。要求达到“综合应用”层次。

6.4 用信号量机制解决生产者—消费者、读者—写者问题。要求达到“综合应用”层次。

6.5 管程的定义、管程的特征、利用管程解决生产者—消费者问题、利用管程解决哲

学家进餐问题。要求达到“领会”层次。

6.6 进程间通信的基本方式：共享存储器系统、管道、消息缓冲队列。要求达到“识记”层次。

第三章 进程调度与死锁

一、课程内容

- 3.1 进程调度的功能与时机。
- 3.2 进程调度算法。
- 3.3 实时系统中的调度。
- 3.4 多处理机调度。
- 3.5 产生死锁的原因和必要条件。
- 3.6 处理死锁的基本方法。

二、学习目的与要求

本章的学习目的是要求考生理解并掌握操作系统进程调度、进程调度时机、进程调度算法、周转时间、死锁等基本概念。理解选择进程调度方式和算法的若干准则；熟练掌握先来先服务调度算法、短进程优先调度算法、时间片轮转调度算法、基于优先权的调度算法、多级队列调度算法、多级反馈队列调度算法的思想和程序实现思路；理解抢占式调度和非强制式调度策略；了解实时系统调度的特殊要求和最早截止时间调度算法、基于最小松弛度的实时调度算法；了解多处理机的体系结构和多处理机进程调度算法；理解引起死锁的原因、死锁的必要条件、解决死锁的基本方法。

本章的重点是理解为什么要进行进程调度，操作系统在什么时候会进行进程调度，进程调度功能由哪部分程序来完成，实现进程调度的算法有哪些，系统在什么情况下会出现死锁现象，如何解决死锁问题。

本章的难点是理解进程调度算法的思想和进程调度算法的程序实现之间的关系、时间片轮转调度算法的实现；使用不同进程调度算法时系统平均周转时间的计算；银行家算法、死锁定理。

三、考核内容与考核要求

1. 进程调度的功能与时机。要求达到“识记”层次。

- 1.1 进程调度的功能。
- 1.2 引起进程调度的时机。

2. 进程调度算法。

2.1 选择调度方式和算法的若干准则。要求达到“识记”层次。

2.2 周转时间、平均周转时间、平均带权周转时间。要求达到“简单应用”层次。

2.3 先来先服务调度算法、短进程优先调度算法、时间片轮转调度算法、基于优先权的调度算法（静态优先权和动态优先权调度算法和两者的比较）、多级队列调度算法、多级反馈队列调度算法的思想和程序实现思路；抢占式调度和非强制式调度策略及抢占式调度策略实现时的抢占时机。要求达到“领会”层次。

2.4 实时系统中的调度算法：实时调度的条件、最早截止时间调度算法、最小松弛度调度算法。要求达到“领会”层次。

2.5 多处理机调度。紧密耦合的多处理机系统和松弛耦合的多处理机系统、对称多处理机系统和非对称多处理机系统、多处理机系统中的进程分配方式、自调度算法、成组调度算法、专用处理机调度算法。要求达到“识记”层次。

3. 产生死锁的原因和必要条件。要求达到“领会”层次。

3.1 死锁的定义。

3.2 产生死锁的原因。

3.3 产生死锁的必要条件。

4. 解决死锁的基本方法。

4.1 死锁的预防。要求达到“识记”层次。

4.2 死锁的避免。安全状态和不安全状态，要求达到“识记”层次。银行家算法，要求达到“简单应用”层次。

4.3 死锁的检测和解除。死锁定理、死锁的解除。要求达到“识记”层次。

第四章 内存管理

一、课程内容

4.1 存储器的层次结构与局部性原理。

4.2 程序的链接和装入。

4.3 连续内存管理方式。

4.4 分页存储管理。

4.5 分段存储管理。

4.6 段页式存储管理。

4.7 基于分页的虚拟存储管理。

4.8 伙伴系统。

二、学习目的与要求

本章的学习目的是要求考生理解计算机内存管理功能实现的基本原理。要求考生学习并理解计算机存储的层次结构、程序执行时具有的局部性原理；链接程序的功能、程序装入内存的方式；连续内存管理方式的单一连续区内存管理方式、固定分区分配、动态分区分配、动态分区分配实现的数据结构和算法、分区合并方法；离散内存管理方式的分页存储管理、分段存储管理、段页式存储管理的实现原理；基于分页的虚拟存储系统的实现原理；伙伴系统的实现原理。

本章的重点是分页存储管理、分段存储管理、段页式存储管理的实现原理及其地址变换过程；虚拟存储管理系统的定义；基于分页的虚拟存储管理系统实现的原理。

本章的难点是分页存储管理的基本原理、二级分页系统的地址映射。

三、考核内容与考核要求

1. 存储器的层次结构与局部性原理。要求达到“识记”层次。

1.1 存储器的层次结构。

1.2 局部性原理。

2. 程序的链接和装入。要求达到“领会”层次。

2.1 绝对装入方式、静态重定位装入方式、动态重定位装入方式。

2.2 静态链接程序的功能。

2.3 动态链接的基本原理、动态链接的优点。

3. 连续内存管理方式。

3.1 单一连续区分配。要求达到“识记”层次。

3.2 固定分区分配的思想、支持固定分区分配的数据结构。要求达到“识记”层次。

3.3 动态分区分配的基本思想、管理空闲分区的数据结构、动态分区分配算法（首次适应算法、循环首次适应算法、最佳适应算法）、空闲分区的合并。要求达到“简单应用”层次。

4. 分页存储管理。

4.1 页、页框、页表、逻辑地址、物理地址的概念。要求达到“识记”层次。

4.2 分页的逻辑地址结构和地址映射过程。要求达到“综合应用”层次。

4.3 快表的定义、快表的作用、具有快表的系统中地址映射的过程、访存性能分析。要求达到“领会”层次。

4.4 二级分页系统的引入、二级分页系统的逻辑地址结构、二级分页系统的地址映射过程。要求达到“综合应用”层次。

4.5 分段存储管理的基本原理、分段机制中的逻辑地址结构、段表、分段的地址映射过程。要求达到“领会”层次。

4.6 段页式存储管理的基本原理、段页式机制中逻辑地址的结构、段表和页表结构、地址映射过程。要求达到“领会”层次。

5. 基于分页的虚拟存储管理。

5.1 虚拟存储系统的定义、虚拟存储系统的特征。要求达到“识记”层次。

5.2 请求分页存储系统的硬件支持、页表结构、缺页异常处理过程。要求达到“领会”层次。

5.3 最小物理块的分配。要求达到“识记”层次。

5.4 页置换策略：固定分配局部置换、动态分配全局置换、动态分配局部置换。要求达到“识记”层次。

5.5 置换算法：先进先出的页置换算法、最佳置换算法、最近最久未使用置换算法、最近最少使用置换算法、CLOCK 置换算法、改进型 CLOCK 置换算法。要求达到“领会”层次。

5.6 缺页率对有效访存时间的影响、工作集的概念。要求达到“领会”层次。

5.7 抖动、引起抖动的原因、抖动的预防。要求达到“识记”层次。

6. 伙伴系统。要求达到“领会”层次。

6.1 Linux 伙伴系统对连续空闲物理块的组织。

6.2 Linux 伙伴系统的工作原理。

6.3 伙伴系统中物理块的释放与合并。

第五章 文件系统

一、课程内容

5.1 文件系统概述。

5.2 目录。

5.3 文件系统的实现。

二、学习目的与要求

本章的学习目的是要求考生了解文件系统的功能、文件的属性、文件的分类、文件的结构、文件的操作；了解目录文件的构成和目录的作用、对目录的操作；理解文件系统实现文件按名访问和文件存储管理的原理。

本章的重点是文件的操作、文件目录的实现方式、目录项的结构、索引结点的结构、文件的存储方式、实现文件按名访问的原理、文件存储空间的管理。

本章的难点是文件的存储方式、文件按名访问的实现、文件存储空间的管理。

三、考核内容与考核要求

1. 文件系统概述。要求达到“识记”层次。

文件的命名规则、文件的分类、文件的结构、文件的存取方式、文件的属性、文件的操作。

2. 文件目录。要求达到“识记”层次。

2.1 目录的类型：单层目录、多级目录、树型目录。要求达到“识记”层次。

2.2 绝对路径和相对路径。要求达到“识记”层次。

2.3 对目录的操作。要求达到“领会”层次。

3. 文件的实现。

3.1 实现文件存储的方式：连续分配、链接分配。要求达到“简单应用”层次。

3.2 目录的实现方式：将文件的属性和地址信息放在目录项中，如 CP/M、MS-DOS 的 FAT12 文件系统；将文件的属性和地址信息放在索引结点（i-结点）中；目录、FAT 表、i-结点如何支持文件的按名访问；目录、FAT 表、i-结点的结构与文件系统管理的单个文件的最大长度的关系。要求达到“综合应用”层次。

3.3 文件系统存储空间的管理，簇块大小的选择、空闲盘块的链接表、位图。要求达到“简单应用”层次。

第六章 I/O 设备管理

一、课程内容

6.1 I/O 系统的组成。

6.2 输入/输出的控制方式。

6.3 缓冲管理。

6.4 设备分配。

6.5 设备管理软件的构成。

6.6 磁盘管理。

二、学习目的与要求

本章的学习目的是要求考生了解输入/输出设备的类型、操作系统的设备管理模块实现的功能、设备管理软件的构成及各部分的功能和层次关系、设备独立性的含义与实现设备独立性的基本原理、设备管理软件与设备硬件的接口；理解输入/输出工作的过程；理解将独占设备变为可共享的虚拟设备的 SPOOLing 技术。

本章的重点是设备管理软件的构成及各部分实现的功能。

本章的难点是对设备管理软件中涉及的数据结构、设备分配过程、设备独立性、设备管理软件与硬件的接口。

三、考核内容与考核要求

1. I/O 系统的组成。要求达到“识记”层次。

1.1 I/O 系统的结构。

1.2 I/O 设备的分类。

1.3 设备控制器的功能、设备控制器的逻辑结构、设备控制器与 CPU 的接口、设备控制器与 I/O 设备的接口。

2. 输入/输出控制方式。要求达到“领会”层次。

2.1 轮询控制方式。

2.2 中断控制方式。

2.3 DMA 控制方式。

3. 缓冲管理。要求达到“领会”层次。

3.1 引入缓冲管理的目的。

3.2 循环缓冲的构成、缓冲区的申请和释放过程、缓冲区使用中的同步控制；公共缓冲池的构成、缓冲区的申请和释放过程。

4. 设备分配。

4.1 设备分配中的数据结构，系统设备表、设备控制表、控制器控制表、通道控制表。要求达到“领会”层次。

4.2 设备分配时应考虑的若干因素，安全分配、不安全分配；独占设备的分配、共享设备的分配、虚拟设备的分配。要求达到“识记”层次。

4.3 设备独立性的含义、实现设备独立性的意义、实现设备独立性的基本原理。要求达到“领会”层次。

4.4 SPOOLing 系统的功能、SPOOLing 系统的构成、SPOOLing 系统的应用。要求达到“识记”层次。

4.5 设备管理软件的层次结构；应用层软件、设备无关层软件、设备驱动程序、中断处理程序分别完成的功能。要求达到“识记”层次。

4.6 磁盘的结构和磁盘上的数据布局、磁盘的物理格式化和逻辑格式化的任务；磁盘的类型、磁盘访问时间的构成。要求达到“识记”层次。

4.7 磁盘调度程序的功能、先来先服务的磁盘调度算法、最短寻道优先的磁盘调度算法、SCAN 算法、CSCAN 算法、NStepSCAN 算法、FSCAN 算法。要求达到“领会”层次。

4.8 提高磁盘访问速度的方法：采用磁盘高速缓存、虚拟磁盘、提前读、延迟写、优化物理块分布。要求达到“识记”层次。

IV. 关于大纲的说明与考核实施要求

一、自学考试大纲的目的和作用

课程自学考试大纲是根据专业自学考试计划的要求,结合自学考试的特点制定的。其目的是对个人自学、社会助学和课程考试命题进行指导和规定。

课程自学考试大纲明确了课程自学内容及其深度和广度,规定出课程自学考试的范围和标准,是编写自学考试教材的依据,是社会助学的依据,是个人自学的依据,也是进行自学考试命题的依据。

二、关于自学教材

《操作系统概论》,全国高等教育自学考试指导委员会组编,张琼声主编,机械工业出版社出版,2017 年版。

三、关于考核内容及考核要求的说明

1) 课程中各章的内容均由若干知识点组成,在自学考试命题中知识点就是考核点。因此,课程自学考试大纲中所规定的考核内容是以分解为考核知识点的形式给出的。因各知识点在课程中的地位、作用以及知识自身的特点不同,自学考试将对各知识点分别按四个认知层次确定其考核要求(认知层次的具体描述请参看 II. 考核目标)。

2) 按照重要性程度不同,考核内容分为重点内容和一般内容。为有效地指导个人自学和社会助学,本大纲已指明了课程的重点和难点,在各章的“学习目的与要求”中也指明了本章内容的重点和难点。在本课程试卷中重点内容所占分值一般不少于 60%。

本课程共 4 学分。

四、关于自学方法的指导

《操作系统概论》作为计算机及应用专业(专科)、计算机信息管理专业(独立本科段)的专业课,内容有一定的难度,需要一定的专业基础,对于考生的记忆、理解和应用能力有较高的要求,要取得较好的学习效果,请注意以下事项:

1) 在学习本课程教材之前应仔细阅读本大纲的第一部分,了解本课程的性质、特点和目标,熟知本课程的基本要求和与相关课程的关系,使接下来的学习紧紧围绕本课程的基本要求。

2) 在学习每一章内容之前,先认真了解本自学考试大纲对该章知识点的考核要求,做到在学习时心中有数。

3) 从计算机系统全局的角度理解本课程的内容。注重基本概念、基本常识的记忆和基本方法、基本原理的理解,反复研读教材,注重结合实例。

4) 在自学过程中应有良好的计划和适当的学习方法,注重资料的查阅,通过阅读参考书和网络资料,加深对教材中内容的理解,在理解的基础上记忆、领悟、应用知识。建议参考 Linux 内核分析方面的书籍,帮助更好地理解操作系统原理。

五、考试指导

在考试过程中应做到卷面整洁,书写工整,段落与间距合理,有助于教师评分。因为阅卷者只能为他能看懂的内容打分,书写不清楚会导致不必要的丢分。回答试卷所提出的问题,不要答非所问,避免超过问题的范围。

保持良好的心态，注重平时知识的积累，考前做必要的总结，避免临时抱佛脚的应试方式。

六、对社会助学的要求

1) 要熟知考试大纲对本课程总的要求和各章的知识点，准确理解对各知识点要求达到的认知层次和考核要求，并在辅导过程中帮助考生掌握这些要求，不要随意增删内容和提高或减低要求。

2) 要结合典型应用，讲清课程的核心知识点，引导学生独立思考，理解课程中的各项内容，掌握解决应用问题的思路 and 技巧，帮助考生真正达到考核要求，并培养良好的学风，提高自学能力。不要猜题、押题。

3) 助学单位在安排本课程辅导时，授课时间建议不少于 60 课时。引导考生通过查阅参考资料中列出的书目帮助理解难点问题。

七、关于考试命题的若干规定

1) 考试方式为闭卷、笔试，考试时间为 150 分钟。考试时只允许携带笔、橡皮和尺，答卷必须使用蓝色或黑色钢笔或圆珠笔书写。

2) 本大纲各章所规定的基本要求、知识点及知识点下的知识细目，都属于考核的内容。考试命题，要注意突出课程的重点，保障重点内容的覆盖度。

3) 不应命制超出大纲中考核知识点范围的题目，考核目标不得高于大纲中所规定的相应的最高能力层次要求。命题应着重考核自学者对基本概念、基本知识和基本理论是否了解或掌握，对基本方法是否会用或熟练。不应命制与基本要求不符的偏题或怪题。

4) 本课程在试卷中对不同能力层次要求的分数比例大致为：识记占 30%，领会占 35%，简单应用占 20%，综合应用占 15%。

5) 要合理安排试题的难易程度，试题的难度可分为：易、较易、较难和难四个等级。每份试卷中不同难度试题的分数比例一般为：3:3:3:1。

必须注意试题的难易程度与能力层次有一定的联系，但二者不是等同的概念，在各个能力层次都有不同难度的试题。

6) 课程考试命题的主要题型一般有单项选择题、填空题、简答题和综合题等。

V. 题型举例

一、单项选择题（在每小题后的四个备选项中只有一个是符合题目要求的，请将其选出并将代码填写在题后的括号内）

1. 假定系统为某进程分配了 3 个页框，采用 FIFO 页面置换算法，当访问页号的序列为 1, 0, 2, 3, 1, 0, 3, 5, 4, 2 时，将产生【 】次页置换。

A. 4

B. 5

C. 6

D. 7

2. 操作系统内核中文件系统模块的主要功能是【 】。

A. 实现虚拟存储

B. 保存系统文档和用户文档

C. 保护系统数据

D. 实现对文件的按名存取和文件的存储

二、填空题（请在题目空白处填写正确内容）

1. 虚拟存储系统是指具有_____功能和_____功能，能从_____上对内存容量

进行扩充的一种存储器系统。

2. 设备控制器的 I/O 逻辑部分完成地址译码和_____的功能。

三、简答题

1. 操作系统在什么时候创建进程？操作系统如何创建新进程？
2. 什么是进程调度算法？引起进程调度的原因有哪些？

四、综合题

进程 p1、p2、p3、p4 到达系统的时刻分别为 0ms、8ms、10ms、20ms。需要的 CPU 服务时间分别为 12ms、30ms、20ms、10ms，进程 p1、p2、p3、p4 的优先权值分别为 10、20、40、5。若系统采用基于优先权的非抢占式进程调度算法，请说明进程的调度顺序并计算系统的平均周转时间。

后 记

本大纲是根据全国高等教育自学考试指导委员会电子电工与信息类专业委员会制定的《高等教育自学考试计算机及应用专业（专科）考试计划》和全国高等教育自学考试指导委员会《关于修订高等教育自学考试课程自学考试大纲的几点意见》的精神制定的。

本大纲提出初稿后，曾聘请专家通审，并由电子电工与信息类专业委员会在北京组织召开审稿会进行审稿，根据审稿会意见由编者作了修改。最后由电子电工与信息类专业委员会定稿。

本大纲由中国石油大学（华东）张琼声副教授负责编写。参加审稿并提出修改意见的有杨兴强教授（山东大学）、武频副教授（上海大学）。

对参与本大纲编写和审稿的各位专家表示感谢。

全国高等教育自学考试指导委员会
电子电工与信息类专业委员会
2017 年 7 月

全国高等教育自学考试指定教材
计算机及应用专业（专科）

操作系统概论

全国高等教育自学考试指导委员会 组编

编 者 的 话

本书是根据全国高等教育自学考试指导委员会制定的《操作系统概论自学考试大纲》编写的自考教材。本书在遵循大纲要求的同时,考虑了计算机信息管理专业(独立本科段)和计算机及应用专业(专科)的教学计划、相关课程的设置及课程之间的衔接,针对专业发展的需要设计并撰写了全书共6章内容。

随着分布式平台、大数据、人工智能技术及移动终端应用的快速发展,计算机与信息技术领域对相关人才的知识结构、学习能力和专业素质提出了更高的要求。本书尽量考虑了当前信息产业对人才的要求,以及计算机与信息相关专业学生应具备的专业素质,扩展了教材涉及知识的广度与深度。同时,以深入浅出、详细说明和增加重要知识点例题的方式降低读者的自学难度。为了帮助读者更直观而不是抽象地理解操作系统原理,缩短操作系统原理与操作系统实现之间的距离,书中结合理论给出了不同操作系统实现代码的实例,读者可以忽略代码的细节,重点理解代码的流程和“模样”。本书希望通过全面介绍操作系统的功能、作用及实现原理,使读者对计算机系统从应用程序层到硬件层工作的原理有一个深刻的理解,为开发复杂、高性能、运行稳定的软件打下良好的专业基础。

本书内容分为6章:第一章介绍操作系统的功能、作用、特征、发展,以及指令的执行;第二章介绍进程的描述与组织、进程的控制、中断、时钟、系统调用、进程同步、线程及进程间通信,第二章是本书的重点和难点之一;第三章主要介绍进程调度算法、死锁产生的原因和解决办法;第四章介绍了内存管理的实现原理,包括程序的链接与装入、单一连续区分配、固定分区分配、动态分区分配、分页存储管理、分段及段页式存储管理,以及基于分页的虚拟内存管理;第五章介绍了文件系统的按名访问和文件存储的实现;第六章介绍了I/O设备的结构、I/O控制方式、I/O缓冲区的管理、I/O设备管理软件的构成,以及I/O设备管理软件各部分的功能和实现原理。

本书由中国石油大学(华东)张琼声副教授编写,刘童璇提供Tinix代码注释,范志东、赵国梁、孟祥奎完成书中插图的绘制,夏守姬、李莹、吴明泉分别参与了操作系统体系结构、线程、文件系统管理部分内容的编写,陈坤、董黎明、赵梦田帮助检查书中的错误。

感谢山东大学杨兴强教授和上海大学武频副教授对全书进行了认真审阅,并提出了详细的修改建议。

因时间仓促、编者水平有限,书中若有不当或错误之处,敬请广大读者批评指正。

编 者

2017年7月

第一章 操作系统简介

计算机操作系统的设计与实现技术是计算机科学领域的一个重要研究课题，也是计算机科学相关领域的工作者理解计算机工作基本原理、深入认识计算机软件驱动硬件完成特定功能过程的关键环节。本章将从操作系统的作用、功能、特征、发展及软件体系结构等不同角度简要介绍操作系统。目的是使读者了解什么是操作系统，操作系统能做什么，为什么计算机系统中需要操作系统的支持。为了使读者更好地理解后续章节的内容，在本章的最后一节还将介绍指令的执行过程。

第一节 什么是操作系统

操作系统(Operating System, OS)是一种复杂的系统软件，是不同程序代码、数据结构、数据初始化文件的集合，可执行。例如，常用的 Linux、Windows 7 和 Windows 10 等。通常所说的装“系统”，就是指装“操作系统”。操作系统提供计算机用户与计算机硬件之间的接口，并管理计算机软件和硬件资源。操作系统本身并不向用户提供可直接使用的功能，一台仅装有操作系统的计算机对于用户而言几乎是没有什么用的，但是它为用户可直接使用的应用程序的运行提供了环境，并为应用程序的开发提供平台，使应用程序的开发变得简单、高效。

操作系统是覆盖在裸机上的第一层软件，编译程序、数据库管理系统及其他应用程序都运行在操作系统之上，操作系统为这些软件提供运行环境。操作系统在计算机系统的位置如图 1-1 所示。

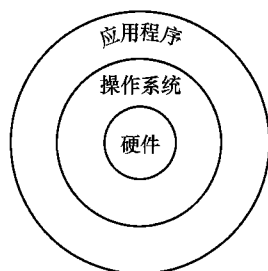


图 1-1 操作系统在计算机系统的位置

下面从不同角度说明什么是操作系统。

一、用户与硬件之间的接口

为什么说：“操作系统提供了计算机用户与计算机硬件之间的接口？”接口是两个不同组成部分的交接面。在计算机专业领域，接口分为硬件接口和软件接口，硬件接口如 USB 接口、串口和并口，软件接口如 C 语言中的函数调用 `printf()`。

计算机的所有功能最终都是由硬件的操作来实现的。但是，通常使用如 C、C++、Java 和 Python 等高级程序设计语言所编写的应用程序中并没有直接操作硬件的语句和指令，那为什么这些程序依然能够操纵显示器、打印机和绘图仪等硬件，通过硬件对程序的执行得到执行结果，完成特定的功能呢？这是因为应用程序是在操作系统上运行的，而对硬件的控制过程都封装在操作系统的核心程序中了。

例如，执行用户程序中的 `printf()`，不需要在用户程序中写直接操作打印机或显卡的指令，但是在操作系统上运行的用户程序通过接口 `printf()`，最终调用到操作系统中实现输出

功能的函数，其中包括了驱动硬件的指令，从而实现了硬件的操作。这个过程比较漫长，实际上在 `printf()` 函数中又调用了其他的库函数，经过库函数的层层调用之后，调用操作系统中的特殊函数和驱动程序，控制硬件完成显示或者打印输出的功能。

因为有了操作系统对硬件的抽象，而且操作系统为应用程序提供了运行环境，所以在操作系统上编写和运行应用程序就简单多了。例如，在 Windows 操作系统下用 C 语言编写从磁盘读文件的程序，只需使用 `fopen()`、`fread()` 和 `fclose()` 这几个函数。但是在没有操作系统的裸机上要写一个可以读文件的程序就复杂多了，必须清楚要读的文件在磁盘上的物理位置，即文件数据在磁盘上的柱面号、磁道号和扇区号，还需要知道读入的数据放在内存的什么位置，然后调用相应的指令序列完成读操作。

接下来通过以下两个实例来说明，在有操作系统的环境中使用高级语言开发应用程序比在无操作系统的裸机上开发相同功能的应用程序要简单、快捷。

例 1-1：在操作系统环境下，用 C 语言编写的运行读磁盘文件 `data.dat` 的程序如下。

```
#include <stdio.h>
#include <stdlib.h>
main()
{ FILE *fp;
  char ch[1000];
  if((fp=fopen("a:\data.dat","r"))==NULL)/* 打开要读的文件 data.dat */
  {
    printf("Cannot open file strike any key exit!");
    exit(1);
  }
  fread(ch, sizeof(int), 30, fp);/* 从文件中读 */
  fclose(fp);
}
```

例 1-2：在无操作系统的裸机上运行的读磁盘文件 `data.dat` 的程序如下。该程序省略了由文件名 `data.dat` 查找根目录获取该文件所在的扇区号部分的程序，所列程序清单仅包括初始化软驱和从文件所在的逻辑扇区号计算出文件扇区所在的物理位置，定位磁头，启动读操作部分的程序代码。

```
xor ah, ah; (对 ah、dl 清零)
xor dl, dl; 软驱复位
int13h; (BIOS 功能调用 int 13h 中断)
mov ax, [wSectorNo]; (要读取的扇区号送 ax 寄存器)
mov cl, 1; 将 1 赋值给 cl 寄存器
call ReadSector; 调用 ReadSector, 从第 19 个扇区开始, 读出扇区内容
; 逐个放入 es:bx 中
...
```

; 以下程序根据逻辑扇区号求扇区在磁盘中的位置, 即柱面号、磁头号和扇区号

```
push bp          ; SP = SP - 2, bp = SP + 1
mov bp, sp
```

```

sub esp, 2          ; 辟出两个字节的堆栈区域保存要读的扇区数: byte [bp-2]
mov byte [bp-2], cl ; 将 cl 中的值放入 bp-2 中, cl 为要读的扇区数
push bx            ; 保存 bx
mov bl, [BPB_SecPerTrk] ; 将每个磁道的扇区数值放入 bl 中
div bl
inc ah             ; z++ (ah 中第一次存放为起始扇区号, 然后进行扇区号递增)
mov cl, ah         ; 起始扇区号送 cl 寄存器
mov dh, al
shr al, 1
mov ch, al         ; 柱面号送 ch
and dh, 1          ; dh & 1 = 磁头号
pop bx             ; 恢复 bx
                  ; 至此, “柱面号, 起始扇区, 磁头号”计算完毕
mov dl, [BS_DrvNum] ; 驱动器号 (0 表示 A 盘)
.   GoOnReading:
mov ah, 2          ; int 13h 功能调用 ah=2, 表示功能号为 2, 进行读操作
mov al, byte [bp-2] ; 读 al 个扇区
int13h             ; 调用 BIOS 的 int 13h 中断

```

上面这段程序, 先执行了一次 int 13h 中断对磁盘进行初始化, 通过对 ah、dl 清零对驱动器复位。然后根据逻辑扇区号计算出文件 data.dat 的第一个扇区在磁盘上的绝对物理地址。这个地址包括了 data.dat 的第一个逻辑扇区在磁盘上的柱面号、磁头号 and 起始扇区号。最后, 程序将这些地址信息和功能号写入指定的寄存器, 然后再次执行 int 13h 中断, 执行一次读操作。

大多数应用软件的开发者可以在不了解计算机硬件接口的情况下编写大量的应用程序, 这是因为有了操作系统和大量的库函数完成了公用的、复杂的、可以被应用程序调用的功能。由以上两个实例的对比可以看出, 操作系统屏蔽了对硬件操作的细节, 提供了计算机用户与计算机硬件之间的接口, 并且通过这个接口使应用程序的开发变得简单、高效。

操作系统必须完成的两个主要目标如下。

1) 与硬件部分相互作用, 为包含在硬件平台上的所有底层可编程部件提供服务。

例如, 要使用显示器, 必须为驱动显示器工作的一些寄存器赋值, 以便让显示器“知道自己要以什么样的背景颜色和前景颜色显示什么内容”。

2) 为运行在计算机系统上的应用程序(即所谓用户程序)提供执行环境。

高级语言的应用程序是不能在裸机上运行的, 必须运行在操作系统上, 由操作系统为应用程序的执行分配必要的软件和硬件资源, 并对这些资源进行有效的管理。

二、资源的管理者

现代计算机系统的一个重要特点就是支持多任务, 即允许在同一个系统内同时驻留多个应用程序。多个应用程序共同使用计算机硬件和软件资源时, 就需要操作系统对这些资源进行有效的管理。一方面保证用户程序的顺利执行, 另一方面使计算机系统资源得到尽可能高效的利用, 保证计算机系统的高性能。操作系统所管理的资源主要包括处理机、内存、设备和文件, 在网络操作系统中还包括网卡、带宽等。下面对操作系统的主要功能进行说明。

1. 处理机管理

本书讲述的是多任务操作系统的实现原理。由于程序的执行必须依靠处理机,任意时刻处理机都只能执行一个程序流。因此,在单处理机系统中执行多个程序流,必须由操作系统的处理机调度程序来管理处理机的分配,以使多个程序共享处理机,从宏观上看多个程序能同时顺利执行。在多处理机系统中,也需要操作系统对多个处理机在多任务的情况下进行有效的管理。

2. 内存管理

在多任务系统中,内存不再是独占资源,而是可能被多个应用程序共同占用。如何为多个应用程序分配内存并使不同应用程序的地址空间互不干扰,如何在程序执行完毕后回收其所占内存,以及完成逻辑地址到物理地址的转换,都是操作系统内存管理程序要完成的功能。

3. 设备管理

设备管理主要完成接受用户的 I/O 请求、为用户分配 I/O 设备、管理 I/O 缓存和驱动 I/O 设备等功能。

4. 文件管理

计算机系统把大量需要长时间保留的数据信息以文件的形式存放在外存储设备中(如硬盘、光盘、磁带和 U 盘),操作系统通过自己的文件管理程序完成外存空间的分配、回收、文件的按名存取、文件的组织、共享与保护等功能。

第二节 操作系统的发展

操作系统是在解决计算机系统所面临的问题中诞生并发展的自然结果。操作系统的发展从时间顺序上经历了从无操作系统到单道批处理系统、多道程序系统(多道批处理系统、分时系统)的发展过程。随着计算机应用领域的扩大、计算机体系结构的多样化,又出现了微机操作系统、网络操作系统、实时操作系统、嵌入式操作系统和物联网操作系统。本节以计算机发展的时间为线索,说明操作系统是如何在解决计算机系统面临的问题中自然发展的。

一、无操作系统

第一代计算机(1945~1955 年)使用电子管作为主要的电子器件,用插件板上的硬连线或穿孔卡片表示程序,没有用来存储程序的内存,无操作系统。

以 1946 年诞生于宾夕法尼亚大学的第一台实用电子计算机“埃尼阿克”(ENIAC)为例,它没有真正的内存,只有 20 个字节的寄存器用来存储数字。每个字节 10 位,也就是只有 200 位的“存储”容量,无法支持存储程序。操作系统需要常驻内存,当时的情况是不仅硬件技术无法支持操作系统的运行,理论研究领域也没有操作系统的概念。使用“埃尼阿克”的方式是程序员用电路的连接方式表示算法,一旦算法发生改变,必须变更电路的连接方式。程序员在使用“埃尼阿克”进行计算工作之前,先用插件板上的电路连接表示程序,同时预约好一段上机时间。然后到机房中将插件板接到计算机中,开启计算机开关,开始一次运算。显然,一个用户程序进入计算机和退出计算机系统都需要人工干预,计算机无法自动完成程序的加载和卸载。因此,整个计算机系统处于运行—因等待人工操作暂停—运行,这样

一种不能连续自动工作的状态。

二、单道批处理系统

第二代计算机(1955 ~ 1965 年)使用的主要电子器件是晶体管, 开始使用磁性存储设备, 内外存容量增加, 计算机运算速度提高, 出现了早期的单道批处理系统。

在本书中, 将用户程序及程序处理的数据统称为作业。在单道批处理系统出现以前, 使用计算机的方式是操作员把作业挂到输入设备上, 启动机器, 运算结束取结果。典型的情况是程序员用穿孔卡片表示程序和数据, 需要运行这些穿孔卡片表示的作业时, 程序员把卡片盒交给输入室的计算机操作员, 然后程序员开始等待输出结果。计算机运行完当前作业后, 将计算结果从打印机上输出。操作员到打印机上撕下运算结果送到输出室, 程序员可以在这里取到自己作业运行的输出结果。

这种对计算机的人工控制模式存在的问题是, 机器速度很快的时候 CPU 长时间等待人工操作, 造成了 CPU 资源的严重浪费。该问题的解决方案是利用磁性存储设备提高输入/输出(简称为 I/O)速度。同时, 系统程序常驻内存并自动装入作业, 撤销作业, 输出运算结果。

这里更详细地描述这一解决问题的思路。操作员在输入室收集全部作业, 然后用一台相对便宜的、专门用来输入的计算机把作业从穿孔卡片输入到磁带上, 一盘磁带可以存储多个作业。然后, 操作员把存有批量作业的磁带装到磁带机上。操作员在专门用于计算的、性能较高的计算机上装入早期的操作系统, 也称监视程序, 它能自动从磁带读入作业并使之运行, 把作业运行结果写入专门存放输出结果的磁带上。每当一个作业运行结束后, 操作系统自动从磁带中读入下一个作业并使之运行。当一批作业运行完毕后, 操作员取下输入和输出磁带, 将输入磁带换成下一批作业, 并把输出磁带装到专门用于输出的计算机上, 打印输出结果。其过程如图 1-2 所示。

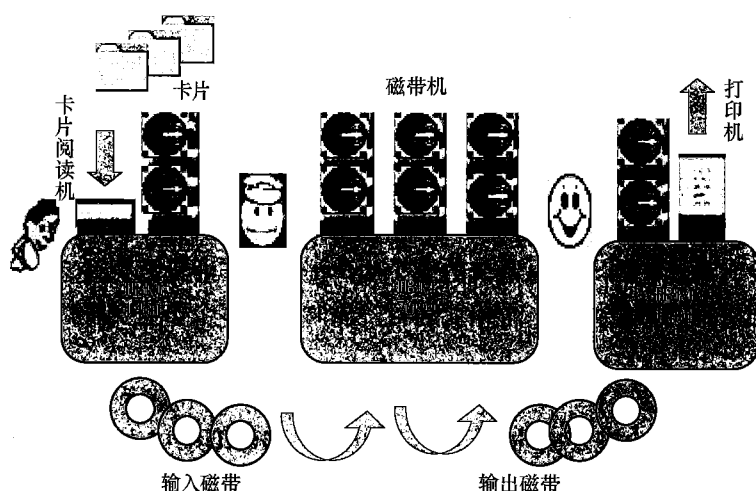


图 1-2 单道批处理系统示意图

这一时期的操作系统是单道批处理系统, 内存中只能驻留一道用户作业, CPU 和内存资源被用户作业独占。程序是指令的集合, 程序的执行是 CPU 依次、逐条执行指令的过程。

当执行到输入/输出指令时,启动设备控制器工作。在设备控制器控制设备完成输入/输出(磁带和内存之间的数据传输)的一段时间内是不需要 CPU 干预的。在单道批处理系统的管理下,此时间段内 CPU 空闲,无事可做。对于租用计算机的用户而言,损失了 CPU 的机时,而计算机系统损失了系统的吞吐量。吞吐量是指单位时间内计算机系统处理的作业量。

三、多道程序系统

随着电子技术的发展,计算机开始采用集成电路芯片作为主要的电子器件,IBM 360 是第一个采用小规模集成电路芯片的主流机型。这个时期,生产厂商试图通过生产通用计算机,把用于科学计算、工程计算及磁带归档、打印处理的两类计算机合二为一。但是通用计算机存在一个问题,对于 CPU 操作密集型的作业(即使用 CPU 频率高的作业),输入、输出操作较少,CPU 利用率比较高。但是,当通用计算机用于运行输入和输出密集型的作业时,由于 CPU 频繁等待输入、输出,而 CPU 在等待输入、输出完成的过程中只能原地踏步,无法进行任何有效的工作,使 CPU 的浪费最多可达到 80%~90%。为了解决这一问题,提高 CPU 的利用率,提出了多道程序系统的概念。OS/360 是 IBM 开发的第一个多道程序系统。

多道程序系统的最初想法是将内存分成几个部分,每一部分存放不同的作业,如图 1-3 所示。当一个作业等待输入/输出操作完成时,另一个作业可以使用 CPU。任何时刻,当一个作业运行完毕,操作系统会自动从外存装入另一个作业到空闲的内存区域。从理论上讲,如果内存中可以同时存放足够多的作业,则 CPU 利用率可以接近 100%。

早期的多道程序系统不具有交互功能,被称为多道批处理系统。程序员提交作业后,在作业运行结束、输出结果之前,无法观察和控制作业的运行。程序员提交作业后可能等待很长的时间,得到的却可能是因为一个小小的语法错误而造成的错误结果。解决这一问题的需求非常迫切,于是出现了分时操作系统。

在分时操作系统的支持下,多个用户可以同时通过不同的终端使用主机,主机可以快速响应常用命令,如程序调试命令。使终端用户感觉自己独占计算机资源,并且实现用户与主机的及时交互。在分时系统中同时登录系统的多个用户提交的作业轮流执行,每个作业都是运行一小段时间就把主机资源让给另一个作业运行一段时间,多个作业交替执行,分时使用主机资源。

第一个通用分时系统 CTSS(Compatible Time Sharing System)是麻省理工学院于 1962 年在一台改装过的 IBM 7094 上开发成功的,后续又出现了 MULTICS(MULTiplexed Information and Computing Service)、UNIX、Linux 和 Windows 等著名的既支持多道批处理功能、又支持分时功能的多道程序系统。

四、微机操作系统

随着个人计算机的出现,微机操作系统应运而生。第一个微机操作系统是 Intel 公司的

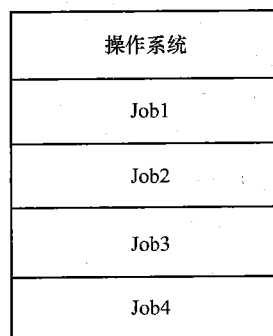


图 1-3 内存中有多道作业的多道程序系统

顾问 Gary Kildall 编写的 CP/M (Control Program for Microcomputer) 系统。这是一个磁盘操作系统, 用于 Intel 8080。1977 年, CP/M 被重写, 使其可以在使用 Intel 8080、Zilog Z80 及其他 CPU 芯片的多种微型计算机上运行。从此, CP/M 在微机操作系统市场风行了几年, 成为了最著名的 8 位操作系统。

20 世纪 80 年代初, IBM 进军个人计算机市场。在 IBM 向 Digital Research 公司提出为 IBM 个人计算机编写 16 位操作系统时, Digital Research 拒绝了 IBM 的请求。这无疑是 Digital Research 最糟糕的一次商业决策。此时, 一直与 IBM 有联系的比尔·盖茨向一家计算机制造商购买了 DOS (Disk Operating System) 操作系统。后来, 经过对 DOS 的修改, 形成了新的 DOS 版本 MS-DOS (MicroSoft Disk Operating System)。以后, 比尔·盖茨的微软公司采取了将 MS-DOS 与硬件捆绑销售的策略。随着 CPU 的不断升级, 微软及时更新 MS-DOS, 使之日臻完善, 成为微机操作系统的主流。

受到 Macintosh 的影响, 1985 年微软开始构建 Windows 操作系统。时至今日, 虽然面临各种挑战, Windows 操作系统依然是微机操作系统的主流。目前, 除 Windows 系列的微机操作系统之外, 还有 Linux、Chrome OS 和 Mac OS 等一些支持个人计算机的操作系统。

五、实时操作系统

随着计算机被广泛应用于各种工业现场的自动控制、智能机器人、海底探测和航空航天等领域, 出现了各种实时系统。实时系统是支持实时计算的系统。实时计算的正确性不仅依赖于系统计算的逻辑结果, 还依赖于产生正确结果的时间。简单的解释就是, 实时系统不仅要求系统能正确地计算出 1 加 1 等于 2, 而且要求必须在规定的时间内计算出 1 加 1 等于 2。如果计算结果正确, 但是时间超过了规定时间, 依然被认为计算出错。例如, 锅炉温度和压力自动控制系统、重病监测系统等都属于实时系统。

实时系统可以使用操作系统, 也可以不用操作系统。在高端的实时系统中通常都有高可靠性和支持实时资源调度的实时操作系统 (Real-time Operating System, RTOS)。VxWorks 操作系统是美国 WindRiver 公司于 1983 年设计开发的一种嵌入式实时操作系统; $\mu\text{C}/\text{OS-II}$ 是一种被广泛应用于微处理器、微控制器和数字信号处理器的开源实时多任务操作系统内核; Linux 系统也支持实时性。

六、批处理系统、分时系统、实时系统的特点

1. 单道批处理系统的特点

单道批处理系统内存中只有一道作业, 可以自动成批处理作业, 其特点如下。

(1) 自动性

单道批处理系统使计算机能够在操作系统控制下, 自动地将作业从外存装入内存运行。当作业运行完毕后, 自动撤销已运行完毕的作业, 并依次从外存装入下一个作业, 使之运行。

(2) 顺序性

存放在外存中的作业按顺序依次被装入内存运行, 先进入内存的作业先运行完毕。

(3) 单道性

任何时刻内存中只有一道作业。即操作系统每次只能从外存装入一个作业进入内存。只

有当该作业正常或异常结束，才能将下一个作业装入内存。

单道批处理系统与无操作系统的计算机系统相比而言，减少了等待人工操作的时间。但是单道批处理系统中，由于作业独占 CPU 和内存，当作业进行 I/O 时，CPU 只能等待 I/O 完成而无事可做，使得 CPU 资源不能得到充分利用。

2. 多道批处理系统的特点

在多道批处理系统中，用户所提交的作业都先存放在外存中并排成一个队列，该队列被称为“后备作业队列”。由操作系统的作业调度程序按一定策略从后备作业队列中选择若干个作业调入内存，使它们共享 CPU 和系统中的各种资源，以达到提高资源利用率和系统吞吐量的目的。与单道批处理系统相比，多道批处理系统支持多道程序驻留内存，CPU 可以不再空闲等待 I/O。多道批处理系统的特点如下。

(1) 多道性

在内存中可同时驻留多道程序，当在 CPU 上运行的作业提出 I/O 请求后，该 CPU 可以执行其他作业，从而有效地提高了系统资源的利用率和吞吐量。

(2) 无序性

多个作业完成的先后顺序与它们进入内存的顺序之间没有严格的对应关系。同时驻留在内存中的作业，其获得 CPU 的顺序和执行的进度无法预知，先进入内存的作业不一定先被调度，也不一定先执行完。

(3) 调度性

多道程序系统必须具有作业调度和进程调度功能。作业调度用来从后备作业队列中选择一个或多个要被装入内存的作业。进程调度程序用来从内存中选择一个(单 CPU 系统)或多个(多 CPU 系统)进程，使其在 CPU 上运行。

(4) 复杂性

由于多道程序系统中的作业共享 CPU、内存、外设和文件，程序并发执行，多道程序系统必须具有处理机管理、内存管理、I/O 设备管理、文件管理和作业管理的功能，分时系统还必须解决终端用户管理的问题。因此，其功能和实现技术都比单道批处理系统复杂得多。

多道批处理系统的优点是能够提高 CPU、内存和 I/O 设备的利用率和系统的吞吐量。

多道批处理系统的缺点是系统平均周转时间长，缺乏交互能力。周转时间是指从作业被提交给系统开始到作业完成为止的时间。平均周转时间是指所有作业的周转时间之和与作业数量的比值。

3. 分时系统的特点

分时操作系统允许多个用户通过终端同时使用计算机。分时操作系统是多道程序系统的自然延伸，支持多个用户任务同时驻留内存，每个用户通过终端与主机交互时都能得到快速响应。

分时系统的特点是多路性、独立性、及时性和交互性。多路性是指允许在一台主机上同时连接多台联机终端。独立性是指各终端用户彼此独立操作，互不干扰。及时性是指用户请求能在很短的时间内获得响应。交互性是指用户可以通过终端与系统进行广泛的人机对话。

分时系统的优点是向用户提供了人机交互的方便性，使多个用户可以通过不同的终端共

享主机。

分时系统的实现需要解决两个关键问题，即及时接收和及时处理。同时连接多个用户终端的分时系统要能及时接收用户的命令或数据，新用户命令到来时，尚未处理的用户命令要通过缓冲区来妥善保存。此外，分时系统要能及时处理用户的交互命令，使系统的响应时间得到保证。为了实现这一目标，应该使所有的用户任务都直接进入内存，并在很短的时间内使每个任务都运行一次。

4. 实时系统的特点

实时系统主要用于实时控制和实时信息处理领域。实时系统必须能及时响应外部事件的请求，在规定的时间内完成对该事件的处理，并控制所有实时任务协调一致地运行。与分时系统相比，实时系统具有以下特点。

(1) 多路性

实时系统的多路性除具有与分时系统相同的特点外，主要表现在经常对多路的现场信息进行采集，以及对多个对象或多个执行机构进行控制。

(2) 独立性

实时信息系统与分时系统一样，每个终端用户在向实时系统提出服务请求时，是彼此独立地操作，互不干扰。而在实时控制系统中，信息的采集和对对象的控制也是彼此互不干扰。

(3) 及时性

实时信息处理系统对实时性的要求是以人所能接受的等待时间来确定的，而实时控制系统的及时性则是以控制对象所要求的开始截止时间或完成截止时间来确定的。

(4) 交互性

实时信息系统的交互性仅限于访问系统某些特定的专用服务程序。

(5) 可靠性

实时系统比分时系统要求有更高的可靠性。因为任何实时系统的错误都可能带来巨大的经济损失，甚至危及生命安全。因此，在实时系统中，往往采取多级容错措施来保证系统安全和数据安全。

需要注意的是，批处理系统、分时系统和实时系统是三种基本的操作系统类型，而一个实际的操作系统可能兼有三者或其中两者的功能特点。

七、操作系统产品现状

1. 主机操作系统

主机操作系统是运行在大型主机上的操作系统，主要提供三类服务：批处理、事物处理和分时处理。批处理系统处理不需要交互式用户干预的周期性作业，保险公司的索赔处理或者连锁商店的销售报告通常以批处理的方式完成。

事务处理系统负责大量的小请求。例如，银行的支票处理或航班预定。每个业务量都很小，但是系统必须每秒处理成百上千个业务，需要很高的处理速度。

分时系统允许多个远程用户同时在计算机上工作，执行命令或处理作业。

一个大型主机操作系统通常同时具有批处理、事务处理和分时处理的功能。IBM 的 OS/360 和 OS/390 是典型的大型主机操作系统。Linux 和 UNIX 的一些版本也是很多大型机常用

的操作系统。

2. 服务器操作系统

服务器操作系统是运行在网络服务器上的操作系统，可以通过网络同时为众多用户服务，允许用户共享硬件和软件资源。服务器可提供打印服务、文件服务和 Web 服务。典型的服务器操作系统有 Solaris、FreeBSD、Linux 和 Windows Server 200X。Linux 作为影响力最大的开源软件，其内核根据不同的需要被定制，广泛地应用于服务器、PC 和移动设备。一些互联网公司（如腾讯、阿里巴巴等）使用 Linux 内核的不同发行版作为自己的网络服务器操作系统。

3. 微机操作系统

微机操作系统也称个人机操作系统，现代微机操作系统都支持多道程序处理，就是通常所说的支持多任务。微机操作系统为单个用户提供良好的应用环境和应用软件开发环境。常见的微机操作系统有 Windows 的个人计算机操作系统系列、Linux 的不同发行版、Macintosh 和 Chrome OS 等。

4. 嵌入式操作系统

嵌入式系统的概念出现在 20 世纪 70 年代，是宿主于非计算机设备中的计算机系统。嵌入式操作系统是运行于嵌入式设备中的操作系统。第一个商用嵌入式操作系统 VRTX32 出现于 1981 年。20 世纪 80 年代的嵌入式操作系统主要用于军事和电信产品，如 VRTX32 和 PSOS。20 世纪 90 年代，微内核设计技术和模块化的设计思想开始渗入嵌入式操作系统。互联网的迅速发展对嵌入式操作系统提出了新的要求，为了方便使用大量现有的软件代码，嵌入式操作系统需要支持标准的 API。这个时期出现了几十种产品，具有代表性的有 VxWork、QNX、WinCE 和 Android 等。嵌入式操作系统在技术上的特征是设计上采用更易于移植的方式，以适应越来越多的新处理器；开放源码，提高性能，保证高可靠性。嵌入式 Linux 在消费电子设备中得以广泛应用。

目前，应用于嵌入式系统的实时操作系统产品非常多。在物联网快速发展的情况下，出现了物联网操作系统。物联网操作系统本质上是连接在物联网的各种可计算设备中的操作系统，也就是嵌入式操作系统。物联网操作系统的产品较为丰富，如苹果公司的 iOS、谷歌的 Android Things、微软的 Windows 10 IoT Core 和开源物联网项目 FreeRTOS 等。

嵌入式操作系统的特征是小巧、实时性、可装卸、代码固化、弱交互性、强稳定性、接口统一、低能耗。

嵌入式操作系统的应用领域有掌上电脑、智能手机、数码相机、自动售货机、自动取款机、工业控制设备、军工装备、游戏机、医疗设备和网络设备等。

第三节 操作系统的特征

现代操作系统都支持多任务，具有并发、共享、虚拟和异步性特征。

1. 并发

并发是指两个或多个事件在同一时间间隔内发生。并发强调“同一时间间隔”，与并行是有区别的两个不同的概念，并行是指多个事件同时发生。

以单 CPU 多任务的操作系统为例，当说两个程序并发执行时，是指从宏观上看这两个

程序同时在向前推进。例如，可以在一台单 CPU 单核的笔记本电脑中一边运行音频播放程序听歌曲，一边用 Micro Office Word 编辑文档，编辑程序在工作的同时音频播放程序也在工作。但是，在单 CPU 单核的系统中，任意时刻只能有一个程序流在 CPU 上执行，要么是编辑文档程序，要么是音频播放程序。之所以用户能感觉到两个程序似乎在同时工作，而且互不干扰，是因为操作系统使两个程序在 CPU 上快速地轮流交替执行，实现了“并发”的功能。

2. 共享

共享是指系统中的资源可供内存中多个并发执行的进程共同使用。资源共享有两种方式，即互斥共享和同时共享。

互斥共享是指任意时刻一种资源只能被一个进程访问，当一个进程访问资源时，其他进程必须等待，直到资源被进程访问完毕，释放访问权。例如打印机的访问，如果不采取互斥访问的方式，可能会把多个进程的输出内容打印到同一页打印纸上。

同时，共享是指从宏观上看，资源可以被多个进程同时访问，如对磁盘的访问。虽然任意时刻只能有一个程序访问某个磁盘。但是，多个程序可以在短时间内轮流访问该磁盘，从用户的角度看，多个用户程序可以同时发送磁盘访问命令，并可以同时得到响应。从宏观上看，磁盘可以被多个用户程序同时访问。

并发与共享的关系是相互依赖的。如果系统不支持并发，资源的使用都是独占的，不需要共享资源。另一方面，如果不能实现有效的资源共享，并发就不可能得到很好的支持。

3. 虚拟

虚拟是指通过某种技术把一个物理实体变成若干逻辑上的对应物。物理实体是实际存在的，而虚拟仅仅是用户感觉到的。如虚拟打印机、虚拟内存和虚拟 CPU 等，都是操作系统通过某种技术把少量的物理资源变成较物理资源多的逻辑资源，使共享资源的每个用户都感觉到自己独占了系统资源。

4. 异步性

进程以不可预知的速度向前推进。内存中的每个程序何时执行、何时暂停、以怎样的速度向前推进，以及每道程序总共需要多少时间才能完成等，都是不可预知的。系统中什么时候会出现需要特殊处理的事件，如键盘中断、鼠标中断等也是随机的、不可预知的。现代操作系统需要很好地处理这些异步事件。

第四节 操作系统的功能

本节简单介绍操作系统的存储器管理、进程管理、设备管理和文件管理等功能，以及操作系统提供的用户接口。

一、内存管理

内存管理的主要任务是为多道程序的运行提供良好的环境，方便用户使用内存，提高内存的利用率，以及从逻辑上扩充内存以实现虚拟存储。为此，内存管理应具有内存分配、内存保护、地址映射和内存扩充功能。

1. 内存分配

内存分配的主要任务是为每道程序分配内存空间,使它们各得其所。提高存储器的利用率,以减少不可用的内存空间。允许正在运行的程序申请附加的内存空间,以适应程序和数
据动态增长的需要。

操作系统可以采用两种方式实现内存分配,即静态分配方式和动态分配方式。采用静态分配方式是把内存划分成固定大小和数量一定的区域,在系统运行过程中各分区的大小和数量不再变化。采用动态分配方式是系统在运行过程中,根据进程的请求分配内存,内存中分区的大小和数量都是动态变化的。

为了实现内存的分配,需要以下数据结构和功能支持。

1) 用于内存分配数据结构。用来记录内存使用状况,如内存空闲区域的大小、空闲区域的起始地址等,为内存分配的实现提供依据。

2) 内存分配功能。系统按照一定的内存分配算法分配内存空间。

3) 内存回收功能。系统需要回收被释放的内存空间。

2. 内存保护

内存保护的任務:一是使操作系统内核的空间不会被用户随意访问,以保证系统的安全和稳定;二是确保每道用户程序都在自己的内存空间中运行,互不干扰。实现的方式可以采用界限寄存器存放允许程序访问的地址区间的上限和下限值。程序运行期间访问内存时,把访问地址与界限寄存器的值相比较,如果发生越界,便发出越界中断请求,以停止该程序的运行。需要说明的是,现代操作系统实现内存保护的方式不仅限于使用界限寄存器。

3. 地址映射

CPU 执行程序过程中访问内存时,需要把程序的逻辑地址转变为物理地址,这个转换的过程称为地址映射。下面首先介绍什么是逻辑地址和物理地址,然后解释地址映射的功能和过程。

(1) 逻辑地址与物理地址

一个应用程序经编译后,通常会形成若干个目标程序,这些目标程序再经过链接而形成可装入程序。这些程序中指令和数据的地址都是相对于编译链接后的机器代码程序的起始地址计算的,由这些地址所形成的地址范围称为地址空间,其中的地址称为逻辑地址。如图 1-4 所示,一个 `main()` 函数经过编译后形成 Intel 指令集的目标文件中,程序第一条指令的地址为 0,指令长度为 1 个字节。第二条指令的起始地址为 1,指令长度为 2 个字节。第三条指令的起始地址为 3,指令长度为 5 个字节。图 1-5 所示是经过链接后形成的可执行程序,第一条指令的起始地址不再是 0,而是 80483ec(用十六进制表示的地址)。图 1-4 和图 1-5 中的地址都是逻辑地址。在本书中,把所有非物理地址的其他各种形式的地址均称为逻辑地址。此外,由内存中的一系列单元所限定的地址范围称为内存空间,其中的地址称为物理地址,CPU 访问内存需要将物理地址送入地址总线,以选中要访问的内存单元。

(2) 地址映射

在多道程序系统中,地址空间中的逻辑地址和内存中的物理地址是不一致的。因此,存储器管理必须提供地址映射功能,以将逻辑地址转换为对应的物理地址。地址映射应在硬件的支持下完成。



图 1-4 目标程序中的逻辑地址



图 1-5 可执行程序中的逻辑地址

举例：高级语言把一个函数调用描述成
A()

```
{ call B
```

编译程序把 call B 翻译成 jump 0x01040 这样一条跳转语句，意思是：调用函数 B 就是跳转到函数 B 的实现代码的起始处开始执行。函数 B 实现代码的起始地址是 0x 01040，编译器给出的这个地址不是物理地址，而是一个逻辑地址。当程序 A 执行到 jump 0x 01040 指令时，系统必须通过内存管理的地址映射机制把逻辑地址 0x 01040 转换成物理地址，将物理地址送到地址总线，才能访问到存放 B 的第一条指令的物理内存单元。

4. 内存扩充

内存扩充的任务是借助于虚拟存储技术，从逻辑上扩充内存容量，使系统能够向用户提供比物理内存大的存储容量。为了能从逻辑上扩充内存，系统必须具有内存扩充机制，以实现下列功能。

1) 请求调入功能。允许系统在装入一部分用户程序时就启动该程序的运行，在程序运行过程中若发现要执行的指令或要访问的数据尚未装入内存，通过请求调入将需要的指令或数据装入内存。

2) 置换功能。在请求调入时，若发现内存空间不足，需要系统先将内存中的一部分内容换出到外存中，以便腾出内存空间装入当前需要的内容。

二、进程管理

进程可以被认为是程序的执行实体(进程的详细介绍见本书第二章)。进程管理功能主要包括进程的描述与组织、进程控制、进程同步、进程通信及进程调度。进程控制完成进程创建、撤销进程、唤醒进程和阻塞进程等操作。进程同步功能完成多个进程(含线程)运行的协调与互斥。进程通信功能用来实现进程之间的信息交换。进程调度功能是从多个可执行的进程中选出一个进程，把处理机分配给它。

三、设备管理

设备管理主要完成用户的 I/O 请求，为用户分配 I/O 设备。为了完成这些任务，设备管理应该具有以下功能。

1) 缓冲管理。管理各种缓冲区。

2) 设备分配。分配用户 I/O 所需要的设备。

3) 设备处理。由设备驱动程序来实现 CPU 与设备控制器之间的通信。

4) 设备独立性和虚拟设备。设备独立性功能使应用程序独立于物理设备。例如，用高级程序设计语言编写打印图形的程序，与使用什么型号、品牌的打印机无关。虚拟设备的功能是把一个物理设备变换为多个对应的逻辑设备，使一个物理设备能供多个用户共享。

四、文件管理

1. 文件存储空间的管理

为每个文件分配必要的外存空间，提高外存利用率，并能有助于提高访问文件的速度。

2. 目录管理

为每个文件建立目录项并对众多目录项进行有效组织。目录项包括文件名、文件属性及

文件的地址等信息。

3. 文件的读、写管理和存取控制

根据用户的请求，从外存中读取数据或将数据写入外存。防止未经核准的用户存取文件，防止冒名顶替存取文件，防止以不正确的方式使用文件。

五、提供用户接口

为了方便用户使用操作系统，操作系统向用户提供了用户与操作系统之间的接口。操作系统向最终用户提供命令行和图形用户接口，向程序员提供应用程序与操作系统之间的接口即系统调用。系统调用是操作系统实现的具有某种功能的程序模块，应用程序可以通过系统调用的接口使用操作系统实现的功能，获得操作系统内核的服务。

1. 命令接口

为了便于用户与计算机系统的交互，早期的操作系统都向用户提供了命令接口。命令接口又可分为联机用户接口和脱机用户接口。

(1) 联机用户接口

这是为联机用户设计的接口，它由一组键盘操作命令和命令解释程序组成。当用户在终端或控制台上每键入一条命令后，系统便立即转入命令解释程序，对该命令进行解释并执行该命令。在完成功能后，控制又回到终端或控制台上，等待用户键入下一条命令。

(2) 脱机用户接口

脱机用户接口是为批处理作业的用户提供的，也称为批处理用户接口。

2. 图形用户接口

20 世纪 90 年代，在操作系统中开始引入图形化用户接口。在提供图形用户接口的操作系统中，用户可以轻松地通过选择窗口、菜单、对话框、滚动条、图标等简单的操作来完成对作业和任务的提交与控制。用户不必记住与操作对应的命令和命令参数的格式。图形用户接口使操作计算机成为相对简单的事情，也使更多的非专业人员能很快掌握计算机和应用程序的操作。

3. 程序接口

操作系统提供给程序员的接口是系统调用。在类 UNIX 系统中，系统调用多使用 C 语言所提供的库函数。例如，在程序中，使用 C 的函数 `fseek()`，而 `fseek()` 的代码调用了相应的系统调用。一般的操作系统都提供进程控制、文件操作、通信管理和系统维护等几类系统调用。关于系统调用的详细内容参见本书第二章第三节。

第五节 操作系统的体系结构

操作系统的体系结构是操作系统作为一种软件的体系结构，对操作系统的可靠性、灵活性和可移植性等方面都有重要影响。

一、软件体系结构简介

随着计算机硬件技术的飞速发展，应用领域日益广泛，软件的规模和复杂性不断增加。人们迫切需要一种新的超越传统算法和数据结构的高层抽象结构来描述整个系统，从而提出

了软件体系结构的概念。

软件体系结构是一个复杂软件系统的高层结构,为软件系统提供了一个结构、行为和属性的高级抽象,包括系统元素的结构、元素间的相互关系,以及指导元素集成的模式和约束三个方面。它采用形式化的方法刻画了软件系统的组织和拓扑结构,显示了系统需求和构成系统的元素之间的对应关系,能够针对系统需求对软件的早期设计进行正确性和完整性分析,从而降低了开发后期产生错误的数量,减少了软件维护的成本,也提高了软件重用的抽象层次和粒度,对提高软件系统的质量和开发效率起着巨大的作用。总的来说,明确清晰的软件体系结构是一条贯穿软件系统整个生命周期的主线,是软件设计成功的基础和关键,也是对软件大型化、复杂化趋势的一种很好的应对决策。

二、操作系统体系结构的分析

操作系统体系结构研究整个操作系统内各组成成分,以及成分间和成分内部存在的相互关系。在操作系统的发展过程中,产生了许多不同类型的体系结构模型,它们各有其适用范围。

应用需求的变化和硬件体系结构的不断发展对操作系统的可靠性、可扩充性、可移植性及开放性都提出了更高的要求。嵌入式应用的发展要求操作系统具有更高的开发效率和较低的开发成本,这些变化必然要求修改和增强现有操作系统的结构。因此,深入理解和认识操作系统内核体系结构的实现模型,从而研究出更优越的操作系统体系结构和开发方法具有重要意义。本节详细介绍几种操作系统体系结构模型。

1. 简单的监控程序模型

最初的计算机中并不存在操作系统这个概念,所有的任务都直接运行于硬件上,由程序员手工操作硬件系统。随着作业控制语言的出现,产生了简单的监控程序。这种系统的功能相当简陋,任意时刻系统中只能运行一个任务,这样保证了对系统信息的互斥访问,保护了系统数据的安全。但是这种设计的主要问题是系统缺乏“可伸缩性”,任务独占整个系统,这将极大地降低系统中潜在的并发性,从而使系统效率严重受损。在结构设计方面,没有任何结构方面的指导,整个系统的所有功能全部集中在一个单一模块中。后来的单体结构模型就是在此基础上发展起来的。这个阶段的典型代表是 FMS(FORTRAN Monitor System)和 IB-SYS(在 IBM 7094 机上运行)。

2. 单体结构模型

单体结构模型是软件工程出现以前的早期操作系统及目前一些小型操作系统采用的体系结构。在单体结构模型中,所有的软件和数据结构都放置在一个逻辑模块中,对外层的用户程序提供一个完整的内核界面——系统调用。整个系统由若干具有一定独立功能的子程序组成,系统各部分之间没有确定的位置概念,彼此间没有信息隐藏,允许任一子程序调用其他子程序。因此,它的特点是结构简单,便于理解和实现,而且系统所有部分都集中在一个内核中,效率较高。缺点也很明显,由于各子程序间可以互相调用,系统结构关系复杂,容易引起循环调用和死锁。修改一个子程序往往会导致若干相关部分的变动,系统的可修改性和可维护性比较差。图 1-6 所示为单体结构操作系统。

随着软件工程的提出和发展,借助于软件工程的模块化思想,内核开发人员根据功能的不同,可以将操作系统内核结构划分成不同的模块,模块间仅能通过各自定义良好的对外接

口进行相互通信，不能访问各模块内部的变量。这样不管一个模块内部如何变化，只要对外提供的接口保持不变，就不会影响到其他模块，从而局部化模块的变更，在一定程度上缓解了单体内核结构的复杂性，也提高了系统的可维护性和可扩充性。

单体内核是操作系统中最早、最常见的体系结构。在计算机发展初期，由于硬件设备和系统资源都极其昂贵和奇缺，单体结构的简单和高效性满足了系统的设计要求，因而得到了广泛的应用。具有单体内核结构的典型操作系统有 UNIX 系统、MS-DOS、Linux、Mac OS X 和 BSD 等系统。

3. 层次结构模型

由于单体结构中各部分之间紧密依赖、相互调用，当系统中的模块数较多时，无序相互调用所造成的系统复杂性将难以驾驭。为了消除这个弊病，就必须先从改变模块间的无序调用着手，实现有序调用，从而提出了层次结构操作系统，如图 1-7 所示。

层次结构的基本思想是将操作系统分解为多个小的、容易理解的层，系统功能被隔离在不同层中，每一层提供对系统功能的部分抽象，然后采用单向调用的顺序，形成一连串彼此连续的对系统功能的“抽象串”，最终形成对整个系统的完整抽象。它为系统中各个模块赋予了位置的理念。在操作系统的系统结构中，各个模块都有

相对固定的位置、相对固定的层次。层与层之间有严格的接口定义，并且每一层只依赖于它下层提供的服务而工作，不能访问上层所提供的服务。这样组织的好处是方便了操作系统的实现、调试和测试。“分层”结构使操作系统中各模块间的组织和依赖关系清晰明了，系统的可读性、可靠性和可移植性都得到了增强。此外，对某一层的修改或替换只影响邻近的两层，便于系统的维护和扩充。分层设计的困难之处在于如何决定各层的顺序和内容，处理不好将会出现“循环依赖”的问题。另一个问题就是性能问题，有时系统完成一个功能需要穿越多个层才能实现，系统分层粒度越小，所花费的额外开销也就越大。出于效率考虑，有些系统允许跨层向下调用。

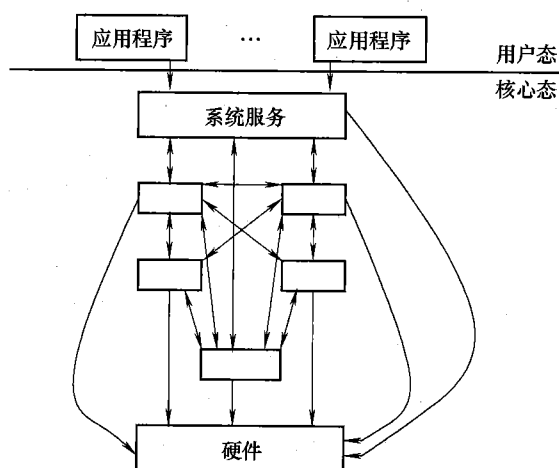


图 1-6 单体结构操作系统

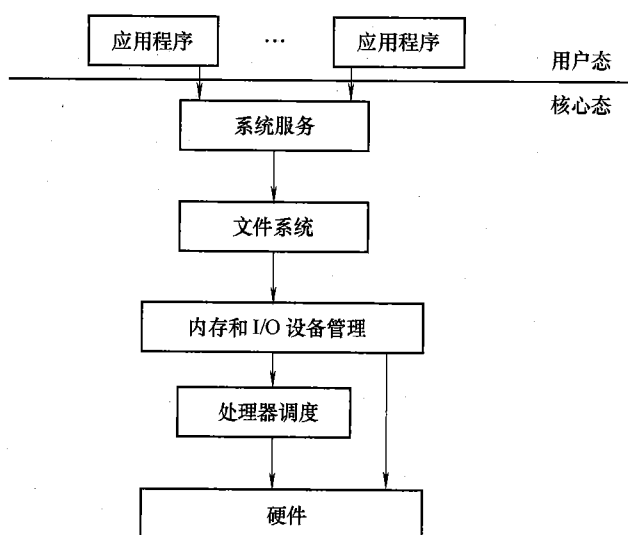


图 1-7 层次结构操作系统

分层结构最经典的例子是 Dijkstra 的 THE 系统，它是 20 世纪 60 年代在 Technische Hogeschool Eindhoven 上开发的。由于操作系统中各功能模块之间的调用关系是网状的，因此分层设计的思想有过度的局限性。要实现一个真正的分层操作系统是非常困难的，所以它难以被商用操作系统采用和推广，不过开发者在设计操作系统时可以借鉴分层的思想，使得系统在抽象层次上努力达到分层的目标。

4. 客户/服务器模型与微内核结构

传统操作系统一般将所有功能模块放在内核模式中完成。随着计算机硬件体系结构的迅速发展，这种实现方式使得操作系统内核的结构变得非常复杂，代码量也迅速增加，这样系统代码的可扩展性、可移植性和可维护性越来越差，错误也随之增多。为了脱离这个困境，人们提出了微内核技术。

微内核技术是操作系统发展的一个里程碑，它产生了一种完全不同的操作系统体系结构，提供了操作系统发展的新途径。它的核心思想是核心功能外移，即把传统操作系统内核中的一些组成部分（如文件系统、网络和驱动程序等内核功能）放到内核之外作为一个独立的服务器进程来实现，在微内核中只保留了操作系统最基本的功能，包括处理机调度、存储管理和消息通信等，这些服务器借助于微内核传递消息来实现相互之间的交互。例如，当用户进程请求系统服务时，内核将用户的请求以消息的形式发送给相应的服务器进程，并将服务器进程返回的信息以消息的形式传送给用户进程。因此，微内核更多的是起着信息交换的作用：验证信息、在服务器间传递信息并授权访问硬件。

微内核结构用一个水平分层的结构代替了传统的纵向分层的结构，这样带来的好处是系统结构清晰，具有较高的灵活性、可靠性和可维护性：首先，只要接口规范，操作系统可以方便地增加或删除服务功能，系统具有很好的灵活性；其次，服务器进程运行在独立的用户进程空间中，如果某个服务器出现问题，不会引起系统其他服务器甚至是系统的崩溃，可靠性好；再者，微内核体积小，便于测试、管理和维护，而且修改服务器的代码也不会影响系统其他部分。另外，由于微内核技术实现了基于客户/服务器体系结构的操作系统，完全适宜于网络环境及分布式计算模型。

但是，微内核设计有一个严重的缺点就是效率不高：微内核采用消息传递机制来实现组件间的通信，消息收发的代价远比传统操作系统直接使用系统调用要大，因为任何消息传递潜在地都有可能跨越机器边界，那么参数就需要被复制而不是从发送者的堆栈进行引用。另外，发送者和接收者之间的同步也将增加系统的开销。为了解决微内核性能问题，目前已经提出了多种解决办法。一是直接优化系统的消息传递机制的性能，二是扩大微内核并把一些关键的服务程序和驱动程序重新加入到内核中，从而减少系统在用户态和核心态之间的切换，以及系统在不同地址空间之间的切换。图 1-8 所示为客户/服务器结构的微内核操作系统。

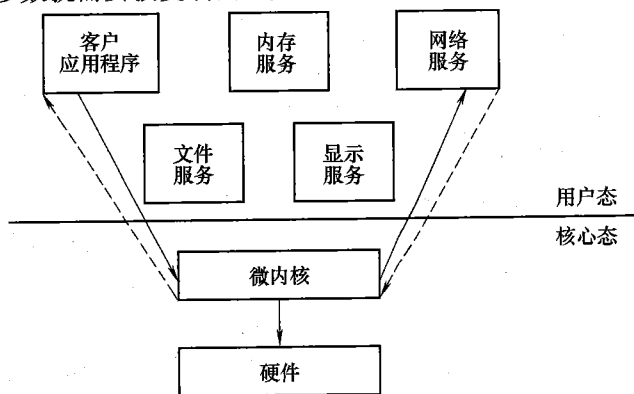


图 1-8 客户/服务器结构的微内核操作系统

微内核技术是现代操作系统的一个发展趋势,结合面向对象设计方法,微内核技术具有更大的应用空间。目前微内核结构的操作系统主要向两个方向发展:一个是向通用操作系统的方向发展,主要代表是美国微软公司研制的 Windows NT 操作系统和我国自行研制的 COS-IX V2.3。另一个就是嵌入式操作系统,由于大多数嵌入式系统要求系统的规模比较小,而微内核结构正好满足这个需求,从而覆盖了几乎整个嵌入式系统的应用领域。典型代表是卡内基梅隆大学研制的 Mach 操作系统、WindRiver 公司的 Vxworks 等。

5. 动态可扩展结构模型

随着应用软件的日益丰富,传统操作系统结构阻碍了应用程序在特定领域方面的优化,为了获得高性能和高灵活性,从 20 世纪 80 年代人们就提出了动态可扩展结构模型。动态可扩展结构的基本思想就是在运行过程中,能够动态地实现系统行为扩展的结构,也可称之为弹性结构。

要具体实现动态可扩展结构,首先要解决的问题就是如何实现系统行为的动态改变,目前实现的方法就是通过动态地增删系统的代码,采用的主要技术有 UPCALL 和 DOWNLOAD。UPCALL 技术就是指在微内核操作系统中通过核心层到用户层服务的调用方式来实现扩展的技术。由于微内核中的部分操作系统服务存在于用户层,用户可通过修改服务来获得不同的行为,但产生的后果是系统性能的下降。由此,人们又提出了外核结构模型,它试图将所有的传统操作系统内核中提供的抽象转移到用户空间,以操作系统库的形式提供服务,核心层只负责对物理设备的控制。应用程序可从用户层库中得到并控制内核抽象,从而实现了操作系统的动态扩展。这样系统具有很好的扩展性和灵活性。缺点是系统的安全性较低,内核抽象的安全必须由用户来保证,这也增加了用户的负担。这类结构的典型系统是 MIT 的 Exo kernel 操作系统。

DOWNLOAD 技术(也称为核心扩展技术)的核心思想就是将软件构件动态地下载到内核中,以达到改变操作系统行为的目的。该技术最大的难点就是系统安全问题。因为扩展程序是动态下载并组装到系统核心中的,因此不安全的程序有可能造成系统的故障甚至崩溃。当前这类系统主要采用软件保护机制进行保护。核心扩展技术是当前操作系统扩展性研究的主要方向。其中,典型代表有 Harvard 大学的 VINO 操作系统。

随着计算技术的发展和 Internet 与网络的普及及应用,产生了移动计算的需求,新的应用需求在一定程度上要求操作系统的动态可扩展性。因此,动态可扩展结构模型是适应当前应用变化的新一代操作系统体系结构模型,其应用前景十分广阔。

第六节 指令的执行

为了更好地理解操作系统原理的内容,在此对程序执行的本质及其与硬件、指令集的关系以一个简化的实例加以说明。程序是指令的集合,程序的执行就是按照某种控制流执行指令的过程。

1. 指令周期

一个单一指令需要的处理称为指令周期,如图 1-9 所示,一个指令周期可以划分成两个步骤,分别是取指周期和执行周期。

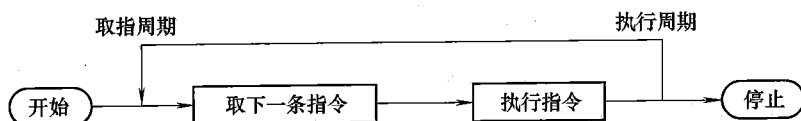


图 1-9 指令的执行

2. 取指令和执行指令

(1) 取指令

在每个指令周期开始时, 处理器从存储器中取一条指令。在典型的固定长度指令的处理器中, 程序计数器(PC)保存有下一次要取的指令的地址。除非接收到别的指示, 否则处理器在每次完成取指令操作后总是对 PC 递增, 使它能够按顺序取得下一条指令, 即位于下一个高端存储器地址的指令。

(2) 执行指令

取到的指令被放置在处理器的指令寄存器 IR 中。指令中包含确定处理器将要采取动作的位, 处理器解释指令并执行要求的动作, 这些动作可分为 4 类。

- 1) 处理器与存储器之间的指令或数据传送操作。
- 2) 处理器与 I/O 设备之间的指令或数据传送操作。
- 3) 算术运算操作或逻辑运算操作。
- 4) 控制操作, 即修改指令的执行顺序的操作。

3. 一个实例

假设一台模型机, 具备图 1-10 中列出的所有特征, 处理器包含一个称为累加器(AC)的数据寄存器。所有指令和数据长度均为 16 位, 故使用 16 位的单元或字来组织存储器。指令中用 4 位二进制位表示操作码, 12 位二进制位表示内存地址。因而最多可有 16 种不同的操作码, 可直接访问的存储器大小为 4096 字节(用 3 位十六进制数表示)。



图 1-10 模型机的指令和数据格式

该模型机内部 CPU 寄存器如下。

- 1) 程序计数器(PC), 存指令地址。
- 2) 指令寄存器(IR), 存正在执行的指令。
- 3) 累加器(AC), 临时存储体和累加操作。

部分操作码的二进制表示与其对应的操作如下。

- 1) 0001: 从内存中读取数据并送入 AC。
- 2) 0010: 把 AC 的内容存储到内存中。

3) 0101: 把从内存中取得的值与 AC 中的值相加, 结果仍存放在 AC 中。

下面举一个程序实例来说明程序的执行。该程序的功能是把内存 940 单元的内容与 941 单元的内容相加, 然后将累加结果送 941 单元。该程序通过执行 3 条指令完成, 指令的汇编形式和二进制形式如图 1-11 所示。假定 3 条指令分别存放于内存地址为 300、301、302 的存储单元内, 地址为 940 和 941 的存储单元内初始存放的值分别是 3 和 2。图 1-11 表示了程序顺序执行 3 条指令的情况。3 条二进制指令的操作码和操作数都用十六进制表示。

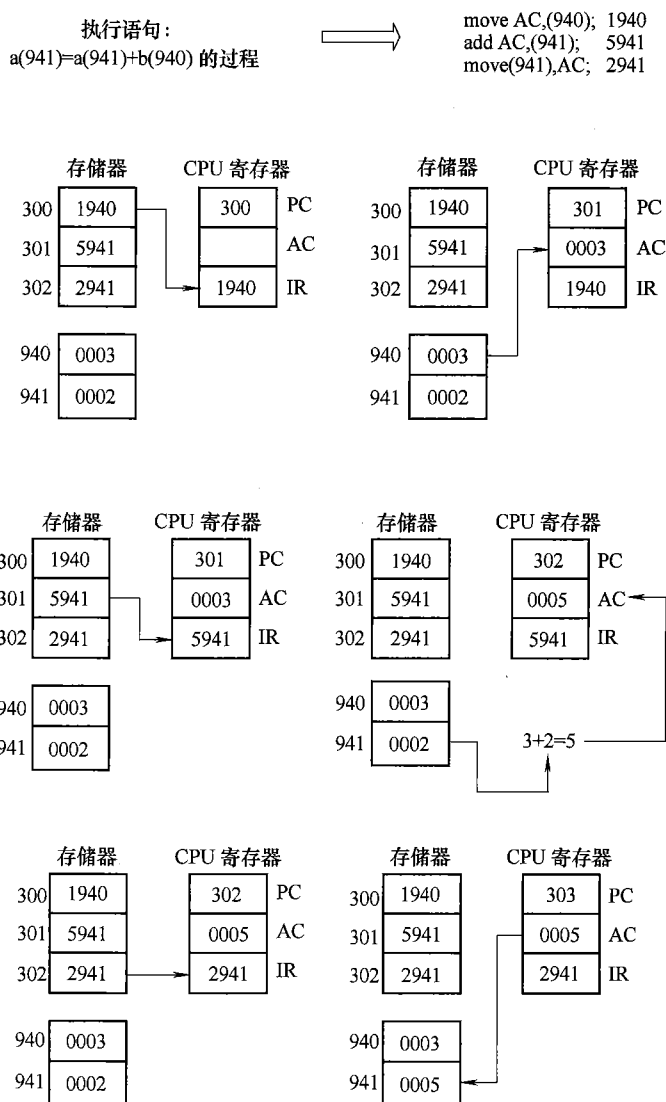


图 1-11 程序执行过程的实例

程序执行开始, PC 的值为 300, 根据该地址, CPU 从地址为 300 的存储单元取到指令 1940, 并将该指令放入指令寄存器 IR, PC 的值自动加 1, 变为 301, 指示下一条要执行的指令在内存中的地址。CPU 对 IR 中的指令 1940 进行译码, 操作码的二进制表示为 0001, 根据该模型机规定, 操作码为 0001 时执行从内存取数送入 AC 的操作, 内存地址是指令操作

数部分的 940。指令执行结果是把地址为 940 的内存单元的值 3 送入 AC，第一条指令至此执行完毕。此时 AC 中的值为 3。

CPU 根据当前 PC 指示的地址 301，到内存单元 301 取要执行的第二条指令 5941，然后 PC 自动加 1，变为 302，指令 5941 放入 IR。指令 5941 的操作码的二进制表示为 0101，表示该指令执行的操作是把从存储器中取得的值与 AC 中的值相加，结果仍存放在 AC 中。执行结果是把地址为 941 的内存单元中的 2 取出，与当前 AC 中的 3 相加，结果 5 仍存放在 AC 中，第二条指令至此执行完毕。此时 AC 中的值为 5。

CPU 根据当前 PC 指示的地址 302，到内存单元取要执行的第三条指令 2941，然后 PC 自动加 1，变为 303。指令 2941 放入 IR，解释该指令执行的结果是把当前 AC 中的值 5 送入地址为 941 的内存单元。至此，各单条指令全部执行完毕，程序也执行完毕。

4. 指令的执行小结

程序执行的过程是反复取指令和执行指令的过程。PC 始终存有下一条待取指令的地址。指令执行的结果就是使寄存器或内存单元的值发生变化，指令执行的过程也就是存储体内容不断变化的过程。取指令和执行指令是由硬件完成的，不同硬件的体系结构支持不同的指令集合，为某一种硬件平台开发的操作系统不能直接在另一种体系结构的硬件上运行。任何高级程序设计语言的程序被编译成指令集合，其中的每一条指令属于计算机体系结构的指令集。

思考题与练习题

一、单项选择题(在每小题后的四个备选项中只有一个是符合题目要求的，请将其选出并将代码填写在题后的括号内)

1. Linux 操作系统是【 】。
 - A. 单用户单任务操作系统
 - B. 单用户多任务操作系统
 - C. 多用户单任务操作系统
 - D. 分时操作系统
2. 操作系统内核中文件系统模块的主要功能是【 】。
 - A. 实现虚拟存储
 - B. 保存系统文档和用户文档
 - C. 保护系统数据
 - D. 实现对文件的按名存取和文件的存储
3. 下列关于批处理系统的叙述中，正确的是【 】。
 - A. 批处理系统允许多个用户与计算机直接交互
 - B. 批处理系统分为单道批处理系统和多道批处理系统
 - C. 单道批处理系统也可能同时是分时系统
 - D. 多道程序系统就是指多道批处理系统

二、填空题(请在题目空白处填写正确内容)

1. 单道批处理系统的内存中只能驻留_____道用户作业，CPU 和内存资源被用户作业独占。
2. 单道批处理系统与无操作系统的计算机系统相比而言，减少了_____的时间。
3. 操作系统是一组控制和管理计算机_____和_____资源、合理地各类作业进行调度，以及方便用户的程序集合。

4. 并发是指两个或两个以上的事件在同一_____内发生。
5. 现代操作系统的特征包括并发、_____、虚拟和异步。

三、简答题

1. 请说明操作系统的作用和功能。
2. 请说明单道批处理系统、多道批处理系统、分时系统的特点及优缺点。
3. 内存管理有哪些主要功能?
4. 请说明指令执行的过程。
5. 操作系统具有哪些特征?

第二章 进程管理

操作系统本身就是一种软件，本质上就是数据结构加算法。所以，要掌握操作系统就要理解算法和实现算法所需要的数据结构。本章介绍操作系统最基本的概念、数据结构、算法和一些程序的控制流程。

操作系统中最核心的概念是进程，操作系统的其他所有内容都是围绕进程展开的。所以，在学习操作系统原理时，首先理解进程是必要的。本章将详细阐述进程的概念、进程的描述、进程的组织 and 进程的控制(包括进程的创建、撤销和进程状态的改变)。

本章为了使讲解原理不至于过于抽象，使用了一部分 Linux 程序的代码来帮助读者理解相关的原理，Linux 内核代码仅作为辅助理解原理的实例，不作为考试内容，读者可根据代码的注释理解代码的流程，以达到理解原理的目的。

第一节 进程的描述

本节由程序的并发执行引入进程的定义，给出与进程有关的数据结构即进程控制块的作用和数据类型、进程状态，以及其在内核中的描述形式。

一、程序的并发执行

为了说明引入进程的原因，首先介绍程序的并发执行及程序并发执行的特点，而为了更好地理解程序的并发执行，本节先介绍程序的顺序执行，再对程序的并发执行与顺序执行进行比较，从而更好地理解程序并发执行的特点。

1. 程序的顺序执行

早期无操作系统及单道批处理系统的计算机中，程序的执行方式都是典型的顺序执行。先进入内存的程序先执行，在一个程序执行完毕之前，不能执行其他程序。程序中的指令也是依照程序的控制流依次执行，一条指令执行完毕之前不能开始执行另一条指令。程序顺序执行时具有以下特点。

(1) 顺序性

处理机的操作，严格按照程序所规定的顺序执行，即只有前一操作结束后，才能执行后继操作。

(2) 封闭性

程序是在封闭的环境下运行的。即程序在运行时独占全机资源，因而各资源的状态(除初始状态外)只有本程序才能改变。程序一旦开始运行，其结果不受外界因素的影响。

(3) 可再现性

只要程序执行时的环境和初始条件相同，当程序多次重复执行时，其执行结果相同。例如一个求两个数之和的加法程序，其可再现性是指无论程序重复执行多少次，在输入的两个加数相同的情况下，和不变，如 $1 + 1$ 永远都等于 2。

2. 程序的并发执行

程序并发执行是指在同一时间间隔内运行多个程序。一个程序执行结束之前，可以运行其他程序。对用户而言，看到的是计算机同时运行多个程序。程序并发执行的确切含义是从宏观上，用户看到多个程序同时向前不间断地推进。而从微观上，任意时刻一个 CPU 上只有一个程序在执行。在现代单 CPU 或多 CPU 的计算机中，支持多任务及支持多用户多任务的操作系统都能够有效管理和支持程序的并发执行。多道程序系统和分时系统都允许用户程序在计算机系统中并发执行，程序并发执行具有以下几个特点。

(1) 间断性

程序在并发执行时，由于它们共享资源，而资源数量又往往少于并发执行的程序数量，系统不能保证每个程序不受限制地占用资源。因而，每个程序在 CPU 上运行，都是时断时续的。当请求某种资源的程序数量大于被请求的资源数量时，就必然有因申请不到资源而暂停执行的程序。当其他程序释放资源后，该程序才可能继续执行。资源的有限使并发执行的程序呈现执行过程的间断性。

(2) 失去封闭性

程序在并发执行时，由于它们共享资源或者合作完成同一项任务，系统的状态不再是只有正在执行的某一个程序可以“看见”和改变。

例如，有一个多用户的售票系统，各终端程序共享存储待售票数量的数据库表中的字段 X，任意第 i 个终端程序设置自己的局部临时变量 Ri，采用下列程序可实施卖票管理。

```
Ri = read(X);    //从数据库中读取剩余机票数量,将该值存入售票进程的局部变量 Ri 中
If(Ri > 0)
|
|
Send(Ri);        //卖票
Ri = -;
X = Ri;          //将更新的机票数量写回数据库表的相应字段
|
```

每一个终端都可以读/写 X。X 的值不再只对某个特定的程序可见。

(3) 不可再现性

程序在并发执行时，由于失去了封闭性，也将导致其失去执行结果的可再现性。同一个程序在输入完全相同的情况下多次运行，可能出现不同的运行结果。

下面为程序并发执行结果不可再现的示例。

例 2-1: counter 是全局变量，初值为 0，进程 p1 和进程 p2 都包含执行对 counter 加 1 操作的代码。

```
进程 p1: ... counter = counter + 1; ...
进程 p2: ... counter = counter + 1; ...
```

用高级程序设计语言表达的程序语句 $\text{counter} = \text{counter} + 1$ ，经过编译后转化为多条指令以完成其功能，CPU 执行的指令单位是编译后的机器语言指令，而不是高级语言的一条语句。

用 r1、r2 表示两个通用寄存器， $\text{counter} = \text{counter} + 1$ 经编译后细化为以下形式的基本指令序列。

进程 p1:

```

r1 = counter;      //内存变量 counter 对应的内存中的数据送寄存器
r1 = r1 + 1;       //执行累加 1 的操作
counter = r1;      //将 r1 中的累加结果送内存变量 counter 对应的内存中
进程 p2:
r2 = counter;      //内存变量 counter 对应的内存中的数据送寄存器
r2 = r2 + 1;       //执行累加 1 的操作
counter = r2;      //将 r2 中的累加结果送内存变量 counter 对应的内存中

```

假设 counter 初始值为 0, p1、p2 各执行一遍, 分别对 counter 做一次加 1 操作。若进程并发执行时出现下面的执行序列。

```

r1 = counter;      //该句执行结果: r1 = 0
r1 = r1 + 1;       //该句执行结果: r1 = 1
r2 = counter;      //该句执行结果: r2 = 0
r2 = r2 + 1;       //该句执行结果: r2 = 1
counter = r2;      //该句执行结果: counter = 1
counter = r1;      //该句执行结果: counter = 1

```

会出现 p1、p2 都执行了一遍, 但是 counter 的值等于 1 而不等于 2。

而当 p1、p2 按下列顺序执行指令时, 在 counter 初值为 0 的条件下, p1、p2 还是都执行了一遍, 但是 counter 的值等于 2, 而不是错误地等于 1。

```

r1 = counter;      //该句执行结果: r1 = 0
r1 = r1 + 1;       //该句执行结果: r1 = 1
counter = r1;      //该句执行结果: counter = 1
r2 = counter;      //该句执行结果: r2 = 1
r2 = r2 + 1;       //该句执行结果: r2 = 2
counter = r2;      //该句执行结果: counter = 2

```

二、进程的概念

当操作系统支持程序并发执行时, 并发执行的程序可能是同一个程序在不同数据集合上的执行, 也可能是不同的程序在不同数据集合上的执行, 它们共享系统资源。用程序已不能描述程序的并发执行, 所以引入了进程的概念。

1. 进程的定义

定义 1: 进程是允许并发执行的程序在某个数据集合上的运行过程。

定义 2: 进程是由正文段、用户数据段及进程控制块共同组成的执行环境。正文段存放被执行的机器指令, 用户数据段存放进程在执行时直接进行操作的用户数据。进程控制块存放程序的运行环境, 操作系统通过这些数据描述和管理进程。

程序装入内存后就可以运行了, 根据指令计数器 PC 的值, 不断将指令从内存取到 CPU 的指令寄存器中, 经过译码后完成各种操作。这些指令控制的对象不外乎各种存储器(内存、外存和各种 CPU 寄存器), 这些存储器中有待运行的指令和待处理的数据, 指令只有得到 CPU 才能发挥其作用。可见, 在计算机内部, 程序的执行过程实际上对应了一个执行环境的总和。这个执行环境包括程序中的各种指令和数据, 还有一些额外数据, 如寄存器的值、用来保存临时数据的堆栈、被打开文件的数量及输入/输出设备的状态等。这个执行环境的动态变化表征程序的运行, 一个进程对应了一个这样的环境。进程代表了程序的执行过

程，是一个动态的实体，它随着程序指令的执行而不断变化，在某个特定时刻的进程内容被称为进程映像。

2. 进程的特征

进程是操作系统管理的实体，对应了程序的执行过程，具有以下几个特征。

1) 并发性。多个进程实体能在一段时间间隔内同时运行。并发性是进程和现代操作系统的重要特征。

2) 动态性。进程是进程实体的执行过程。进程的动态性表现在因执行程序而创建进程、因获得 CPU 而执行进程的指令、因运行终止而被撤销的动态变化过程。此外，进程在被创建后还有进程状态的变化。

3) 独立性。在没有引入线程概念的操作系统中，进程是独立运行和资源调度的基本单位。

4) 异步性。是指进程的执行时断时续，进程什么时候执行、什么时候暂停都无法预知，呈现一种随机的特性。在中断技术出现以后，系统还需要处理随时可能出现的中断事件。

5) 结构特征。进程实体包括用户正文段、用户数据段和进程控制块。

3. 进程与程序的比较

(1) 进程与程序的区别

程序是静态的，进程是动态的。程序是存储在某种介质上的二进制代码，进程对应了程序执行的过程，系统不需要为一个不执行的程序创建进程，一旦进程被创建，就处于不断变化的动态过程中，对应了一个不断变化的上下文环境。

程序是永久的，进程是暂时存在的。程序的永久性是相对于进程而言的，只要程序的拥有者不去删除它，程序可以永久地存在于磁盘、U 盘等外存储介质中。进程因程序的执行而被创建，因程序执行的结束而被撤销，有一个相对短暂的生命期。

程序与进程的存在实体不同。程序是指令的集合，而进程是包括了正文段、用户数据段和进程控制块的实体。

(2) 进程与程序的联系

进程是程序的一次执行，进程总是对应至少一个特定的程序，执行程序的代码。

一个程序可以对应多个进程。同一个程序可以在不同的数据集合上运行，因而构成若干个不同的进程。几个进程能并发地执行相同的程序代码，而同一个进程能顺序地执行几个程序。

《现代操作系统》的作者 Andrew S. Tanenbaum 在其著作中对进程和程序的关系给出了一段很形象的比喻：“一位计算机科学家正在为她的女儿烘制生日蛋糕。他有做蛋糕的食谱，以及做蛋糕的各种原料，如面粉、鸡蛋、糖和香草汁等。做蛋糕的食谱好比是程序，计算机科学家好比是处理器 CPU，而做蛋糕的各种原料就是输入数据。进程就是计算机科学家阅读食谱利用原料烘制蛋糕的过程。”

三、进程控制块

操作系统中的进程不仅是一个概念，而且是真实存在的实体。应用程序对应的进程由程序、用户数据和操作系统管理进程所需要的进程控制块构成。进程实体存在的标志是操作系统管理进程所使用的数据结构——进程控制块。

1. 什么是进程控制块(Process Control Block, PCB)

进程控制块是进程实体的一部分，是操作系统中最重要的数据结构。进程控制块中记录了操作系统所需要的、用于描述进程情况及控制进程运行所需的全部信息。

在使用 C 语言实现的操作系统中，这个数据结构是一个结构类型的变量。该结构类型中包含描述和管理进程需要的字段，这些字段的类型可以是简单类型，也可以是复杂类型。操作系统在创建进程时，首先要为进程创建进程控制块，也就是生成一个进程控制块类型的变量，以存储所创建进程的描述信息。每个进程有唯一的进程控制块，进程控制块是操作系统感知进程存在的唯一标志。下面是 Linux 2.6.11 的进程控制块结构的定义和其中的部分字段。

```
struct task_struct {
    volatile long state;      /* -1 unrunnable, 0 runnable, >0 stopped */
    struct thread_info * thread_info;
    atomic_t usage;
    unsigned long flags;      /* per process flags, defined below */
    unsigned long ptrace;
    int lock_depth;          /* Lock depth */
    int prio, static_prio;
    struct list_head run_list;
    prio_array_t * array;
    unsigned long sleep_avg;
    unsigned long long timestamp, last_ran;
    int activated;
    unsigned long policy;
    cpumask_t cpus_allowed;
    unsigned int time_slice, first_time_slice;
    ...
}
```

2. 进程控制块中的信息

不同操作系统进程控制块中包含的描述信息不完全相同，一般操作系统中的进程控制块中通常包含以下信息。

(1) 进程标识符信息

进程标识符用于唯一标识一个进程。进程控制块中除了存有本进程的标识符外，还存放其父进程、子进程的标识符。

(2) 处理机状态信息

处理机是被进程共享的资源，当一个进程在 CPU 上执行时，CPU 寄存器的值随进程的执行不断变化，当发生进程切换时，当前的进程执行要被暂停，新进程占用 CPU。为了使当前进程的 CPU 值不被新进程覆盖而丢失，必须将当前进程的 CPU 寄存器值保存到内存中，以便该进程再次获得 CPU 时，能从内存中加载寄存器的值，恢复进程上次被暂停时的 CPU 环境，使进程从上次被中断处继续执行。进程控制块中保留的处理机状态信息通常包括以下几个。

1) 通用寄存器。用户程序可以访问的寄存器，用于暂存信息。

2) 指令计数器。其中存放了 CPU 要访问的下一条指令的地址。

3) 程序状态字 PSW。其中包含状态信息，如条件码、执行方式和中断屏蔽标志等。

4) 用户栈指针。每个用户进程都有一个与之相关的系统栈，用于存放过程和系统调用参数及调用地址，栈指针指向该栈的栈顶。

(3) 进程调度信息

进程调度信息包括进程状态信息、进程优先级和进程调度所需的其他信息。

(4) 进程控制信息

进程控制信息包括程序和数据的地址、进程同步和通信机制、资源清单，以及链接指针。

本书中的数据结构就是为程序的需要而定义的简单类型或复杂类型的变量，变量用来存放随着程序的执行不断发生变化的数据。进程控制块是复杂类型的变量。

操作系统如何利用进程控制块？换句话说，进程控制块这种数据结构在操作系统这种复杂的软件中有什么作用呢？下面以 Linux 的进程控制块结构中的变量为例来说明该问题。

例 2-2：进程控制块 `task_struct` 的 `prio` 字段，用来存放进程的优先权。在操作系统决定为系统中的多个可执行进程中的哪个进程分配资源时，可以利用这个字段来比较进程的优先权，把资源分配给 `prio` 字段值最大或最小的进程。

四、进程的状态

进程控制块的状态字段描述了进程当前所处的状态。它由一组标志组成，其中每个标志描述一种可能的进程状态。Linux 中进程状态字段是 `task_struct` 中的 `state` 字段。

不同操作系统在设计和实现时，定义的进程状态数量和含义有所不同，不同版本的 Windows、UNIX 和 Linux 的进程状态数都不同，对每种状态的定义也不同。为了说明操作系统实现的原理，可以把操作系统的进程状态归结为几种基本的状态，而不影响对操作系统基本实现原理解析。

1. 进程的 3 种基本状态

(1) 就绪态

就绪态是进程一旦获得 CPU 就可以投入运行的状态。在多任务系统中，可以有多个处于就绪态的进程，这些进程被组织成一个或多个就绪进程队列。

Linux 2.4 把所有就绪进程组织成一个先进先出的就绪队列，Linux 2.6.11 根据进程的优先权不同将就绪进程组织成多个不同的就绪队列。优先权相同的进程放在同一个就绪队列中，每个不同的优先权对应不同的就绪队列。

(2) 执行态

执行态是进程获得 CPU 正在运行的状态。系统中执行态的进程数量受 CPU 数量的限制。单 CPU 系统中，任意时刻只能有一个进程处于执行态。有 N 个 CPU 的多 CPU 系统中，任意时刻系统中最多有 N 个进程处于执行态。

(3) 阻塞态

阻塞态是进程由于等待资源或某个事件的发生而暂停执行的状态，系统不会为处于阻塞态的进程分配 CPU。阻塞态进程在获得其等待的资源或其等待的事件发生之后，转变为就绪态。处于阻塞态的进程数量可能很多，系统可以根据不同的阻塞原因将进程组织成不同的

阻塞队列。例如，将因为等待打印机而阻塞的进程排列成一个打印阻塞队列，将等待输入数据的进程排列成等待输入数据的队列。

2. 进程状态的转换

进程的 3 个基本状态之间的转换关系如图 2-1 所示。

新创建进程的状态一般被设置为就绪态，当操作系统为处于就绪态的进程分配 CPU 时，进程开始在 CPU 上运行，进程的状态就由就绪态变为执行态。执行态进程可能变为就绪态，也可能变为阻塞态。在多任务系统中，CPU 是被多个进程共享的资源，操作系统通常会为普通进程规定一个在 CPU 上连续运行的时间长度，称这个时间长度为时间片。当进程在 CPU 上运行的时间片长度递减为 0 时，系统把 CPU 分

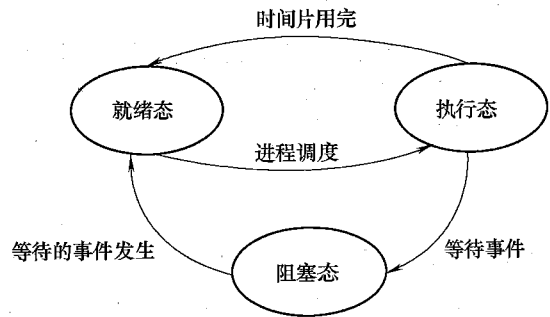


图 2-1 进程状态转换图

配给其他就绪态进程。如果进程在 CPU 上运行的时间片递减为 0，系统将该进程的状态由执行态变为就绪态。如果执行态进程在运行过程中因为申请某种资源或等待某事件的发生而不宜继续占用 CPU，系统将该进程的状态改变为阻塞态，并暂停该进程的执行，将 CPU 分配给就绪态进程。阻塞态进程在其等待的事件发生或申请到所需资源后，系统将其状态转变为就绪态。阻塞态进程等待的事件最容易理解的一个实例是进程提出 I/O 请求后等待 I/O 结束。

进程在创建后到撤销前，其状态可能多次在 3 个状态之间转换。进程状态不能由阻塞态直接变为执行态，进程状态由阻塞态变为就绪态的过程称为唤醒过程，由执行态变为阻塞态的过程称为阻塞过程。

3. Linux 的进程状态

(1) Linux 2.4.30 进程状态及状态定义。

- 1) 可运行状态(TASK_RUNNING)。
- 2) 可中断的等待状态(TASK_INTERRUPTIBLE)。
- 3) 不可中断的等待状态(TASK_UNINTERRUPTIBLE)。
- 4) 暂停状态(TASK_STOPPED)。
- 5) 僵死状态(TASK_ZOMBIE)。

```
#define TASK_RUNNING 0
```

```
#define TASK_INTERRUPTIBLE 1
```

```
#define TASK_UNINTERRUPTIBLE 2
```

```
#define TASK_ZOMBIE 4
```

```
#define TASK_STOPPED 8
```

(定义文件：include/linux/sched.h)

(2) Linux 2.6.11 进程状态及其定义。

- 1) 可运行状态(TASK_RUNNING)。
- 2) 可中断的等待状态(TASK_INTERRUPTIBLE)。

- 3) 不可中断的等待状态(TASK_UNINTERRUPTIBLE)。
- 4) 暂停状态(TASK_STOPPED)。
- 5) 跟踪状态(TASK_TRACED)。
- 6) 僵死状态(TASK_ZOMBIE)。
- 7) 僵死_撤销状态(EXIT_DEAD)。

```
#define TASK_RUNNING          0
#define TASK_INTERRUPTIBLE    1
#define TASK_UNINTERRUPTIBLE  2
#define TASK_STOPPED          4
#define TASK_TRACED           8
#define EXIT_ZOMBIE           16
#define EXIT_DEAD              32
```

(定义文件: include/linux/sched. h)

在 Linux 2. 6. 11 中, 状态字段的值通常用一个简单的赋值语句设置。例如, procdesc_ptr -> state = TASK_RUNNING。

五、进程的组织

在操作系统中任意时刻都可能存在很多进程, 每个进程对应一个进程控制块, 操作系统组织和管理进程是通过管理和组织进程控制块来实现的。管理和组织的主要目的就是要使系统中的程序在需要时能够以有效的方式检索到系统中的每一个进程。实际上对进程的组织是通过定义数据结构来实现的。

1. 链接方式

把系统中具有相同状态的进程的进程控制块 (PCB) 用其中的链接字链接成一个队列, 如图 2-2 所示。

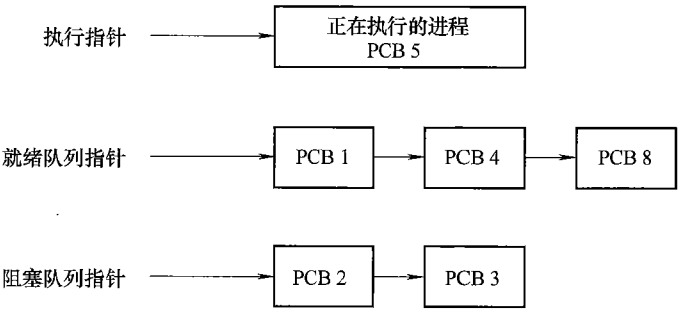


图 2-2 进程队列

2. 索引方式

系统根据所有进程的状态, 建立几张索引表, 索引表的每一个表项指向一个 PCB 的物理块, 如图 2-3 所示。

3. 进程队列

前面讲解了进程控制块, 当系统中有很多进程时, 可以把进程控制块用队列组织起来, 形成进程队列。把具有相同状态的进程放在同一个队列中, 具有不同状态的进程就形成了不同的进程队列。处于就绪态的进程构成的进程队列称为就绪队列, 处于阻塞态的进程构成的进程队列称为阻塞队列。

根据算法的需要, 又可以把就绪队列按照优先权的不同分成几个优先权不同的就绪队列, 把阻塞进程根据阻塞原因的不同分成不同的阻塞队列, 阻塞原因相同的进程在同一个阻塞队列中, 如图 2-4 所示。

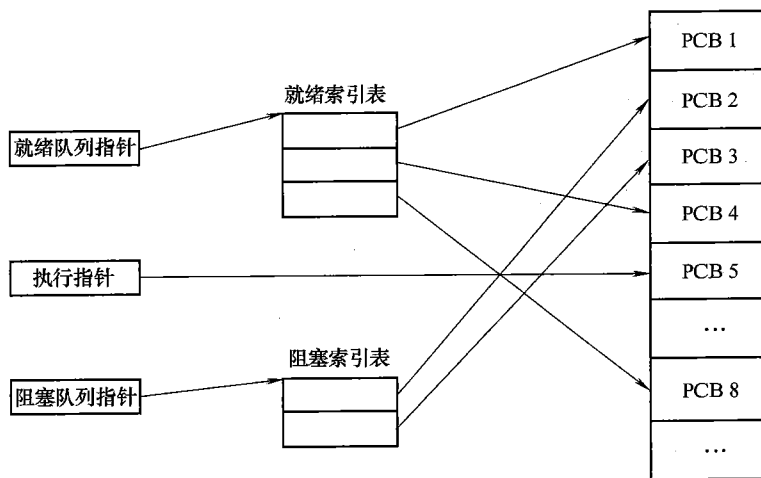


图 2-3 采用索引方式组织进程

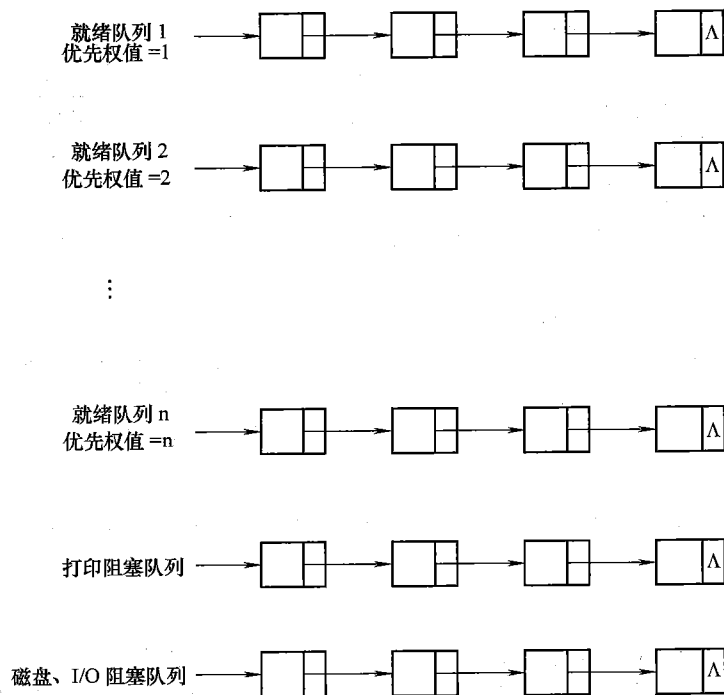


图 2-4 多个就绪队列和阻塞队列的进程组织

第二节 进程的控制

一、进程的创建

创建新进程包括为进程分配必要的资源，建立操作系统用于管理进程的数据结构(如进程控制块)等操作。通常在下列情况下需要创建新进程。

1) 用户登录。在交互式系统中，当一个新用户登录系统时会创建进程。

2) 作业调度。当把位于外存储器中的作业调入内存准备运行时，操作系统会为作业的运行创建进程。

3) 提供服务。操作系统可以创建一个进程，代表用户程序执行某种功能，使用户无需等待。例如，用户请求打印一个文件，操作系统可以创建一个管理打印的进程，进而可以使请求进程继续执行，与完成打印任务的时间无关。

4) 应用请求。当用户发出执行某程序的命令，或者用户程序在执行中基于并行的要求在程序中显式调用进程创建的系统调用时，都会引起进程的创建。

在 Linux 系统中，除了 0 号进程(swapper 进程)外的其他进程都是由其父进程创建的。在操作系统启动并正常工作后，系统中的已有进程在执行过程中都可以通过系统调用创建新进程。被创建的新进程成为创建该新进程的进程的子进程，创建者进程和被创建进程成为父子进程。一个进程可以创建多个子进程，由同一个进程创建的多个子进程之间的关系称为兄弟进程。系统中所有的进程通过父子关系和兄弟关系被组织成一棵进程树，如图 2-5 所示。

进程 1 是进程 3 的父进程，进程 3 是进程 1 的子进程，进程 3 和进程 4 互为兄弟进程。

通常，进程需要一定的资源(如 CPU 时间、内存、文件和 I/O 设备)，以完成其任务。在一个进程创建子进程时，子进程可以从操作系统那里直接获得资源，也可能只从其父进程资源子集那里获得资源。父进程可能必须在其子进程之间分配资源或共享资源(如内存或文件)。

进程被创建时，除了得到各种资源外，它还能从父进程那里得到所需要的输入数据。例如，考虑一个进程 p，其功能是在终端屏幕上显示文件 F1 的状态。在该进程被创建时，会得到来自父进程的输入——F1 的文件名。进程 p 能使用该文件名数据执行以获得所需要的 F1 的其他信息。

当新进程被创建时，有两种执行可能。

- 1) 父进程与子进程并发执行。
- 2) 父进程等待，直到某个或全部子进程执行完毕。

新进程的地址空间也有两种可能。

- 1) 子进程共享父进程的地址空间。
- 2) 子进程拥有独立地址空间。

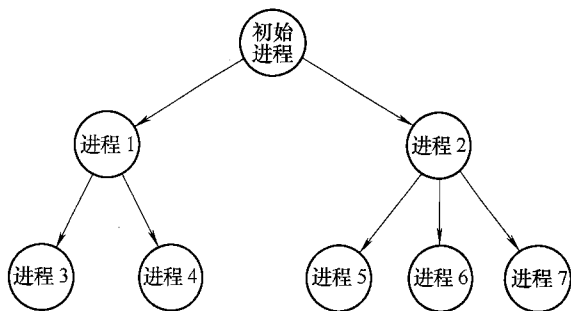


图 2-5 进程树

调用创建新进程的系统调用来创建进程的一般步骤如下。

- 1) 申请空白 PCB。
- 2) 为新进程分配资源。
- 3) 初始化进程控制块。
- 4) 将新进程插入就绪队列。

Linux 2.6.11 中创建进程的常用系统调用在函数库中的接口函数是 `fork()`。

下面以一般用户进程的创建为例说明进程创建的过程。用户在 shell 中输入命令后，shell 大多数情况下调用函数 `fork()` 来创建进程。

在 Linux 内核文件 `linux/include/asm-x86_64/unistd.h` 中定义其系统调用号。

```
#define __NR_fork                                57
__SYSCALL(__NR_fork, stub_fork)
```

函数 `fork` 对应的系统调用是在 `/linux + v2.6.11/arch/i386/kernel/process.c` 中定义的 `sys_fork`，在这个系统调用中调用内核函数 `do_fork()`，由 `do_fork()` 完成进程创建。简单而言，`do_fork()` 主要完成下列功能。

- 1) 为新进程分配唯一的进程 `pid`。
- 2) 为新进程获取自己的进程描述符。
- 3) 为新进程指定内存地址空间。
- 4) 完成进程的一些初始化工作。
- 5) 返回进程的 `pid`。

Linux 创建进程的系统调用还有 `clone` 和 `vfork`，它们对应的系统调用例程分别是 `sys_clone` 和 `sys_vfork`。这两个系统调用在 `/linux2.6.11/arch/i386/kernel/process.c` 中定义，都调用了 `do_fork` 函数。

与进程创建密切相关的函数还有 `execve()`，与其对应的系统调用是 `sys_execve()`，主要功能是使进程执行新的程序代码。

二、进程的阻塞

操作系统在下列情况下可能进行进程的阻塞和唤醒操作。

- 1) 请求系统服务。例如打印服务，进程在提出打印请求后，系统创建打印服务进程为该进程服务，而提出打印请求的进程可能被阻塞起来等待打印的完成。
- 2) 启动某种操作。例如，进程发出启动 I/O 或启动打印机的命令后，等待启动完成时处于阻塞态。
- 3) 新数据尚未到达。对于一个计算进程，如果新的输入数据还没有到达，计算进程需要阻塞等待。
- 4) 无新工作可做。发送消息之后等待时。

完成进程阻塞的简化过程如下。

- 1) 将进程的状态改为阻塞态。
- 2) 将进程插入相应的阻塞队列。
- 3) 转进程调度程序，从就绪进程中选择进程为其分配 CPU。

处于执行态的进程被阻塞后，CPU 空闲，需要执行进程调度程序，从就绪态进程中为

CPU 选择一个进程运行。

三、进程的唤醒

操作系统通过下列过程将阻塞态进程唤醒，使其变成就绪态进程。进程唤醒的过程如下。

- 1) 将进程从阻塞队列中移出。
- 2) 将进程状态由阻塞态改为就绪态。
- 3) 将进程插入就绪队列。

四、进程的终止

进程的终止也称进程的撤销，在下列情况下，进程会被终止。

- 1) 当进程正常执行完毕，调用终止进程的系统调用，请求操作系统删除该进程。
- 2) 一个进程调用适当的系统调用，终止另外一个进程。通常，操作系统规定只允许父进程通过这种方式终止其子进程。

父进程终止其子进程的原因可能有以下几个。

- 1) 子进程使用了超过它所分配到的一些资源。
- 2) 分配给子进程的任务已不再需要。
- 3) 父进程退出，如果父进程终止，那么操作系统不允许子进程继续。对于这类系统，如果一个进程终止，那么它的所有子进程也被终止。这种现象称为级联终止，通常是由操作系统进行的。在 UNIX 系统中，如果父进程终止，那么其他所有子进程会以 init 进程作为它们新的父进程。因此，子进程仍然有父进程可以收集它们的状态和执行统计。

操作系统通过系统调用完成进程终止的一般过程如下。

- 1) 从进程 PCB 中读进程状态。
- 2) 若进程正在执行，则终止进程的执行。
- 3) 若进程有子孙进程，在大多数情况下需要终止子孙进程。
- 4) 释放资源。
- 5) 将终止进程的 PCB 移出。

五、操作系统的启动和系统中进程的出现

当打开计算机电源后，计算机会先进行加电自检，然后寻找启动盘。如果是选择硬盘启动，计算机会检查硬盘的 0 柱面 0 磁道 1 扇区。如果发现该扇区以 0xAA55 结束，则 BIOS 认为它是引导扇区，一旦发现引导扇区，BIOS 会执行程序将其装入到内存地址 0000:7c00 处，然后跳转到该地址处执行这段引导程序代码，开始加载操作系统。

当硬盘被划分为多个分区，同时安装了多个操作系统时，每个分区都有自己的引导扇区，但是整个硬盘有一个主引导扇区，主引导扇区就是硬盘的 0 柱面 0 磁道 1 扇区。通过执行主引导扇区的代码，判断当前被激活的分区，然后加载被激活分区的引导扇区，通过该引导扇区代码的执行加载该激活分区的操作系统。

Linux 系统从硬盘引导、加载、初始化和创建进程的简化过程如图 2-6 所示。

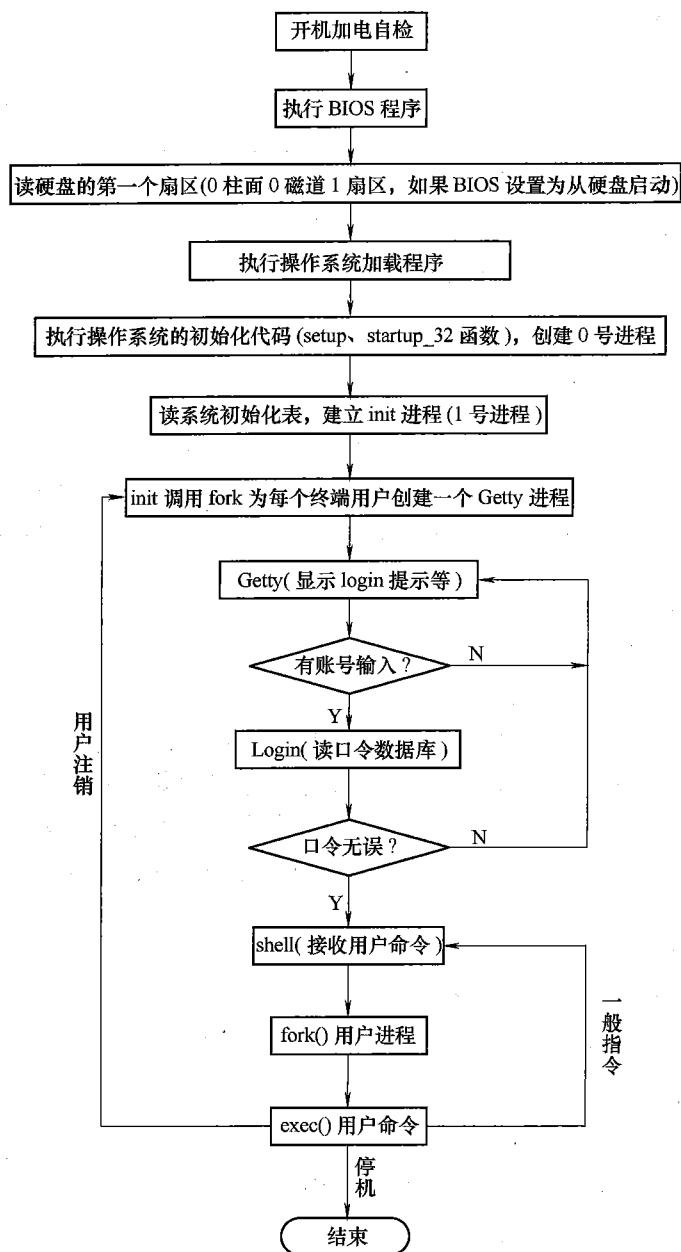


图 2-6 系统初始化流程

第三节 操作系统内核

操作系统内核是计算机硬件的第一次扩充, 内核执行操作系统与硬件关系密切, 执行频率高的模块, 常驻内存。不同的操作系统内核具有的功能可能是不同的。一般操作系统内核包括下列功能。

3. 中断的类型

中断分为同步中断(也称内部中断或异常)和异步中断(也称外部中断)两种。

(1) 同步中断(内部中断或异常)

同步中断是当指令执行时由 CPU 控制单元产生的,之所以称为同步,是因为只有在一条指令终止执行(注意:此时指令并不一定已经执行完毕)后 CPU 才会发出中断,如除法出错、调试、溢出和浮点出错等。

(2) 异步中断(外部中断)

异步中断是由其他硬件设备随机产生的。在 Intel 微处理器手册中,把同步中断和异步中断分别称为异常(Exception)和中断(Interrupt)。Linux 中采用这种分类,有时也用术语“中断信号”来表示这两种类型的中断(同步中断及异步中断)。

外部中断又可以分为外部可屏蔽中断和外部不可屏蔽中断。

1) 外部可屏蔽中断。外部可屏蔽中断是由 I/O 设备产生的中断,有两种方式可以屏蔽中断。一是如在 Intel 80x86 的 CPU 上,把 EFLAGS 寄存器的 IF 标志置 0,表示关中断。此时, CPU 忽略所有可屏蔽中断。二是可以对可编程中断控制器(Programmable Interrupt Controller, PIC)编程来禁止中断请求(Interrupt Request, IRQ)。也就是说,可以告诉 PIC 停止对给定的 IRQ 线发布中断。

2) 外部不可屏蔽中断。外部不可屏蔽中断是由紧急事件引起的中断,如硬件故障。

4. 引起中断的原因

引起中断的原因可能有以下几个。

1) 人为设置中断。在程序中人为设置中断。

2) 程序性事故。例如,计算中出现除数为 0。

3) 硬件故障。插件接触不良、电源掉电等。

4) I/O 设备。I/O 设备被启动以后,一旦其准备就绪或完成一次输入/输出,便向 CPU 发出中断请求。

5) 外部事件。如用户通过键盘和鼠标来中断现执行程序。

5. 中断响应

(1) 响应中断的条件

对于可屏蔽中断,开中断是响应中断的前提。例如,在 Intel 80x86 CPU 上,当 EFLAGS 寄存器的 IF 标志置 1 表示开中断。

(2) 响应中断的时机

对于外部中断, CPU 每执行完一条指令都会检测是否有外部中断信号的到来。若有,则转中断处理过程。

6. 单重中断的处理过程

下面以单重中断为例,简单说明外部中断处理的过程。假定在中断处理的过程中不再响应新近产生的中断信号,直到本次中断处理完毕。同时假定中断处理完毕后, CPU 返回被中断的程序断点处继续执行被中断的程序。

外部中断的处理过程如图 2-8 所示。CPU 在反复执行指令的过程中,每执行完一条指令,都会检测是否有外部中断信号的到来。如果检测到有中断信号,则转中断处理过程。

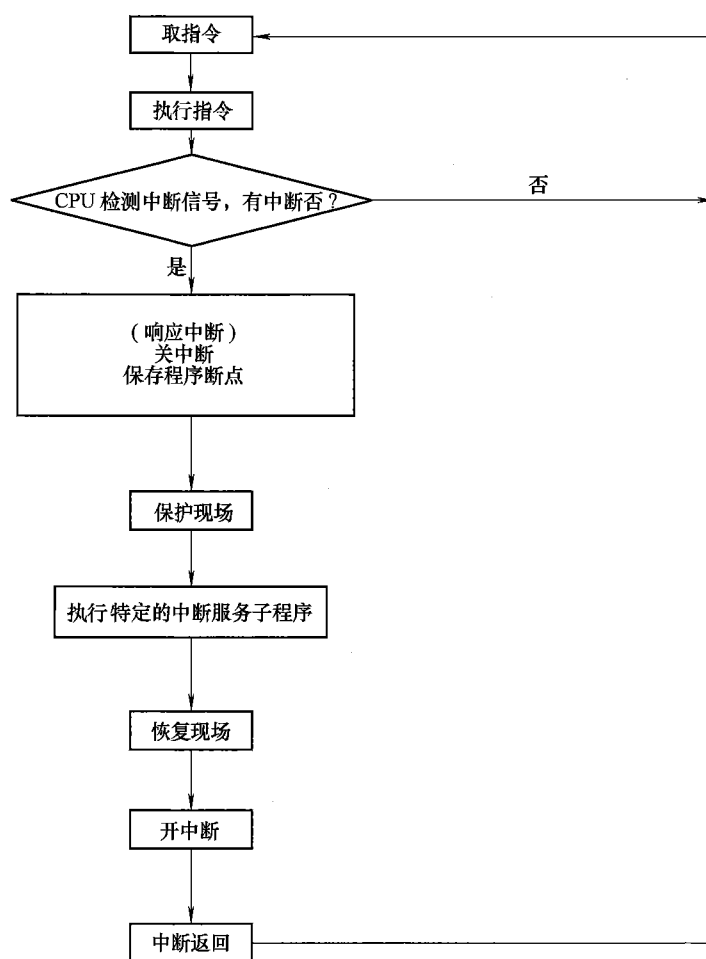


图 2-8 单重中断的处理流程

1) 系统关闭中断, 保护断点, 把当前要执行的下一条指令的地址保存到内存中, 以便中断返回时, 能把这个地址恢复到程序计数器 PC 中, 使被中断的程序从断点处开始继续执行。

2) 转中断处理程序。在中断处理程序中完成保护现场的工作, 就是把相关的硬件上下文信息保存到内存中。硬件上下文就是中断返回恢复被中断程序的执行时, 需要写回 CPU 寄存器的值。

3) 保护完现场后, 要根据中断向量到中断向量表中(在 Linux 中是到中断描述符表中)找到与中断处理子例程入口地址相关的信息, 由这些信息得到中断处理子例程的入口地址, 以执行中断处理子例程, 完成本次中断处理的特定处理工作。

4) 最后, 恢复现场, 开中断, CPU 返回断点处继续执行被中断的程序。

7. 如何找到中断服务子程序

下面以 Intel 80x86 和 Linux 系统为例说明获取特定中断(键盘、鼠标和时钟中断等)服务子程序入口地址的原理。

1) 中断向量。中断向量是对不同中断源到来的信号编号，该编号是一个无符号整数(0 ~255)，称为中断向量。

不可屏蔽中断的向量和异常的向量是固定的，而可屏蔽中断的向量可以通过对中断控制器的编程来改变。

可编程中断控制器的 IRQ 线是从 0 开始顺序编号的，因此，第一条 IRQ 线通常表示成 IRQ0。与 IRQn 关联的 Intel 的默认向量是 $n + 32$ 。如前所述，通过向中断控制器端口发布合适的指令，就可以修改 IRQ 与向量之间的映射。图 2-9 所示为级联的中断机构示意图。

Linux 中的中断向量与中断源的对应关系如表 2-1 所示。

2) 中断描述符表 (Interrupt Descriptor Table, IDT) 是一个系统表，每一个中断或异常与向量相联系。每一个向量在表中有唯一对应的表项，其中存有与中断或异常处理子例程入口地址相关的信息。内核在允许中断发生前，必须正确地初始化 IDT。在操作系统初始化时，由操作系统执行汇编指令 `lidt` 加载进入内存。

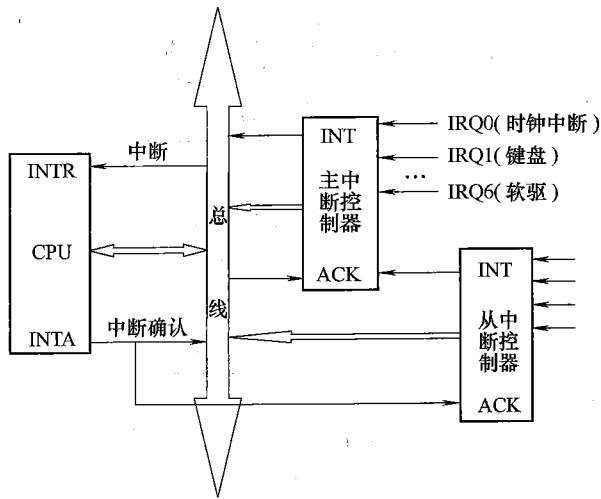


图 2-9 级联的中断机构示意图

表 2-1 Linux 中的中断向量与中断源的对应关系

向量范围	用途
0-19(0x0-0x13)	非屏蔽中断和异常
20-31(0x14-0x1f)	Intel 保留
32-127(0x20-0x7f)	外部中断(IRQ)
128(0x80)	用于系统调用的可编程异常
129-23(0x81-0xee)	外部中断(IRQ)
239(0xef)	本地 APIC 时钟中断
240(0xf0)	本地 APIC 高温中断(在 Pentium 4 模型中引入)
241-250(0xf0-0xfa)	由 Linux 留作将来使用

在 Linux 中，IDT 的每一项对应一个中断或异常向量，每个表项由 8 个字节(64 位)组成。因此，最多需要 $256 \times 8 = 2048$ 字节来存放 IDT。IDT 表的起始地址由操作系统执行 `lidt` 汇编指令存放在 CPU 的 `idtr` 寄存器中。`idtr` 寄存器使 IDT 可以位于内存的任何地方，它指定 IDT 的基地址及其最大长度。在允许中断之前，必须用 `lidt` 汇编指令初始化 `idtr`。

例如，有如图 2-10 所示的 IDT。

当中断发生时，CPU 根据中断号获取中断向量值，再根据中断描述符表的起始地址和中断向量值，得到形成中断服务子程序(中断服务例程)入口地址的相关信息。

中断子程序的入口地址相关信息在内存中的地址 = `idtr` 中的地址 + $8 \times$ 中断向量的值。

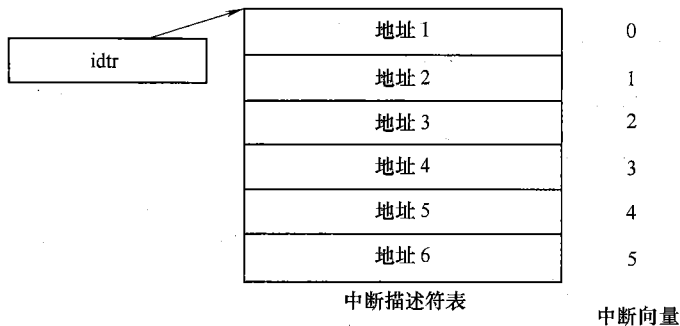


图 2-10 中断描述符表

二、时钟管理

1. 时钟的重要性

时钟是计算机系统的脉搏。计算机的很多活动都是由定时测量(Timing Measurement)来驱动的。计算机系统的定时测量功能对用户通常是不可见的。例如，屏幕保护功能的实现就利用了定时器。操作系统的内核跟踪用户最后一次按键或移动鼠标后经过了多少时间，如果这个时间达到一个用户设定的阈值，屏幕会自动进入被保护状态。

编译系统为了缩短一个比较大的工程文件重新编译和链接的时间，对于一个工程文件中的所有源程序，并不是每次修改某一个源文件后都需要重新编译所有的源文件。通常只编译那些源文件的最后修改时间晚于相应目标文件生成时间的源程序。例如，有一个文件 a.c 和它的目标文件 a.obj，如果 a.c 的生成时间晚于 a.obj 的生成时间，需要重新编译 a.c 以形成新的 a.obj。为了进行这些操作，程序必须能从每个文件中检索到文件的最后修改时间，即时间戳(Timestamp)。因此，这样的时间标记必须由内核自动设置。

更重要的是，操作系统的内核可以利用时钟机制防止一个进程垄断 CPU 或其他资源。系统可以利用时钟机制限制一个用户进程在 CPU 上连续执行的时间，当进程一次连续运行的时间超过限定的时间时，由系统将 CPU 分配给其他进程。

计算机中当前时间显示功能、即时通信程序等与时间相关的软件都需要时钟机制的支持。

2. 计算机系统时钟

大部分 PC 中有两个时钟源，分别称为实时时钟(Real Timer Clocker, RTC)和 OS 时钟。RTC 时钟也称 CMOS 时钟，是一块时钟芯片，靠电池供电，为计算机提供计时标准，是最原始、最底层的数据。OS 时钟产生于 PC 主板上的定时/计数芯片，在开机时有效，由操作系统控制。

计算机开机加电后，操作系统通过 BIOS 获取当前 RTC 时钟的值作为系统的初始时间，操作系统初始化后启用自己的时钟硬件，即可编程间隔定时器(Programmable Internal Timer, PIT)。PIT 可以按照一定的频率产生时钟中断(Timer Interrupt)，以告知内核又一个时间间隔过去了。

RTC 时钟、OS 时钟和应用程序之间的关系如图 2-11 所示。

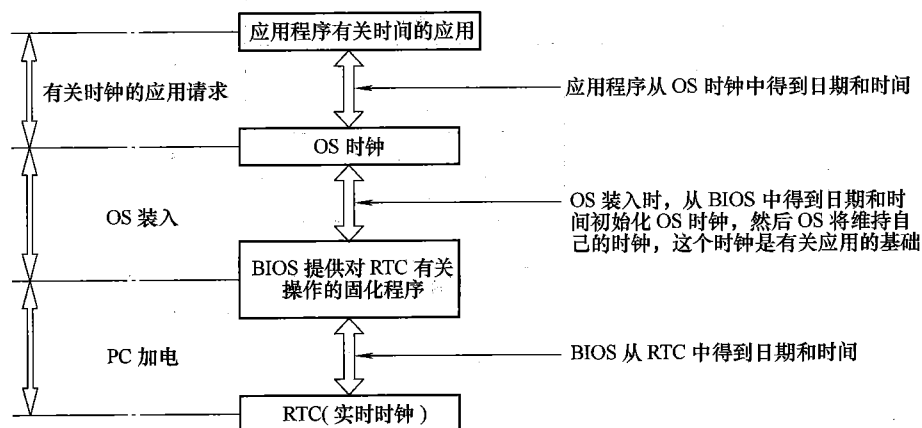


图 2-11 RTC 时钟、OS 时钟和应用程序之间的关系

3. 操作系统的时钟机制

操作系统内核需要完成两种主要的定时测量，一是保存当前的日期和时间，以便能通过系统调用把它们返回给用户程序，让用户程序获得当前的日期和时间，也可以由内核本身把当前时间作为文件和网络包的时间戳。二是维持定时器，这种机制能够告诉内核或用户程序某一时间间隔已经过去了。

操作系统依靠时钟硬件(可编程间隔定时器 PIT)和时钟驱动程序完成上述两种定时测量功能。

(1) OS 时钟管理硬件(可编程间隔定时器 PIT)

可编程间隔定时器的功能是按指定的时间间隔产生时钟中断，测量逝去的时间，并触发与时间有关的操作。主要由 3 部分构成：晶振、计数器和保持寄存器，如图 2-12 所示。

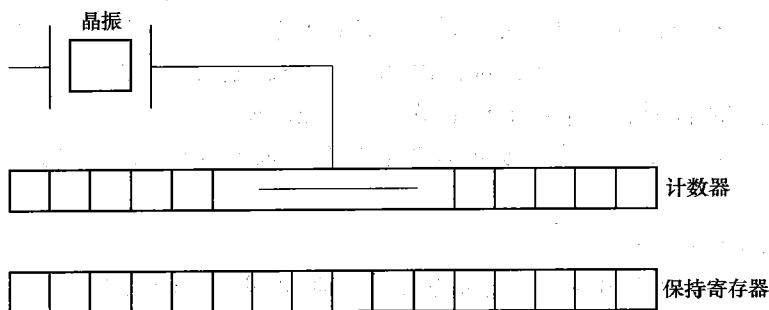


图 2-12 可编程间隔定时器逻辑结构

晶振能够产生固定频率的脉冲，每产生一次脉冲，计数器的值减 1，当计数器的值减为 0 时，产生一次时钟中断信号，保持寄存器的值再次送计数器。由可编程间隔定时器产生的时钟中断信号送到可编程中断控制器的时钟中断信号引脚上。

可编程间隔定时器产生时钟中断的过程如图 2-13 所示。

(2) 时钟软件——时钟驱动程序

时钟驱动程序也称为时钟中断处理程序，每产生一次时钟中断信号，操作系统内核要执

行时钟驱动程序，时钟驱动程序完成下列功能。

1) 维护日期和时间。

2) 递减当前进程在一个时间片内的剩余执行时间，并检查是否为零，防止进程运行超时。当前进程在一个时间片内的剩余执行时间通常记录在进程控制块中的一个字段中。该字段在进程被调度后初始化为一个时间片的长度，这个值通常用时钟中断的次数来表示。每次处理时钟中断时，对当前进程在 CPU 上的剩余时间值做递减操作，并检查该值递减后是否为零。若进程在 CPU 上的剩余时间值为零，系统执行进程调度程序，将 CPU 分配给其他进程；若不为零，进程可继续执行。

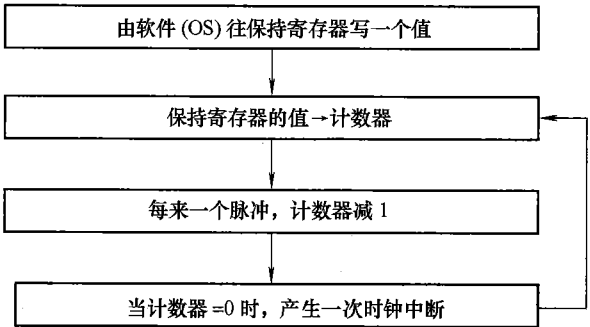


图 2-13 时钟中断信号的产生

3) 对 CPU 的使用情况记账。

4) 递减报警计数器。

4. Linux 时钟

本节内容帮助读者获得对时钟软件的直观认识，帮助理解 OS 时钟软件实现的原理。

1) 时钟滴答。OS 时钟是由可编程定时/计数器输出脉冲触发中断而产生的，输出脉冲的周期称为一个时钟滴答。Linux 2.4 的设计者将一个时钟滴答定义为 10 ms(每秒产生 100 次时钟中断)。

2) 时钟基准。Linux 的时间基准是 1970 年 1 月 1 日凌晨 0 点。

3) Linux 用全局变量 jiffies 表示自系统启动以来的时钟滴答数。

4) Linux 提供的时间格式举例。

```

struct timespec { time_t tv_sec; 存放自 1970 年 1 月 1 日
                  (UTC)午夜以来经过的秒数
                  long tv_nsec; 存放自上一秒开始经过
                  的纳秒数。} xtime;
  
```

5) 与时钟中断相关的函数。

下面是 Linux 2.4 中几个与时钟相关的函数的简化代码。

```

① Do_timer_interrupt(...)
{ do_timer(regs);
  if(xtime.tv_sec > last_rtc_update + 660)
    update_RTC(); /* 每 11 分钟更新一次实时时钟 */
② do_timer(struct pt_regs *regs)
{ jiffies++;
  Update_process_times(user_mode(regs));
  // 为本地 CPU 执行几个与定时相关的计数操作
}
③ Void _init time_init(void)
  
```

```

}
xtime.tv_sec = get_cmos_time(); 获得自 1970 年 1 月 1 日
                                (UTC)午夜以来经过的秒数
xtime.tv_nsec = 0;
setup_irq(0, &irq0); //创建与 IRQ0 相应的中断门(中断门: 包含段选择符和中断或异常处理程序在
段内的偏移量)
}

```

三、系统调用

1. 什么是系统调用

系统调用是一群预先定义好的模块，它们提供一条管道让应用程序或一般用户能由此得到核心程序的服务。系统调用是系统程序与用户程序之间的接口，在类 UNIX 系统中，系统调用多使用 C 语言提供的库函数作为接口。例如，在 Linux 操作系统中使用 C 语言编程，在程序中使用 C 的函数 `getpid()` 以获取进程的內部标识符，而 `getpid()` 的代码调用了在系统态运行的系统调用 `sys-getpid()`。

2. 系统调用与一般函数的区别

为了说明系统调用与一般函数的区别，先说明什么是用户态执行，什么是系统态执行。

(1) 用户态执行

用户空间是指用户进程所处的地址空间，一个用户进程不能访问其他进程的用户空间，只有系统程序才能访问其他用户空间。当 CPU 执行用户空间的代码时，称该进程在用户态执行。

(2) 系统态执行

系统空间是指含有一切系统核心代码的地址空间，当 CPU 执行系统核心代码时，称进程处于系统态执行。

系统调用与一般函数调用的区别如下。

- 1) 系统调用运行在系统态(核心态)，而一般函数运行在用户态。
- 2) 系统调用与一般函数调用的执行过程不同。系统调用执行时，当前进程被中断，由系统找相应的系统调用子程序，并在系统态下执行，执行结果返回进程。
- 3) 系统调用要进行“中断处理”，比一般函数调用多了一些系统开销。

3. 普通函数执行过程实例

例如，调用 C 的库函数求一个整数的绝对值。当执行求绝对值的函数调用时，程序执行的控制流只需要转移到求绝对值的函数代码的入口处，图 2-14 所示的实例就是跳转到地址为 100 的内存单元开始执行一段求绝对值的子程序。在函数调用前后，进程都处于用户态。求绝对值的函数和用户程序一样处于用户地址空间。

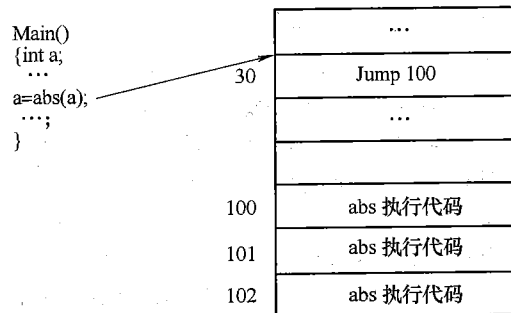


图 2-14 一般函数调用的执行

4. 系统调用执行过程实例

下面以 Linux 系统中用户程序调用 `getpid()` 求当前进程的內部标识符为例, 说明系统调用执行过程的细节。调用 `getpid()` 求当前进程内部标识符的用户程序如下。

```
#include <linux/unistd.h>

int main()
{
    int i = getpid();
    printf("Hello world! This is My pid:%d\n", i);
}
```

该程序代码对 `getpid()` 的调用, 经过 `unistd.h` 中的宏展开后, 转化为以下代码。

```
int getpid(void)
{
    long __res
    __asm__ volatile("int $ 0x80"
                     : "=a" (__res)
                     : "NR_getpid")
    __syscall_return(int, __res)
}
```

这段代码的汇编程序相当于以下汇编代码。

```
movl $ __NR_getpid, %eax;
int $ 0x80
movl %eax, __Res
```

这段代码看起来很简单, 先执行 `movl $ __NR_getpid, %eax`, 把系统调用号 `__NR_getpid` 保存到 `eax` 寄存器中, 以便将来给内核程序使用。因为内核实现了很多不同的系统调用, 所以进程必须传递一个名为系统调用号的参数来识别所需的系统调用。系统调用号是一个整数, 与系统调用一一对应。`__NR_getpid` 是 `getpid()` 的系统调用号。内核需要根据 `__NR_getpid` 来找到与 `getpid()` 对应的系统调用实现程序 `sys_getpid`。

然后执行 `int $ 0x80`, 使进程进入内核态, 并转去执行与 `0x80` 中断对应的处理程序 `system_call`。最后通过执行 `movl %eax, __Res`, 将系统调用结果返回给用户。

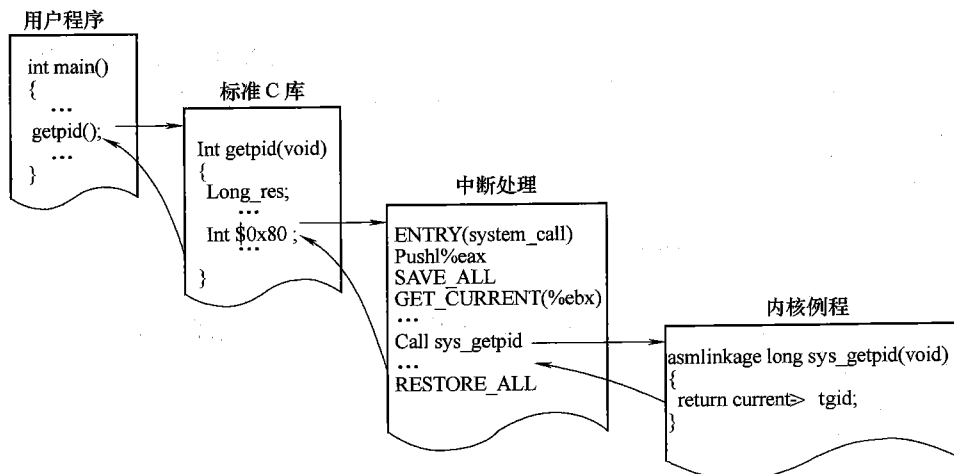
从图 2-15 中可以看到, 在进程处于系统态执行 `system_call` 的过程中, 利用 `getpid()` 的系统调用号 `__NR_getpid` 找到 `getpid()` 对应的系统调用实现程序 `sys_getpid`, 然后执行 `sys_getpid` 获得当前进程的內部标识符, 最后将程序返回到用户态。

为了把系统调用号与相应的系统调用实现程序关联起来, Linux 内核利用了一个系统调用分派表 (Dispatch Table)。这个表存放在 `sys_call_table` 数组中, 有 `NR_syscalls` 个表项 (在 Linux 2.6.11 内核中这个值是 289)。系统调用分派表的第 `n` 个表项包含系统调用号为 `n` 的系统调用实现程序的地址。`getpid()` 对应的系统调用号是 `__NR_getpid`, 所以在系统调用分派表的第 `__NR_getpid` 个表项中存有 `sys_getpid` 程序的起始地址。

5. 系统调用的类型

根据系统调用的功能把系统调用总结为以下几种类型。

- 1) 进程控制类系统调用。创建、撤销进程; 获得、改变进程属性。
- 2) 文件操纵类系统调用。创建文件、删除文件、打开文件、关闭文件和读/写文件。
- 3) 设备管理类系统调用。请求、释放设备。
- 4) 通信类系统调用。打开、关闭连接, 交换信息。

图 2-15 系统调用 `getpid()` 的执行过程

5) 信息维护类系统调用。返回系统当前日期、时间、版本号、用户数、空闲内存和磁盘空间大小等信息。

6. Linux 中的系统调用举例

- 1) `fork` 创建一个新进程。
- 2) `clone` 按指定条件创建子进程。
- 3) `execve` 运行可执行文件。
- 4) `exit` 中止进程。`_exit` 立即中止当前进程 `getdtablesize` 所能打开的最大文件数。
- 5) `getpgid` 获取指定进程组标识号。
- 6) `open` 打开文件。
- 7) `creat` 创建新文件。
- 8) `close` 关闭文件描述符。
- 9) `read` 读文件。
- 10) `write` 写文件。

7. 操作系统提供系统调用的优点

- 1) 使编程更加容易，把用户从学习硬件设备的低级编程特性中解放出来。
- 2) 极大地提高了系统的安全性。

第四节 进 程 同 步

多任务操作系统支持多个进程并发执行，并发执行的进程共享系统的软件和硬件资源。进程访问资源是通过执行程序指令实现的，在这种情况下，操作系统可以也必须对进程访问共享资源的过程进行控制和管理。否则，系统会出现程序运行结果的错误。操作系统同步机制的主要任务就是要保证在多任务共享系统资源的情况下，程序执行能得到正确的结果。此外，有些进程之间具有相互合作关系，同步机制需要解决这些进程执行的协调问题。

一、进程同步的基本概念

在多道程序环境下，进程之间可能存在资源共享关系和相互合作关系。进程同步有两个任务，一是对具有资源共享关系的进程，保证诸进程以互斥的方式访问临界资源。临界资源是必须以互斥方式访问的共享资源。二是对具有相互合作关系的进程，保证相互合作的诸进程协调执行。相互合作的进程可能同时存在资源共享的关系。

为了说明什么是临界资源，先回忆前面 p1、p2 两个进程并发执行，counter 是全局变量，初始值为 0，p1、p2 两个进程分别对 counter 做加 1 操作，代码如下。

```
p1
{ ...
  counter = counter + 1
...
}

p2
{ ...
  counter = counter + 1
...
}
```

若当前 counter = 0，p1 和 p2 各执行一次后，counter 的正确值应该是 2，但是当 p1 和 p2 在经过编译后，必须分别经过下列指令的执行，才能完成 counter = counter + 1 的功能。

```
p1
{ ...
  register1 = counter;
  register1 = register1 + 1;
  counter = register1;
...
}

p2
{ ...
  register2 = counter;
  register2 = register2 + 1;
  counter = register2;
...
}
```

p1、p2 并发执行时，指令的执行序列可能出现各种组合，当按下列顺序执行时，会发生 counter 计数错误。

register1 = counter	//执行结果: register1 = 0
register1 = register1 + 1	//执行结果: register1 = 1
register2 = counter	//执行结果: register2 = 0
register2 = register2 + 1	//执行结果: register2 = 1
counter = register1	//执行结果: counter = 1

```
counter = register2
```

```
//执行结果: counter = 1
```

执行结果是 $\text{counter} = 1$ ，而正确结果应该是 $\text{counter} = 2$ 。

如果 p_1 和 p_2 以互斥的方式去访问 counter ，也就是说，如果 p_1 先开始对 counter 的访问， p_2 就必须等待 p_1 对 counter 的访问完全结束，再开始对 counter 的访问。反之，如果 p_2 先开始对 counter 的访问， p_1 就必须等待 p_2 对 counter 的访问完全结束，再开始对 counter 的访问。那么 p_1 和 p_2 对 counter 访问指令的交错执行的情况就不会出现，计算结果的错误也就不会出现了。 p_1 和 p_2 以互斥的方式去访问 counter 的含义是在 p_1 和 p_2 中的任意一个进程执行 $\text{counter} + 1$ 操作的一系列指令的过程中，不允许另一个进程执行访问 counter 的操作。类似 counter 这样必须以互斥方式访问的共享资源称为临界资源。

在设计操作系统时，设计人员必须严格界定系统中的哪些资源是临界资源，通过同步机制保证系统运行过程中，对临界资源的访问进行有效控制。

临界区是进程中访问临界资源的那段代码。访问临界资源是通过执行临界区代码来实现的，如果能使程序以互斥的方式进入临界区，就能够实现对临界资源的互斥访问。这一点可以通过在临界区前面加进入区代码，在临界区后面加退出区代码来实现。进入区代码在临界区代码之前执行，检查进程是否可以进入临界区并对临界区“加锁”。退出区代码在临界区代码之后执行，完成释放临界区访问权的功能。

在进程中，可以采用下列模式的代码来实现对临界资源的互斥访问。

```
{...
    进入区(Entry Section);
    临界区(Critical Section);
    退出区(Exit Section);
}
```

二、同步机制应遵循的准则

当存在多种可选的同步方案时，可能是硬件或软件的不同，也可能是算法的不同，根据什么来权衡利弊，做出恰当的选择呢？对于一种同步技术，如何对它的正确性和性能做出评价呢？同步机制应遵循的准则可以为人们提供判断、选择和评价的参考依据。

1. 空闲让进

当没有进程处于临界区时，表明临界资源处于空闲状态，应允许一个请求进入临界区的进程立即进入自己的临界区，以有效地利用临界资源。

2. 忙则等待

当已有进程进入临界区时，表明临界资源正在被访问，因而其他试图进入临界区的进程必须等待，以保证对临界资源的互斥访问。

3. 有限等待

对要求访问临界资源的进程，应保证在有限时间内能进入自己的临界区，以免进程陷入无限等待的状态。

4. 让权等待

当进程申请不到共享资源的访问权时，应立即释放处理机，以免进程陷入“忙等”状态，浪费 CPU 资源。

三、信号量机制

在信号量机制中，用某种类型的变量，即信号量的取值来表示资源的使用状况，或某种事件是否发生，以此为基础实现进程的同步。本节将介绍整型信号量机制、记录型信号量机制、AND 型信号量机制。

交通信号灯的作用是通过灯的不同颜色告知车辆应如何行驶，信号量机制类似这种信号灯的作用。对不同的共享资源设置被称为信号量的变量，用信号量的取值来表示资源的使用状况，或某种事件是否发生。通过信号量的取值来判断进程是否能访问与信号量对应的共享资源。

1. 整型信号量机制

整型信号量是表示共享资源状态且只能由特殊的原子操作改变的整型量。其完成同步功能的原理是定义一个整型变量，用整型变量值来标记资源的使用情况。如果整型量 >0 ，说明有可用资源；如果整型量 ≤ 0 ，说明资源忙，进程必须等待。对于一次只允许一个进程访问的临界资源，可定义一个用于互斥的整型信号量，并将其初始化为 1。整型信号量的值只能通过两个特定的原子操作 wait 和 signal 来改变。

(1) 整型信号量的 wait 和 signal 操作

s 定义为整型信号量。

```
Var s integer;
wait(s) //用于申请资源
{
while s ≤ 0 do no-op ;    //整型信号量值 ≤ 0 时循环执行空操作
    s = s - 1
}
signal(s) //用于释放资源
{ s = s + 1;
}
```

(2) 用整型信号量实现进程互斥

用整型信号量实现进程互斥的思想是：为必须互斥访问的临界资源 CS 定义一个互斥信号量 mutex，将初始值置为 1，然后将 CS 放入 wait(mutex) 和 signal(mutex) 之间。当 CS 可访问时，wait(mutex) 才能正常结束使进程进入 CS，如图 2-16 所示。

例 2-3：在前面进程 p1 和 p2 访问共享变量 counter 的程序中，使用整型信号量机制实现对 counter 的互斥访问。设置互斥信号量

mutex，初值为 1，说明初始状态是没有进程访问 counter 变量。在进程 p1 和 p2 的临界区代码前后分别加 wait(mutex) 和 signal(mutex) 操作，就可以实现进程 p1、p2 对 counter 的互斥访问。进程 p1 和 p2 互斥的同步代码描述如下。

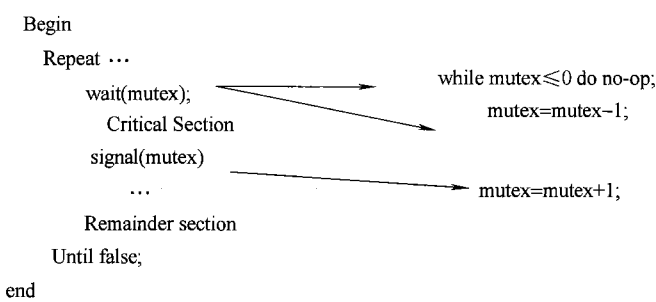


图 2-16 用整型信号量实现互斥

```

p1:
{ ...
    wait(mutex);
    counter = counter + 1;
    signal(mutex);
    ...
}

p2:
{ ...
    wait(mutex);
    counter = counter + 1;
    signal(mutex);
    ...
}

```

假定 p1 进程先执行对 counter 的访问。因为在进入临界区 $\text{counter} = \text{counter} + 1$ 前, p1 必须先执行 $\text{wait}(\text{mutex})$ 操作。而 mutex 此时的值为 1, 所以 $\text{while } \text{mutex} \leq 0 \text{ do no-op}$ 中的 while 条件不满足, while 循环结束, 执行 $\text{mutex} = \text{mutex} - 1$ 操作。结果信号量 mutex 的值为 0。p1 在结束 $\text{counter} = \text{counter} + 1$ 和 $\text{signal}(\text{mutex})$ 操作之前, 如果 p2 也试图进入临界区执行 $\text{counter} = \text{counter} + 1$, 由于 p2 在进入临界区代码前必须先执行 $\text{wait}(\text{mutex})$ 操作, 而此时 $\text{mutex} = 0$, 进程 p2 的 $\text{wait}(\text{mutex})$ 操作中的 $\text{while } \text{mutex} \leq 0 \text{ do no-op}$ 的 while 条件满足, 进程 p2 只能执行 while 循环中的空操作, 无法进入自己的临界区。只有当 p1 执行完临界区代码, 并通过执行 $\text{signal}(\text{mutex})$ 操作, 完成 $\text{mutex} = \text{mutex} + 1$, 使 $\text{mutex} = 1$ 后, 当 CPU 再次执行 p2 时, 才会由于 p2 执行的 $\text{while } \text{mutex} \leq 0 \text{ do no-op}$ 中的 while 条件不满足, 而结束循环。p2 执行完 $\text{wait}(\text{mutex})$ 的最后一条语句 $\text{mutex} = \text{mutex} - 1$ 后, 得以进入自己的临界区执行 $\text{counter} = \text{counter} + 1$ 。

对于任意进程 p, 要访问临界资源 CS, 如果与 CS 对应的整型信号量设置为 mutex, mutex 初值为 1, 以如下形式在进程中添加同步代码可以实现进程对 CS 的互斥访问。

```

P
{
    wait(mutex);
    CS;
    signal(mutex);
}

```

例 2-4: p1 和 p2 两个进程都访问同一个临界资源——打印机(用 CS 表示), 对应的整型信号量为 s, s 初值为 1。

p1 和 p2 两个进程中就应该包含以下两段同步代码。

```

p1
{ ...
    wait(s);
    CS;
    signal(s);
}

```

```

...}
p2
{...
    wait(s);
    CS;
    signal(s);
...
}

```

假设初始条件为打印机空闲， $s = 1$ ，若 $p1$ 先请求打印机，在 $p1$ 释放资源之前， $p2$ 也请求打印机。

```

p1: wait(s)
    因为  $s = 1$ 
    所以 while 循环结束
    执行  $s = s - 1$ ，结果  $s = 0$ 
    执行 CS 代码访问打印机

```

此时，若 $p2$ 也试图访问打印机。因为此时信号量 $s = 0$ ，故 $p2$ 进入 while 循环，不能继续执行 while 语句后的代码，无法进入 $p2$ 的临界区访问打印机，因此 $p2$ 不能访问 $p1$ 正在访问的打印机。由此，实现了 $p1$ 和 $p2$ 进程对打印机的互斥访问。

(3) 用整型信号量实现进程的协调

例 2-5: $p1$ 、 $p2$ 两个进程，要求 $p2$ 必须在 $p1$ 结束后执行。为此，设置一个信号量 s ，初始值为 0。同步代码如下。

```

parbegin
    begin p1; signal(s); end
    begin wait(s); p2; end
parent

```

如果 $p1$ 不执行，或者还没有执行完时，要执行 $p2$ ，由于执行 $wait(s)$ 时， s 等于 0， $wait(s)$ 中的 while 循环条件满足，因此一直循环，使得控制流不能执行到 $p2$ 。只有当 $p1$ 执行完，并通过执行 $signal(s)$ 操作，使得 $s = 1$ ， $wait(s)$ 中的 while 循环结束， $wait(s)$ 执行完毕，才能执行 $p2$ 。

(4) Linux 中的整型信号量

Linux 2.4 使用的自旋锁可以看作是整型信号量机制的一个应用实例。自旋锁是用来在多处理器环境中工作的一种特殊的锁。如果内核控制路径发现自旋锁“开着”，就是相应的整型信号量的值大于 0，就获取锁并继续自己的执行。相反，如果内核控制路径发现锁由运行在另一个 CPU 上的内核控制路径“锁着”，即整型信号量的值等于 0 或者小于 0，就反复执行一条紧凑的循环指令，直到锁被释放。

(5) 对整型信号量机制的总结

- 1) 整型信号量的值只能由 $wait$ 和 $signal$ 操作改变。
- 2) $wait$ 和 $signal$ 操作都是原子操作，即在这两个操作中对信号量的访问是不能被中断的。

在单 CPU 系统中，如果把“ $wait$ 和 $signal$ 操作都是原子操作”理解成 $wait$ 和 $signal$ 操作执

行的过程是不可以被打断的，当信号量 $s \leq 0$ 时，系统会在 `while s <= 0 do no-op` 中死循环。因为要保证 `wait` 操作的原子性，只能关中断。程序的执行流如下所示。

```
{ ...
  关中断
  wait(s)
  开中断
  ...
}
```

在这种情况下，如果当前 $S=0$ ，一直执行 `while` 循环，在关中断的情况下，CPU 不可能执行其他进程， S 的值无法被改变，系统会在 `while s <= 0 do no-op` 中死循环。

3) 原子操作可以通过关中断来实现。

4) 整型信号量机制的实例：Linux 中的自旋锁 `SpinLock`。

5) 不同的资源对应不同的信号量，并不是系统中所有的资源都用同一个信号量表示。

2. 记录型信号量机制

(1) 记录型信号量的数据类型

```
Type semaphore = record
    Value: integer;           //资源数量
    L: list of process;       //阻塞队列
end
```

(2) 记录型信号量的 `wait(s)` 和 `signal(s)` 操作

```
procedure wait(s)
    var s: semaphore;
    begin
        s.value = s.value - 1;
        if s.value < 0 then block(s.L);
    end.
procedure signal(s)
    var s: semaphore;
    begin
        s.value = s.value + 1;
        if s.value <= 0 then wakeup(s.L);
    end.
```

(3) 对记录型信号量 `wait(s)` 和 `signal(s)` 的说明

1) 当 $s.value \geq 0$ 时， $s.value$ 的值表示资源数量。当 $s.value < 0$ 时， $s.value$ 的绝对值等于某资源的等待队列中阻塞进程的数量。

2) 每次的 `wait(s)` 操作，意味着进程请求一个单位的资源，描述为 $s.value = s.value - 1$ 。当 $s.value < 0$ 时，表示资源已分配完毕。因而，进程调用 `block` 原语，进行自我阻塞，放弃处理机，并自我阻塞进程，其进程控制块被插入到阻塞队列 $s.L$ 中。

3) 每次的 `signal(s)` 操作，意味着进程释放一个资源，故 $s.value = s.value + 1$ 操作表示系统可用的资源数目加 1。若加 1 后 $s.value \leq 0$ ，则表示在该信号量的阻塞队列中，仍有等待该资源的进程被阻塞，故还应该调用 `wakeup` 原语，唤醒 $s.L$ 队列中的一个阻塞进程。

4) 如果 $s.value$ 的初值为 1, 表示只允许一个进程访问临界资源, 此时的信号量转化为互斥信号量。

5) 记录型信号量机制的优点是不存在“忙等”, 采取了“让权等待”的策略。

(4) 利用记录型信号量实现互斥

利用记录型信号量实现互斥的模型从实现互斥的代码片段上看与整型信号量一致, 都是在访问临界资源的进程中定位临界区。在临界区前面加 $wait(s)$, 在临界区后面加 $signal(s)$, 不同的是信号量的类型和 $wait(s)$ 、 $signal(s)$ 实现代码不同。实现进程互斥的代码模型描述如下。

```
var s: semaphore;
s.value = 1;
Begin
  Repeat
    wait(s);
    Critical Section;
    signal(s);
    Remainder section;
  Until false;
End
```

例 2-6: 图 2-17 表示了 3 个进程访问同一个临界资源时, 利用记录型信号量实现互斥的情况。图 2-17 仅以一种可能的执行流为例说明互斥实现的具体情形。3 个进程 $p1$ 、 $p2$ 、 $p3$ 访问同一个临界资源, 互斥信号量为 s , $s.value$ 初始值为 1。进程 $p1$ 先通过执行 $wait(s)$ 申请信号量, 得以进入临界区 CS, 访问临界资源。若此时进程 $p2$ 要访问临界资源, 因为执行进程 $p2$ 中的 $wait(s)$ 操作, 使 $s.value = -1$, 因而 $p2$ 被阻塞在 $s.L$ 队列中。在进程 $p1$ 退出临界区并执行 $signal(s)$ 操作之前, 若进程 $p3$ 也试图进入自己的临界区访问临界资源, 在 $p3$ 执行 $wait(s)$ 时, 使 $s.value = -2$, 因而, $p3$ 也被阻塞在 $s.L$ 中。此时, $s.L$ 队列中有两个被阻塞的进程, 即 $p2$ 和 $p3$ 。

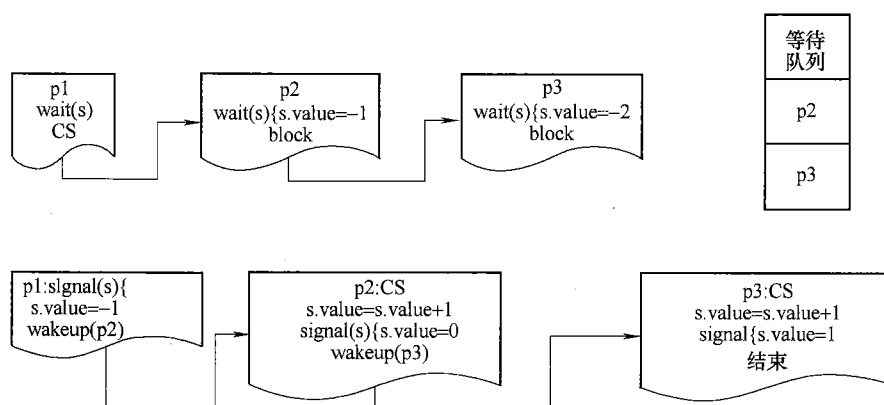


图 2-17 记录型信号量机制实现互斥的实例

当进程 $p1$ 退出临界区, 执行 $signal(s)$ 时, 先唤醒进程 $p2$, $p2$ 得以进入临界区。当 $p2$ 也退出临界区后, 执行 $signal(s)$ 操作, 唤醒进程 $p3$ 。

(5) 利用记录型信号量实现“协调”的应用举例

下面以一个简单的相互合作的问题为例，说明如何利用记录型信号量机制实现“协调”功能。

例 2-7：爸爸擀饼，妈妈烙饼。面板上只能容纳两张擀好的饼，只有当面板上有空闲空间时，爸爸才能把擀好的饼放在面板上。只有当面板上有饼时，妈妈才能从面板上取饼。试采用记录型信号量机制实现爸爸与妈妈进程的同步。

此问题中，爸爸进程和妈妈进程是一对相互合作的进程。爸爸进程需要的资源是面板上的空闲空间，妈妈进程需要的资源是爸爸擀好的饼。为实现爸爸、妈妈进程的同步，设置两个资源信号量为

```
var empty, full: semaphore
```

`empty` 信号量对应的资源是面板上的空间，`full` 信号量对应的资源是饼。因为在两个进程开始工作之前，面板为空，有可放两个饼的空间，饼的数量为 0。因此，初始化两个资源信号量，使得

```
empty.value = 2
```

```
full.value = 0
```

协调爸爸、妈妈两个进程的目的是使爸爸进程仅当在面板上有空闲空间时才能把擀好的饼放在面板上，然后继续工作。否则，爸爸进程要自我阻塞在空闲空间这个资源的阻塞队列中。妈妈进程仅当面板上有饼时才能进行从面板上取饼的操作，然后继续工作。否则，要被自我阻塞在饼这个资源的阻塞队列中。下面是爸爸进程和妈妈进程实现协调的同步代码的描述。

爸爸进程如下。

```
|
| 擀饼；
|    wait(empty)；
|    放饼；
|    signal(full)；
|
```

妈妈进程如下。

```
{ wait(full)；
  取饼；
  signal(empty)；
  烙饼；
|
```

(6) Linux 2.4 内核记录型信号量实例

Linux 2.4 内核的 semaphore 信号量的定义文件：`/include/asm-i386/semaphore.h`。

1) 信号量结构的定义。

```
struct semaphore { atomic_t count;    //记录资源数量
                  int sleepers;    //记录等待队列中是否有因等待资源被阻塞的进程
                  wait_queue_head_t wait; //等待队列队首的地址
};
```

2) 对信号量的操作。

①用于申请资源的 down 函数 static inline void down(struct semaphore * sem) 等价于下面的汇编语言程序。

```
down:
movl $ sem,%ecx
    lock; decl (%ecx)
    jns 1f          //如果大于等于0, 转到1: 处结束
    pushl %eax      //如果小于0, 保存寄存器值
    pushl %edx
    pushl %ecx
    call _ _down    //阻塞进程
    popl %ecx       //恢复寄存器的值
    popl %edx
    popl %eax
```

1:

down() 函数减少 * sem 信号量的 count 字段的值(信号量结构的 0 偏移处), 然后检查该值是否为负。如果 count 大于或等于 0, 当前进程获得资源并继续正常执行。否则, count 为负, 当前进程必须挂起。把一些寄存器的内容保存在栈中, 然后调用 _ _down(), 把当前进程阻塞在信号量的等待队列上。所以这里 down() 函数相当于记录型信号量机制中的 wait 操作。

②用于释放资源的 up 函数 static inline void up(struct semaphore * sem) 等价于下面的汇编语言程序。

```
movl $ sem,%ecx;
lock; incl (%ecx)
jg1f          //大于0, 什么也不做, 转到1: 结束
pushl %eax    //小于等于0, 保存寄存器值
pushl %edx
pushl %ecx
call _ _up
popl %ecx
popl %edx
popl %eax
```

1:

这里的 _ _up 是下列 C 函数。

```
void _ _up(struct semaphore * sem)
{
wake_up(&sem -> wait); //唤醒进程
}
```

up() 函数增加 * sem 信号量 count 域的值, 然后检查它的值是否大于 0。如果 count 大于 0, 说明没有进程被阻塞在队列上。因此, 什么事也不做。否则, 调用 _ _up() 函数以唤醒一个阻塞的进程。可见, up() 函数相当于记录型信号量机制中的 signal 操作。

3. AND 型信号量机制

(1) AND 型信号量机制的引入

一个进程在运行过程中往往需要申请多个共享资源，如果使用整型或记录型信号量，可能会出现因为申请资源的顺序不当而导致进程死锁（死锁的相关内容见第三章）。下面以一个实例说明这种情况是如何发生的。

假定 A、B 两个进程，都要求访问临界资源 D 和 E。若设置两个用于互斥的信号量 Dmutex 和 Emutex，并令它们的初值为 1，相应地，两个进程要包含两个对 Dmutex 和 Emutex 的操作，即

```
processA:                processB:
wait(Dmutex);            wait(Emutex);
wait(Emutex);            wait(Dmutex);
```

在这种互斥方式下，若进程 A 和 B 按下列顺序交替执行 wait 操作，会造成死锁。

processA: wait(Dmutex); 于是 Dmutex = 0

processB: wait(Emutex); 于是 Emutex = 0

processA: wait(Emutex); 于是 Emutex = -1, 进程 A 阻塞

processB: wait(Dmutex); 于是 Dmutex = -1, 进程 B 阻塞

进程 A 因等待进程 B 释放临界资源 E 的访问权而被阻塞，进程 B 因等待进程 A 释放临界资源 D 的访问权而被阻塞。为了解决这样的问题，引入了 AND 型信号量机制。

AND 信号量机制的基本思想是将进程在整个运行过程中所需要的所有资源一次性地全部分配给进程，待该进程使用完后再一起释放。只要还有一个资源不能分配给该进程，其他所有可能为之分配的资源也不分配给它。

(2) AND 型信号量机制的实现

为不同的共享资源设定相应的信号量 $s_1, s_2, \dots, s_n$ ，调用 Swait 操作申请资源，调用 Ssignal 操作释放资源。Swait 和 Ssignal 实现代码描述如下。

```
Swait( $s_1, s_2, \dots, s_n$ )
```

```
    if  $s_1 \geq 1$  and  $\dots s_n \geq 1$  then
```

```
        for  $i = 1$  to  $n$  do  $si = si - 1$ 
```

```
    end for
```

```
else
```

把进程插入 S_i 阻塞队列，并把程序计数器的值赋为 Swait 的起始地址。

```
end if
```

```
Ssignal( $s_1, s_2, \dots, s_n$ )
```

```
    For  $i = 1$  to  $n$  do
```

```
         $Si = Si + 1$ ;
```

将阻塞在 S_i 队列中的进程唤醒，插入就绪队列。

```
end for
```

四、经典的进程同步问题

本节将介绍如何用记录型信号量机制解决两个经典的进程同步问题。

1. 生产者—消费者问题的描述

生产者—消费者问题是相互合作进程关系的一种抽象。例如，输入进程和计算进程的关系中，输入进程是生产者进程，计算进程是消费者进程。计算进程和输出进程的关系中，计

算进程是生产者进程，输出进程是消费者进程。

(1) 问题描述

生产者进程生产消息，并将消息提供给消费者进程消费。在生产者进程和消费者进程之间设置了一个具有 n 个缓冲区的缓冲池，生产者进程可以将它所生产的消息放入缓冲池的一个缓冲区中，消费者进程可以从一个缓冲区中取得一个消息消费。任意两个进程必须以互斥的方式访问公共缓冲池。当缓冲池空，没有可供消费的消息时，消费者进程必须阻塞等待。当缓冲池装满消息，没有空闲缓冲区时，生产者进程必须阻塞等待。

如图 2-18 所示，out 指针指向装有消息的缓冲区，消费者进程从 out 指向的缓冲区中取消息。in 指针指向空缓冲区，生产者进程总是把新生产的消息放入 in 指向的空缓冲区。

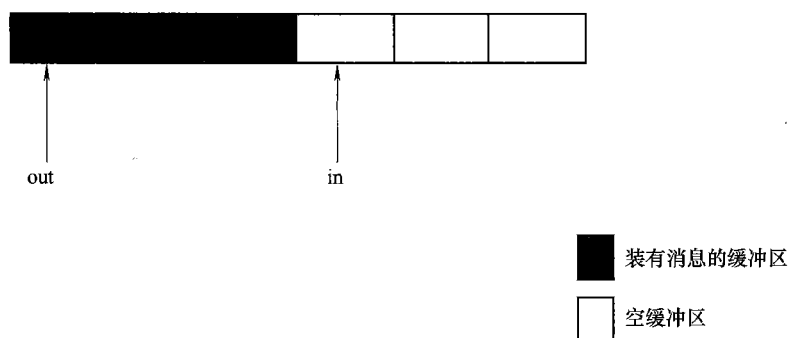


图 2-18 生产者—消费者进程共享的缓冲池

(2) 需要解决的问题

1) 实现任意两个进程对缓冲池的互斥访问。

2) 实现对生产者进程和消费者进程的“协调”，即缓冲池中有消息时消费者进程才能执行取消息的操作。无消息时，阻塞消费者进程。缓冲池中有空闲缓冲区时，生产者进程才能执行放消息的操作。无空闲缓冲区时，阻塞生产者进程。

(3) 信号量的设置

1) 设置一个互斥信号量 `mutex`，用于实现对公共缓冲池的互斥访问，初值为 1。

2) 设置两个资源信号量，分别表示可用资源数。

`empty`：表示缓冲池中的空缓冲区数，初值为 n 。

`full`：表示装有消息的缓冲区数，初值为 0（一个缓冲区中放一个消息）。

(4) 同步程序

利用记录型信号量机制实现生产者—消费者问题的同步代码描述如下。

1) 生产者进程同步代码的描述。

```

Producer:
    begin
repeat
    ...
    produce an item in nextp;
    wait(empty);           //申请空缓冲区

```

```

wait( mutex);           //申请公共缓冲池的互斥访问权
buffer( in ) = nextp;    //将消息放入 in 指针指向的缓冲区
in = ( in + 1) mod n;     //in 指针指向下一个空缓冲区
signal( mutex);         //释放对公共缓冲池的互斥访问权
signal( full);           //释放消息资源
until false;
end

```

2) 消费者进程同步代码的描述。

```

Consumer:
begin
repeat
...
wait( full);           //申请消息
wait( mutex);         //申请公共缓冲池的互斥访问权
nextc = buffer( out);  //从 out 指针指向的缓冲区中取消息
out = ( out + 1) mod n; //out 指针指向下一个装有消息的缓冲区
signal( mutex);        //释放对公共缓冲池的互斥访问权
signal( empty);        //释放空缓冲区
consume item in nextc;
until false;
end

```

(5) 说明

1) wait 和 signal 操作必须成对出现。

2) wait 操作的顺序不能颠倒。必须先对资源信号量(即 empty 和 full)进行 wait 操作,然后再对互斥信号量进行 wait 操作。

3) 用记录型信号量机制解决生产者—消费者问题,对具有相互合作关系的进程,提供了解决问题的模型。

2. 读者—写者问题

(1) 问题描述

D 是多个进程共享的数据区,允许多个进程同时读 D 区,仅允许一个进程写 D 区,且有进程写 D 区时,不能有任何其他进程读或写 D 区。

数据库管理中存在这种同步问题的实例,系统允许多个用户同时读一个数据库表,但是任意时刻只允许一个用户修改它,当数据库表被用户(通常是有特殊权限的数据库管理员)修改时,任何其他用户不能读或者写这一数据库表。

(2) 信号量的设置

1) 全局变量 readcount 用于对进入共享区的读进程计数。

2) 互斥信号量 rmutex 用于对多个进程共享的全局变量 readcount 的互斥访问。

3) 互斥信号量 wmutex 用于实现读操作与写操作的互斥,以及写操作与写操作的互斥。

(3) 同步程序

1) 写进程同步代码的描述。

```

writer:
begin;
    wait( wmutex );
...
writing operation;
...
    signal( wmutex );
end;

```

2) 读进程同步代码的描述。

```

reader:
begin;
    wait( rmutex );
    if readcount = 0 then wait( wmutex );
    readcount ++ ;
    signal( rmutex );
    ...
    reading file from D;
    ...
    wait( rmutex );
    readcount -- ;
    if readcount = 0 then signal( wmutex );
    signal( rmutex );
end;

```

五、管程

信号量机制的缺陷是每个访问共享资源的进程都必须自备同步操作 `wait(s)` 和 `signal(s)`。这就使大量的同步操作分散在各个进程中，这不仅给系统的管理带来麻烦，而且还会因同步操作的使用不当而导致系统出错，因此引入了管程的概念。

1. 管程的基本概念

(1) 管程的定义

管程是描述共享资源的数据结构和在数据结构上的共享资源管理程序的集合。其中包括变量的定义、变量的初始化代码，以及管理共享资源的过程。管程的语法描述如下。

```

type monitor = name = monitor
    variable declarations;
    procedure entry1(…)
begin…end;
    ...
    procedure entrypn(…)
        begin…end;
    begin initialization code;
end

```

(2) 对管程的说明

1) 管程是可供程序员调用的软件包。管程是一个由过程、变量及数据结构等组成的集合，它们组成一个特殊的模块或软件包。进程可以在任何需要的时候调用管程中的过程，但它们不能在管程外的过程中直接访问管程内的数据结构。

2) 每次只有一个进程调用管程执行，任意时刻管程中只能有一个活跃进程。若多个进程同时调用一个管程中的过程，只有一个进程得以进入管程继续运行，其他进程则被阻塞。

3) 管程是一种编程语言的构件，所以编译器知道它们很特殊，并可以调用与其他过程不同的方法来处理它们。典型的，当一个进程调用管程中的过程时，前几条指令将检查在管程中是否有其他的活跃进程。如果有，调用进程将等待直到另一个进程离开管程；如果没有，则调用进程进入管程。

(3) 条件变量

为了进行并发处理，管程必须包含同步工具。例如，假设一个进程调用了管程，并且当它在管程中时必须被阻塞，直到满足某些条件。这就需要一种机制，使得该进程不仅能被阻塞，而且以后当条件满足时能够唤醒这个进程，并允许它在阻塞点重新进入管程。管程通过使用条件变量提供同步支持，这些条件变量包含在管程中，并且只有在管程中才能被访问。条件变量的定义形式是：Var x, y: condition。

一个管程过程，可以用在某条件变量上执行 wait 操作，将调用管程的进程阻塞并插入该条件的阻塞队列；用在条件变量上执行 signal 操作，唤醒在该条件上阻塞的进程。管程中的 signal 操作不同于信号量机制中的 signal 操作。如果某个条件变量的阻塞队列为空，调用该条件变量的 signal 操作将不产生任何结果。

2. 管程的应用

(1) 利用管程解决生产者—消费者问题

利用管程机制解决生产者—消费者问题要先建立一个管程，把对共享资源访问的功能都放在管程中。建立的管理名为 PC (Producer—Consumer)，其中包括了两个过程，一个是 enter(item) 过程，生产者进程调用该过程向缓冲池中投放消息；另一个是 remove(item) 过程，消费者进程调用该过程从公共缓冲池中取消息。管程 PC 的定义如下。

```

type PC = monitor
condition full, empty;           //定义两个条件变量
integer count;                   //缓冲池中的消息数
procedure enter(item)
begin
    if count = N then wait(full); //如果缓冲池满，则将生产者进程阻塞在条件变量 full 的阻塞队
                                //列中
    enter = item;
    count = count + 1;
    signal(empty);               //如果有因缓冲池空而被阻塞的消费者进程，则唤醒该进程
end
procedure remove(item)
begin
    if count = 0 then wait(empty); //如果缓冲池为空，将消费者进程阻塞在条件变量 empty 的阻塞队

```

列中

```

remove - item;
count = count - 1;
signal(full);           //如果有因缓冲池满而被阻塞的生产者进程，则唤醒该进程
end
count = 0;
end monitor

```

调用管程的生产者进程和消费者进程可描述如下。

```

Producer ;
begin
    repeat
        produce an item in nextp;
        PC. enter(item);
    until false;
end
Consumer;
begin
    repeat
        PC. remove(item);
        consume the item in nextc;
    until false;
end

```

(2) 利用管程解决哲学家进餐问题

1) 问题描述。

有 5 个哲学家围坐在一个圆桌旁，每两人之间放一支筷子。每个哲学家的行为是思考、感到饥饿，然后用餐。为了能进餐，每个哲学家必须拿到两支筷子，并且每个人只能直接从自己的左边或右边去取筷子。

哲学家进餐问题是多个进程竞争有限资源的一类问题的模型。

2) 解决哲学家进餐问题的管程定义如下。

```

type dining-philosophers = monitor
var state: array[0..4] of (thinking, hungry, eating);
var self: array[0..4] of condition;
Procedure entry pickup(i: 0..4)
    begin
        state[i] = hungry;
        test(i);
        if state[i] ≠ eating then self[i].wait;
    end
Procedure entry putdown(i: 0..4)
    begin
        state[i] = thinking;
        test(i + 4 mod 5);
    end

```

```

test(i + 1 mod 5);
end
Procedure test(k: 0..4)
begin
    if state[k + 4 mod 5] ≠ eating and state[k] = hungry and state[k + 1 mod 5] ≠ eating
    then begin
        state[k] = eating;
        self[k].signal;
    end;
end
begin
    for i = 0 to 4
        Do state[i] = thinking;
    end

```

3) 哲学家进程描述如下。

```

void philosopher(int i)
{
    while(true)
    {
        think();
        dining-philosophers.pickup(i);
        eat();
        dining-philosophers.putdown(i);
    }
}

```

第五节 进 程 通 信

操作系统提供进程通信功能，以支持进程之间的信息交换。进程之间的高级通信机制分为共享存储器系统、消息传递系统和管道通信系统。

1. 共享存储器系统

在共享存储器系统中，相互通信的进程共享某些数据结构或共享存储区，进程之间能够通过这些空间进行通信。共享存储系统可分为两种类型。

1) 基于共享数据结构的通信方式。在这种通信方式中，要求诸进程公用某些数据结构，以实现进程间的信息交换。例如在生产者—消费者问题中，使用有界缓冲区这种数据结构来实现进程间的通信。

2) 基于共享存储区的通信方式。为了传输大量数据，在存储器中划出一块共享存储区，进程可以通过对共享存储区中的数据的读或写来实现通信。

2. 消息传递系统

在消息传递系统中，进程间通过操作系统提供的一组通信程序传递格式化的消息。这种方式对应用程序隐藏了通信实现的细节，使通信过程对用户是透明的。

消息传递系统中，根据源进程向目标进程传递消息方式的不同，可分为直接通信方式和

间接通信方式。

- 1) 直接通信方式。操作系统利用发送程序直接把消息发送给目标进程。
- 2) 间接通信方式。进程之间的通信需要通过用于暂存消息的共享数据结构来实现，如信箱。该方式既可以实现实时通信，又可以实现非实时通信。

3. 管道通信

管道(Pipeline)是连接读写进程的一个特殊文件，也被称为管道文件。管道文件存在于外存中，其中的消息没有固定长度，能用于进程间大量的信息通信。向管道提供输入的发送进程以字符流的形式将大量的数据送入管道(写)。接受管道输出的接收进程，从管道中接收数据(读)。

4. 消息缓冲队列

消息缓冲队列机制广泛用于本地进程之间的通信。该机制包括数据结构、发送原语和接收原语，每个进程都有自己的消息缓冲队列和消息缓冲区。发送进程发送消息时，先申请一个消息缓冲区，将要发送的消息从发送进程的发送区放入消息缓冲区。然后，调用发送原语将消息发送给接收进程，发送原语将发送缓冲区插入接收进程的消息缓冲队列。接收消息的进程通过调用接收原语将该进程消息缓冲队列中的消息复制到自己的消息接收区。

消息缓冲区是一个结构型数据结构。通常包括发送进程标志符、消息长度、消息正文和指向下一个消息缓冲区的指针。在采用消息缓冲队列的系统中，进程控制块中要增加指向消息缓冲队列的指针、消息队列的互斥信号量和消息队列的资源信号量等字段。消息缓冲队列需要被当作临界资源，在发送原语和接收原语中对消息缓冲队列的访问需要进行互斥与同步。

第六节 线 程

在传统的操作系统中，进程是进行资源分配和独立执行的基本单位。为了进一步提高程序的并发性，减少系统开销，在操作系统中引入了线程的概念。

由于进程既是独立执行的基本单位，又是资源拥有者，在进程创建、撤销和切换时需要较大的时空开销。所以，系统中所设置的进程数和进程切换的频率都受到了限制，影响了操作系统并发程度的提高。引入线程作为独立调度和分派的单位，不独立拥有资源(仅有少量基本资源)，而与其他线程共享同一进程的资源，减小了系统的时空开销。

线程的实质是把进程的任务划分成更小、具有独立功能的单位，以线程的形式来并发执行，以提高程序并发执行的程度。

一、线程的描述

1. 线程的概念和分类

(1) 线程的概念

线程是进程中的一个实体，是被系统独立调度和分派的基本单位。线程只拥有在运行中必需的资源，包括程序计数器、一组寄存器和栈，但它可与同属一个进程的其他线程共享进程所拥有的全部资源。一个线程可以创建和撤销另一个线程。同一进程中的多个线程可以并发执行。线程在运行中呈现间断性，也有就绪、阻塞和执行3种基本状态。

(2) 线程的分类

线程的实现可以分为两类，即用户级线程和内核级线程。内核级线程依赖于内核，用户进程和系统进程中的线程，它们的创建、撤销和切换都由内核实现。在内核中为线程创建线程控制块，内核根据该控制块感知线程的存在并对线程进行控制。

用户级线程不依赖于内核，用户级线程的创建、撤销和切换都与内核无关。

下面从不同方面对内核级线程和用户级线程进行比较。

1) 线程的调度与切换速度。

内核级线程的调度由内核的线程调度程序完成，用户级线程的调度则由用户线程包中的一个过程来完成。内核级线程的调度程序运行在系统态，用户级线程的调度程序运行在用户态。

内核级线程的调度规则与进程调度相似，用户级线程的调度规则相对简单。内核级线程切换慢，用户级线程切换快。

2) 系统调用。

内核级线程进行系统调用，只阻塞该线程。用户级线程的系统调用，要阻塞线程所属的进程。

3) 线程执行时间的分配。

内核级线程的 CPU 时间以线程为单位分配，每一个线程都可以独享一个 CPU 时间片。用户级线程的 CPU 时间以进程为单位，同一进程的多个线程共享一个 CPU 时间片。

2. 线程的 3 种基本状态

与进程一样，线程的主要状态有就绪、运行和阻塞 3 种。就绪态是线程一旦获得 CPU 就可以投入运行的状态。运行态是线程获得 CPU 正在运行的状态。阻塞态是线程由于等待某个事件的发生而暂停执行的状态。

线程的 3 种状态转换如图 2-19 所示。

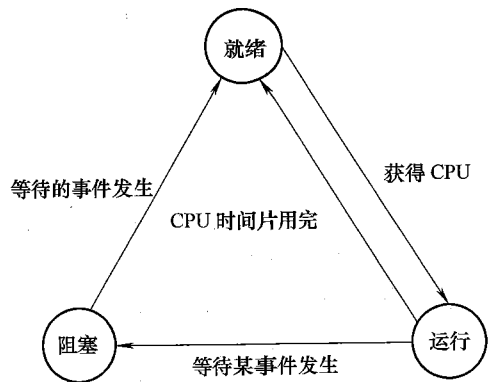


图 2-19 线程状态转换图

3. 线程控制块

(1) 线程控制块的定义

每个线程都由一个数据结构表示，包括它的基本状态、标识及记账信息。这个数据结构就是线程控制块 (Thread Control Block, TCB)。TCB 记录了操作系统所需要的、用于描述线程情况及控制线程运行所需的全部信息。

(2) 线程控制块中的信息

线程控制块中包含的信息有线程标识符信息、处理机状态信息、线程调度信息和线程控制信息。

(3) 线程控制块的组织方式

线程控制块通常采用链接方式来组织，把同一进程中具有相同状态的 TCB 用指针链接成队列，如图 2-20 所示。

4. 线程与进程的关系

由于线程与进程密切相关，可从以下几个角度来说明线程与进程的关系。

1) 资源和调度。线程是程序执行的基本单位，进程是拥有资源的基本单位。

2) 地址空间资源。不同进程的地址空间是相互独立的，而同一进程中的各线程共享同一地址空间。

3) 通信关系。进程之间的通信必须使用操作系统提供的进程间通信机制，而同一进程中的各线程间可以通过直接读或写全局变量来进行通信，甚至无需操作系统的参与。

4) 并发性。多进程之间可以并发执行，多线程之间也可以并发执行，而且同一进程中的多个线程之间也可并发执行。

5) 系统开销。由于创建进程或撤销进程时，系统都要为之分配或回收资源，操作系统所付出的开销远大于创建或撤销线程时的开销。在进行进程上下文切换时，涉及整个当前进程的 CPU 环境的保存及新调度到进程的 CPU

环境的设置。而线程上下文切换时，只需保存和设置少量寄存器内容，因此开销很小。另外，由于同一进程内的多个线程共享进程的地址空间，因此，同一进程中的线程的上下文切换要更快。

图 2-21 和图 2-22 分别从执行的角度和进程管理的角度来描述线程和进程的关系。

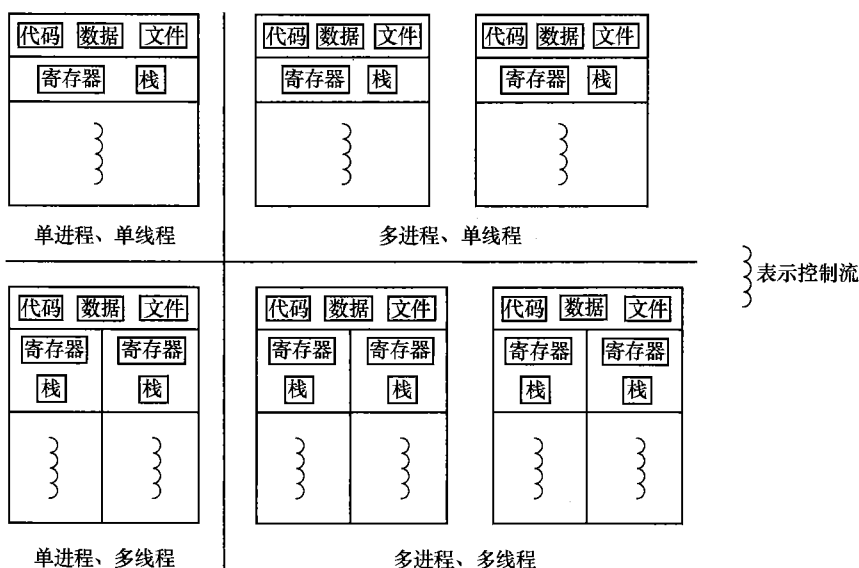


图 2-21 从执行角度看进程与线程的关系

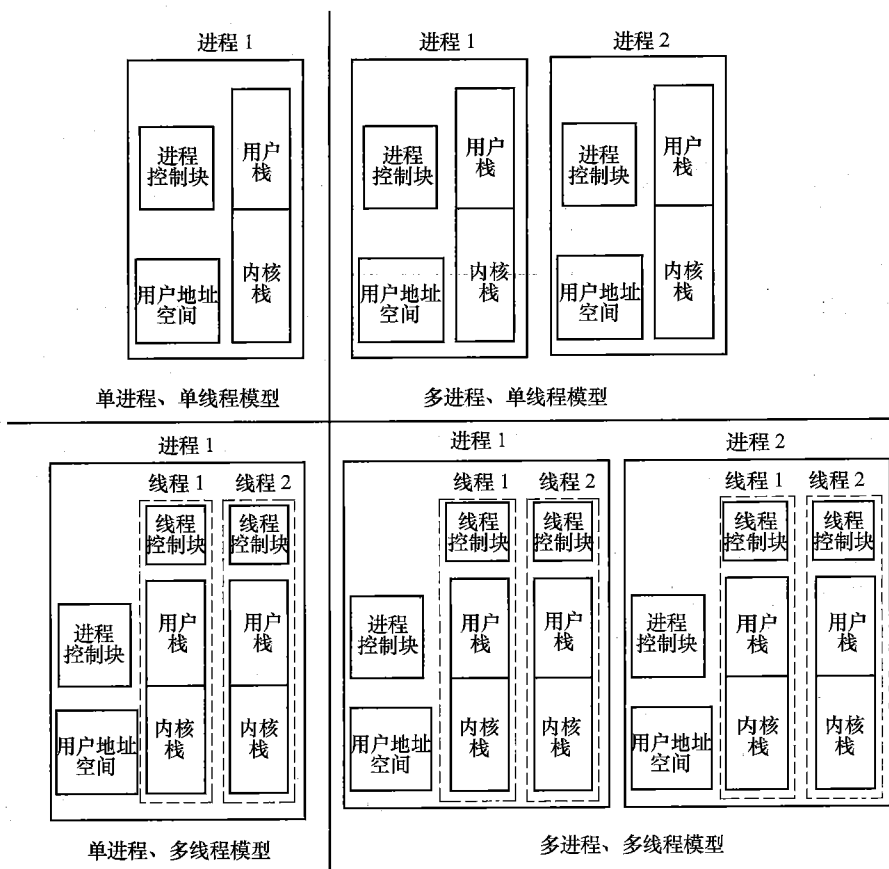


图 2-22 从进程管理角度看线程与进程的关系

二、线程的控制

线程控制是线程实现中最基本的功能。它包括创建新线程、终止线程、线程调度和线程切换,以及线程由于等待某个事件的发生而被阻塞与该事件发生后线程被唤醒。

1. 线程创建

(1) 用户线程的创建

用户线程的创建是通过调用线程库中的实用程序完成的。创建线程的实用程序为新线程申请空白线程控制块,并初始化线程控制块,然后将新线程插入其所属进程的就绪线程队列。

(2) 内核线程的创建

内核线程的创建是由内核完成的。内核为新线程申请空白线程控制块,并初始化线程控制块,然后将新线程插入其所属进程的就绪线程队列。

2. 线程的终止

(1) 引起线程终止的原因

1) 正常结束。

2) 异常结束。

3) 外界干预。

(2) 线程的终止过程

如果系统发生了上述要求终止线程的事件，用户线程由线程库的实用程序来调用线程终止原语，而内核线程由内核来调用线程终止原语，然后按下列过程终止指定的线程。

1) 根据被终止线程的标识符，从 TCB 集合中检索出该线程的 TCB，从中读出该线程的状态。

2) 若被终止线程正处于运行状态，应立即终止该线程的执行，并置调度标志为真，用于指示该线程被终止后应重新执行线程调度程序。

3) 将被终止线程的 TCB 从所在队列(或链表)中移出，等待其他程序来搜集信息。

3. 线程的调度与切换

(1) 用户线程的调度与切换

由于内核没有意识到用户线程的存在，所以内核继续以进程为单位进行调度，并且给该进程指定一个状态(就绪、运行或阻塞等)。用户线程的调度在应用程序内部进行，通常采用非抢占式和更简单的规则，如时间片轮转规则，也无需用户模式和内核模式之间的切换，所以速度特别快。调度算法可以是应用程序专用的，可以去适应应用程序而不会扰乱底层的操作系统调度程序。CPU 时间片分配给进程，当进程内有多个线程时，每个线程的执行时间相对较少。

一个多用户线程的应用程序不能充分利用多 CPU 技术。内核一次只把一个 CPU 分配给一个进程，因此一个进程中只有一个用户线程可以执行，无法享用多 CPU 带来的好处。

由于所有线程管理数据结构都在一个进程的用户地址空间中，线程切换不需要内核模式特权，所以也不需要用户模式和内核模式之间的切换，节省了两种模式间切换的开销。

(2) 内核线程的调度与切换

内核线程由内核来维护其上下文信息，调度是由内核以线程为单位进行的。内核线程的调度和切换都需要用户模式和内核模式之间的切换。

调度可以为一个进程中的多个内核线程分配多个 CPU，使多个内核线程达到并行。

4. 线程的阻塞与唤醒

(1) 引起线程阻塞的事件

1) 请求系统服务。当运行态线程请求操作系统提供服务时，由于某种原因，操作系统并不立即满足该线程的要求，该线程只能被阻塞。

2) 启动某种操作。当线程启动某种操作后，如果该线程必须在该操作完成之后才能继续执行，则必须先将该线程阻塞，以等待该操作完成。

3) 新数据尚未到达。对于相互合作的线程，如果其中一个线程先获得另一个线程提供的数据才能运行以对数据进行处理，则只要其所需数据尚未到达，该线程只能被阻塞。

(2) 用户线程的阻塞与唤醒

如果进程中的一个用户线程被阻塞，则整个进程都必须等待，即使还有其他用户线程可以在应用程序内运行。

正在执行的用户线程，当需要等待某事件的发生时，该线程便调用线程阻塞原语把自己阻塞，该线程所属的进程也调用进程阻塞原语把自己阻塞。用户线程的阻塞过程如下。

- 1) 停止该线程的执行，将该线程的状态改为阻塞态。
- 2) 将该线程控制块插入相应的线程阻塞队列。
- 3) 将该线程所属进程的状态改为阻塞态。
- 4) 将该线程所属进程的进程控制块插入相应的进程阻塞队列。
- 5) 将控制传递给进程调度程序，重新进行进程调度。

当被阻塞的线程所等待的事件发生时，则由有关进程唤醒原语将该线程所属的进程唤醒，然后由有关线程唤醒原语将该线程唤醒。用户线程的唤醒过程如下。

- 1) 将该线程所属进程的状态由阻塞改为就绪。
- 2) 将该线程所属进程的进程控制块从进程阻塞队列中移出。
- 3) 将该线程所属进程的进程控制块插入进程就绪队列。
- 4) 将该线程状态由阻塞改为就绪。
- 5) 将该线程的线程控制块从线程阻塞队列中移出。
- 6) 将该线程的线程控制块插入线程就绪队列。

(3) 内核线程的阻塞与唤醒

如果进程中的一个内核线程被阻塞，内核可以调度同一个进程中的另一个内核线程运行。

正在执行的内核线程，当需要等待某事件的发生时，该线程便调用线程阻塞原语进行自我阻塞。内核线程的阻塞过程如下。

- 1) 停止该线程的执行，将该线程的状态改为阻塞态。
- 2) 将该线程控制块插入相应的线程阻塞队列。
- 3) 将控制传递给线程调度程序，重新进行线程调度。

当被阻塞的线程所等待的事件发生时，则由有关线程唤醒原语将该线程唤醒。内核线程的唤醒过程如下。

- 1) 将该线程状态由阻塞态改为就绪态。
- 2) 将该线程的线程控制块从线程阻塞队列中移出。
- 3) 将该线程的线程控制块插入线程就绪队列。

阻塞和唤醒是一对作用相反的原语，因此，如果在某线程中调用了阻塞原语，则必须在与之相合作的另一个线程或其他相关的线程中安排唤醒原语，用于唤醒阻塞线程。

三、线程的同步

一个进程中的所有线程共享同一个地址空间和诸如打开的文件之类的其他资源。一个线程对资源的任何修改都会影响同一个进程中其他线程的环境。因此，需要对各种线程的活动进行同步，保证诸线程以互斥的方式访问临界资源，以使它们互不干扰且不破坏数据结构。线程同步的机制有原语操作和信号量机制。

四、线程通信

线程通信是指线程之间的信息交换。由于同一进程中线程间共享内存和文件资源，各线程间可以通过直接读/写全局变量来进行通信，甚至无需操作系统内核的参与。对于不同进程的线程间通信，则必须使用操作系统提供的线程间通信机制。

思考题与练习题

一、单项选择题(在每小题后的四个备选项中只有一个是符合题目要求的,请将其选出并将代码填写在题后的括号内)

1. 下列选项中,操作系统提供给应用程序的接口是【 】。
A. 系统调用 B. 中断 C. 库函数 D. 原语
2. `wait(s)` 操作中的 `block(s, L)` 阻塞的进程是【 】。
A. 申请资源的进程 B. 申请临界资源的进程
C. 调用 `wait(s)` 的进程 D. 提供 `wait(s)` 操作的系统进程
3. 在一个单处理机系统中存在 5 个进程,处于就绪队列中的就绪进程数最多为【 】。
A. 0 个 B. 4 个 C. 5 个 D. 6 个
4. 若记录型信号量的 `wait(s)` 和 `signal(s)` 操作的信号量 `s` 初值为 2,当前值为 -1,则在 `s` 的阻塞队列中处于阻塞态的进程数为【 】。
A. 0 个 B. 1 个 C. 2 个 D. 3 个

二、填空题(请在题目空白处填写正确内容)

1. 进程是由正文段、_____和_____构成的实体。
2. 进程的 3 种基本状态是执行态、_____态和_____态。
3. 整型信号量的值只能被_____和_____改变,不允许系统中的其他程序改变信号量的值。
4. 临界区是指访问_____的代码。

三、简答题

1. 什么是进程?说明进程与程序的区别和联系。
2. 操作系统在什么时候创建进程?操作系统如何创建一个进程?
3. 时钟中断信号是如何产生的?时钟中断处理程序(时钟驱动程序)的功能是什么?
4. 请说明单重中断的处理过程。
5. 进程具有哪些特征?
6. 什么是线程?为什么要引入线程?
7. 实现进程互斥的基本原理是什么?

四、综合应用

有两个进程 `pA`、`pB` 合作解决文件打印问题:`pA` 将文件记录从磁盘读入主存的缓冲区,每执行一次读一个记录;`pB` 将缓冲区的内容打印出来,每执行一次打印一个记录。缓冲区的大小等于一个记录大小。请用记录型信号量机制的 `wait(s)` 和 `signal(s)` 操作来保证文件的正确打印,并写出同步代码。

第三章 进程调度与死锁

当多个进程在系统中并发执行时，所有进程共享 CPU。当某一 CPU 上运行的进程因阻塞或进程运行结束而使 CPU 可以分配给其他进程使用时，如何从众多就绪的可运行进程中选择一个进程，将 CPU 分配给该进程，使系统有效运行，是多任务操作系统必须解决的问题之一。本章介绍进程调度的算法，以及选择进程调度算法的原则。

在多任务系统中，进程共享系统资源，当多个进程竞争有限的共享资源时会发生进程无限阻塞，不能继续运行的僵死状态，即本章所述的进程死锁问题。死锁发生的原因和解决死锁的方式是本章阐述的另一个重要知识点。

第一节 进程调度的功能与时机

一、进程调度的功能

进程调度功能由操作系统内核的进程调度程序完成，在 Linux 内核中，进程调度功能的实现从调用内核函数 `schedule()` 开始。进程调度的功能是按照某种策略和算法从就绪态进程（在 Linux 中是可执行进程）中为当前空闲的 CPU 选择在其上运行的新进程。在后续章节中将阐述进程调度的策略和算法。

二、进程调度的时机

当一个进程运行结束（包括正常结束和异常结束）、进程阻塞、中断返回、在支持抢占式调度的系统中有比当前运行进程优先级更高的进程到来、当前运行进程的时间片用完时，系统都会通过执行进程调度程序重新进行进程调度。

第二节 进程调度算法

进程调度算法是指从就绪态进程中选择一个或几个进程为其分配 CPU，使其进入执行态的算法。也就是说进程调度算法要“决定”把 CPU 分配给就绪队列中的哪个进程。进程调度由操作系统内核中的进程调度程序完成。

一、选择调度方式和算法的若干准则

不同的调度算法有不同的特点，为了根据特殊应用领域的要求选择合适的调度算法，可以下面的准则作为选择依据。

1. 周转时间短

周转时间是指从作业被提交给系统开始，到作业完成为止的这段时间间隔。它包括 4 部分时间：作业在外存后备队列上等待调度的时间，进程在就绪队列上等待进程调度的时间，

进程在 CPU 上执行的时间，以及进程等待 I/O 操作完成的时间。

对每个用户而言，都希望自己的作业周转时间尽可能短。而计算机系统的设计者和管理者使用平均周转时间和带权平均周转时间来衡量系统的时间性能。

如果系统中有 n 个作业，系统的平均周转时间 T 等于 n 个作业的周转时间之和除以 n ，其表达式如式(3-1)所示。

$$T = \frac{1}{n} \left[\sum_{i=1}^n T_i \right] \quad (3-1)$$

作业的周转时间 T 与系统为它提供的服务时间 T_s 之比为 W ， $W = T/T_s$ 被称为带权周转时间， n 个作业的平均带权周转时间表达式如式(3-2)所示。

$$W = \frac{1}{n} \left[\sum_{i=1}^n \frac{T_i}{T_s} \right] \quad (3-2)$$

服务时间 T_s 是一个作业在 CPU 上执行的总时间。

2. 响应时间快

响应时间是指从用户提交一个请求开始直至系统首次产生响应的时间为止的一段时间，它包括 3 部分时间：从输入设备（如键盘、鼠标等）输入的请求信息传送到处理机的时间、处理机对请求信息进行处理的时间，以及将所形成的响应信息回送到终端显示器的时间。对于交互式系统，响应时间是衡量系统时间性能的重要指标。进程调度算法和输入/输出设备的速度都会影响系统的响应时间。

3. 截止时间的保证

截止时间是指某个任务必须开始执行的最迟时间，或必须完成的最迟时间。截止时间是评价实时系统性能的重要指标。在实时系统中，若实时任务的开始截止时间或完成截止时间不能得到保障，实时系统的可靠性就无法保证。因为实时系统计算的正确性不仅取决于计算的逻辑结果的正确性，而且取决于得到计算结果的时间。实时系统必须采取特殊的调度策略和调度算法来满足对截止时间的要求。

4. 系统吞吐量高

吞吐量是指单位时间内完成的作业数。系统的吞吐量是评价系统性能的重要指标之一，调度算法影响系统的吞吐量。

5. 处理机利用率好

CPU 是计算机系统中影响时间性能的最重要的硬件资源，在多任务系统中，进程调度算法对 CPU 的利用率有很大影响。因此，在选择和设计进程调度算法时应该考虑使 CPU 的利用率尽可能高。

二、调度算法

1. 先来先服务调度算法 (First-Come, First-Served, FCFS)

(1) 调度算法

在进程调度中，FCFS 就是从就绪队列的队首选择最先到达就绪队列的进程，为该进程分配 CPU。

(2) 性能分析

FCFS 适合长进程，不利于短进程，短进程等待时间相对运行时间而言太长。FCFS 使短

进程的周转时间过长，系统的平均周转时间也比较长。FCFS 有利于 CPU 繁忙型进程(如科学计算)，不利于 I/O 繁忙型进程(如多数的事务处理)。考虑以下实例。

有 3 个进程 p1、p2 和 p3，分别在 0、1、2 时刻进入系统，需要的运行时间长度分别为 24、3、3，如果按 FCFS 调度算法，3 个进程的等待时间和周转时间如表 3-1 所示。

表 3-1 FCFS 算法的性能-1

进 程 名	进入系统时间	开始运行时间	服务时间	等待时间	周转时间
p1	0	0	24	0	24
p2	1	24	3	23	26
p3	2	27	3	25	28

系统的平均周转时间 $T = (24 + 26 + 28) / 3 = 26$ 。

平均带权周转时间 $W = (24/24 + 26/3 + 28/3) / 3 \approx 6.33$ 。

如果进程进入系统的顺序是 p3、p2 和 p1，3 个进程进入系统的时间分别是 0、1、2，那么 3 个进程的等待时间和周转时间则如表 3-2 所示。

表 3-2 FCFS 算法的性能-2

进 程 名	进入系统时间	开始运行时间	服务时间	等待时间	周转时间
p1	2	6	24	4	28
p2	1	3	3	2	5
p3	0	0	3	0	3

此种情况下系统的平均周转时间为 $T = (28 + 5 + 3) / 3 = 12$ 。

平均带权周转时间 $W = (28/24 + 5/3 + 3/3) / 3 \approx 1.28$ 。

对比可见，让短进程先运行能显著降低系统的平均周转时间和平均带权周转时间。

2. 短进程优先调度算法 (Shortest-Process-First, SPF)

(1) 调度算法

短进程优先 (SPF) 的调度算法是从就绪队列中选择估计运行时间最短的进程，将处理机分配给它，使它立即执行并一直执行完成，或发生某事件而被阻塞放弃处理机时，再重新调度。

(2) 算法优点

与 FCFS 算法相比，短进程优先的算法能有效降低进程的平均等待时间，提高系统的吞吐量。

(3) 算法的缺陷

- 1) 对长进程不利。如果系统中不断有短进程到来，长进程可能长时间得不到调度。
- 2) 不能保证紧迫进程的及时处理，因为该算法不考虑进程的紧迫程度。
- 3) 进程的长短根据用户的估计而定，故不一定能真正做到短进程优先。

(4) 性能分析

从表 3-1 和表 3-2 的比较中可以看出短进程优先的调度算法与先来先服务调度算法相比，能降低系统的平均周转时间和带权平均周转时间。

3. 优先权调度算法 (Priority-Scheduling Lgorithm)

(1) 调度算法

在使用优先权调度的系统中，每个进程都有一个与之关联的优先权。优先权值通常是固定区间的数字，如 0 ~ 127 中的一个数。系统可能设计为优先权值越大，优先权越高；也可能设计为优先权值越小，优先权越低。优先权可以通过内部或者外部方式来定义。内部定义优先权可使用一些可测量数据以计算进程的优先权值。例如，时间极限、内存要求、打开文件的数量、平均 I/O 服务时间与平均 CPU 服务时间之比都可以用于计算优先权。外部优先权是通过操作系统之外的准则来设置的，如进程的重要性、用于支付使用计算机的费用和数量等。

当使用优先权调度算法进行进程调度时，系统将 CPU 分配给就绪队列中优先权值最高的进程。

(2) 优先权调度算法的类型

用于进程调度的优先权调度算法可以分为下列两种类型。

1) 非抢占式 (Nonpreemptive) 优先权调度算法。

在支持非抢占式调度的系统中，高优先权进程一旦得到处理机，则该进程便一直运行下去，直到完成或由于某事件使该进程主动放弃处理机。在进程运行期间，即使有高优先权进程到来，系统也不能剥夺当前进程（即正在运行的进程）的 CPU 使用权。高优先权进程到来时如果有别的进程正在运行，高优先权进程只能先进入就绪队列。在对截止时间要求严格的实时系统中，非抢占式调度难以保证高优先权进程得到及时调度。

2) 抢占式 (Preemptive) 优先权调度算法。

在支持抢占式调度的系统中，如果新到达进程的优先权高于当前正在运行进程的优先权，那么系统会抢占 CPU，把它分配给新到达的高优先权进程，而正在执行的低优先权进程暂停执行。在抢占式调度算法中，每当系统中出现一个新的就绪进程，就将其优先权与正在执行的进程的优先权相比较，如果其优先权大于正在执行的进程的优先权，可采用立即抢占策略，立刻进行进程切换，使其投入运行。也可以采用基于时钟中断的抢占策略，在最近一次时钟中断到来时进行进程切换。

(3) 优先权的类型

如何确定进程的优先权是实现优先权调度算法的一个关键问题，优先权的计算方法对优先权调度算法的性能有很大的影响。根据优先权的变化特点可以将优先权的类型分为静态优先权和动态优先权。

1) 静态优先权。静态优先权在创建时确定，在进程的整个运行期间保持不变。静态优先权值通常可以根据进程的类型、进程需要的资源数量和用户的要求来设定。

2) 动态优先权。进程创建时被赋予的优先权，随进程的推进或随其等待时间的增加而改变。动态优先权调度算法可以使系统获得更好的调度性能。

(4) 优先权调度算法存在的问题和解决方案

1) 问题。优先权调度算法的一个主要问题是无穷阻塞，或称饥饿 (Starving) 问题。这里的阻塞是指就绪态进程因得不到 CPU 而等待的状态。优先权调度算法会使某个低优先权进程无穷等待 CPU，在一个重负载的计算机系统中，平稳的高优先权进程流可以阻止低优先权进程获得 CPU。据说，MIT 在 1973 年关闭 IBM 7094 时，发现有一个低优先权进程是于

1967 年提交但一直未运行的。

2) 解决方案。低优先权进程无穷等待问题的解决方案之一是老化(Aging)技术。老化技术以逐渐增加在系统中等待时间很长的进程的优先权,使低优先权进程在等待时间很长的情况下,优先权变高而获得 CPU 执行。

4. 时间片轮转调度算法(Round-Robin, RR)

时间片轮转调度算法是在现代分时系统中广泛使用的进程调度算法,UNIX、Linux 和 Windows 操作系统都采用基于时间片轮转、支持优先权和抢占式调度的混合式进程调度算法。

(1) 时间片轮转调度算法

在早期的时间片轮转调度算法中,系统将所有的就绪进程按先来先服务的原则,排成一个队列,每次调度时把 CPU 分配给队首进程,并令其执行一个时间片。当时间片用完时,调度程序终止当前进程的执行,并将它送到就绪队列的队尾。时间片是一个较小的时间单元,通常为 10 ~ 100 ms。Linux 2.4 内核给用户进程分配的时间片大小一般为 50 ms。

在采用时间片轮转调度算法的系统中,进程需要在 CPU 上运行的时间总数(这里称之为时间区间)可能小于一个时间片,也可能大于一个时间片。对于进程的时间区间小于一个时间片的情况,进程在 CPU 上运行结束后,进程本身会自动释放 CPU。然后,由操作系统执行进程调度程序为另一个就绪进程分配 CPU。对于进程的时间区间大于一个时间片的情况,进程可能需要执行若干个时间片。每当进程在 CPU 上连续运行的时间等于一个时间片长度时,操作系统在时钟中断处理过程中会抢占 CPU,进行进程切换,用新的就绪进程替代当前进程,被替换的当前进程重新回到就绪队列中。

(2) 时间片大小的确定

在设计时间片轮转调度算法时,时间片大小的确定是一个重要的环节。时间片太长,可以使多数进程在一个时间片内处理完,能够降低进程的周转时间,但是可能造成交互用户的响应时间过长。时间片太短,一个进程要经过多次调度运行才能执行完毕,进程切换和进程调度的开销会增加,系统的平均周转时间也会比较长。例如,假定进程切换需要 10 ms,如果时间片也是 10 ms,那么,CPU 至少把 50% 的时间花费在进程切换上。因此,要根据用户和系统的需要选择大小合适的时间片。选择时间片时既要考虑交互用户的需求,也要平衡系统的效率。在为调度程序确定时间片的大小时,通常要考虑以下几个因素。

1) 系统对响应时间的要求。响应时间是分时系统性能的重要指标,设计时间片大小时首先要考虑其对响应时间的影响和系统对响应时间的要求。系统响应时间为 T ,进程数目为 N ,时间片为 q ,有 $T = Nq$,也就是说响应时间与进程数和时间片成比例。因此在系统允许的最大进程数一定的情况下,时间片的长短取决于系统要求的响应时间。响应时间越短,时间片取值应该越小。

2) 就绪队列中进程的数目。在分时系统中,系统允许的最大进程数往往是给定的,系统中的进程数会影响系统的响应时间,进程越多,响应时间越长。但是,操作系统的设计者希望系统的响应时间有一个上限值,不能出现因为进程多而使系统响应时间很长的情况。当设定了系统的最长响应时间值后,时间片的大小就与系统允许的最大进程数成反比。

3) 系统的处理能力。系统的处理能力是必须保证系统支持的基本命令能在一个时间片内执行完,否则将无法保证得到满意的响应时间,而且会使平均周转时间及带权周转时间都

很长。

(3) 时间片轮转调度算法的性能评价

时间片轮转调度算法的性能很大程度上依赖于时间片的大小。在极端情况下，如果时间片很大，那么时间片轮转调度算法就与先来先服务算法一样(如果就绪队列按先进先出对进程排序)。如果时间片很小，进程需要经过多次上下文切换和进程调度，会大大增加 CPU 用于进程切换和进程调度的开销。

5. 多级队列调度

在现代操作系统中，进程可以根据不同的特点进行分类，每一类进程属于一个就绪队列。因此，系统中的就绪队列往往不止一个，而是多个，这种系统通常根据不同进程对响应时间要求的不同，采用多级队列调度算法。

(1) 多级队列调度算法(Multilevel Queue-Scheduling Algorithm)

将就绪队列分成多个独立队列，根据进程的某些属性，如需要占用的内存大小、进程优先权或进程类型，进程会被永久地分配到一个队列。每个队列有自己的调度算法。不同的队列优先权不同，调度算法也可能不同。

(2) 多级队列调度算法应用举例

多级队列调度举例：Minix 操作系统的进程调度。

Minix 把进程按类型不同分为 3 类，分别对应 3 个优先权不同的就绪队列：优先权最高的任务进程队列、优先权次之的服务进程队列和优先权最低的用户进程队列。对任务进程和服务进程采用基于优先权的非抢占式调度，优先权高的进程一旦获得处理机将一直运行下去，直到阻塞。对优先权最低的用户进程采用时间片轮转的抢占式调度。

调度程序按如图 3-1 所示的流程完成进程调度。

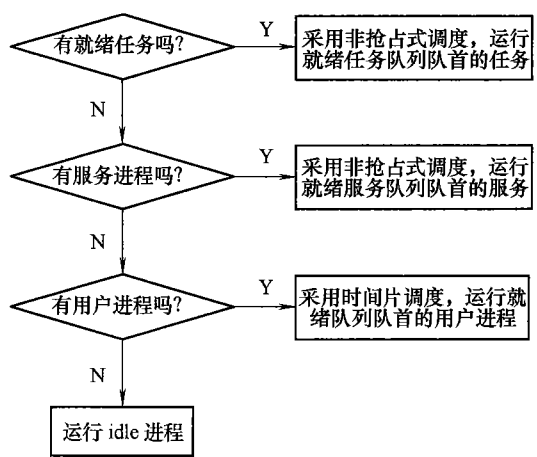


图 3-1 Minix 系统的多级队列调度

6. 多级反馈队列调度

采用多级队列调度，一旦进程进入系统，就被固定地分配到一个就绪队列中，进程在被撤销前不会在不同队列之间移动。这种算法的优点是降低了进程调度的开销，但是不够灵活，对低优先权进程会存在无穷阻塞(饥饿)问题。而多级反馈队列调度算法(Multilevel Feedback Queue Scheduling)可以弥补这些不足。

在采用多级反馈队列调度的系统中建立多个优先权不同的就绪队列，为每个队列赋予大小不同的时间片。有一种反馈策略可以规定：队列优先权越高，时间片越短，时间片通常成倍增长。新进程被创建后，先插入优先权最高的队列。仅当高优先权队列空时，才调度优先权次之的队列。同一队列中，采用时间片轮转调度算法。使用 CPU 时间过多的进程会被移到优先权较低的队列中，在较低优先权队列中等待时间过长的进程会被转移到较高的优先权队列中。这样就通过使用老化技术阻止了饥饿的发生。但是，不同的操作系统在设计多级反馈队列调度时采取的策略可能是不同的。多级反馈队列算法的设计要考虑以下几个方面的

问题。

- 1) 就绪队列的数量。
- 2) 根据进程优先权确定进程应该进入哪个就绪队列的算法。
- 3) 用以确定进程何时转移到较高优先权队列的算法。
- 4) 用以确定进程何时转移到较低优先权队列的算法。
- 5) 用以确定进程在需要服务时应该进入哪个队列的算法。

多级反馈队列调度算法的一个典型实例是 Linux 2.6.11 内核的进程调度算法。

第三节 实时系统中的调度

在第一章介绍了实时系统的特点，实时系统对处理器操作或者数据流动有着严格的时间限制，实时系统的进程调度对保证时间的要求具有重要作用。

一、实现实时调度的基本条件

为了满足实时系统对截止时间的要求，实现实时调度应具备下列几个条件。

1. 提供必要的调度信息

为了实现实时调度，系统可能需要为调度程序提供下列一些信息。

- 1) 就绪时间。是一个实时任务成为就绪态的起始时间。
- 2) 开始截止时间和完成截止时间。
- 3) 处理时间。指一个实时任务从开始执行直至完成所需要的时间。
- 4) 资源要求。关于任务执行所需要的资源信息。

5) 优先级。根据实时任务紧迫程度的不同，可以给实时任务赋予不同的优先权，使高优先权任务能优先获得系统资源，尽快得到执行。系统中实时进程的优先权不能动态降低。

2. 系统处理能力强

在实时系统中，通常有多个实时进程（通常也称其为实时任务），若处理机的处理能力不够强，处理速度不够快，从而使某些实时进程不能得到及时处理，可能发生难以预料的后果。

(1) 单处理机情况下必须满足的限制条件

假定系统中有 m 个周期性的硬实时进程，它们的处理时间可表示为 C_i ，周期时间表示为 P_i ，则在单处理机情况下，必须满足如式(3-3)所示的限制条件。

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1 (1 \leq i \leq m) \quad (3-3)$$

例如，有 6 个实时进程，它们的周期时间都是 50 ms，也就是说每个进程都必须每隔 50 ms 完成一次运行。处理机的处理能力为处理一个进程的时间是 10 ms。所以该系统是无法保证所有的 6 个实时进程都能在截止时间内完成的，这种情况称为系统不可调度。要保证该系统是可调度的，可以提高处理机的处理能力以缩短每个实时进程的处理时间，也可以增加处理机数量。

(2) n 个处理机情况下必须满足的限制条件

采用多处理机系统可以提高实时系统的处理能力。若系统的处理机个数为 n ，处理能力

的限制条件改变如式(3-4)所示。

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq n (1 \leq i \leq m) \quad (3-4)$$

操作系统在提高处理能力方面是可以有所作为的,比如,设计适合特定实时应用的进程调度算法,规定实时进程具有最高的优先权,以及使调度程序和进程切换的延迟时间尽可能短。

3. 采用抢占式调度机制

在实时系统中广泛采用抢占式调度。当一个优先权更高的进程到达时,允许将当前进程暂时挂起,而令高优先权进程立即投入运行。这样可以满足实时进程对截止时间的要求。

为了使调度过程更简单,调度开销更小,对于支持需要服务时间很短的小实时进程的系统,也可以采用非抢占式调度。

抢占式调度算法根据抢占 CPU 的时机不同,可以分为基于时钟中断的抢占和立即抢占。

(1) 基于时钟中断的抢占式优先权调度算法

在高优先权的实时进程到达后,虽然该进程的优先权大于正在执行的进程,系统也并不立即抢占当前进程的处理机,而是等到最近一次时钟中断到来时,系统才剥夺当前进程的 CPU,将 CPU 分配给新到来的优先权更高的实时进程。

(2) 立即抢占的优先权调度算法

在这种调度策略中,一旦接收到触发实时进程运行的信号,这通常是一个外部中断信号,系统立即剥夺当前进程的 CPU,把它分配给请求中断的新的实时进程。这种算法能获得比基于时钟中断的抢占式优先算法更快的响应速度。

4. 具有快速切换机制

为保证对截止时间要求较高的实时进程能及时运行,在实时系统中还应具有快速切换机制,以保证进程的快速切换。该机制应具有以下两个方面的能力。

(1) 对外部中断的快速响应能力

为使在紧迫的外部事件请求中断时系统能及时响应,要求系统具有快速的硬件中断机构,还应使禁止中断的时间间隔尽可能短。

(2) 快速的进程切换能力

在完成进程调度后进行进程切换。为了提高进程切换时的速度,应使系统中的每个运行功能单位适当地小,以减少进程切换的时间开销。

二、常用的几种实时调度算法

1. 最早截止时间优先 EDF(Earliest Deadline First, EDF) 算法

该算法是根据进程的开始截止时间确定进程的优先级。截止时间越早,进程的优先级越高,越优先获得处理机。该算法要求在系统中保持一个实时进程的就绪队列,该队列按各进程截止时间的早晚排序,具有最早截止时间的进程排在队列的最前面。调度程序在选择进程时,总是选择就绪队列中的第一个进程,为之分配处理机,使之投入运行。最早截止时间优先的算法既可用于抢占式调度,也可用于非抢占式调度。

2. 最低松弛度优先 LLF(Least Laxity First, LLF) 算法

松弛度用来表示一个实时进程的紧迫程度。如果一个进程的完成截止时间为 T,当前时

间为 T_c ，处理完该任务还需要的时间为 T_s ，则松弛度 L 的计算式表示为

$$L = T - T_c - T_s$$

在使用最低松弛度优先算法时，调度程序在调度时机到来时，每次选择松弛度 L 最小的进程，把 CPU 分配给该进程。该算法在实现时，把进程按松弛度排序，让松弛度最小的进程处于就绪队列队首，这样调度程序执行时只需选择就绪队列队首的进程执行即可。

下面是一个采用最低松弛度优先调度的实例。

假如在一个实时系统中，有两个周期性实时进程 A 和 B，要求进程 A 每 20 ms 执行一次，执行时间为 10 ms。要求进程 B 每 50 ms 执行一次，执行时间为 25 ms。由此可知进程 A 和 B 每次必须完成的时间分别为：A1、A2、A3……和 B1、B2、B3……，如图 3-2 所示。

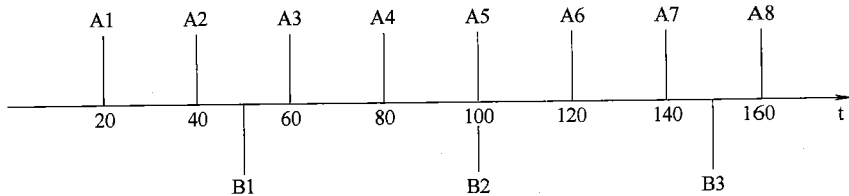


图 3-2 任务 A 和任务 B 每个周期必须完成的时间

对进程 A 和进程 B 采用最低松弛度优先算法，保证两个进程在每个周期都能执行一次。

采用最低松弛度优先算法的调度时机和调度过程示例如图 3-3 所示。图 3-3 中 A1、A2、A3、A4 分别表示进程 A 在第一、二、三、四个周期内的一次连续执行时间，B1、B2 分别表示进程 B 在第一、二个周期内的一次连续执行时间。

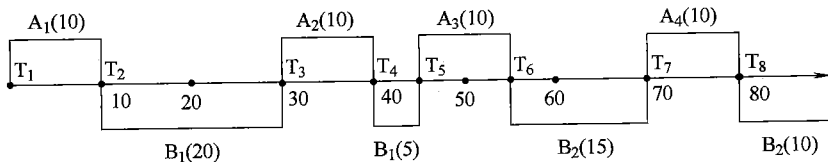


图 3-3 采用 LLF 调度任务 A 和任务 B 的情况

相应的调度过程说明如下。

在 $T_1 = 0$ 的时刻， A_1 必须在 20 ms 时完成，而它本身运行需要 10 ms，可计算出 T_1 时刻进程 A 的松弛度 $L(a)$ 为 10 ms。 B_1 必须在 50 ms 时完成，而它本身运行需要 25 ms，可计算出 B 在 T_1 时刻的松弛度 $L(b)$ 为 25 ms，所以在 T_1 时刻应该调度进程 A 先执行。

在 $T_2 = 10$ ms 的时刻，根据公式 $L = T - T_c - T_s$ ，进程 A 的松弛度 = 任务的完成截止时间 - 当前时刻 - 处理完该进程还需要的时间。A 的松弛度 $L(a) = 40 \text{ ms} - 10 \text{ ms} - 10 \text{ ms} = 20 \text{ ms}$

同样，计算进程 B 的松弛度为 15 ms，所以在 T_2 时刻应该调度进程 B 运行。

在 $T_3 = 30$ ms 时，进程 A 在第二周期的松弛度降为 0，而 B_1 的松弛度为 15 ms。此时，调度程序抢占 CPU，让进程 A 运行。

在 $T_4 = 40$ ms 时，进程 A 的松弛度为 10 ms，而进程 B 的松弛度为 5 ms。因此，重新调度进程 B 执行。

在 $T_5 = 45$ ms 时, 进程 B 完成其在第一周期的执行, 其在第二周期的松弛度为 30 ms, 而此时 A 的松弛度为 5 ms, 故调度进程 A 执行。

在 $T_6 = 55$ ms 时, 进程 A 的松弛度为 15 ms, 进程 B 的松弛度为 20 ms。虽然进程 A 的松弛度小于进程 B 的松弛度, 但进程 A 还没有进入新的执行周期, 而 B 已经进入自己的新的执行周期, 所以此时调度进程 B 执行。

在 $T_7 = 70$ ms 时, 进程 A 在其第四周期的松弛度已经为 0 ms, 进程 B 的松弛度为 20 ms, 此时系统再次抢占 CPU, 转而执行进程 A。

下面是在 T_1 、 T_2 、 T_3 、 T_4 、 T_5 、 T_6 、 T_7 、 T_8 时刻两个进程的松弛度计算结果和调度过程。 $L(A)$ 、 $L(B)$ 分别表示任务 A 和任务 B 的松弛度。

$$T_1 = 0$$

$$L(A) = 20 - T_1 - 10 = 10$$

$$L(B) = 50 - T_1 - 25 = 50 - 0 - 25 = 25$$

$$L(A) < L(B) \text{ 调度进程 A 执行 } 10 \text{ ms}$$

$$T_2 = 10 \text{ ms}$$

$$L(A) = 40 - T_2 - 10 = 20 \text{ ms}$$

$$L(B) = 50 - T_2 - 25 = 50 - 10 - 25 = 15$$

$$L(B) < L(A) \text{ 调度进程 B 执行 } 20 \text{ ms}$$

$$T_3 = 30 \text{ ms}$$

$$L(A) = 40 - T_3 - 10 = 40 - 30 - 10 = 0$$

$$L(B) = 50 - T_3 - (25 - 20) = 50 - 30 - 5 = 15$$

$$L(A) < L(b) \text{ 调度进程 A 执行 } 10 \text{ ms}$$

$$T_4 = 40 \text{ ms}$$

$$L(A) = 60 - T_4 - 10 = 60 - 40 - 10 = 10$$

$$L(B) = 50 - T_4 - (25 - 20) = 50 - 40 - 5 = 5$$

$$L(B) < L(A) \text{ 调度进程 B 执行 } 5 \text{ ms}$$

$$T_5 = 45 \text{ ms}$$

$$L(A) = 60 - T_5 - 10 = 60 - 45 - 10 = 5$$

$$L(B) = 100 - T_5 - 25 = 100 - 45 - 25 = 30$$

$$L(A) < L(B) \text{ 调度进程 A 执行 } 10 \text{ ms}$$

$$T_6 = 55 \text{ ms}$$

$$L(A) = 80 - T_6 - 10 = 80 - 55 - 10 = 15$$

$$L(B) = 100 - T_6 - 25 = 100 - 55 - 25 = 20$$

因为 A 进程还没有进入新的执行周期, 所以虽然此时 $L(a) < L(b)$, 但是调度进程 B 执行 15 ms

$$T_7 = 70 \text{ ms}$$

$$L(A) = 80 - T_7 - 10 = 80 - 70 - 10 = 0$$

$$L(B) = 100 - T_7 - (25 - 15) = 100 - 70 - (25 - 15) = 20 \quad (\text{执行进程的松弛度不变})$$

$$L(A) < L(b) \text{ 调度进程 A 执行 } 10 \text{ ms}$$

$$T_8 = 80 \text{ ms}$$

$$L(A) = 100 - T_8 - 10 = 100 - 80 - 10 = 10$$

$$L(B) = 100 - T_8 - (25 - 15) = 100 - 80 - (25 - 15) = 10$$

当两个进程的松弛度相同时，应该调度哪个进程，就需要其他条件的支持，比如进程的优先权、截止时间等。

第四节 进 程 切 换

当进程调度程序选择到一个新的进程后，要进行进程切换，用新选择的进程替换原来的执行进程，也就是把 CPU 的控制权交给由调度程序所选择的进程。

进程切换使当前正在执行的进程成为被替换进程，出让其所使用的 CPU，以运行被进程调度程序选中的新进程。进程切换通常包括以下几个步骤。

- 1) 保存包括程序计数器和其他寄存器在内的 CPU 上下文环境。
- 2) 更新被替换进程的进程控制块。
- 3) 修改进程状态，把执行态改为就绪态或者阻塞态。
- 4) 将被替换进程的进程控制块移到就绪队列或阻塞队列。
- 5) 执行通过进程调度程序选择的新进程，并更新该进程的进程控制块。
- 6) 更新内存管理的数据结构。
- 7) 恢复被调度程序选中的进程的硬件上下文。

第五节 多处理器调度

随着对计算机性能的要求，尤其是对计算速度、处理能力的要求越来越高，多处理器系统得到更加广泛的应用，各类大、中、小型主机系统，网络服务器，以及高端的工作站都采用多处理器结构，甚至个人微机都使用双 CPU。操作系统与计算机体系结构有着天然的联系，面对多处理器系统，必须解决多处理器的进程调度问题。

一、多处理器系统 (MultiProcessor Systems, MPS) 的类型

对处理器系统有多种不同的分类方式，根据处理器的耦合程度，可以把多处理器系统分为紧密耦合多处理器系统和松弛耦合多处理器系统；根据处理器结构是否相同，可以把多处理器系统分为对称多处理器系统和非对称多处理器系统。

1. 紧密耦合的多处理器系统和松弛耦合的多处理器系统

(1) 紧密耦合的多处理器系统

紧密耦合的多处理器系统通常通过高速总线或高速交叉开关实现多个处理器之间的互连，它们共享主存储器系统和 I/O 设备，并要求将主存储器划分为若干个独立访问的存储器模块，以便多个处理器能同时对主存进行访问。系统中的所有资源和进程都由操作系统实施统一的控制和管理。

(2) 松弛耦合的多处理器系统

松弛耦合的多处理器系统通常通过通道或通信线路来实现多台计算机之间的互连。每台计算机都有自己的存储器和 I/O 设备，并配置了操作系统来管理本地资源和在本地运行的进

程。因此，每一台计算机都能独立工作，必要时可通过通信线路与其他计算机交换信息，以及协调它们之间的工作。

2. 对称多处理器系统和非对称多处理器系统

对称多处理器系统属于同构的多处理器系统，其中所包含的各处理单元，在功能和结构上都是相同的，当前的绝大多数多处理器系统都是对称多处理器系统。

非对称多处理器系统中有多种类型的处理单元，它们的功能和结构各不相同。其中只有一个主处理器，有多个从处理器。

二、多处理器系统中的进程分配方式

1. 对称多处理器系统中的进程分配方式

在对称多处理器系统中，进程到处理器的分配可以采用以下两种方式。

(1) 静态分配

在采用这种分配方式时，操作系统为每个处理器建立一个专门的就绪队列，该就绪队列的每个进程都只能在与就绪队列对应的处理器上运行。一个进程无论经过多少次调度，操作系统都把同一个处理器分配给该进程。静态分配方式的优点是进程调度的开销小，缺点是不能动态地平衡各处理器的负载，使系统存在各处理器忙闲不均的情况。

(2) 动态分配

动态分配的基本特征就是每个进程经过多次调度，每次获得的不一定是同一个处理器。为实现动态分配，可以在系统中设置一个公共的就绪队列，系统中的所有就绪进程都被放在该队列中。可将进程分配到任何一个处理器上，这样，对一个进程的整个运行过程而言，在每次被调度执行时，都是随机地被分配到当时处于空闲状态的某一处理器上去执行。例如，某进程一开始被分配到处理器 A 上执行，后来因为阻塞或时间片用完而暂停执行，当再次回到公共就绪队列中，重新被调度时，该进程可能被分配到 B 处理器或 C 处理器上运行。动态分配的优点是可以在每次调度时考虑处理器的负载平衡问题，总是把进程分配给当前空闲的处理器。此外，对于共享存储器的紧密耦合系统，所有的处理器可以共享保存在内存中的进程信息。

2. 非对称多处理器系统(MPS)中的进程分配方式

对于非对称多处理器系统，大多采用主—从式操作系统，即操作系统的核心部分驻留在一台主机上，而从机上只运行用户程序，只有主机执行调度程序，所有从机的进程都是由主机分配的。每当从机空闲时，便向主机发送一个请求分配进程的信号，然后便等待主机为它分配进程。在主机中保持有一个就绪队列，只要就绪队列不为空，主机便从其队首摘下一个进程分配给请求分配进程的从机。从机接收到分配的进程后便运行该进程，该进程结束后从机又向主机发出请求。

在非对称多处理系统中，主、从式的进程分配方式的主要优点是系统处理比较简单。这是因为所有的进程分配都由一台主机独自处理，使进程间的同步问题得以简化，并且进程调度程序也很易于从单处理器的进程调度程序演化而来。但由一台主机控制一切，也存在着不可靠性，主机一旦出现故障，会导致整个系统瘫痪，而且也容易因主机太忙而形成系统瓶颈。克服这些缺点的有效方法是利用多台处理器来管理整个系统。这样，当其中一台处理器出现故障时，可由其他处理器来接替其完成任务，不影响系统正常运行。而且用多台处理器

还具有更强的执行管理任务的能力，也不容易形成系统瓶颈。

三、进程(线程)调度方式

多处理器的调度比单处理器调度机制要复杂，下面简要介绍几种比较典型的多处理器调度方式。

1. 自调度

自调度算法是当前多处理器系统中最常用的调度方式之一，也是最简单的一种调度方式。采用自调度的系统中设置有一个公共的就绪队列，任何一个空闲的处理器都可以自行从该就绪队列中选取一个进程或者一个线程运行。在自调度方式中，可采用在单处理器环境下所用的调度算法，如先来先服务(First-Come, First-Service, FCFS)调度算法、最高优先权调度算法等。在多处理器环境下，FCFS 是一种较好的自调度算法，算法简单，开销小。

(1) 自调度算法的优点

1) 易移植。在采用自调度方式的系统中，公共就绪队列可按照单处理器系统中所采用的各种组织方式加以组织，其调度算法也可沿用单处理器系统所用的算法，因此，很容易将单处理器环境下的调度机制移植到多处理器系统中。

2) 有利于提高 CPU 的利用率。只要系统中有任务，或者说只要公共就绪队列不为空，就不会出现处理器空闲的情况，也不会发生处理器忙闲不均的现象，因而有利于提高处理器的利用率。

(2) 自调度方式的缺点

1) 瓶颈问题。采用自调度，在整个系统中只有一个必须互斥访问的公共就绪队列。在系统中有多处理器情况下，对公共就绪队列的访问很容易形成瓶颈。当系统中处理器的数目在数十个，甚至数百个时，采用自调度算法瓶颈问题会非常严重。

2) 低效性。采用自调度算法，CPU 上的高速缓存的命中率较低。当进程阻塞后再重新就绪时，它只能进入唯一的公共就绪队列，经过进程调度后，不一定能在阻塞前的处理器上运行。如果在每台处理器上都配有高速缓存，则这时在其中保留的该进程的数据已经失效，而在该进程新获得的处理器上又需要重新建立这些数据的备份。由于一个进程的整个生命期中，可能要多次更换处理器，因而使高速缓存的命中率很低。

3) 线程切换频繁。在多线程系统中，通常一个应用中的多个线程是相互合作的关系。而采用自调度时，相互合作的线程很难同时获得处理器运行，这将会使某些线程因其合作线程未获得处理器运行而阻塞，进而被切换下来。

2. 成组调度

成组调度方式是由系统将一组相互合作的进程或线程同时分配到一组处理器上运行，进程或线程与处理器一一对应。

(1) 成组调度的优点

1) 减少线程切换，改善系统性能。如果一组相互合作的线程或进程能并行执行，则能有效减少进程(线程)阻塞情况的发生，从而减少线程的切换，使系统性能得到改善。

2) 减少调度开销。因为每次调度都可以解决一组线程的处理器分配问题，因而可以显著减少调度频率，从而减少调度开销。

(2) 成组调度中的时间分配

在成组调度中可以采用两种方式为应用程序分配处理器时间：一是面向所有的应用程序平均分配处理器时间；二是面向所有的线程平均分配处理器时间。

3. 专用处理器分配

1989年由Tucker提出了专用处理器分配方式。该方式是在一个应用程序执行期间，专门为该应用程序分配一组处理器，每个线程一个，这组处理器供该应用程序专用，直至应用程序完成。这种方式会造成处理器资源的严重浪费。因为当一个处理器上的线程被阻塞时，处理器不能分配给别的线程使用，只能空等。例如，有一个线程为了与另一个线程同步而被阻塞起来，则该线程所分配到的处理器就会空闲。

之所以可以将这种专用处理器调度方式用于并发程度高得多的处理器环境，是因为在具有数十个乃至上百个处理器的并行系统中，每个处理器的投资费用在整个系统中仅占很小的比重。对整个系统的性能和效率而言，单个处理器的使用效率并不像在单处理器系统中那么重要。其次，在一个应用程序的整个运行过程中，由于每个进程或线程专用一台处理器，因此可以完全避免进程或线程的切换，从而可以大大加速程序的完成。

总的来说，专用处理器的优点一是加速了应用程序的运行速度，二是避免了进程切换。

第六节 死 锁

一、产生死锁的原因和必要条件

在多道程序系统中，多个进程可能竞争数量有限的资源。如果一个进程所申请的资源被其他处于阻塞状态的进程占有，该进程就会因为不能获得所申请的资源而被阻塞。若此时该进程恰好又占有了前述其他进程所需要的资源，那么这一组进程就可能因为等待释放自己所需要但被其他进程已占有的资源而无法向前推进。这种由于多个进程竞争共享资源而引起的进程不能向前推进的僵死状态称为死锁。

1. 产生死锁的原因

进程访问资源是通过执行程序实现的。一般来说，进程按照申请资源、访问资源和释放资源的顺序使用资源。

产生死锁的原因为：竞争共享资源且分配资源的顺序不当。

2. 产生死锁的必要条件

死锁产生时，必须同时满足下列4个条件。

(1) 互斥条件

指一个进程在访问资源的过程中，其他进程不能访问该资源。如果一个资源正在被访问时，有其他进程也提出对该资源的访问请求，必须把请求该资源的进程阻塞起来，直到资源被进程释放。

(2) 请求和保持条件

进程已经保持了至少一个资源，又提出了新的资源要求，而新请求的资源已经被其他进程占有，此时进程阻塞，但又对已经获得的资源保持不放，使得其他进程无法使用被保持的

资源。举例说明：进程 p1 获得了资源 A 的访问权，然后又请求资源 B。但是资源 B 因为被别的进程占用而无法分配给进程 p1。通常，操作系统会把 p1 阻塞起来，插入资源 B 的等待队列中。p1 被阻塞却没有释放对资源 A 的访问权。这时就出现了 p1 请求资源 B 而保持资源 A 的条件。

(3) 不剥夺条件

进程已经获得的资源不能被剥夺，只能由进程自己释放。例如，进程 p1 获得了资源 A 的访问权，进程 p2 再申请访问资源 A，即使进程 p1 因为某种原因被阻塞，暂时不需要使用资源 A，进程 p2 也不能通过系统剥夺资源 A 的访问权，只能等待进程 p1 释放对资源 A 的访问权。

(4) 环路等待条件

在发生死锁时，必然存在一个进程申请资源的环形链。即进程集合 {p0, p1, p2..., pn} 中的 p0 正在等待一个 p1 占用的资源；p1 正在等待 p2 占用的资源，..., pn 正在等待已经被 p0 占用的资源。

注意：只有当上述 4 个条件同时满足时才会发生死锁。

3. 实例

下面以生产者和消费者问题为例，说明产生死锁的必要条件，如图 3-4 所示。

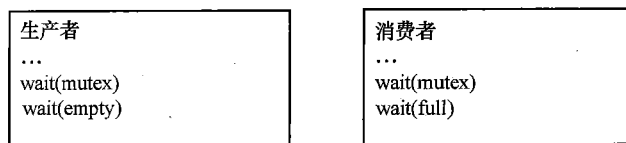


图 3-4 可能导致生产者-消费者进程死锁的同步操作

如果生产者和消费者进程都先通过执行 wait(mutex) 申请公共缓冲池的互斥访问权，当缓冲池满且生产者和消费者进程几乎同时访问公共缓冲池时，可能发生死锁，出现 4 个必要条件同时满足的情况。

1) 互斥条件。若生产者进程先占用公共缓冲池访问权，消费者请求缓冲池访问权时会被阻塞。即缓冲池访问权只能被任意两个进程中的一个所占有。所有的生产者进程和消费者进程对公共缓冲池的访问是互斥的。

2) 请求和保持条件。生产者进程获得缓冲池使用权后，又提出申请空缓冲区的请求。因为缓冲池装满了消息，没有空缓冲区。因此，生产者进程无法获得空缓冲区资源而被阻塞，但生产者进程仍保持缓冲池的访问权。

3) 不剥夺条件。虽然缓冲池中有消费者进程需要的消息资源，只要消费者进程能够获得缓冲池的互斥访问权就可以执行取消息的操作，但消费者进程不能剥夺生产者已获得的缓冲池访问权。

4) 环路等待。存在生产进程等待消费者进程释放空缓冲区，消费者进程等待生产者进程释放公共缓冲池互斥访问权的资源申请环路，如图 3-5 所示。

二、处理死锁的基本方法

处理死锁的基本方法有预防死锁、避免死锁、检测并解除死锁和忽略死锁问题(即假定

死锁不可能在系统内发生而忽略死锁)。

为确保不发生死锁,操作系统可以采用死锁预防或死锁避免方案。

1. 死锁的预防

死锁预防是根据前面讨论的死锁必要条件,通过保证至少其中一个条件不成立来达到预防发生死锁的目的。

由于有些共享资源必须被定义为临界资源,对于这些资源的访问必须是互斥的。因此,对这类资源的访问不能通过摒弃互斥条件来预防死锁的发生,如对打印机的访问。但是,有些共享资源是允许进程同时访问的,如允许进程同时读一个共享文件。如果进程访问的都是这些资源,是不会出现死锁问题的。在操作系统中无法预知进程是否一定不访问临界资源,所以通常不能采用摒弃互斥条件来预防死锁的发生。

预防死锁可以通过摒弃下列三个必要条件之一来实现。

(1) 摒弃请求和保持条件

可以通过摒弃请求和保持条件来预防死锁的发生。摒弃请求和保持条件的一种方法是系统要求所有进程执行前要一次性地申请在整个运行过程中所需要的全部资源,只要有一个资源申请不成功,其他所有资源也不分配给该进程,并阻塞该进程。例如生产者—消费者问题中,生产者进程执行前必须申请到公共缓冲池的互斥访问权和空缓冲区资源,只要有一种资源申请不到,就阻塞生产者进程。

还有一种方法是对某些进程在申请其他资源前要求该进程必须释放已经分配给它的所有其他资源。例如,一个进程要从磁盘读数据送打印机,可以规定,必须在进程释放磁盘驱动器之后,才能申请并获得打印机。

(2) 摒弃不剥夺条件

摒弃不剥夺条件的方法是一个已保持了某些资源的进程,当它再提出新的资源要求而不能立即得到满足时,必须释放它已经保持的所有资源。具体实现时可以采取这样的策略:当一个进程 p_1 申请某资源 R_1 时,如果 R_1 可用,就将 R_1 分配给 p_1 。如果 R_1 不可用,系统就检查是否有其他进程占用了 R_1 ,而又申请其他资源不成功而被阻塞。如果有这样的阻塞进程,系统抢占被该进程占用的 R_1 资源分配给 p_1 进程。这种方法的缺点是实现复杂而且代价高。

(3) 摒弃环路等待条件

摒弃环路等待的方法是指进程必须按规定的顺序申请资源。对所有不同类型的资源排序,要求每个进程按规定的顺序申请资源。

打印机和磁盘驱动器是两种不同类型的资源,一个系统中可能存在多个打印机和多个磁盘驱动器。把某台打印机称为打印机这种类型资源的一个实例,把某个磁盘驱动器称为磁盘驱动器类型资源的一个实例。

设 $R = \{R_1, R_2, \dots, R_n\}$ 为资源类型的集合。为每个资源类型分配一个唯一的整数,以比较资源的先后顺序。可定义一个函数 $F: R \rightarrow N$, 其中 N 是自然数集合。例如, R 的

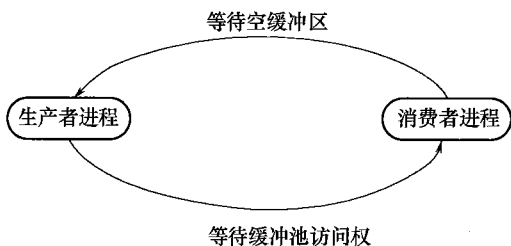


图 3-5 生产者—消费者问题中的环路等待

集合包括打印机和磁盘驱动器，那么函数 F 可以按以下来定义。

$$F(\text{disk drive}) = 1$$

$$F(\text{printer}) = 2$$

用这样的方法来摒弃环路等待的策略是：每个进程按资源排序的递增顺序申请资源。即一个进程开始可申请任何数量的资源类型 R_i 的实例。之后，当且仅当 $F(R_j) > F(R_i)$ 时，该进程可以申请资源类型为 R_j 的实例。例如，对于以上给定的函数，一个进程需要同时使用磁盘驱动器和打印机，那么就必须先申请磁盘驱动器，后申请打印机，因为 $F(\text{printer}) > F(\text{disk drive})$ 。换言之，当一个进程申请资源类型 R_j 的实例时，它必须先释放所有资源 R_i ，其中 R_i 使 $F(R_i) \leq F(R_j)$ 满足。如果需要同一资源类型的多个实例，那么同时申请这多个实例。这种方法的缺点如下。

- 1) 限制了新设备的增加。
- 2) 系统为资源分配的序号与进程实际使用资源的顺序不同，造成资源浪费。
- 3) 给用户编程带来了麻烦。

2. 死锁的避免

避免死锁的方法是把系统的资源分配状态分为安全状态和不安全状态，只要资源分配使系统资源分配状态处于安全状态，死锁就不会发生。在避免死锁的方法中，允许进程动态地申请资源。系统在资源分配之前，先计算资源分配的安全性。若本次资源分配不会导致系统进入不安全状态，便将资源分配给进程。否则拒绝进程的资源请求，将进程阻塞起来。

(1) 系统的安全状态

当系统能找到一个进程执行序列，使系统只要按此序列为每个进程分配资源，就可以保证进程的资源分配和执行顺利完成，不会发生死锁时，称系统处于安全状态。若系统不存在这样的安全序列，则称系统处于不安全状态。

不安全状态不一定是死锁状态，但当系统进入不安全状态之后，便可能进入死锁状态。反之，只要系统处于安全状态，系统可避免进入死锁状态。因此，避免进程死锁的实质在于使系统处于安全状态。

(2) 安全状态举例

假定系统有 3 个进程 p_1 、 p_2 、 p_3 ，共有 12 个某类资源 R 。进程 p_1 总共要求 10 个 R 类资源， p_2 和 p_3 分别要求 4 个和 9 个 R 类资源。设在 T_0 时刻，系统分配资源情况如表 3-3 所示。

表 3-3 安全状态举例

进 程	最大需求	已分配	可 用
p_1	10	5	3
p_2	4	2	
p_3	9	2	

经分析可以发现，在 T_0 时刻系统是安全的。因为 T_0 时刻存在一个安全序列 $\langle p_2, p_1, p_3 \rangle$ ，即只要系统按此进程序列分配资源，每个进程都可以顺利完成。 T_0 时刻系统资源 R 的可用数量为 3，如果先将其中的 2 个资源分配给进程 p_2 ， p_2 就可以顺利执行完，然后释放 p_2 已

经占用的全部 4 个资源。此时，系统的可用资源数量变为 5。把 5 个 R 类资源全部分配给进程 p1，p1 也可以顺利执行完毕。然后释放 p1 占用的全部资源，使系统可分配的资源数量达到 10 个。将其中的 7 个分配给进程 p3，p3 即可顺利执行完毕。

(3) 不安全状态举例

设在 T_1 时刻，系统分配资源情况如表 3-4 所示，则系统处于不安全状态，因为当系统处于表 3-4 所示的状态时，不存在安全序列，无论进程按什么顺序推进，都无法避免死锁。

表 3-4 不安全状态举例

进 程	最大需求	已分配	可 用
p1	10	5	2
p2	4	2	
p3	9	3	

(4) 安全状态可以向不安全状态转换

处于安全状态的系统，可能由于一次资源分配，由安全状态进入不安全状态。
安全状态向不安全状态转换举例如表 3-5 所示。

表 3-5 安全状态向不安全状态转换

进 程	最大需求	已分配	可 用		进 程	最大需求	已分配	可 用
p1	10	5	3	⇒	p1	10	5	2
p2	4	2			p2	4	2	
p3	9	2			p3	9	3	

T_1 时刻的状态

T_0 时刻的状态

T_0 时刻处于安全状态的系统，在进程 p3 提出申请一个 R 类资源后，把系统可用的 3 个资源中的 1 个分配给了 p3，系统状态由 T_0 时刻的安全状态转换为 T_1 时刻的不安全状态。

三、银行家算法

1965 年 Dijkstra 提出了一种能够避免死锁的资源分配算法。其基本思想是一个进程提出资源请求后，系统先进行资源的试分配。然后检测本次的试分配是否使系统处于安全状态，若安全则按试分配方案分配资源，否则不分配资源。

1. 数据结构

为了实现银行家算法，需要数据结构的支持，用 m 表示系统中资源的种类数，n 表示系统中的进程数。需要的数据结构如下。

1) available[] 是一个一维数组。表示系统中某种资源的可用数量，也就是这种资源可分配的数量，available[j] = k 表示 j 类资源的可用数量为 k，系统还可以为进程分配的 j 类资源为 k 个。

2) max[] 是一个 n 行 m 列的二维数组。表示各进程需要各类资源的最大数量。max[i,

$j] = k$ 表示进程 p_i 需要 j 类资源的最大数量为 k 个。

3) $allocation[]$ 是二维数组, 表示某时刻已分配给进程的某类资源数。 $allocation[i, j] = k$ 表示进程 p_i 已经占有 j 类资源 k 个。

4) $need[]$ 是二维数组, 表示某个进程还需要某类资源的数量。 $need[i, j] = k$, 表示进程 p_i 还需要 j 类资源 k 个。

2. 银行家算法的说明

银行家算法分为两个过程, 一是进行资源试分配的过程; 二是对试分配后系统的状态做安全性检测的过程。经安全性检测, 若试分配后系统状态是安全的, 则分配资源。若不安全, 则阻塞申请资源的进程, 暂不为它分配资源。

为了简化银行家算法的描述, 采用一些记号。设 X 和 Y 是长度为 n 的向量, 当且仅当对所有的 $i=1, 2, \dots, n$, $X[i] < Y[i]$, 则有 $X < Y$ 。例如, 如果 $X = (0, 3, 2, 1)$ 而 $Y = (1, 7, 3, 2)$, 那么 $X < Y$ 。当且仅当对所有的 $i=1, 2, \dots, n$, $X[i] = Y[i]$, 则有 $X = Y$ 。

将矩阵 $allocation_i$ 和 $need$ 的每行作为向量, 并分别用 $allocation_i$ 和 $need_i$ 来表示。向量 $allocation_i$ 表示已经分配给进程 p_i 的各类资源数, 向量 $need_i$ 表示进程 p_i 为完成其任务可能仍然需要申请的各类资源数。

(1) 资源试分配算法

设 $request_i$ 为进程 p_i 的请求向量。如果 $request_i[j] = k$, 那么进程 p_i 向系统请求 j 类资源 k 个。当进程 p_i 提出资源请求后, 系统从资源的试分配程序开始执行, 资源的试分配算法如下。

1) 如果 $request_i \leq need_i$, 那么转到第 2) 步。否则, 产生出错条件。

2) 如果 $request_i \leq available$, 说明系统可以满足进程提出的资源请求, 那么转到第 3) 步。否则, 说明系统缺乏可用资源, 不能满足进程的资源请求, 则阻塞进程 p_i 。

3) 执行资源的试分配。

$available = available - request_i$

$allocation_i = allocation_i + request_i$

$need_i = need_i - request_i$

4) 转安全性检测过程, 如果检测结果是安全的, 则确认本次的试分配, 否则, 进程 p_i 必须等待, 系统取消资源的试分配, 恢复到原来的资源分配状态。

资源分配流程如图 3-6 所示。

(2) 安全性检测算法

安全性检测算法用来判断系统的资源分配状态是否安全。算法需要的数据结构如下。

$work[]$ 是一个大小为 m 的一维数组, 进入安全性检测过程时被初始化为当前的 $available$ 。

$finish[]$ 是一个大小为 n 的一维数组, 进入安全性检测过程时, 对于所有的 $i=1, 2, \dots, n$, $finish[i] = false$ 。

安全性检测算法对 T 时刻的分配方案进行检测,

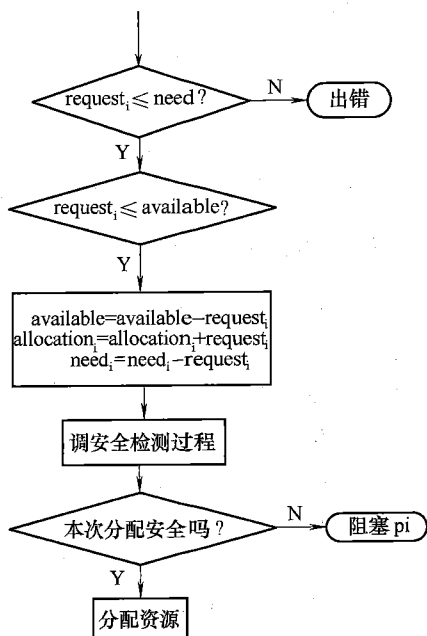


图 3-6 银行家算法流程图

确定是否存在至少一个安全序列。检测方法是：对进程集中的每一个进程，只要还有一个进程未“执行”时，在未“执行”进程集中找一个当前可以运行的进程，其所需资源数量 $need$ 小于当前可用资源量 $work$ 。若能找到这样的进程，则置其可执行标志 $finish[i] = true$ ，并令进程释放资源，增加系统可用资源量，进入下一次循环。若找不到符合条件的进程，说明该次分配方案不安全，因此结束循环。当找到可执行进程时，检测算法每次都在“考虑”：这个进程如果可以投入运行的话，它运行完并释放资源后，系统中可供分配的总资源数 $work$ 在获得一次累加之后，能否满足另一个进程的资源要求（存在一个进程 p_i ，其 $finish[i] = false$ 且 $need_i \leq work$ 吗？）

安全性检测算法的过程如下。

1) 查找 i ，使其满足条件。

$finish[i] = false$ 并且 $need_i \leq work$

如果没有满足条件的 i 存在，则跳转到第 3) 步。

2) 如果找到满足条件的 i ，执行下列操作。

$work = work + allocation_i$

$finish[i] = true$;

3) 如果对所有的 i ， $finish[i] = true$ ，那么系统处于安全状态。否则，系统处于不安全状态。

安全性检测算法流程如图 3-7 所示。

3. 实例

假设一个系统，有 5 个进程 p_0 、 p_1 、 p_2 、 p_3 和 p_4 ，有 3 种类型的资源 A、B 和 C。A 类资源有 10 个，B 类资源有 5 个，C 类资源有 7 个。假定在 T_0 时刻，系统的资源分配状态如表 3-6 所示。

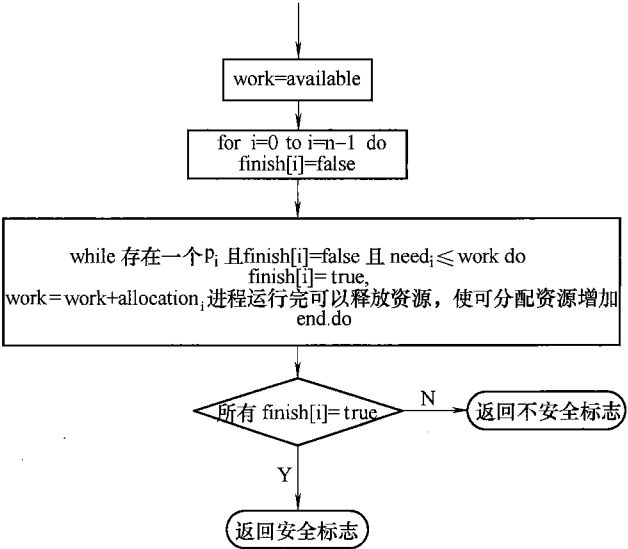


图 3-7 安全性检测算法流程图

表 3-6 T_0 时刻的资源分配状态

进程名称	allocation _i (A B C)	Max (A B C)	Need (A B C)	Available (A B C)
p0	0 1 0	7 5 3	7 4 3	3 3 2
p1	2 0 0	3 2 2	1 2 2	
p2	3 0 2	9 0 2	6 0 0	
p3	2 1 1	2 2 2	0 1 1	
p4	0 0 2	4 3 3	4 3 1	

在 T_0 时刻，可以找到一个安全序列 $\langle p_1, p_3, p_4, p_2, p_0 \rangle$ ，系统在 T_0 时刻处于安全状态。

若此时进程 p_i 提出资源请求 $request_i = (1, 0, 2)$ ，通过银行家算法，先进行资源试分

配, 然后检测试分配后系统在 T_1 时刻的状态是否安全。

1) 对 p_1 的请求进行试分配。

$\therefore$ (因为) T_0 时刻 $request_1 = (1, 0, 2)$, $available = (3, 3, 2)$, $need_1 = (1, 2, 2)$

可见满足 $request_1 \leq need_1$ 并且 $request_1 \leq available$

$\therefore$ (所以) 执行资源试分配算法的第 3) 步。

$available = available - request_1 = (3, 3, 2) - (1, 0, 2) = (2, 3, 0)$

$allocation_{i1} = allocation_{i1} + request_1 = (2, 0, 0) + (1, 0, 2) = (3, 0, 2)$

$need_1 = need_1 - request_1 = (1, 2, 2) - (1, 0, 2) = (0, 2, 0)$

试分配结束后系统的资源分配状态转变为 T_1 时刻状态, 如表 3-7 所示。

表 3-7 T_1 时刻的资源分配状态

进程名称	$allocation_i$ (A B C)	Max (A B C)	Need (A B C)	Available (A B C)
p_0	0 1 0	7 5 3	7 4 3	2 3 0
p_1	3 0 2	3 2 2	0 2 0	
p_2	3 0 2	9 0 2	6 0 0	
p_3	2 1 1	2 2 2	0 1 1	
p_4	0 0 2	4 3 3	4 3 1	

2) 执行试分配算法的第 4) 步, 转安全性检测过程, 检测系统在 T_1 时刻的状态是否安全。

先对 work 和 finish 变量进行初始化。

$work = available = (2, 3, 0)$

$finish[0] = finish[1] = finish[2] = finish[3] = finish[4] = false$

然后执行安全性检测过程的第 1)、2) 步, 循环检测是否能找到一个进程 p_i , 满足 $finish[i] = false$, 并且 $need_i \leq work$ 。

执行这个过程。

1 $\therefore need_1 < work$ 并且 $finish[1] = false$

$\therefore finish[1] = true$, $work = work + allocation_{i1} = (2, 3, 0) + (3, 0, 2) = (5, 3, 2)$, 按序执行的第一个进程为 p_1 。

2 $\therefore need_3 < work$ 并且 $finish[3] = false$

$\therefore finish[3] = true$, $work = work + allocation_{i3} = (5, 3, 2) + (2, 1, 1) = (7, 4, 3)$, 按序执行的第二个进程为 p_3 。

3 $\therefore need_4 < work$ 并且 $finish[4] = false$

$\therefore finish[4] = true$, $work = work + allocation_{i4} = (7, 4, 3) + (0, 0, 2) = (7, 4, 5)$, 按序执行的第三个进程为 p_4 。

4 $\therefore need_2 < work$ 并且 $finish[2] = false$

$\therefore finish[2] = true$, $work = work + allocation_{i2} = (7, 4, 5) + (3, 0, 2) = (10, 4, 7)$, 按序执行的第四个进程为 p_2 。

5 $\therefore need_0 < work$ 并且 $finish[0] = false$

$\therefore \text{finish}[0] = \text{true}$, $\text{work} = \text{work} + \text{allocation}_{i_0} = (10, 4, 7) + (0, 1, 0) = (10, 5, 7)$, 按序执行的第五个进程为 p_0 。

到此, 再也找不到满足 $\text{finish}[i] = \text{false}$ 并且 $\text{need}_i \leq \text{work}$ 条件的进程 p_i , 因此循环结束。执行安全性检测算法的第 3) 步。由于条件“对所有的 i , $\text{finish}[i] = \text{true}$ ”是满足的, 所以 T_1 时刻系统处于安全状态, 通过安全性检测算法找到的安全序列为 $\langle p_1, p_3, p_4, p_2, p_0 \rangle$ 。

由资源的试分配和安全性检测程序的运行结果可以看出, 在 T_0 时刻, 进程 p_1 提出的资源请求 $\text{request}_1 = (1, 0, 2)$, 系统是可以满足的。

4. 银行家算法应用情况的说明

虽然 Dijkstra 在 1965 年就提出了银行家算法, Andrew S. Tanenbaum 在其所著《操作系统: 设计与实现》(《Operating Systems Design and Implementation》)一书中指出, 几乎每本操作系统专著都详细地描述了银行家算法, 许多论文的内容也围绕该算法, 但很少有作者指出该算法缺乏实用价值。因为很少有进程能够在运行之前就知道其所需资源的最大值, 而且进程数不是固定的, 往往在不断发生变化, 况且原本可用的资源也可能突然间变成不可用(如磁带机可能会坏掉)。

四、死锁的检测和解除

操作系统可以不采取事先预防和避免的方法来解决死锁问题, 而是检测是否有死锁发生。如果检测到系统中有死锁的进程, 则解除死锁。

应用检测死锁算法必须面对的问题是: 何时调用检测算法? 如何检测死锁? 下面先对第一个问题进行说明, 然后介绍用于检测和解除死锁的资源分配图及死锁定理。

1. 何时调用检测算法

关于何时调用检测算法? 答案取决于两个因素: 一是死锁可能发生的频率, 二是当死锁发生时受影响的进程数量。

如果死锁经常发生, 就应该经常调用检测算法。只有当某个进程提出资源请求且得不到满足时, 才会出现死锁。在极端情况下, 当进程请求资源分配而不能立即被满足的情况下, 就调用死锁检测算法。但是, 对于每个请求都调用死锁检测算法会引起相当大的计算开销。另一种开销相对较小的方法是只在一个不频繁的时间间隔里调用检测算法, 如每小时一次, 或当 CPU 使用率低于 40% 时。因为死锁最终会使系统性能下降, 并造成 CPU 使用率下降。

2. 资源分配图

系统死锁可利用资源分配图来描述, 该图由一组结点和一组边组成。如图 3-8 所示, 用圆圈代表一个进程, 用方框代表一类资源。由于一种类型的资源可能有多个, 用方框中的一个小圆圈代表某一类资源中的一个资源。此时, 请求边由进程指向方框中的 R_j 。而分配边则应始于方框中的一个点。图 3-8 所示的资源分配图中, p_1 进程已分得了两个 R_1 资源, 并又请求一个 R_2 资源。 p_2 进程分得了 一个 R_1 和一个 R_2 资源, 并请求一个 R_1 资源。

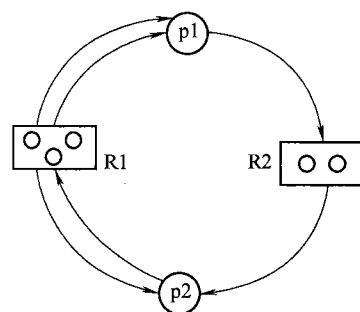


图 3-8 资源分配图示例

3. 死锁定理

死锁定理用于检测系统所处的资源分配状态 S 是否为死锁状态。

死锁定理为： S 为死锁状态的充分条件是当且仅当 S 状态的资源分配图是不可完全简化的。

下面用一个实例阐述资源分配图的简化问题，说明什么是不可完全简化的。

实例图如 3-9a 所示，为了判断该图所表示的资源分配状态是否是死锁状态，对该图进行简化，简化方法如下。

在资源分配图中，找出一个既不阻塞又非独立的进程结点 p_i 。在顺利的情况下， p_i 可获得所需资源而继续执行，直至运行完毕，再释放其所占有的全部资源。这相当于消去 p_i 所有的请求边和分配边，使之成为孤立的结点。在图 3-9a 中，将 p_1 的两个分配边和一个请求边消去，便形成如图 3-9b 所示的情况。

p_1 释放资源后，便可以使 p_2 获得资源而继续运行，直到 p_2 完成后又释放它所占有的全部资源，而形成如图 3-9c 所示的情况。

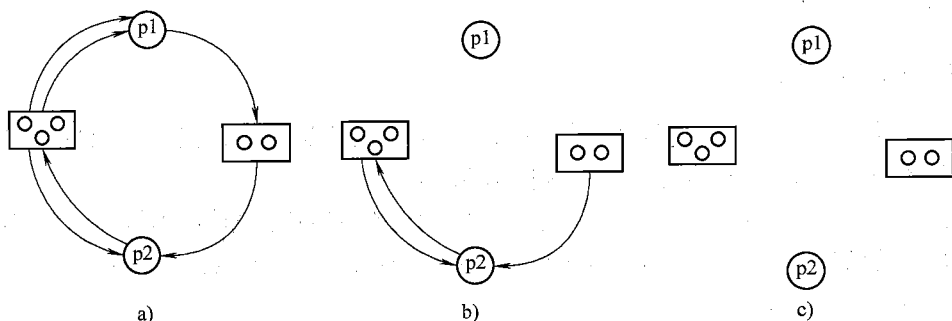


图 3-9 资源分配图的简化

在进行一系列的简化后，若能消去图中所有的边，使所有的进程都成为孤立结点，则称该图是可完全简化的。若不能通过任何过程使该图完全简化，则称该图是不可完全简化的。

对于较复杂的资源分配图，可能有多个既未阻塞又非孤立的进程结点，不同的简化顺序是否会得到不同的简化图呢？有关文献已经证明，所有的简化顺序都将得到相同的不可简化图。

4. 死锁的解除

检测到系统出现死锁后，可以由系统管理员人工处理死锁，也可以让系统自动解除死锁。解除死锁的途径有两个：一是终止处于死锁状态的进程，二是抢占死锁进程占有的资源。

(1) 进程终止

终止死锁的进程后，系统回收进程占有的资源。可以采用下面两种方式终止进程。

1) 终止所有死锁进程。

2) 一次只终止一个处于死锁的进程，直到死锁解除。

在采用终止部分进程的方法时，每次应选择终止哪个进程呢？原则上应该选择终止代价最小的进程。需要考虑的因素包括以下几个。

- 1) 进程的优先级是什么?
- 2) 进程已执行了多久? 进程在完成其指定任务之前还需要多长时间?
- 3) 进程使用了多少资源? 分别是什么类型的资源? 这些资源是否容易抢占?
- 4) 进程需要多少资源才能完成?
- 5) 需要终止多少进程才能解除死锁?
- 6) 进程是交互的还是批处理的?

(2) 资源抢占

逐步从进程中抢占资源给其他进程使用, 直到死锁环被打破为止。如果要求使用抢占来处理死锁, 那么有 3 个问题需要处理。

1) 抢占哪个进程和哪些资源? 必须确定抢占的顺序以使代价最小。影响因素有: 死锁进程所拥有的资源数量、死锁进程到目前为止已经消耗的执行时间等。

2) 回滚。如果从一个进程那里抢占一个资源, 那么该进程因缺少资源不能从抢占点正常执行, 必须将进程回滚到某个安全状态, 以便从该状态重启进程。最简单的方法是完全回滚, 就是终止进程并重启进程。

3) 饥饿。饥饿是进程因长时间不能获得所需要的资源而无限等待的状态。比如系统采取基于静态优先权的进程调度算法, 当系统不断有高优先权进程到来时, 优先权低的进程就可能长时间得不到 CPU, 也就不能运行, 这种状态被称为饥饿状态。

处于饥饿状态的进程不影响其他进程获得资源, 死锁的进程与饥饿进程一样无法继续获得资源。但是死锁一旦发生, 便涉及竞争资源的一组进程。系统中不会出现单个进程死锁的现象, 但是可能会只有一个进程饥饿。

在抢占资源的过程中如何保证不会总是从同一个进程抢占资源而使该进程处于饥饿状态? 一个解决方案是限制对同一进程进行资源抢占的次数。最常用的方法是在影响代价的因素中加上回滚次数。

思考题与练习题

一、单项选择题(在每小题后的四个备选项中只有一个是符合题目要求的, 请将其选出并将代码填写在题后的括号内)

1. 影响时间片大小选择的主要因素有【 】。
 - A. 系统的响应时间
 - B. 系统的周转时间
 - C. 系统在单位时间内处理的作业量
 - D. 进程需要 CPU 服务的总时间
2. 关于多级队列调度算法下列说法正确的是【 】。
 - A. 每个就绪队列的优先权不同, 调度算法相同
 - B. 每个就绪队列的优先权相同, 调度算法不同
 - C. 各就绪队列调度算法和优先权都可能不相同
 - D. 一个进程在运行过程中可能被插入不同的就绪队列
3. 某系统中有 4 个并发进程, 都需要同类资源 3 个, 试问该系统不会发生死锁的最少资源数是【 】。
 - A. 6
 - B. 7
 - C. 8
 - D. 9

二、填空题(请在题目空白处填写正确内容)

1. FCFS 适合_____进程, 不利于_____进程。
2. 采用基于静态优先权的调度算法时, 若不断有高优先权进程进入就绪队列, 低优先权进程可能进入_____状态。
3. _____算法是用来避免死锁的算法。
4. S 为死锁状态的_____条件是当且仅当 S 状态的资源分配图是不可完全简化的。

三、简答题

1. 进程调度的功能是什么?
2. 说明什么是时间片轮转调度算法?
3. 什么是多级队列调度算法?
4. 什么是自调度方式? 自调度有什么优缺点?
5. 什么是死锁? 引起死锁的原因是什么?

四、简单应用

考虑下面的一个系统在某一时刻的状态, 如表 3-8 所示。

表 3-8 某一系统在某一时刻的状态

进程名称	allocation _i	Max	Available
	(A B C D)	(A B C D)	(A B C D)
p0	0 0 1 2	0 0 1 2	1 5 2 0
p1	1 0 0 0	1 7 5 0	
p2	1 3 5 4	2 3 5 6	
p3	0 6 3 2	0 6 5 2	
p4	0 0 1 4	0 6 5 6	

用银行家算法回答下列问题。

- (1) need 矩阵的内容是怎样的?
- (2) 系统是否处于安全状态?
- (3) 如果进程 p1 提出资源请求(0, 4, 2, 0), 这个请求能否立刻被满足?

第四章 内存管理

内存是计算机系统的重要组成部分，当操作系统接收到运行某程序的命令后，要为该程序的运行分配内存资源，创建进程，并把进程的全部或部分调入内存。进程运行结束，系统要回收被撤销进程的内存空间。内存管理的目标一方面是实现内存分配、内存回收等基本内存管理功能，另一方面是要提高内存空间的利用率和内存的访问速度。每个程序员都希望自己的程序能运行在一个独占的、容量充分大、访问速度足够快的存储器上。然而，事实上计算机的内存是有限的，而且是被许多应用程序共享的。处于同一内存中的不同进程之间必然会存在竞争与相互影响。操作系统的内存管理正是解决这一问题的，其目标是充分利用现有的内存资源，为应用程序提供方便的内存使用方式和一个快速、安全且充分大的存储器。

关于内存管理，需要掌握：①计算机硬件为程序的运行提供了什么样的存储器使用方式。②操作系统为应用程序的运行提供了什么样的存储服务。③操作系统怎样将多个不同的用户进程放在一个内存中，并使它们不相互干扰地并发执行。④操作系统如何为进程提供远比物理内存大得多的虚拟内存空间。带着这些问题来学习内存管理，就会理解计算机硬件、操作系统和应用程序在内存的使用上各自的角色，以及如何相互合作，从而有一个内存管理的大局观。本章将详细介绍内存管理的基本原理和实现技术。

第一节 存储器的层次结构

内存是计算机存储系统的一部分，为了更好地理解后续章节的内容，有必要先了解存储器系统的层次结构。

存储器系统是一个具有不同容量、成本和访问时间的存储设备的层次结构，如图 4-1 所示。

在这个层次系统中，从高层到低层（L0 ~ L5），较低层的存储设备速度更慢、容量更大、价格更便宜。在最高层（L0 层），是少量的快速 CPU 寄存器，CPU 可以在一个时钟周期内访问它们。接下来是一个或多个小型或中型的基于 SRAM 的高速缓存存储器，可以在几个 CPU 时钟周期内访问它们。然后 L3 层是一个大的基于 DRAM 的主存，可以在几十或几百个时钟周期内访问它们。L3 的下层 L4 是慢速但容量很大的本地磁盘。L5 表示有些系统可能还包括一层附加的远程服务器上的磁盘，需要通过网络来访问它们。例如 NTF 网络文件系统这样的分布式文件系统，允许程序访问存储在远程网络服务器上的文件。

CPU 寄存器保存最常用的数据。靠近 CPU 的容量小、速度快的高速缓存存储器作为速度相对较慢、容量较大的主存中数据和指令子集的缓冲区。主存暂时存放存储容量更大、速度更慢的磁盘上的数据。而这些磁盘常常又作为存储在通过网络连接的其他机器的磁盘或磁带上的数据的缓冲区。

如果程序需要的数据是存放在 CPU 寄存器中的，程序执行期间在零个周期内就可以访

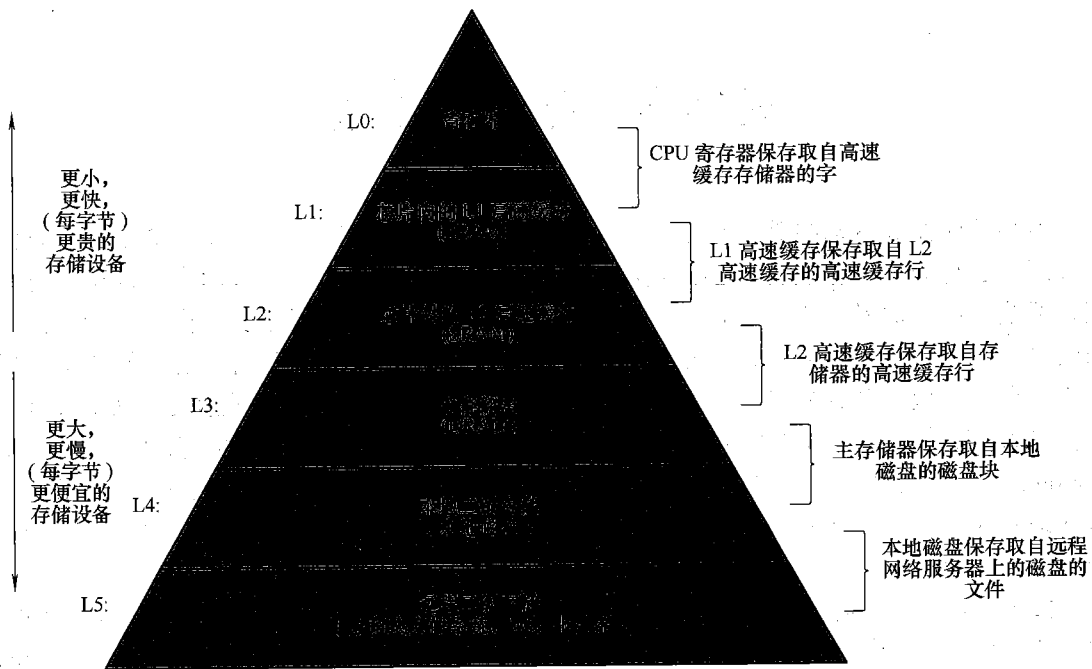


图 4-1 存储器的层次结构

问到它们。如果存储在高速缓存中，需要 1~10 个周期，如果这些数据存放在主存中，访问它们就需要 50~100 个周期。而如果这些数据是存放在磁盘中的，要访问它们需要大约 2000 万个周期。因此，在编写程序、设计算法时要尽可能把最近将要访问的指令或数据存储在层次较高的地方，以便让 CPU 更快地访问到它们。

程序的执行遵循局部性原理。程序执行的局部性原理指出，程序在执行时呈现出局部性规律，即在一段较短的时间内，程序的执行仅局限于某个部分，相应地，它所访问的存储空间也局限于某个区域。关于程序执行的局部性原理有以下几个论点。

1) 程序在执行时，除了少部分的转移和过程调用指令以外，在大多数情况下是顺序执行的。

2) 过程调用将会使程序的执行轨迹由一部分内存区域转到另一部分内存区域。但研究表明，在大多数情况下，过程调用的深度都不超过 5。这就是说，程序将会在一段时间内局限在这些过程的范围内运行。

3) 程序中存在很多循环结构，它们虽然由少数指令构成，但多次执行。

4) 程序中往往包括许多对数据结构的处理。例如对数组进行操作，它们往往都局限在很小的范围内。

总的来说，局部性原理表现为时间和空间的局部性。

1) 时间局部性。如果程序中的某条指令一旦执行，则不久后该指令可能再次执行。如果某个数据结构被访问，不久以后该数据结构可能被再次访问。

2) 空间局部性。一旦程序访问了某个单元，在不久之后，其附近的存储单元也将被访问。

具有良好局部性的程序会经常访问相同的数据集合或相邻的数据集合。具有良好局部性

的程序比局部性差的程序能更好地利用处于高层次的存储器，因此运行速度更快。例如，不同的矩阵初始化程序，执行相同数量的赋值操作，由于程序的不同，具有不同的局部性，其运行速度可能相差很大。

第二节 程序的链接和装入

高级语言程序必须经过编译、链接才能成为可执行程序，操作系统需要为程序的执行分配内存空间。下面介绍链接程序的功能和程序被装入内存的几种方式。

一、程序的链接

链接程序不属于操作系统的构成部分，但是它为操作系统提供可装入的程序模块。链接程序要解决的问题是将编译后的目标模块装配成一个可执行的程序。根据链接进行的时间和实现方式的不同，可以把链接分为静态链接和动态链接。

1. 静态链接

静态链接 (Static Linking) 是在程序运行前，用链接程序将目标模块链接成一个完整的装入模块。静态链接程序的任务一是对逻辑地址进行修改，二是变换外部调用符号。

(1) 对逻辑地址进行修改

如图 4-2 所示，链接前模块 A、B、C 的相对地址范围分别是 $0 \sim L-1$ 、 $0 \sim M-1$ 、 $0 \sim N-1$ ，通过链接程序把三个目标模块链接成一个可执行程序后，模块 B 和 C 的逻辑地址范围变成了 $L \sim L+M-1$ 、 $L+M \sim L+M+N-1$ 。三个独立目标模块的逻辑地址空间链接成了一个连续的地址空间。

(2) 变换外部调用符号

将每个模块中所用的外部调用符号都变换为逻辑地址。在图 4-2 中，经过链接后，j 将模块 B 的外部调用 CALL B 变成了一条跳转到模块 B 在相对地址空间中的起始地址 L 处的指令 JSR " L"。对模块 B 的外部调用 CALL C 变成了一条跳转到模块 C 在逻辑地址空间中的起始地址 $L+M$ 处的指令 JSR " L+M"。

静态链接相对于动态链接而言，程序运行速度较快。但是无论程序在本次运行中会不会被执行，都将全部被链接到一个可执行文件中，使可执行文件比较大，占用的内、外存空间较大，使存储开销较大。另外，使用静态链接的方式，程序开发不够灵活、方便，修改某一个模块会导致整个程序的重新链接。

2. 动态链接

采用动态链接 (Run-time Dynamic Linking)，可将某些目标模块的链接推迟到这些模块中的函数被调用执行时才进行。即在程序执行时，若发现一个被调用模块尚未链接，再把它

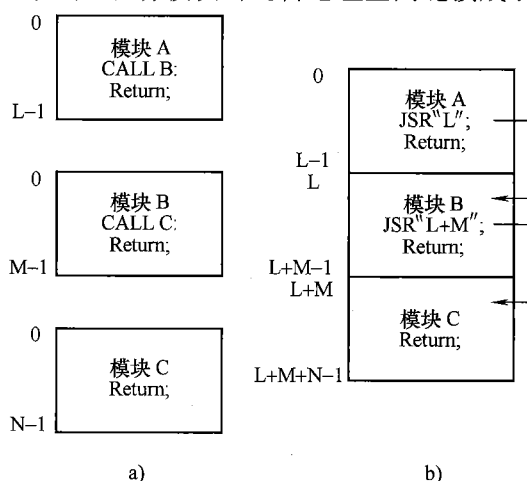


图 4-2 静态链接
a) 目标模块 b) 装入模块

链接到调用者模块上。采用动态链接的优点是节省内存和外存空间，方便了程序开发。例如开发一个图形控件，若让图形控件以动态链接库的形式存在，这个控件就可以独立开发、独立编译和链接。但由于动态链接是在程序运行过程中从外存将被调用的模块调入内存并链接到调用者模块上，这需要运行时的时间开销，会使程序运行时的速度变慢。

二、程序的装入

在多道程序环境下，程序要运行必须为之创建进程，而创建进程后，不可避免地要为进程分配内存，并将进程的程序和数据装入内存。将一个用户的源程序变为一个可在内存中执行的程序，通常要经过编译、链接和装入3个阶段。

通常，可执行程序以二进制可执行文件的形式存储在磁盘上，为执行程序，操作系统需要把程序调入内存。

多数系统允许操作系统将用户进程放在物理内存的任意位置。因此，虽然计算机的地址空间从0开始，但用户进程的起始地址不一定是0。在绝大多数情况下，源程序需要经过编译、链接和装入几个阶段才能执行。在不同阶段，程序地址有不同的表示形式。源程序中的地址通常是符号表示，如一个整型变量 `counter`。编译器将这些符号地址变成可重定位地址，通常是相对于本模块开始位置的地址。例如，被编译模块的起始地址为0，所有符号地址都转变成相对于0开始的一个逻辑地址。链接程序把几个目标模块的逻辑地址转变为相对于整个可执行程序的起始地址的逻辑地址。每一次的地址变化都是从一个地址空间到另一个地址空间的映射。根据形成在内存中物理地址的时机不同，把程序的装入方式分为绝对装入方式、可重定位装入方式（静态重定位）和动态运行时装入方式。

1. 绝对装入方式

编译程序事先已知程序在内存中的驻留位置，编译时产生物理地址的目标代码，绝对装入程序按照装入模块的物理地址将程序和数据装入内存。因此装入模块被装入内存后，无需对程序和数据地址进行修改。

2. 可重定位装入方式（静态重定位）

在多道程序系统中，众多用户进程共享内存空间，什么时候空闲、内存的哪一块区域空闲，可以让操作系统在此装入一个新进程是无法预知的。因此，编译器在编译程序时无法形成程序的物理地址。如果编译时不知道目标程序将驻留在内存的什么位置，那么编译时就必须生成可重定位的代码，其中的地址都是逻辑地址，在程序被装入内存时，再把这些逻辑地址映射为物理地址。在程序装入时对目标程序中的指令和数据地址的修改过程称为重定位。

可重定位方式的两个特点如下。

- 1) 编译程序使目标模块的起始地址从0开始。
- 2) 程序装入时，装入程序根据内存的使用情况将装入模块装入到内存的某个位置，并对模块进行重定位。

物理地址 = 有效逻辑地址 + 程序在内存中的起始地址。

在采用可重定位装入方式将程序装入内存后，程序在内存中的实际物理地址与逻辑地址通常是不相同的。如图4-3所示，程序在逻辑地址（相对地址）为1000的单元中有一条指令 `LOAD 1, 2500`。该指令的语义是将逻辑地址为2500处的内容取出装入1号寄存器，逻辑地址为2500的单元装有数据365。若采用重定位装入方式将该用户程序装入内存实际物理

地址从 10000 开始的内存单元，就必须根据逻辑地址和用户程序在内存中的起始地址计算并修改用户程序在物理内存中的实际地址。指令 LOAD 1, 2500 的实际物理地址改为 1000 + 10000，即 11000。数据 365 所在的实际物理内存单元地址改为 2500 + 10000，即 12500。用户程序装入内存后，指令中的逻辑地址也需要修改为实际物理地址，指令 LOAD 1, 2500 要修改为 LOAD 1, 12500。

3. 动态运行时装入（动态重定位）

进程在装入内存后，还可能从内存的一个区域移动到另一个区域，这种情况可能发生在支持虚拟存储的系统中。一个进程在被换出之前所在的内存位置与后来被从外存重新调入内存时所在的内存位置不同，在这种情况下，地址映射必须延迟到进程执行时再进行，把这种装入方式称为动态运行时装入。

在采用动态运行时装入方式的系统中，系统将进程装入内存后，由于进程在内存中的位置可能发生移动，所以此时并不计算物理地址，而是在进程运行访存的过程中才进行地址转换，这种方式需要重定位寄存器的支持。当进程获得 CPU 运行时，系统把该进程在内存的起始地址存入重定位寄存器，进程在运行过程中访存时，通过重定位寄存器与被访问单元的逻辑地址计算出被访问单元的物理地址，如图 4-4 所示。

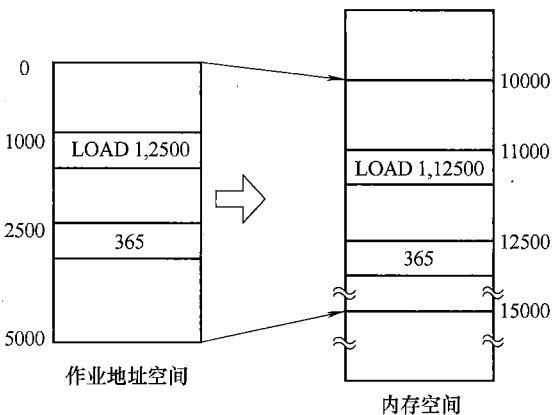


图 4-3 采用可重定位方式装入作业的情况

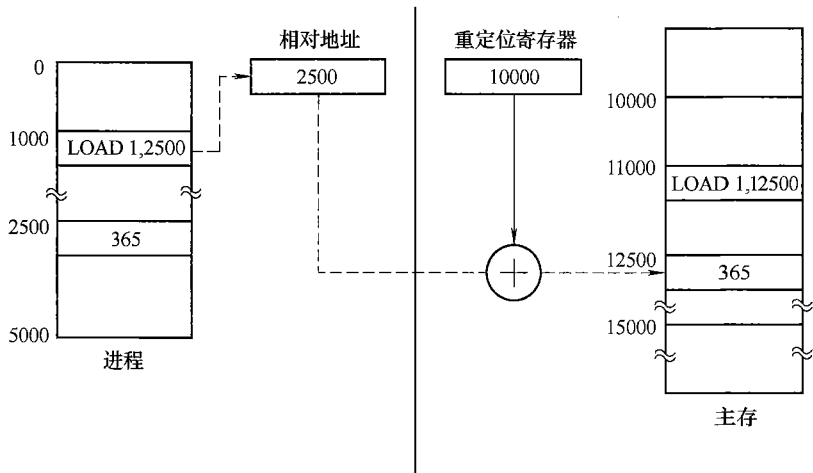


图 4-4 采用动态运行时装入方式装入进程的情况

进程获得 CPU 运行时，操作系统将进程在内存中的起始地址 10000 存入重定位寄存器，作业运行的过程中，当要执行逻辑地址为 1000 的单元中的指令时，用逻辑地址 1000 与重定位寄存器中的地址 10000 相加，得到实际物理内存地址 11000。CPU 从 11000 物理单元取到指令 LOAD 1, 2500。当执行该指令访问逻辑地址为 2500 的单元取 365 时，先将 2500 与重定位寄存器的值 10000 相加，得到 365 所在的物理单元地址 12500，然后从 12500 物理单元

读取数据 365。

重定位寄存器是每 CPU 一个的, 当发生进程切换时, 要用获得 CPU 的进程在内存的起始地址更新重定位寄存器。

第三节 连续分配存储管理方式

连续分配是指操作系统分配内存时, 为每个进程分配一块物理地址连续的内存空间。连续分配方式有 3 种类型。

1. 单一连续区分配方式

内存中只有一个用户区, 任意时刻内存中只能装入一道程序, 这种分配方式仅适用于单用户、单任务的系统。

2. 固定分区分配方式

将内存用户区划分成若干个固定大小的区域, 每个区域中驻留一道程序。

3. 动态分区分配方式

系统动态地对内存进行划分, 根据进程需要的空间大小分配内存。内存中分区的大小和数量是变化的。动态分区方式比固定分区方式显著地提高了内存利用率。

一、单一连续分配

单一连续分配方式适用于单用户、单任务的操作系统, 它把内存分为系统区和用户区。系统区仅供操作系统使用, 用户区供用户使用, 如图 4-5 所示。

系统区用于驻留操作系统, 用户区用于分配给用户进程使用。为了防止用户程序对操作系统的破坏, 保证系统的安全、可靠, 在操作系统中应该考虑设置存储器保护机制。在单用户、单任务操作系统中较常用的方法是设置一个基址寄存器和一个界限寄存器。基址寄存器中存放程序在物理内存中的最小地址, 界限寄存器中存放装入用户区程序的地址范围。在 CPU 访问内存时, 检查 CPU 要访问的内存单元的地址是否大于界限寄存器的值。如果大于界限寄存器中的地址范围, 则是非法地址, 表示用户进程访存越界, 程序执行会被中断。

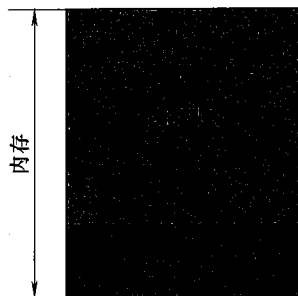


图 4-5 单一连续分配

在有些单用户、单任务的操作系统, 如 CP/M 和 MS-DOS 中, 没有设置存储器保护机制。其理由为: 一是节省硬件; 二是单任务单用户系统中, 用户独占机器, 对系统的破坏只可能是用户自己造成的, 后果也不严重, 不会影响其他用户程序的执行, 而且操作系统很容易重装和再次启动。

二、固定分区分配

固定分区分配将用户内存空间划分为若干个固定大小的区域, 在每个用户区中可以装入一道用户程序, 如图 4-6 所示。

内存的用户区被划分成几个分区, 便允许几个进程并发运行。当有一个空闲分区时, 可从外存的后备队列中选择一个大小适当的作业装入该分区。当该作业结束时, 释放所占用的

分区，系统又可从后备作业队列中找出另一个作业调入该分区。

1. 划分分区的方法

固定分区分配的用户分区数量是固定的，每个分区的大小也是固定的。但是每个分区的大小可以相等，也可以不相等。

(1) 分区大小相等

在这种设计中，把用户区划分成大小相等的若干个分区。这种设计的缺点是内存利用率比较低。当程序太小时，该程序所占用的分区有很大一部分空间是空闲的。而程序较大时，可能找不到一个分区足以装下该程序。这种设计主要用于利用一台计算机去控制多个相同对象的场合，因为这种情况下，各用户进程需要的空间大小相同，例如，用一台计算机控制多台相同的冶炼炉。

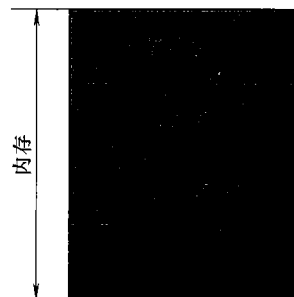


图 4-6 固定分区分配

(2) 分区大小不等

为了更好地利用内存，可以将用户区划分成大小不同、数量固定的若干个分区。为用户进程分配空间时，把大小最接近进程大小的空闲分区分配给申请内存空间的进程。使小进程占小分区，大进程占大分区，减少内存浪费。

2. 支持固定分区分配的数据结构

操作系统为了完成对固定分区的管理，必须定义一个记录用户分区大小和使用情况的数据结构。可以使用一个如表 4-1 所示的记录内存状况的内存分区说明表来管理内存分配。

表 4-1 固定分区说明表

分区编号	分区大小 (KB)	分区起始地址 (KB)	分区状态
1	15	30	已分配
2	30	45	空闲
3	50	75	已分配
4	100	125	已分配

用 C 语言来描述这个分区表，可以定义一个有 4 个元素的结构数组。每个元素对应一个分区。结构体包括 4 个字段：分区编号、分区大小、分区起始地址和分区状态。数据结构定义示例如下。

```
struct {
    int num;      //分区编号
    int length;   //分区大小
    int addr;     //分区起始地址
    int state;    //分区状态，该值为 0 表示分区空闲；该值为 1 表示分区被占用
} mem_block[4];
```

写程序时数据结构的定义根据具体系统的需要和算法的需要来调整。

3. 固定分区分配的过程

需要为进程分配内存时，操作系统执行内存分配程序，搜索内存分区使用表。当找到一个大小大于或等于进程需要的内存空间而且处于空闲状态的用户分区时，将该分区分配给进程，并将该分区状态改为“已占用”。在上面的数据结构中，就是将相应分区的 state 字段置 1。

4. 固定分区的回收

当进程运行结束后，系统要回收进程占用的分区。通过执行内存回收程序完成回收操作，只要把回收分区的使用状态改为“空闲”即可，即把上面数据结构中相应分区的 state 字段置 0。

固定分区分配实现简单，但是由于每个分区的大小固定，必然造成存储空间的浪费，使内存利用率低下。现在的多道程序系统很少使用固定分区分配的内存管理方式。但是，在一些实时控制系统中，使用固定分区分配还是简单而有效的。

三、动态分区分配

由于固定分区分配将内存空间划分成大小固定的分区，当运行需要的内存空间比分区大的进程时，固定分区无法满足要求。而当运行只需要很小空间的进程时，内存空间浪费大。使用固定分区分配内存利用率低，难以提高系统的多道程序度。动态分区分配是根据进程的实际需要，为进程分配大小合适的内存区域。系统中用户分区的数量和大小都是动态变化的。

本节主要介绍下列内容。

- 1) 动态分区分配使用的数据结构。
- 2) 动态分区分配算法。
- 3) 动态分区的分配和回收操作。

1. 动态分区分配原理

动态分区分配算法的原理：系统初始只有一个大空闲区，当进程请求空间时，由系统根据进程需要的空间大小划分出一片空闲区分配给进程。系统运行一段时间后，内存的空闲区可能散布在不连续的区域。系统维护一个记录当前空闲分区情况的数据结构，当进程请求内存时，系统从所有空闲区中找到大小合适的空闲分区进行分配。系统中分区的大小和数量都是变化的，空闲区的大小和数量也是变化的。

2. 动态分区分配中的数据结构

为实现动态分区分配，系统需要建立并维护记录空闲分区情况的数据结构。常用的数据结构有空闲分区表和空闲分区链。

(1) 空闲分区表

如表 4-2 所示，系统在空闲分区表中为每一个空闲分区建立一个表项，每个表项中包括分区编号、分区大小和分区起始地址。在具体程序实现时，可以用结构数组来实现空闲分区表。数组的每一个元素对应一个表项，记录一个空闲分区的情况。每个表项包含 3 个字段，即分区编号、分区大小和分区起始地址。使用空闲分区表的缺点是，若设置太多表项，会浪费内存空间；设置太少的表项，当空闲分区较多时，无法记录所有空闲分区的情况。在实现时，结构数组的大小不容易确定。

表 4-2 空闲分区表

分区编号	分区大小 (KB)	分区起始地址 (KB)
1	12	30
2	30	100

在程序中可以用下列结构数组表示空闲分区表。

```
struct {
    int num;        //分区编号
    int length;     //分区大小
    int addr;       //分区起始地址
} FreeMem_block[N]
```

数组大小根据系统管理的最大分区数确定。

(2) 空闲分区链

使用空闲分区链可以动态地为每一个空闲分区建立一个结点，每个结点包括分区大小、分区起始地址、指向前一个空闲分区结点的指针，以及指向后一个空闲分区结点的指针。空闲分区链中的每个结点占用的内存可以动态分配、动态回收。使用空闲分区链可以克服空闲分区表存在的缺点。

空闲分区链数据结构如图 4-7 所示。图中有 5 个分区，其中有 3 个分区是已占用分区。两个大小分别为 10 KB 和 20 KB 的分区是空闲分区，其起始地址分别为 100 KB 和 130 KB。系统建立的空闲链表如图 4-7 所示，包括两个与空闲分区对应的结点。每个结点中包含了空闲分区的大小、空闲分区的起始地址和指向前一个空闲分区结点的指针、指向后一个空闲分区结点的指针。在实际系统中，也可以采用更复杂的双向链表或者双向循环链表。

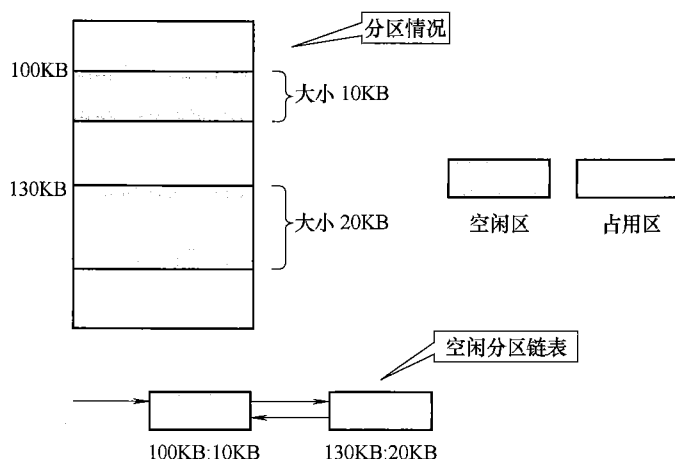


图 4-7 动态分区与空闲区链表

使用双向空闲链表的数据结构如下所示。

```
Struct FreeMem
{
    int length;    //空闲分区大小
    int addr;      //空闲分区起始地址
    TFreeMem *prev, *next; //指向前一个结点的指针和指向后一个结点的指针
}
```

3. 动态分区分配算法

当进程需要内存空间时，操作系统通过执行动态分区分配算法从多个空闲分区中选定一

个合适的分区分配给进程。不同的分区分配算法有不同的特点，需要不同数据结构的支持。

下面介绍 3 种动态分区分配算法，即首次适应算法、循环首次适应算法和最佳适应算法。

(1) 首次适应算法 FF(First Fit, FF)

在采用空闲分区链作为数据结构时，首次适应算法要求空闲分区链以地址递增的顺序链接。在进行内存分配时，从链首开始顺序查找，直至找到一个能满足进程大小要求的空闲分区为止。然后，再按照进程请求内存的大小，从该分区中划出一块内存空间分配给请求者，余下的空闲分区仍留在空闲链中。

该算法总是先分配低地址部分的内存空间，容易使低地址部分留下小分区，而高地址部分大空闲区较多。当进程请求大内存空间时，要找到合适的空闲分区，搜索空闲分区链需要的时间开销比较大。

此外，由于低地址部分的空闲分区反复被划分，可能留下许多难以利用的很小的空闲分区，这种难以被利用的小空闲区也被称为外部碎片或外碎片。分配给进程的分区若大于进程请求的分区，分区内会存在一部分不被利用的空间，这部分被浪费的空间称为内部碎片或内碎片。

(2) 循环首次适应算法 NF(Next Fit, NF)

该算法是由首次适应算法演变而形成的。在为进程分配内存空间时，不再每次从链首开始查找合适的空闲分区，而是从上次找到的空闲分区的下一个空闲分区开始查找，直至找到第一个能满足要求的空闲分区，并从中划出一块与请求的大小相等的内存空间分配给进程。为实现该算法，应设置一个起始查找指针，以指示下一次起始查找的空闲分区，并采用循环查找方式，如图 4-8 所示。

循环首次适应算法的优点是：空闲区分布均匀、查找开销较小。缺点是容易使系统缺乏大空闲区。

(3) 最佳适应算法 BF(Best Fit, BF)

该算法每次为作业分配内存，总是把大小与进程所请求的内存空间大小最接近的空闲分区分配给进程，避免了“大材小用”。为了加速寻找，该算法要求将所有的空闲区按分区大小递增的顺序形成一个空闲区链。这样，第一次找到的满足要求的空闲区必然是大小最接近进程需要的内存空间大小的。

最佳适应算法的优点是避免了大材小用，能提高内存利用率。但是，采用最佳适应算法容易留下难以利用的小空闲区。

下面通过一个例题来说明首次适应算法、循环首次适应算法和最佳适应算法。

例 4-1：当前空闲链如图 4-9 所示。

空闲链图示的含义如下。

假设系统中现有 3 个空闲分区，第一个空闲分区起始地址为 20 KB，大小为 120 KB；第

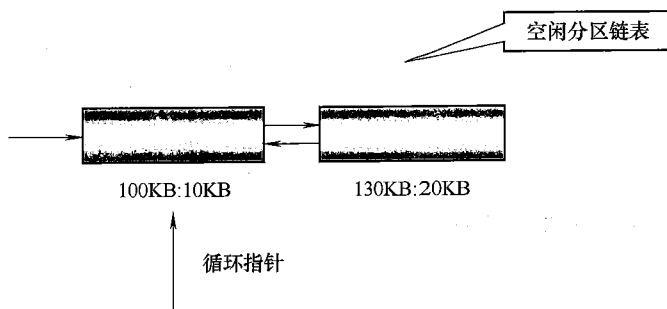


图 4-8 增加了循环指针的空闲区链表

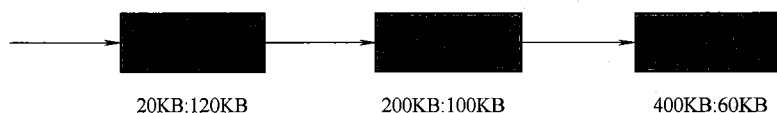


图 4-9 动态分区与空闲区链表

二个空闲分区起始地址为 200 KB，大小为 100 KB；第三个空闲分区起始地址为 400 KB，大小为 60 KB。若某进程 p1 先请求大小为 30 KB 的内存空间，随后进程 p2 再请求大小为 20 KB 的内存空间。画出分别采用首次适应算法、循环首次适应算法和最佳适应算法时，为进程 p1 分配完空间之后系统的空闲链和在此之后为进程 p2 分配完空间后的空闲链(当空闲分区大小 $m.size - 进程请求空间的大小 u.size > 1 KB$ 时，从空闲分区中为进程分配 $u.size$ 大小的空间，将 $m.size - u.size$ 大小的空闲空间作为新的空闲区使用)。

解答：

1) 若系统采用首次适应算法，空闲区应该按地址递增顺序排列，空闲区链如题所示。按照首次适应算法，从第一个大小合适的空闲区中划分一块给进程 p1，从起始地址为 20 KB、大小为 120 KB 的空闲区中划分 30 KB 空间分配给进程 p1。为进程 p1 分配完空间之后，空闲链变为如图 4-10 所示。

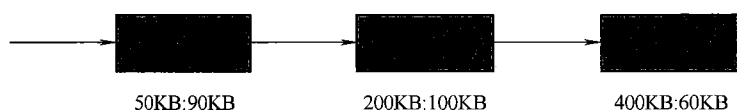


图 4-10 采用 FF 算法为 p1 分配 30 KB 空间后的空闲区链表

从这个链表中可以看到，为进程 p1 分配 30 KB 空间后，第一个起始地址为 20 KB、大小为 120 KB 的分区因给进程 p1 划分了 30 KB 空间，变为起始地址为 50 KB，大小为 90 KB 的空闲分区。若在此后，进程 p2 申请 20 KB 空间，则空闲分区链中的第一个分区大小也满足 p2 的需要，从起始地址为 50 KB、大小为 90 KB 的空闲分区中再划分 20 KB 空间分配给 p2 后，系统空闲区链变为如图 4-11 所示。

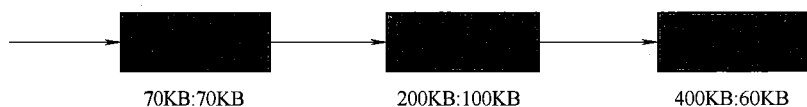


图 4-11 采用 FF 算法为 p2 分配 20 KB 空间后的空闲区链表

2) 若采用循环首次适应算法。系统维护的空闲区链表中要增加一个循环指针，每次为进程分配内存都从循环指针指向的结点对应的空闲分区开始。

初始空闲区链表为如图 4-12 所示。



图 4-12 采用 NF 算法分配空间的初始空闲区链表

进程 p1 申请 30 KB 空间后, 循环指针移到下一个结点, 下一次分配空闲区时, 从循环指针指向的结点开始搜索适合进程请求的空闲分区。系统为进程 p1 分配 30 KB 空间后, 系统空闲分区链表变为如图 4-13 所示。

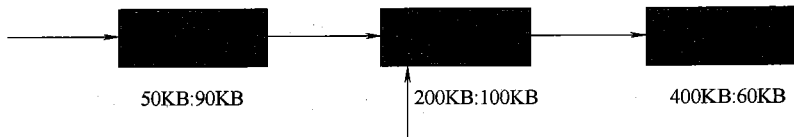


图 4-13 采用 NF 算法为 p1 分配 30 KB 空间后的空闲区链表

此后, p2 再申请 20 KB 空间, 由于循环指针指向起始地址为 200 KB、大小为 100 KB 的分区对应的结点, 因此系统从该结点开始搜索大小大于或等于 20 KB 的空闲分区, 从中划分出 20 KB 空间给进程 p2, 为进程 p2 分配 20 KB 空间后, 循环指针后移, 系统空闲分区链变为如图 4-14 所示。

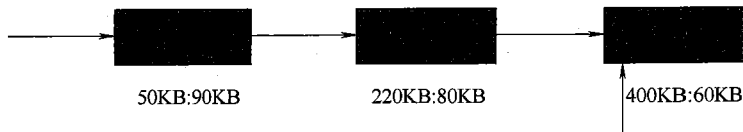


图 4-14 采用 NF 算法为 p2 分配 20 KB 空间后的空闲区链表

3) 若采用最佳适应算法。系统维护的空闲区链表中的结点要按照空闲区的大小由小到大排序, 因此在进程 p1、p2 申请内存空间之前, 空闲区链如图 4-15 所示。

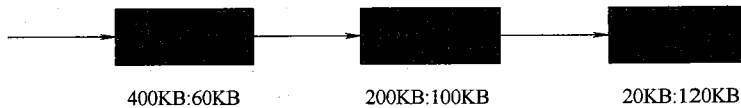


图 4-15 采用 BF 算法分配空间的初始空闲区链表

进程 p1 申请 30 KB 空间后, 空闲链变为如图 4-16 所示。



图 4-16 采用 BF 算法为 p1 分配 30 KB 空间后的空闲区链表

该链表在每一次分配空间后都要重新排序, 以保证空闲分区链是按分区大小递增的顺序排序的。

此后, 系统再为 p2 分配 20 KB 空间后, 系统空闲分区链表变为如图 4-17 所示。

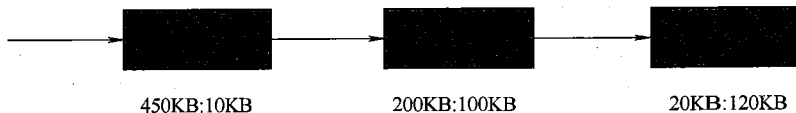


图 4-17 采用 BF 算法为 p2 分配 20 KB 空间后的空闲区链表

为了更好地说明问题,假设进程 p1 申请 30 KB 空间后, p2 申请大小为 110 KB 的空间,在图 4-16 所示的链表中查找大小合适的空闲区,只有起始地址为 20 KB、大小为 120 KB 的分区能满足进程 p2 的请求。系统从该分区中划分 110 KB 的空间给进程 p2,之后该分区剩余 10 KB 的小分区作为新的空闲分区。空闲分区链变为如图 4-18 所示。



图 4-18 采用 BF 算法为 p2 分配 110 KB 空间后的空闲区链表

对空闲分区链重新排序,得到的空闲分区链如图 4-19 所示。

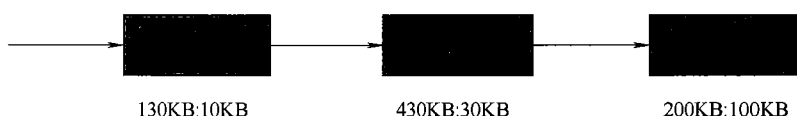


图 4-19 采用 BF 算法为 p2 分配 110 KB 空间后按分区大小排序后的空闲区链表

4. 动态分区分配的流程

采取动态分区分配的内存管理方式,内存分配功能由内存分配程序完成,内存分配程序采用某种内存分配算法为进程分配内存。内存不再被应用程序需要时,由系统调用内存回收程序回收原来被占用的内存分区。下面说明采用动态分区分配时内存分配的流程和内存回收的流程。

(1) 内存分配流程

如果进程申请的内存空间大小为 $u.size$,当前结点对应的空闲分区大小为 $m.size$, $size$ 是系统规定的一个阈值。当系统接收到申请内存的请求后,按照如图 4-20 所示的流程分配空闲分区。

1) 检索空闲分区链。找到满足条件 $m.size \geq u.size$ 的空闲区(可以采用首次适应算法、循环首次适应算法或最佳适应算法)。

2) 如果 $m.size - u.size \leq size$,则直接把该空闲分区分配给进程。

否则,从 $m.size$ 中划出大小为 $u.size$ 的空间分配给进程,把剩余的大小为 $m.size - u.size$ 的空闲空间作为新的空闲分区。

3) 将分配给进程的分区起始地址返回给内存分配程序的调用者。

4) 修改空闲分区链表。

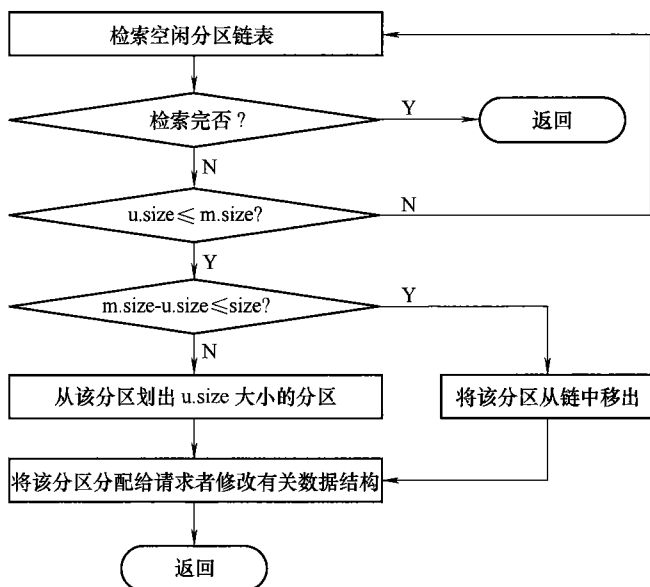


图 4-20 内存分配流程图

(2) 内存回收流程

内存回收的任务是释放被占用的内存区域, 如果被释放的内存空间与其他空闲分区在地址上相邻接, 还需要进行空间合并。

下面以采用动态分区链管理空闲分区的情况为例, 说明内存回收的流程。

- 1) 释放一块连续的内存区域。
- 2) 如果被释放区域与其他空闲区间相邻, 则合并空闲区。
- 3) 修改空闲分区链。

如果被释放的内存区域(回收区)与其他任何的空闲区都不相邻, 则为该回收区建立一个空闲区链的结点。使新建结点的起始地址字段等于回收区起始地址, 空闲分区大小字段等于回收区大小。根据内存分配程序使用的算法要求(按地址递增顺序或按空闲分区大小由小到大排序), 把新建结点插入空闲分区链的适当位置。

如果被释放区域与其他空闲区间相邻, 需要进行空间合并, 在进行空间合并时需要考虑以下 3 种情况。

1) 仅回收区的前面有相邻的空闲分区, 如图 4-21a 所示。在这种情况下, 把回收区与空闲分区 R1 合并成一个空闲分区, 把空闲链中与 R1 对应的结点的分区起始地址作为新空闲区的起始地址, 将该结点的分区大小字段修改为空闲分区 R1 与回收区大小之和。

2) 仅回收区的后面有相邻的空闲分区, 如图 4-21b 所示。在这种情况下, 把回收区与空闲分区 R2 合并成一个空闲分区。把空闲链中与 R2 对应的结点的分区起始地址改为回收区起始地址, 将该结点的分区大小字段修改为空闲分区 R2 与回收区大小之和。

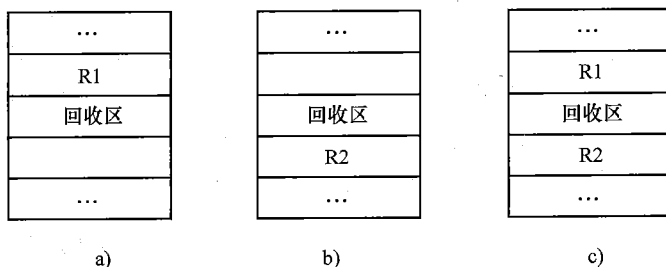


图 4-21 内存回收区与其他空闲区相邻的情况 1

3) 回收区的前、后都有相邻的空闲分区, 如图 4-21c 所示。在这种情况下, 把回收区与空闲分区 R1、R2 合并成一个空闲分区。把空闲链中与 R1 对应的结点的分区起始地址作为合并后新空闲分区的起始地址, 将该结点的分区大小字段修改为空闲分区 R1、R2 与回收区三者大小之和, 删去与 R2 分区对应的空闲分区结点。当然, 也可以修改分区 R2 对应的结点, 而删去 R1 对应的结点。还可以为新合并的空闲分区建立一个新的结点, 插入空闲分区链表, 删除 R1 和 R2 对应的分区结点。

为了进一步说明回收流程来看下面的例题。

例 4-2: 当前空闲链如图 4-22 所示。

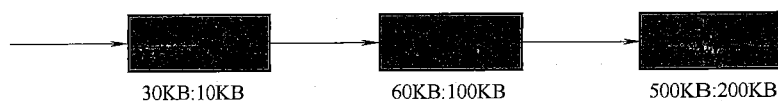


图 4-22 内存回收区与其他空闲区相邻的情况 2

空闲链结点分别对应起始地址为 30 KB, 大小为 10 KB; 起始地址为 60 KB, 大小为 100 KB; 起始地址为 500 KB, 大小为 200 KB 的空闲分区。

假如系统采用首次适应算法分配内存，请画出分别回收如表 4-3 中所示的空闲区之后的空闲链。

表 4-3 例 4-2 回收分区

回收区	起始地址	分区大小	回收区	起始地址	分区大小
F1	50 KB	5 KB	F3	50 KB	10 KB
F2	40 KB	10 KB	F4	40 KB	20 KB

解答：

当回收区是 F1 时，F1 前面的空闲区起始地址为 30 KB，大小为 10 KB，地址范围是 30 ~ 40 KB。F1 后面(高地址部分)的空闲区起始地址为 60 KB，大小为 100 KB，地址范围是 60 ~ 160 KB。而 F1 的起始地址为 50 KB，大小为 5 KB，F1 的地址范围为 50 ~ 55 KB。可见，回收区 F1 前后都没有相邻的空闲分区，因此，要为回收 F1 创建新的空闲分区链表结点，结点的起始地址字段为 F1 的起始地址 50 KB，结点的空闲分区大小字段为 F1 的大小 5 KB。按首次适应算法的要求，将该结点按空闲分区起始地址递增的顺序排序插入，得到回收 F1 后的系统空闲分区链表如图 4-23 所示。

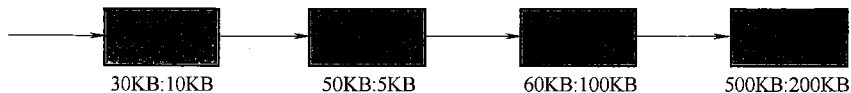


图 4-23 内存回收区与其他空闲区相邻的情况 3

当回收区是 F2 时，F2 前面的空闲区起始地址为 30 KB，大小为 10 KB，地址范围是 30 ~ 40 KB。F2 后面(高地址部分)的空闲区起始地址为 60 KB，大小为 100 KB，地址范围是 60 ~ 160 KB。而 F2 的起始地址为 40 KB，大小为 10 KB，F2 的地址范围为 40 ~ 50 KB。可见，回收区 F2 与前面起始地址为 30 KB，大小为 10 KB，地址范围是 30 ~ 40 KB 的空闲分区相邻。因此，将该空闲分区与 F2 合并，合并后的空闲区起始地址为 30 KB，大小为 20 KB，得到回收 F2 后的系统空闲分区链表如图 4-24 所示。

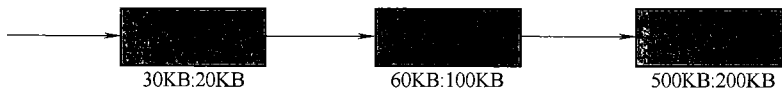


图 4-24 内存回收区与其他空闲区相邻的情况 4

当回收区是 F3 时，F3 前面的空闲区起始地址为 30 KB，大小为 10 KB，地址范围是 30 ~ 40 KB。F3 后面(高地址部分)的空闲区起始地址为 60 KB，大小为 100 KB，地址范围是 60 ~ 160 KB。而 F3 的起始地址为 50 KB，大小为 10 KB，F3 的地址范围为 50 ~ 60 KB。可见，回收区 F3 与后面起始地址为 60 KB，大小为 100 KB，地址范围是 60 ~ 160 KB 的空闲分区相邻。因此，将该空闲分区与 F3 合并，合并后的空闲区起始地址为 50 KB，大小为 110 KB。得到回收 F3 后的系统空闲分区链表如图 4-25 所示。

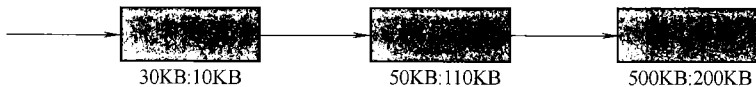


图 4-25 内存回收区与其他空闲区相邻的情况 5

当回收区是 F4 时, F4 前面的空闲区起始地址为 30 KB, 大小为 10 KB, 地址范围是 30 ~40 KB。F4 后面(高地址部分)的空闲区起始地址为 60 KB, 大小为 100 KB, 地址范围是 60 ~160 KB。而 F4 的起始地址为 40 KB, 大小为 20 KB, F4 的地址范围为 40 ~60 KB。可见, 回收区 F4 与前面起始地址为 30 KB, 大小为 10 KB, 地址范围是 30 ~40 KB 的空闲分区相邻。与后面起始地址为 60 KB, 大小为 100 KB, 地址范围是 60 ~160 KB 的空闲分区也相邻。因此, 将 F4 前后两个空闲分区与 F4 合并, 合并后的空闲区起始地址为 30 KB, 大小为 130 KB。得到回收 F4 后的系统空闲分区链表如图 4-26 所示。



图 4-26 内存回收区与其他空闲区相邻的情况 6

第四节 基本分页存储管理方式

把进程离散地存储在内存中物理地址不连续的区域中, 这种内存管理方式称为离散内存管理方式。为了支持虚拟内存管理, 需要引入离散内存管理方式。根据离散内存管理分配内存空间的基本单位的不同, 将其分为 3 种不同的管理方式: 分页存储管理、分段存储管理和段页式存储管理。

本节详细说明分页存储管理的基本原理。

一、分页存储管理的基本原理

1. 基本概念

(1) 页(Page)

将一个进程的逻辑地址空间分成若干个大小相等的片, 称为页。

(2) 页框(Page Frame)

将物理内存空间分成与页大小相同的若干个存储块, 称为页框或页帧。

(3) 分页存储

在为进程分配内存时, 以页框为单位将进程中的若干页分别装入多个可以不相邻接的页框中。如图 4-27 所示, 一个进程的逻辑地址空间包括两个页: 0 号页和 1 号页。0 号页存放在 0 号页框中, 1 号页存放在 2 号页框中。

(4) 页内碎片

进程的最后一页一般装不满一个页框, 而形成了不可利用的碎片, 称为“页内碎片”, 是一种内部碎片。

(5) 页表(Page Table)

页表是系统为进程建立的数据结构, 页表的作用是实现从页号到页框号的映射。在进程地址空间内的所有页(0 ~ n), 依次在页表中有一个页表项, 其中记录了相应页在内存中对应的页框号。如图 4-27 所示, 一个进程的逻辑地址空间划分成了两个页: 0 号页和 1 号页。0 号页存放在 0 号页框中, 1 号页存放在 2 号页框中。该进程的页表中有两个表项, 0 号表项中存放了页框号 0, 指示 0 号页存放在 0 号页框中。1 号表项中存了页框号 2, 指示 1 号页存放在 2 号页框中。在基本的分页机制中, 每个进程有一个页表, 进程的每一个页在页表中有一个对应的页表项。页表在内存中连续存放。

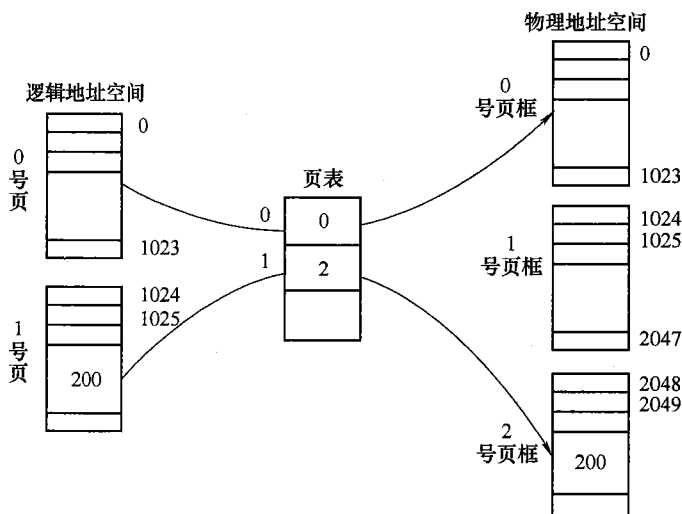


图 4-27 分页存储管理

2. 基本分页存储管理方式中的地址结构

基本分页的逻辑地址结构包含两部分：页号 P 和页内偏移量 W 。若用 m 位表示逻辑地址，页大小为 2^n 字节（如 512 字节，1024 字节），则用低 n 位表示页内偏移量 W ，用高 $m - n$ 位表示页号 P 。

以 32 位地址为例：可用 0 ~ 11 位表示页内偏移， $n = 12$ ，页大小 = 页框大小 = 4KB。12 ~ 31 位(20 位)表示页号，共可有 2^{20}

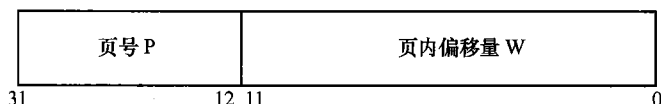


图 4-28 分页的地址结构

个页即 1 M 个页。这种地址结构可以表示 4G 的逻辑地址空间，地址结构如图 4-28 所示。

若 A 为逻辑地址， L 为页大小， P 为页号， W 为页内偏移量，则有以下计算关系。

$$P = \text{INT}(A/L)$$

$$W = \text{MOD}(A/L)$$

例 4-3：逻辑地址为十进制的 5236，系统页大小为 4 KB，该逻辑地址所在的页号 P 和页内偏移地址 W 计算如下。

$$P = \text{INT}(5236/4096) = 1$$

$$W = \text{MOD}(5236/4096) = 1140$$

在实际系统中，是根据系统设计好的页大小和地址结构，由硬件从逻辑地址 A 中分离出页号和页内偏移地址。若系统的地址结构为 32 位，页大小为 4 KB，则硬件将逻辑地址 A 的高 20 位解释为页号，低 12 位解释为页内偏移，如图 4-29 所示。

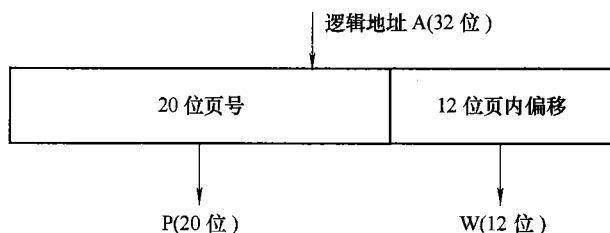


图 4-29 硬件由逻辑地址分离出页号和页内偏移

例 4-4：已知逻辑地址 A 为 0x 503200A0，若地址结构如图 4-30 所示，请问该地址对应

的页号和页内偏移地址分别是什么？

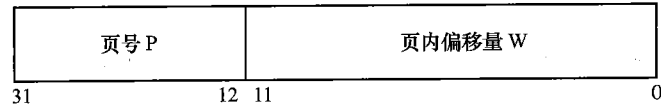


图 4-30 例 4-4 地址结构示意图

逻辑地址 A 在题中以十六进制表示，其对应的二进制地址为 0101 0000 0011 0010 0000 0000 1010 0000，高 20 位为 0101 0000 0011 0010 0000，此为页号。低 12 位为 0000 1010 0000，此为页内偏移地址。页号的十六进制数为 0x 50320，页内偏移地址的十六进制数为 0x A0。

3. 分页地址变换

为了能将用户地址空间中的逻辑地址变换为内存空间中的物理地址，在系统中必须设置地址变换机构，该机构的基本任务是实现逻辑地址到物理地址的变换。

分页地址变换原理如图 4-31 所示。

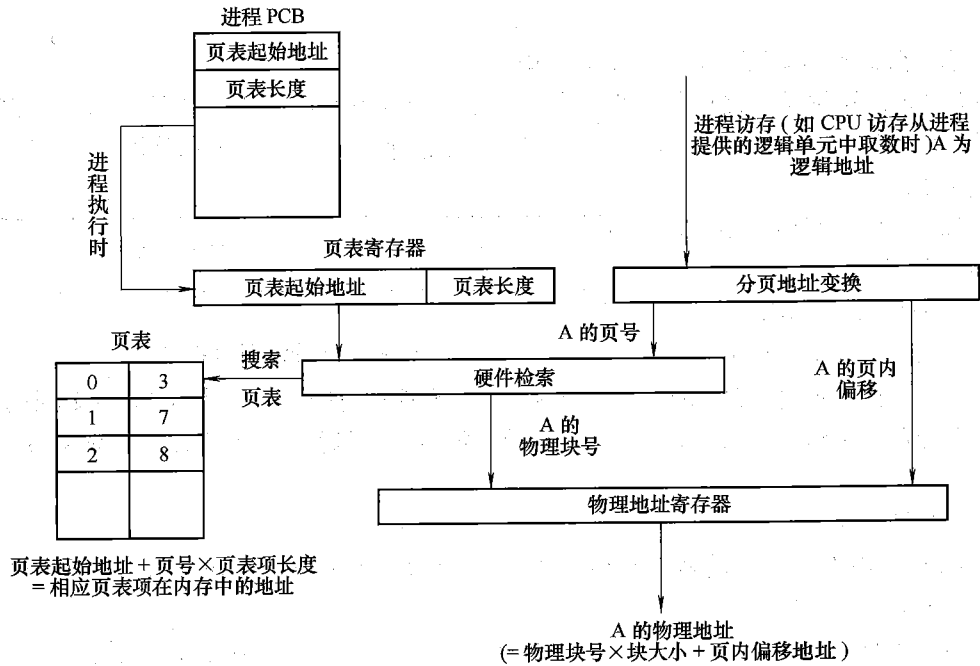


图 4-31 基本分页地址变换机构

图 4-31 所示的地址变换过程如下。

- 1) 进程执行，PCB 中页表起始地址和页表长度送 CPU 的页表寄存器。
- 2) CPU 访问逻辑单元 A。
- 3) 由分页地址变换硬件自动将 A 分为页号和页内偏移两部分。
- 4) 由硬件检索页表，得到 A 所在的页对应的页框号。

页号对应的页表项起始地址 = 页表起始地址 + 页表项长度 × 页号 (页表项中存有页框号)。从该地址指示的内存单元中读取页框号。

5) 页框号和页内偏移地址送物理地址寄存器，计算物理地址。物理地址 = 页框大小 × 页框号 + 页内偏移量。

例 4-5：在这个实例中假定页大小为 1 KB，进程的逻辑地址空间有 1026 个单元，逻辑地址为 0 ~ 1025。将逻辑地址空间划分为两个页，即 0 号页和 1 号页。逻辑单元 1025 落在 1 号页中。0 号页存放在 0 号页框中，1 号页存放在 2 号页框中。此时页表存储了这一对应关系。可以很显然地看出，逻辑单元 1025 中的内容实际上存放在物理地址为 2049 的物理单元内，如图 4-32 所示。

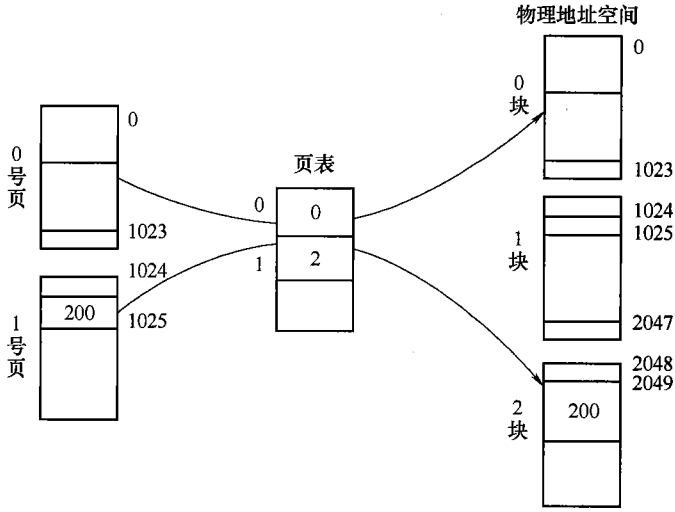


图 4-32 分页中的逻辑地址和物理地址的对应关系实例

当执行汇编指令 `Move eax, (1025)` 时，如何根据程序中给出的逻辑单元 1025 计算出对应的物理地址呢？

1) 计算页号 p 。

由题意可知，页大小为 1 KB，即 1024 个字节。

$$P = \text{INT}(A/L) = \text{INT}(1025/1024) = 1。$$

2) 搜索页表可知，1 号页存放在 2 号页框中。

3) 页内偏移地址 $W = \text{MOD}(A/L) = \text{MOD}(1025/1024) = 1。$

4) 逻辑单元 1025 所对应的物理单元地址 = 页框号 × 页框大小 + 页内偏移地址 = $2 \times 1024 + 1 = 2049。$

由图 4-32 可见逻辑单元 1025 内存放整数 200，逻辑单元 1025 相应的物理内存地址为 2049。

例 4-6：某计算机系统按字节编址，采用一级页表的分页存储管理方式，分页逻辑地址格式如图 4-33 所示。



图 4-33 例 4-6 中的分页逻辑地址格式

页表如图 4-34 所示。

请问逻辑地址 0x 0100 0001 经过地址转换后的物理地址是什么？

解答：

由题中给出的地址结构可知：硬件将逻辑地址的高 20 位解释为页号，低 12 位解释为页内偏移。逻辑地址 0x 0100 0001 的二进制表示为 0000 0001 0000 0000 0000 0000 0000 0001。

该逻辑地址的高 20 位为 0000 0001 0000 0000 0000，此为页号，其十六进制表示为 0x 01000。

该逻辑地址的低 12 位为 0000 0000 0001，此为页内偏移，其十六进制表示为 0x 001。
由页号检索页表可知该逻辑单元所在的页对应的页框号为 0x 00110。
物理地址 = 0x 00110 × 0x1000 + 0x001 = 0x00110001。

4. 页大小的选择

在分页系统中，页的大小是由机器的体系结构和操作系统共同决定的。对于某一种机器只能支持几种固定的页大小，一旦操作系统选定了一种页大小，系统中所有进程的页大小就固定不变了(例如，分页单元把低 12 位逻辑地址解释为页内偏移地址，则页大小就是 4 KB)。

在设计系统时，如何选择页大小呢？选择较小的页，可以减小页内碎片及所有进程页内碎片的总和。但较小的页会使进程的页数较多，页表较长，页表所需占用的连续内存空间比较大。而且较小、较多的页会导致进程的缺页率高，页换入、换出频繁。若选择较大的页，页内碎片会增大。为了保证系统有较好的时间和空间性能，要慎重选择大小合适的页，一般页的大小为 2 的整数次幂。在目前的计算机系统中，大多选择 4 KB 大小的页。

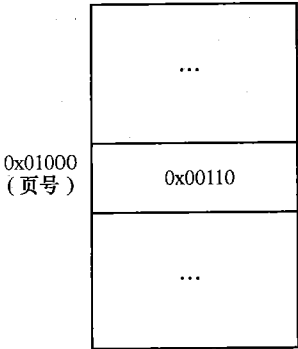


图 4-34 页表

归纳影响页大小设计的因素如下。

(1) 管理内存的开销

选择较小的页，会导致进程被划分为较多的页，页表过长，占用大量内存空间。同时，进程的缺页率和置换率都会比较高，内存管理的时间开销相对大。

(2) 内存的利用率

选择较小的页，有利于提高内存的利用率，但存在(1)所述的缺点。选择较大的页，可克服(1)所述的缺点，但页内碎片大，空间利用率降低。

页大小一般是 2 的整数次幂，大小为 512 B ~ 4 KB。现在硬件可以支持多种不同的页大小，页大小可以选择 4 KB、16 KB、2 MB、4 MB、8 MB 和 16 MB 等。如果系统支持的通常是大作业，可以选择较大的页。而支持较小的交互式应用的系统可以选择较小的页，现在的交互式系统大多选择 4 KB 的页。

二、快表

页表的硬件实现有多种方法，最为简单的一种方法就是将页表存放在一组专用寄存器中，进程调度程序选中新的进程执行时，更新这组寄存器的值，把当前进程的页表装入寄存器。操作系统使用特权级指令装入或修改页表寄存器的值。早期的 DEC PDP - 11 就采用这种结构。但是，目前使用的计算机系统都允许非常大的进程页表，一个页表中的表项可能多达上百万个，把这么大的页表存放在寄存器中显然是不合适的，所以，现在的计算机系统基本上都把页表存放在内存中。在这种情况下，CPU 要访问内存读写数据或读取指令，必须访问两次内存。第一次访问内存，从内存页表中获取访存单元所在的页框号，以形成访存单元的物理地址。第二次访存是根据计算出的物理地址实现对内存单元的访问，读写数据或读取指令。每一次的有效访存都必须要访问页表，因此访问页表的效率很重要。而两次访存都有时间开销，为了减少 CPU 在有效访存上的时间开销，提高访存速度，在硬件上引入了快表机制。

1. 什么是快表

快表也称转换后援缓冲(Translation Look-aside Buffer, TLB), 是为了提高 CPU 访存速度而采用的专用缓存, 用来存放最近被访问过的页表项。TLB 是关联的快速闪存。TLB 的条目由两部分组成: 键和值。键部分对应页号, 值部分对应页所在的页框号。当关联内存查找 TLB 中的页表项时, 会同时与所有键进行比较, 如果找到条目, 就得到相应的值域, 从而得到页的页框号。这种查找方式比较快, 但是硬件比较昂贵。通常, TLB 的条目数很有限, 在 64 ~ 1024 个之间。

2. 引入 TLB 之后的地址变换过程

引入 TLB 之后的地址变换过程可以概括为以下 3 个步骤。

- 1) CPU 产生分页的逻辑地址页号和页内偏移后, 将该逻辑地址的页号提交给 TLB。
- 2) 查找 TLB, 如果找到页号, 则把该页所在的页框号用于形成物理地址。否则(TLB 失效)查找内存页表, 从内存页表中找到相应的页表项, 读取页所在的页框号, 以形成物理地址。
- 3) 如果所查找的页表项不在 TLB 中, 在访问完内存页表后, 要把找到的页表项中的页号和页框号写到 TLB 中。如果 TLB 中的条目已满, 系统会根据某种策略(如最近最少使用替换)选择一个 TLB 中的条目, 用刚访问的页表项信息替换选中的这个 TLB 条目。

有些系统中允许 TLB 中的某些条目是固定不变的, 也就是说这些条目是永远不会被更新和替换的。这些被固定的 TLB 条目通常是与操作系统内核代码相关的条目。

带 TLB 的分页硬件如图 4-35 所示。

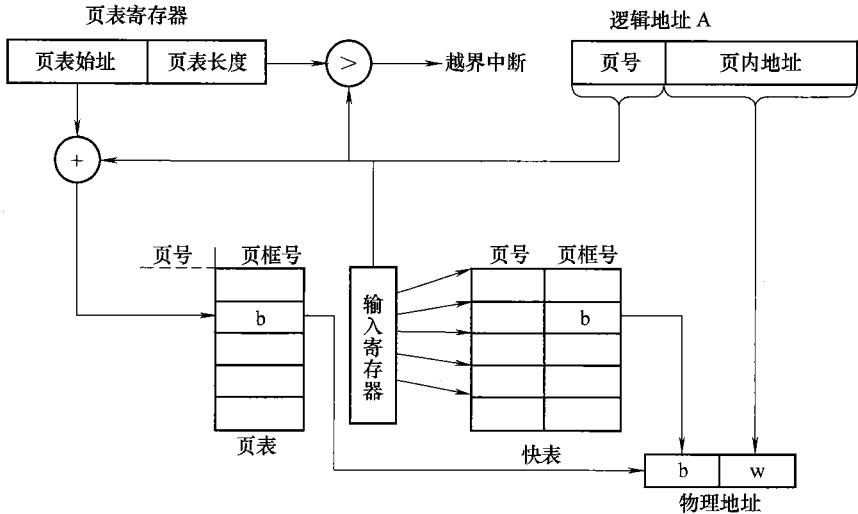


图 4-35 带 TLB 的分页硬件

使用 TLB 还有一个很重要的问题是, TLB 是系统中所有进程公用的一套硬件缓存, 如何保证在发生进程切换后, 新的当前进程不会错用了 TLB 中保存的旧进程的页表项? 下面用一个实例来说明这个问题。系统中有两个进程 p1 和 p2, 进程 p1 的 1 号页存放在第 100 号页框中, 在进程 p1 的页表中有一个相应的页表项如下。

1	100
---	-----

进程 p2 的 1 号页存放在 200 号页框中, 在进程 p2 的页表中有一个相应的页表项如下。

1	200
---	-----

当进程 p1 在 CPU 上运行时, 页表项【1, 100】在进程访问过 1 号页后会存入 TLB。当 TLB 中还存留有进程 p1 的页表项【1, 100】时, 如果发生进程切换, p2 开始在 CPU 上运行。当 CPU 访问进程 p2 的 1 号页时, 如何保证不会错误地找到【1, 100】这个条目, 而能正确地找到【1, 200】这个条目?

解决途径如下。

1) 每当进程切换时就刷新一次 TLB, 保证 TLB 中只有当前进程的页表项。

2) 在每个 TLB 页表项中保存地址空间标识符 (Address-Space Identifier, ASID)。ASID 用来唯一标识进程。当 TLB 试图命中一个页表项时, 为了确保这个页表项属于当前进程, TLB 会确保当前进程的 ASID 与被查找的 TLB 页表项的 ASID 相同。

3. 引入 TLB 的性能分析

在 TLB 中找到某一个页号对应的页表项的百分比称为 TLB 命中率。当能在 TLB 中找到所需要的页表项时, 有效访存时间等于一次访问 TLB 的时间加上一次访问内存的时间。当没有在 TLB 中找到所需要的页表项时, 访存时间等于一次访问 TLB 的时间加上两次访问内存 (一次访问内存页表, 一次访问内存读写数据或指令) 的时间。

下面以一个实例来说明引入 TLB 对改善有效访存速度所起的作用。

例 4-7: 若 CPU 访问内存的速度为 120 ns, 访问 TLB 的速度为 20 ns, 试比较有 TLB 和无 TLB 系统的平均有效访存时间。假定 TLB 的命中率为 90%。

1) 有 TLB 系统的有效访存时间 = $(120 + 120 + 20) \times 10\% + (120 + 20) \times 90\% = 152 \text{ ns}$

2) 无 TLB 系统的有效访存时间 = 两次访问内存的时间 = $120 + 120 = 240 \text{ ns}$

无 TLB 系统的有效访问时间比有 TLB 系统的有效访问时间慢 $(240 - 152)/152 = 57.9\%$ 。

三、两级和多级页表

现代计算机都支持很大的逻辑地址空间, 如 32 位系统的 4 GB, 64 位系统的 2^{24} TB。使用前面所讲的基本分页存储, 回忆以下 3 个知识点。

1) 每个进程有一个页表。

2) 每个页表至少包含与进程页数相同的页表项数。

3) 页表必须连续存放。

当系统支持的逻辑空间很大时, 意味着系统允许每个进程的逻辑地址空间很大。大进程包含的页数多, 其页表中的页表项数就多。因为一个系统的页表项长度是固定的, 所以, 页表项数越多, 页表的长度越大, 存放页表所需要的连续地址空间也越大。大多数系统都支持 32 位逻辑地址空间, 即 4 GB 的逻辑地址空间。随着计算机硬件的更新换代, 逻辑地址空间和物理内存空间都会更大。如果系统支持的页大小为 4 KB, 那么在 32 位系统中每个进程可以拥有 $4 \text{ GB} / 4 \text{ KB} = 1 \text{ M}$ 个页。相应的, 页表中有至少需要 1 M 个页表项, 如果每个页表项的长度为 4 个字节 (存放一个页表项需要 4 个字节的物理内存空间), 那么一个进程的页表需要 $1 \text{ M} \times 4 \text{ B} = 4 \text{ MB}$ 的连续物理空间来存放。从系统性能考虑, 不希望用这么大的连续地

址空间存放页表，解决的办法就是把页表再分页，形成两级或多级页表。由此，可以将页表离散地存放在物理内存中。页表离散存放，也可以在支持虚拟存储管理时，将一部分页表存放在外存中。

1. 两级页表

两级页表是将页表再进行分页，使每个页表分页的大小与内存页框的大小相同，并为它们编号。将这些页表分页分别放入不同的、不一定相邻的页框中，为离散分配的页表再建立一张外层页表，本书称之为页目录表，页目录表中的每个表项中记录了页表分页所在的页框号。

以前面系统支持大小为 4 GB 的逻辑地址空间、页大小为 4 KB 的情况为例，一个逻辑地址空间为 4 GB 的进程，拥有 1 M 个页，页表中有 1 M 个页表项，如果每个页表项的长度为 4 个字节，那么一个进程的页表需要 $1\text{ M} \times 4\text{ B} = 4\text{ MB}$ 的连续物理空间来存放。使用二级页表时，把 4 M 大小的页表分成 1024 个页表分页，这 1024 个页表分页中包含的是页表项，本书中把这种页表分页称为页表。每个页表占 4 KB 空间，连续存放。不同的页表可以离散地存放在页框号不连续的物理空间内。总结一下，大小为 4 GB 的进程，其页表有 1 K 个，每个页表大小为 4 KB，1 K 个页表离散存放，如图 4-36 所示。

(1) 两级页表的逻辑地址结构

在二级分页系统中，为了能在地址映射时得到离散存放的页表在物理内存中的地址，需要为页表再建立一个连续存放的外层页表，本书也称之为页目录表。页目录表的表项中存放了每一个页表在物理内存中所在的页框号。为了能够实现逻辑地址到物理地址的映射，必须把要访问的逻辑地址解析成如图 4-37 所示的结构。

页目录号 p_1 实际上是一个索引值，根据 p_1 从页目录表中找到页表所在的页框号。页号 p_2 是页表中的偏移量，根据 p_2 可以知道应该从页表的第 p_2 项中找到进程页所在的页框号。

根据如图 4-38 所示的实例做简单说明：如果由逻辑地址 A 得到 $p_1 = 0$ ， $p_2 = 2$ ，那么 A 所在的进程页存放在哪个页框中？

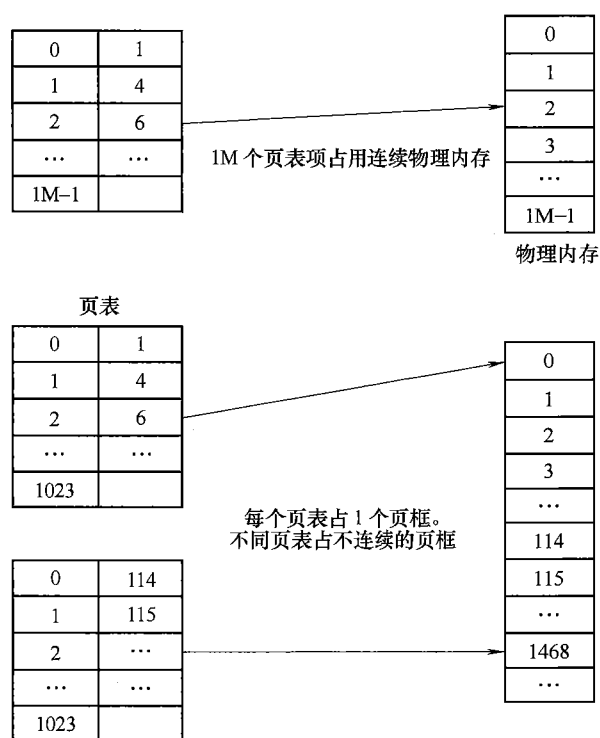


图 4-36 两级分页

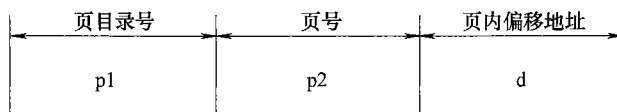


图 4-37 逻辑地址解析

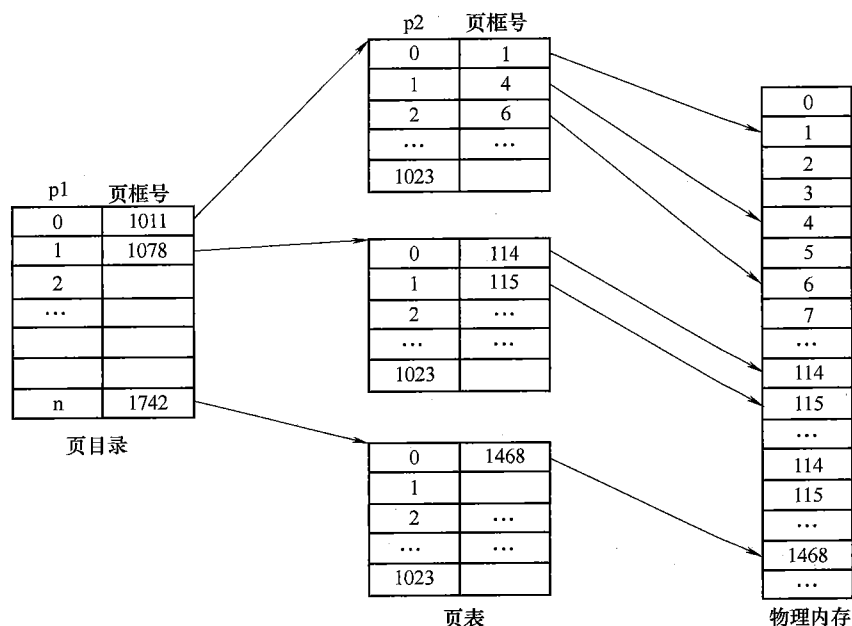


图 4-38 二级页表结构

$p1 = 0$ 时，根据 $p1$ 值在页目录表中找到 0 号表项，该表项中存放的页框号是 1011，说明 0 号页表存放在 1011 号页框中。系统根据这个数据，可以在内存中找到 0 号页表。然后根据 $p2 = 2$ ，找到 0 号页表的第 2 个表项。其中存放的页框号是 6，说明 A 所在的进程页存放在 6 号页框中。得到这个页框号后就可以计算 A 对应的物理地址了。物理地址 = A 所在的页框号 $\times$ 页框大小 + 页内偏移地址 d 。

(2) 两级页表的寻址

使用两级页表的系统，当进程切换时，要运行的进程的页目录表起始地址被写入 CPU 寄存器，可以称之为页表寄存器，地址映射的过程如下。

1) 对于给定的逻辑地址 A，由硬件从中分离出页目录号 $p1$ 、页号 $p2$ 和页内地址 d 。

2) 由页表寄存器的值和页目录号 $p1$ ，从存放页目录的页框中找到页表所在的页框号。页表所在的页框号在内存中的地址 = 页目录起始地址 + $p1 \times$ 页表项长度，从该地址指示的物理内存单元中读取页表所在的页框号

3) 由页表所在的页框号和页号 $p2$ ，从存放页表的页框中找到进程页所在的页框号。进程页所在的页框号在内存中的地址 = 页表的起始地址 + $p2 \times$ 页表项长度。

页表的起始地址 = 页表所在的页框块号 $\times$ 页框大小。

4) A 的物理地址 = 进程页所在的页框号 $\times$ 页框大小 + 页内地址 d 。

可以结合图 4-39 理解上述寻址过程。

例 4-8：某系统逻辑地址和物理地址均为 32 位，采用两级分页，页大小为 4 KB，地址结构如图 4-40 所示。

已知逻辑地址为 $0x\ 00801004$ ，2 号页表如图 4-41 所示。求其对应的物理地址。

答案：逻辑地址 $0x\ 00801004$ 的二进制地址为 0000 0000 1000 0000 0001 0000 0000 0100

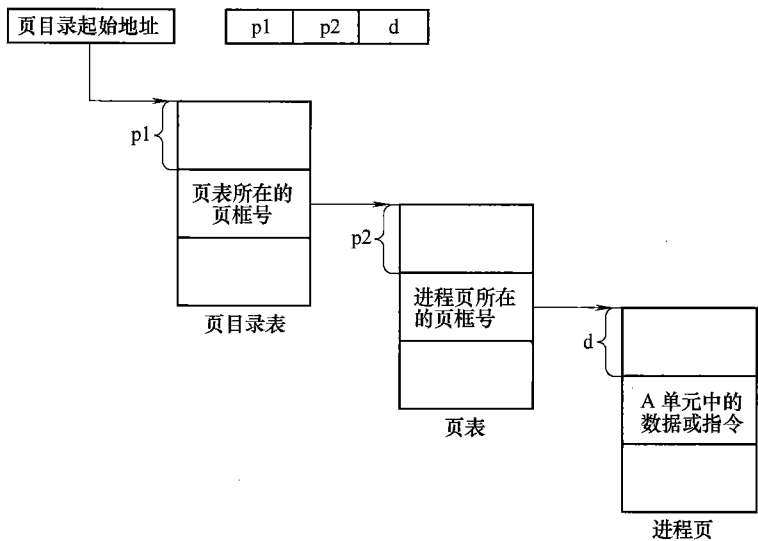


图 4-39 二级分页的寻址

页目录号 (10 位)	页表索引 (10 位)	页内偏移量 (12 位)
-------------	-------------	--------------

图 4-40 例 4-8 地址结构

由图中给出的地址结构可知，取二进制逻辑地址的高 10 位作为页目录号，得到页目录号为 0x 2。

取二进制逻辑地址的中间 10 位作为页号，得到页号为 1。

取二进制逻辑地址的低 12 位作为页内偏移，得到页内偏移值为 0x 4。

由 2 号页表可得到页号为 1 的页表项中存放的页框号为 0x 208，即进程页所在的页框号。

页号	页框号
0 × 0	0 × 10A
0 × 1	0 × 208
0 × 2	0 × 40B

图 4-41 2 号页表

逻辑地址 0x 00801004 对应的物理地址为：0x208 × 0x1000 + 0x4 = 0x208004。

(3) 减少页表占用内存的方法

为了减少页表所占内存，可以将当前所需要的页目录表和页表存放在内存中，其余页表存放在外存中，当所需页表不在内存中时，产生中断，将请求的页表调入内存。

2. 多级页表结构

对于 64 位的机器，使用二级页表，仍存在连续占用大量内存的问题，可以采用多级页表结构，将外层页表再分若干页，然后为外层页表建立连续存放的索引表。

例 4-9：64 位机器若仍然采用二级页表，页表项占 4 个字节，页大小为 4 KB，低 12 位表示页内偏移，中间 10 位表示页表项在页表中的表项号，那么外层页号就可以使用 42 位。表示系统中可以有 2^{42} 个页表分页，外层页表就有 4096 G 个页表项，需要 4 × 4096 G 字节的连续物理内存来存放外层页表。因此，需要对外层页表再进行分页，建立更外层的页表，形成多级页表。

四、反置页表

1. 反置页表

在前面所介绍的分页系统中，每个进程都有一个页表，进程逻辑地址空间的每个页在页表中都有一个相应的页表项用来存放页所在的页框号。现代系统中可能存在大量进程，每个进程都允许很大的逻辑地址空间，因而进程可能拥有一个很大的页表，这些页表会占用大量的物理内存空间。为了解决这个问题，可以使用反置页表，为每一个页框设一个表项，表项中存放进程号和页号，系统只维护一张反置页表即可。由于物理存储空间小于逻辑存储空间，所以使用反置页表减少了页表占用的内存空间。

2. 反置页表的地址映射

在利用反置页表进行地址变换时，是用进程标志符(进程号)和页号去检索反置页表以获取页框号。地址映射过程如下。

- 1) 根据进程号和页号找到页框号。
- 2) 物理地址 = 页框号 × 页框大小 + 页内偏移地址。

五、空闲页框的管理

操作系统需要跟踪和记录空闲页框的信息，下面介绍两种管理空闲页框的方式。

1. 使用位图管理空闲页框

使用位图管理空闲页框时，使位图中的每一位对应一个页框，具有 N 个页框的内存需要至少有 N 个二进制位的位图。当某个二进制位对应的页框被占用时，将该位置 1。当该页框空闲时，该位置 0。当操作系统为进程分配页框时，检索位图，找对应位为 0 的页框分配给进程(具体实现时用 0 表示空闲，1 表示被占用，反之也可)。

2. 使用空闲页框的链表

使用空闲页框链表时，系统维护记录空闲页框信息的链表。空闲页框链表可以按地址递增的顺序排序，每个结点中包含页框的地址信息、指向后面结点的指针和指向前面结点的指针。

第五节 基于分页的虚拟存储系统

虚拟存储器是指具有请求调入功能和置换功能，能从逻辑上对内存容量进行扩充的一种存储器系统。在虚拟存储器系统中，进程无需全部装入，只要装入一部分即可运行。

虚拟存储技术实现的基本思想是，只把进程的一部分装入内存。进程执行过程中，CPU 访问内存时如果发现所访问的内容不在内存中，则通过异常处理将所需要的内容从外存调入内存。也就是说先将进程的一部分装入内存，其余的部分什么时候需要，什么时候请求系统装入，这就是请求调入。如果在请求系统装入进程在外存中的某一部分时，没有足够的内存，则由操作系统选择一部分内存中的进程内容换出到外存，以腾出内存空间把当前需要装入的内容调入内存，这就是置换。

虚拟存储技术至少能带来以下 3 点好处。

- 1) 提高内存利用率。因为虚拟存储技术允许只把进程的一部分装入内存，原则上尽量

把必须或常用的部分装入内存。

2) 提高多道程序度。因为只把每个进程的一部分装入内存, 因此可以在内存中装入更多的进程。

3) 把逻辑地址空间和物理地址空间分开, 使程序员不用关心物理内存的容量对编程的限制。

虚拟存储系统具有以下几个主要特征。

1) 离散性。离散性是指进程可以分散地存储在物理内存中。分页、分段和段页式存储都属于离散的内存管理方式。离散性是实现虚拟存储管理的基础。

2) 多次性。多次性是指不必把进程一次性全部装入内存, 可以先将执行当前进程所必需的部分代码和数据装入内存, 其余部分等进程运行需要时再装入, 可以将进程分多次装入内存。

3) 对换性。对换性是指在内存中的进程可以换出, 以腾出内存空间换入外存中的进程。为了提高内存的利用率, 为程序员提供足够大的虚拟空间, 在进程运行期间, 系统可以将内存中暂时不用的程序代码或数据换出到外存, 以后需要这些程序和代码时再由系统调入内存。

4) 虚拟性。虚拟性是指虚拟存储系统为用户提供了比实际物理内存大的逻辑内存空间, 使程序员不必在编程时受物理内存空间大小的限制。虚拟性是实现虚拟存储系统的最重要的目标。

请求分页系统是最基本、最常用的虚拟存储系统的实现方式。其基本原理是, 把进程的逻辑地址空间分成大小相同的页, 操作系统创建进程时只把进程的一部分页调入内存。进程运行过程中访问内存, 若发现所访问的页不在内存中, 则产生一个缺页异常信号, 系统响应缺页异常, 请求调入缺页。若调入缺页时内存已满, 则需要先从内存中选择一个或若干个页换出到外存空间, 以腾出内存空间容纳请求调入的缺页。本节将详细说明请求分页存储管理的实现原理。

一、请求分页中的硬件支持

为了实现请求分页, 需要特殊的页表(相对于基本分页存储的页表而言)、缺页异常机构和请求分页的地址变换机构。

1. 页表

页表是支持请求分页系统最重要的数据结构, 其作用是记录描述页的各种数据, 包括在实现逻辑地址到物理地址映射时需要的页号与页框号的对应关系。除了页号和页框号之外, 页表中增加了请求换入和页置换时需要的数据。例如, 表示页是否在内存中的状态位, 以及记录页被访问情况的字段。不同的操作系统可能为页表项定义不同的字段, 在支持请求分页的系统中一般包含以下基本字段, 页表项的构成如图 4-42 所示。

页框号	状态位 P	访问字段 A	修改位 M	保护位
-----	-------	--------	-------	-----

图 4-42 支持请求分页的页表项结构

对页表项中的各字段说明如下。

1) 页框号：存放页所在的页框号。

2) 状态位 P：用来标识页是否在内存中。例如，可以规定 P 为 0 时表示页不在内存中，需要请求调页；P 为 1 时表示页在内存中，可以获取该页所在的页框号。

3) 访问字段 A：用于记录页最近被访问的情况。A 中可以存放页最近被访问的次数，也可以存放最近未被访问的时间长度。也可以简单地用 $A = 1$ 表示页最近被访问过，用 $A = 0$ 表示页最近没有被访问过。操作系统实现置换程序时需要根据 A 的值来选择被换出的页，出于效率的考虑，系统总是希望根据 A 的值把最近、最久未访问的页换出到外存。

4) 修改位 M：用于标识页最近是否被修改过。由于内存中的每个页在外存中都保存有一个副本，如果页没有被修改过，则外存中的页副本和内存中的页完全一致。因此，换出页时就不用把页信息写回外存，只需要把该页所占用的页框标识为空闲可用即可。如果页最近被修改过，那么内存中的页和外存中该页的副本就不一致了。在换出页时，必须把最近修改过的页写回外存。往外存中写信息，需要请求磁盘操作，为了减少系统开销和启动磁盘的次数，在进行页置换时，尽量选择最近没有被修改过的页。

5) 保护位：用于标识页的访问权限，比如，该位值为 1，表示页是既可读又可写的；该位为 0，表示页是只读的。

2. 缺页异常机构

缺页异常机构的主要作用是在访问内存过程中发现缺页时产生缺页异常信号，使 CPU 中断当前控制流的执行，转去执行操作系统的缺页异常处理程序，完成请求调页。具体过程如下。

1) 分页硬件通过页表完成逻辑地址到物理地址的映射时，通过检查页表中的状态位 P，判断当前被访问的页是否在内存中。如果不在，则产生缺页异常信号。

2) 执行操作系统的缺页异常处理过程。先在内存中为请求调入的页找一个空闲页框。然后，调度磁盘操作，把需要的页装入找到的空闲页框中。

3) 修改页表，更新已经调入页的存在位、在内存中的页框号、访问位和保护位等字段的值。

4) 重新开始执行因缺页而被中断的指令。

缺页异常发生时，系统保存了被中断的进程状态，包括寄存器和程序计数器的内容。所以，缺页异常处理完成返回时，可以按照与中断前完全相同的位置和地址重新开始执行进程。

由于单条指令可能涉及多个不同的页，所以，从理论上说，执行单条指令的过程中可能发生多次缺页异常。例如指令：copy A to B，指令本身可能涉及两个不同的页，A、B 两个变量也可能分别涉及不同的页，如图 4-43 所示。在这种情况下，如果该指令和 A、B 涉及的所有页都不在内存中，执行这条指令就会发生 6 次缺页异常。

3. 地址变换

请求分页系统中的地址变化过程如下。

1) 由分页地址变换机构从逻辑地址中分离出页号和

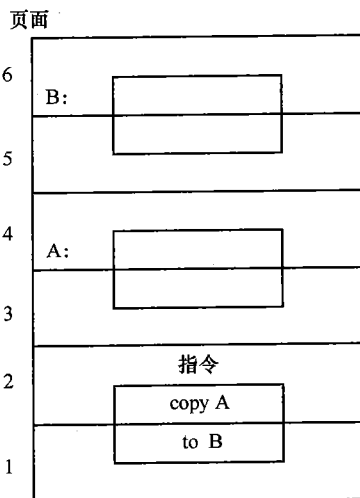


图 4-43 涉及 6 次缺页异常的指令

页内偏移地址。

2) 以页号为索引查找快表, 若快表中有该页的页表项, 则读出页框号, 计算物理地址。

3) 若快表中无该页信息, 转到内存页表中查找。若页表中的状态位 P 显示该页已调入内存, 则从相应的页表项读出页所在的页框号, 并计算物理地址, 然后把该页表项写入快表。

4) 若该页尚未调入内存, 则产生缺页异常, 请求操作系统从外存中把该页调入内存, 然后修改页表, 重新执行被中断的指令。该过程可以用图 4-44 来表示。

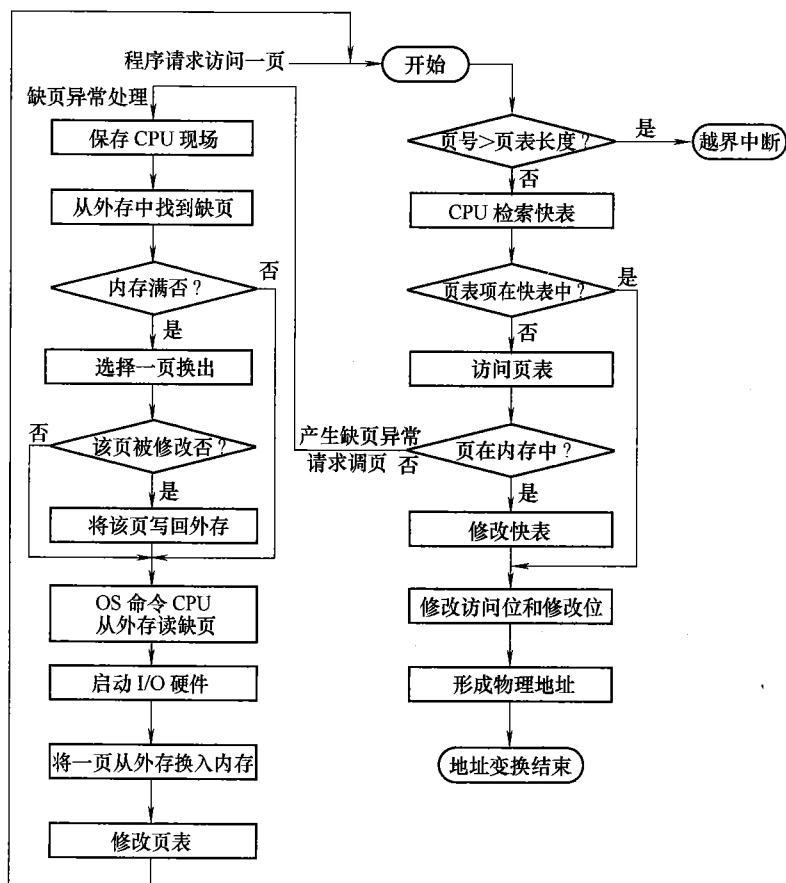


图 4-44 请求分页中的地址变换过程

二、页分配策略

本节要解决的问题是：

1) 只要装入进程的一部分页就可以运行进程, 那么究竟为进程分配多少个页框, 在内存中装入进程的多少个页呢? 我们通过阐述最小页框数, 也就是至少要为进程分配多少个页框才能使进程正常运行, 回答上述问题。

2) 操作系统在需要请求调入并进行页置换时, 是从缺页进程本身的页中选择淘汰页, 还是从系统中所有的进程页中选择被淘汰的页?

3) 采用什么样的算法为不同的进程分配页框? 系统如何在多个进程之间合理分配页

框数？

下面对上述问题进行详细阐述。

1. 最少页框数

最少页框数，是指能保证进程正常运行所需要的最少的页框数。如果系统为进程分配的页框数少于这个值，进程将无法正常运行。保证进程正常运行所需要的最少页框数与进程的大小没有关系，它与计算机的硬件结构有关，取决于指令的格式、功能和寻址方式。

例如，一个支持 16 位指令格式的机器，用高 8 位表示操作码，低 8 位表示操作数，如图 4-45 所示。

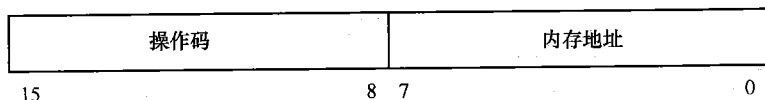


图 4-45 某机器指令格式

如果内存单元以一个 8 位字节作为一个编址单元，当采用直接寻址方式时，上面的指令本身可能涉及两个页，操作数部分的地址可能涉及第三个页，这种情况下，至少要为进程分配 3 个页框，才能保证进程正常执行。

如果采用间接寻址方式，操作数部分的地址指向的内存单元中存放的是另外一个地址，这时指令涉及的页数最多可能达到 4 个（假设地址和数据都只占一个字节），即指令本身可能涉及两个页，操作数部分的地址可能涉及第三个页，操作数地址中存放的内存地址可能涉及第四个页。这时，系统必须为进程分配至少 4 个页框才能保证进程正常执行。因为缺页异常发生在执行指令的过程中，缺页异常处理完毕，调入所缺页后，进程要重新执行原来被中断的指令。因此，必须保证这条指令涉及的全部页都在物理内存中，该指令才能被顺利执行完毕。为了保证进程顺利执行，操作系统为进程分配的初始页数应该大于或等于最少页框数。

2. 页分配和置换策略

在请求分页系统中，从分配给进程的页框数量上来看，可以采用固定分配和可变分配策略。固定分配策略是指在进程从创建到撤销的过程中，为进程分配的页框数保持不变，而可变分配是指为进程分配的页框数是可变的。从选择淘汰页的候选页是请求调入页的进程页还是系统中的所有用户进程页来看，可以采用局部置换和全局置换两种策略。局部置换是指发生置换时，只从请求调页进程本身的内存页中选择一个被淘汰的页，以腾出内存页框，装入请求调入的页。全局置换是指置换发生时，从系统中所有进程的内存页中选择被淘汰的页。固定分配和可变分配策略、局部置换和全局置换两种策略在实际系统中可以组合成以下 3 种页分配和置换策略。

(1) 固定分配局部置换

在进程创建时为每个进程分配一定数量的页框，在进程运行期间，进程拥有的页框数不再改变。当进程发生缺页时，系统从该进程在内存中的页中选择一页换出，然后再调入请求的页，以保证分配给该进程的内存空间保持不变。

(2) 可变分配全局置换

这是在操作系统中被广泛使用的策略。在采用这种策略时，先为系统中的每个进程分配

一定数量的页框，同时，操作系统保持一个空闲页框队列。当某进程发生缺页时，由系统从空闲页框队列中取出一个页框分配给该进程，并将欲调入的缺页装入其中。任何产生缺页的进程都可以由系统获得新的页框，以增加本进程在物理内存中的页数。当系统总空闲页框数小于一个规定的阈值时，操作系统会从内存中选择一些页调出，以增加系统的空闲页框数，调出的页可能是系统中任何一个进程的页。

(3) 可变分配局部置换

进程创建时，为进程分配一定数目的页框。当进程发生缺页时，只允许从该进程在内存中的页中选出一页换出。只有当进程频繁发生缺页时，操作系统才会为该进程追加页框，以装入更多的进程页，直到该进程的缺页率降低到适当程度。反之，若一个进程在运行过程中的缺页率特别低，则可在不引起进程缺页率明显增加的前提下，适当减少分配给该进程的页框数。

3. 分配算法

操作系统为进程分配的页框数通常都是大于最少页框数的，究竟为每个进程分配多少个页框，可以通过执行页框的分配算法来确定。可采用的页框分配算法如下。

(1) 平均分配算法

采用平均分配算法，如果系统中有 n 个进程， m 个可供分配的内存页框，则为每个进程分配 $\text{INT}[m/n]$ 个页框，其余的 $\text{MOD}[m/n]$ 个页框可以放入空闲页框缓冲池中。例如，系统中有 26 个空闲页框，有 3 个进程，那么为每个进程分配 8 个页框，即共为 3 个进程分配 24 个页框，剩下的 2 个页框可以作为系统可用的空闲页框放在空闲页框缓冲池中。

(2) 按比例分配算法

采用平均分配算法为进程分配页框的缺点是算法不考虑进程规模，可能使大进程分配到的页框与小进程一样多。大进程能进入内存的页数占本进程页总数的比例远远小于小进程，大进程可能因此频繁缺页。为了解决这个问题，可以采用按比例分配的算法。

为进程分配的页框数 = 进程页数 / 所有进程页数的总和 $\times$ 页框数。

例 4-10：在两个进程之间按比例分配 40 个页框，进程 p_1 大小为 80 页，另一个进程 p_2 大小为 240 页，则为 p_1 分配 10 个页框，为 p_2 分配 30 个页框。计算式如下。

为 p_1 分配的页框数 = $80 / (80 + 240) \times 40 = 10$ 。

为 p_2 分配的页框数 = $240 / (80 + 240) \times 40 = 30$ 。

(3) 考虑优先权的分配算法。

这种分配算法的思想是为优先权高的进程分配较多的页框数，为优先权低的进程分配较少的页框数。

三、页调入策略

实现请求分页的虚拟存储系统，还需要解决系统应在何时、从何处调入所需页的问题。当系统产生缺页异常，调入请求页的同时可以只把该页调入内存，也可以在调入该页的同时，把与该页相邻的页也调入内存。外存中的页可以存放于对换区，也可能在文件区。

1. 何时调入页

系统可以设计成只有在进程需要时才将页调入内存。在一种极端的情况下，进程的所有页都不在内存中就开始执行进程。当操作系统将指令指针指向进程的第一条指令时，由于所

在的页不在内存中，立即产生缺页异常。当完成缺页异常处理，指令所在的页调入内存后，进程继续执行，执行过程中不断发生缺页异常，并不断将所缺页调入内存。当调入足够多的页后，多数需要的页都在内存中了，进程执行将不再出现缺页。这种策略有利于提高内存的利用率，但是对系统的时间性能不利。

大多数系统都采用预先调入页的策略，将预计不久之后会被访问的页预先调入内存，而不是缺哪一页时再调入该页。在实际系统中，通常是在调入缺页时，把与所缺页相邻的若干页也调入内存。

2. 从何处调入页

(1) 从对换区调入

当系统拥有足够的对换空间时，若发生缺页请求，则从对换区调入页。从对换区调入页比从文件区调入页的速度快。对换区中的页是进程运行前从文件区复制到对换区的。

(2) UNIX 方式

进程运行前，与进程有关的程序代码、数据等都存放在文件区，所以，没有被访问过的页都直接从文件区调入。换出页都存放在对换区，曾经运行过而又被换出的页从对换区调入。对于可以共享的页，如果已经由于其他进程的需要而被调入内存，就无需再次从外存调入。

3. 页调入过程

请求调页和页调入的过程如图 4-46 所示。

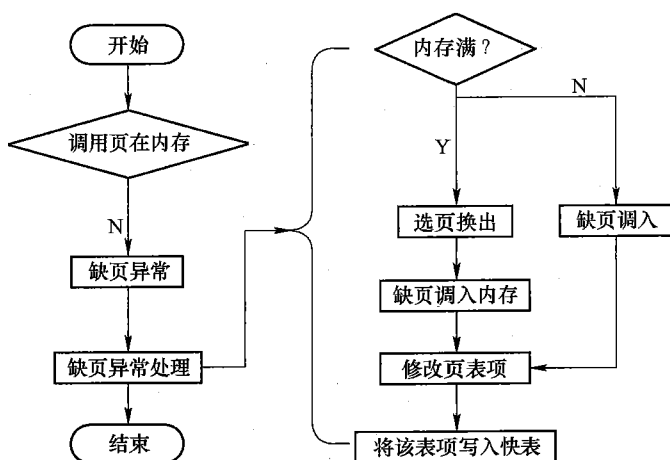


图 4-46 页调入过程

四、页置算法

本小节主要阐述页置算法，同时介绍抖动及缺页率的相关问题。页置算法是从内存页中选择换出页的算法。

1. 最佳置换算法和先进先出置换算法

(1) 最佳置换算法 (Optimal page-Replacement Algorithm)

最佳置换算法是 Belady 于 1966 年提出的一种页置换算法，该算法选择以后永远不会被访问的页或者在未来最长时间不再被访问的页作为换出页。因此，该算法主要用于理论研究。例如，如果知道一个算法不是最优，但是与最优相比最坏情况下的缺页率比最佳置换算法最多高 10%，平均缺页率与最佳置换算法相比最多高 5%，可以此说明该置换算法的性能。

假定系统为某进程分配了 3 个页框，并考虑有以下的页引用序列。

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

进程运行时，先将 7、0、1 这 3 个页装入内存。当进程访问 2 号页时，由于 2 号页不在内存中，将产生缺页异常，请求调入 2 号页。由于 3 个页框已满，需要选择一个换出页，以腾出空闲页框装入 2 号页。若操作系统根据最佳置换算法选择被换出的页，由于 7 号页是未

来最长时间不会被访问的页，所以选择7号页作为换出页，然后将2号页调入内存。访问完2号页后，访问0号页，由于该页已在内存中，所以不会产生缺页异常。当进程访问3号页时，因为3号页不在内存中，又要产生缺页异常，并且由于内存满，又要执行页置换算法选择换出页。根据最佳置换算法，这次选择2、0、1这3个页中的1号页换出，因为1号页是未来最长时间不会被访问的页，所以换出1号页，调入3号页。图4-47所示是采用最佳置换算法的置换图。由图可见，在此例中，采用最佳置换算法发生了6次页置换。

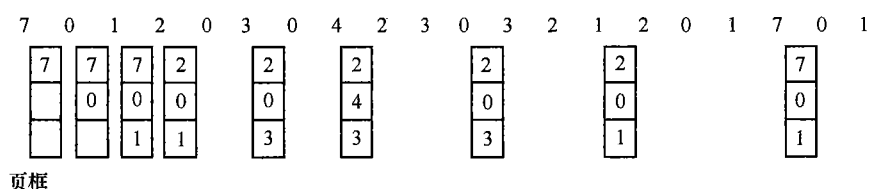


图 4-47 采用最佳置换算法时的置换图

(2) 先进先出页置换算法 (First In First Out, FIFO)

FIFO 是最简单的页置换算法。实现这种算法的一种方式是为每个页记录该页调入内存的时间，当选择换出页时，选择进入内存时间最早的页。最简单的实现方法是创建一个 FIFO 的队列来管理内存中的所有页，选择队首的页作为换出页。新调入的页被加入到队尾。

FIFO 算法实现简单，但是效率较低，会导致较高的缺页率。因为 FIFO 算法不考虑页被引用的频繁程度，例如有的页刚被换出，可能立刻又要被访问。在进程中有些页经常被访问，如包含全局变量、常用函数等的页，FIFO 算法不能保证这些经常被访问的页不被换出。

仍然采用上面的例子来说明采用 FIFO 算法进行页置换的过程。如图 4-48 所示，当进程第一次访问 2 号页时，将把 7 号页换出，因为它是最早被调入内存的。访问 0 号页时，因为 0 号页已经在内存中，所以不会产生缺页异常，也不会发生页置换。当第一次访问 3 号页时，因为内存中的 3 个页 2、0、1 中，0 号页是最早被调入内存中的，所以选择 0 号页作为淘汰页。由图 4-48 可见，在该例中，利用 FIFO 算法时，进行了 12 次页置换，比最佳置换算法的置换次数高一倍。

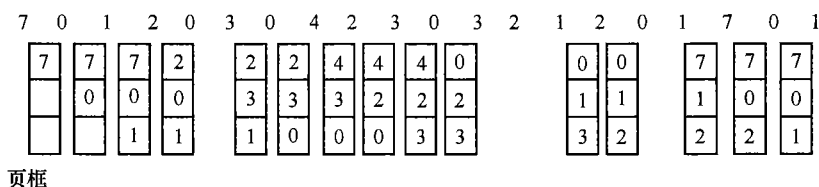


图 4-48 采用 FIFO 算法时的置换图

2. 最近最久未使用 LRU 置换算法

FIFO 算法实现简单，但是导致很高的缺页率和置换次数，性能较差。而最佳置换算法有最低的页置换次数，性能很好，却难以实现。因此，这里试图用“最近的历史”来预测“最近的将来”，实现最佳置换算法的近似算法。LRU 算法是广泛使用的性能较好的算法。

(1) LRU (Least Recently Used, LRU) 算法的描述

LRU 置换算法是选择最近最久未使用的页换出（最近最久未使用的页在最近的将来被访问的可能性也比较小）。该算法赋予每个页一个访问字段，用来记录一个页自上次被访问以来所经历的时间 t 。当需要淘汰一个页时，选择现有页中 t 值最大的页换出。

仍然采用上面的例子来说明采用 LRU 算法进行页置换的过程。当进程第一次访问 2 号页时，由于内存满，需要从 7、0、1 这 3 个页中选择一页换出，由于 7 号页是最近最久没有被访问的页，所以选择 7 号页换出。当进程第二次访问 0 号页时，由于 0 号页已在内存中，所以不会发生缺页异常和页置换。当进程第一次访问 3 号页时，内存满，要从在内存中的 3 个页 2、0、1 中选择一个页换出，由于 1 号页是这 3 个页中最近最久没有被访问的页，所以换出 1 号页，以装入 3 号页。由图 4-49 所示的结果可见，对该访问序列采用 LRU 置换算法发生了 9 次置换，性能优于 FIFO 算法，但较最佳置换算法差。

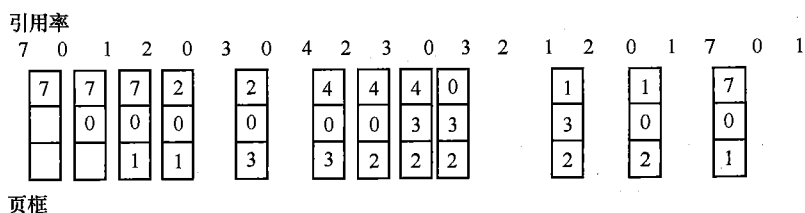


图 4-49 采用 LRU 算法时的置换图

(2) LRU 算法的实现

LRU 算法是常用的页置换算法，性能较好，也有较好的实现方法。其实现主要解决两个问题，一是内存中的各个页各有多久时间未被进程访问；二是如何快速地知道哪一页是最近最久未使用的页。其可行的实现方法如下。

1) 寄存器。

为每个在内存中的页配置一个移位寄存器，可表示为 $R = R_{n-1} \cdots R_2 R_1 R_0$ 。当进程访问某页框时，要将相应的寄存器的最高位置成 1。此时，每隔一定时间将寄存器右移一位。如果把 n 位寄存器的数看作是一个页对应的整数，那么具有最小数值的寄存器所对应的页就是最近最久未使用的页。图 4-50 所示的例子说明了一个具有 8 个页的进程，通过为每个页配备一个 8 位寄存器实现 LRU 算法的情况。在这个例子中，3 号页的 8 位寄存器值最小。所以发生缺页、需要置换时，选择 3 号页换出。图中的实页是指在内存中的页。

实页 \ R	R7	R6	R5	R4	R3	R2	R1	R0
1	0	1	0	1	0	0	1	0
2	1	0	1	0	1	1	0	0
3	0	0	0	0	0	1	0	0
4	0	1	1	0	1	0	1	1
5	1	1	0	1	0	1	1	0
6	0	0	1	0	1	0	1	1
7	0	0	0	0	0	1	1	1
8	0	1	1	0	1	1	0	1

图 4-50 用寄存器实现 LRU 算法的示例

2) 栈。

可利用一个特殊的栈来保存当前使用的各个页的页号。每当进程访问某页时，便将该页的页号从栈中移出，将它压入栈顶。因此，栈顶始终是最新被访问页的编号，而栈底则是最近最久未使用的页号。

图 4-51 所示的例子中，页访问序列为 4, 7, 0, 7, 1, 0, 1, 2, 1, 2, 6，在访问 6 号页时发生缺页。此时栈底的页为 4，是最近最久没有被访问的页，将 4 号页换出，调入 6 号页。

4	7	0	7	1	0	1	2	1	2	6
							2	1	2	6
				1	0	1	1	2	1	2
		0	7	7	1	0	0	0	0	1
	7	7	0	0	7	7	7	7	7	0
4	4	4	4	4	4	4	4	4	4	7

图 4-51 用栈实现 LRU 算法的示例

3) 计数器。

为每个页表项增加一个时间字段，并为 CPU 增加一个逻辑时钟或计数器。每次访问内存中的某个页时，就增加这个页对应的页表项的时间字段的值，每次置换时选择时间字段值最小的页作为换出页。

3. 其他置换算法

计算机系统要提供足够的硬件来支持 LRU 算法是比较困难的，许多系统在实现时都采用 LRU 的近似算法，如附加引用位算法、简单 Clock 算法和改进型 Clock 算法等。

(1) 附加引用位算法

采用这种算法时，为每个内存中的页增加一个 8 位的字节，每隔一段时间（如 100 ms），操作系统把每个页自己的访问位（请求分页管理的页表项中的 A 字段）移到 8 字节附加引用位的最高位，将其他位向右移，并抛弃最低位。进行页置换时，选择附加引用位值最小的页换出。因为具有最小附加引用位值的页可能不止一个，既可以换出所有这些页，也可以按 FIFO 选择部分具有最小附加引用位值的页换出。

(2) 简单 Clock 置换算法

利用简单 Clock 算法时，为每一页设置一位访问位，再将内存中的所有页都通过链接指针链接成一个循环队列。当某页被访问时，其访问位被置 1。置换算法在选择一页换出时，按照 FIFO 算法，检查各页的访问位。如果是 0，就选择该页换出；若为 1，则重新将它置 0，暂不换出而给该页第二次驻留内存的机会，再按照 FIFO 算法检查下一个页。

当检查到队列中的最后一个页时，若其访问位仍为 1，则再返回到队首去重新从第一个页开始检查。

(3) 改进型 Clock 算法

简单 Clock 算法在选择一个换出页时，不考虑该页被修改的情况，而选择最近既没有被访问过又没有被修改过的页换出，能大大提高页置换的效率。在将一个页换出时，如果该页

已被修改过，便应将它重新写到磁盘上，但如果该页未被修改过，则不必将它写回磁盘。换言之，修改过的页在换出时所付出的时间开销将比未修改过的页开销大。在改进型 Clock 算法中，除了必须考虑页的使用情况外，还考虑了置换代价这一因素，这样选择换出页时，既要是未使用过的页，又要是未被修改过的页。把同时满足这两个条件的页作为首选的淘汰页。改进型 Clock 算法描述如下。

在该算法中，由访问位 A 和修改位 M 可以组合成下面 4 种类型的页。

- 1) 第 1 类 ($A=0, M=0$)。表示该页最近既未被访问，又未被修改，是最佳换出页。
- 2) 第 2 类 ($A=0, M=1$)。表示该页最近未被访问，但已被修改，并不是很好的换出页。
- 3) 第 3 类 ($A=1, M=0$)。最近已被访问，但未被修改，该页有可能再被访问。
- 4) 第 4 类 ($A=1, M=1$)。最近已被访问且被修改，该页可能再被访问。

改进型 Clock 置换算法可分为以下 3 步。

1) 从指针所指示的当前位置开始，扫描循环队列，寻找 $A=0$ 且 $M=0$ 的第一类页。将所遇到的第一个页作为所选中的换出页。在第一次扫描期间不改变访问位 A 。

2) 如果第一步失败，即遍历页的链表后未遇到第一类页，则开始第二轮扫描。寻找 $A=0$ 且 $M=1$ 的第二类页，将所遇到的第一个这类页作为换出页。在第二轮扫描期间，将所有经过的页的访问位 A 置 0。

3) 如果第二步也失败，即未找到第二类页，则将指针返回到开始的位置。然后再次寻找 $A=0$ 且 $M=0$ 的第一类页，如果仍失败，再寻找 $A=0$ 且 $M=1$ 的页，此时就一定能找到被换出的页。

改进型 Clock 置换算法最坏情况下，需 4 次扫描页队列，才能选中被换出的页。

(4) 最少使用置换算法 (Least Frequently Used Page-Replacement Algorithm, LFU)

这种算法选择最近时期内使用次数最少的页作为淘汰页。这种算法为每个页保留一个用于记录页被访问次数的计数器，每次选择其访问计数器值最小的页换出。这种算法可能存在的问题是：有些页在进程开始时被访问的次数很多，但以后这些页不再被访问，这些页不应该长时间留在内存中，而应该在不再被访问时被换出。解决这个问题的方法之一是定期地将计数器右移，以形成指数衰减的平均使用次数。

(5) 页缓冲算法

页缓冲算法采用 FIFO 算法选择换出页，并为换出页建立两个链表，被换出的页要放入两个链表中的一个。如果页没有被修改过，则将其放入空闲页链表中。如果页被修改过，则将其放入已修改页的链表中。页在内存中并不做物理上的移动，而只是将页表中的表项移到上述两个链表之一中。

空闲页链表实际上是一个空闲页框链表，其中的每个页框都是空闲的，因此可以在其中装入程序或者数据。当需要读入一个页时，便可利用空闲页框链表中的第一个页框来装入该页。当有一个没有被修改的页要换出时，实际上并不将它换出内存，而是把该页所在的页框挂在空闲页链表的末尾。在置换一个已修改的页时，将其所在的页框挂在修改页链表的末尾。采用这种方式可以使没有被修改的页和已经被修改的页都留在内存中。当进程以后再次需要访问这些页时，就不用从磁盘而只需从内存中再次使这些页回到进程在内存的驻留集中，使这些页所在的页框成为“被占用”状态即可，由此减小了请求调入和页置换的开销。

已修改页的链表周期性地或者当已修改页的数量达到一定阈值时被延迟写入磁盘，减少了换出时写磁盘的时间开销。

五、请求分页系统的性能分析

前面已经阐述了基于请求分页的存储管理的基本原理，在设计和选择虚拟存储管理的策略和算法时，不可避免地要考虑访存的性能。请求调入和置换技术都是以时间换空间的技术，如何在应用这种技术时尽量提高时间性能是在设计系统时需要考虑的问题。

1. 缺页率对有效访问时间的影响

进程执行中访存发生缺页时，需要请求从外存调入缺页。如果内存中没有空闲页框，还需要进行页置换，调入缺页后，指令需要重新执行。因此，一旦发生缺页，进程会存在因为访存而带来的时间延迟。下面分析缺页率对有效访问时间的影响。

有效访问时间是成功访存所用的时间。假设 P 为缺页率， m_a 为存储器访问时间。这里根据实际系统的可能性，取 $m_a = 100 \text{ ns} = 0.1 \mu\text{s}$ 。缺页异常服务时间、缺页读入时间及进程重新执行时间近似地取 25ms 是合理的。由此：

有效访问时间 = $(1 - P) \times m_a + P \times \text{缺页异常时间}$ 。

缺页异常时间 = 缺页异常服务时间 + 缺页读入时间 + 进程重新执行时间
 $\approx 25 \text{ ms (毫秒)} = 25000 \mu\text{s (微秒)}$

所以，有效访问时间 = $0.1 \times (1 - P) + 25000 \times P$
 $= 0.1 + 24999.9 \times P$

由上式可见，有效访问时间正比于缺页率。下面以实例数据说明缺页率对有效访问时间的的影响。

例 4-11:

$P = 0$ 时 有效访问时间 = $0.1 \mu\text{s}$

$P = 1/1000$ 时 有效访问时间 = $0.1 + 24999.9 \times 1/1000$
 $= 0.1 + 24.9999 \approx 25 \mu\text{s}$

可见，缺页率 $P = 1/1000$ 时，有效访问时间延长为 $P = 0$ 时的 250 倍。

例 4-12: 要使有效访问时间延长不超过 10%，缺页率 P 为多少？

因为 $P = 0$ 时，有效访问时间 = $0.1 \mu\text{s}$ 。

P 不为 0 时，有效访问时间为 $(0.1 + 24999.9P) \mu\text{s}$ 。

P 不为 0 时比 P 为 0 延长的时间为： $(0.1 + 24999.9P) - 0.1 = 24999.9P$

要使： $\frac{24999.9P}{0.1} < 10\%$ 必须使 $P < 0.000.0004$

即使有效访问时间延长不超过 10%，缺页率应该小于 250 万分之一。

2. 工作集

(1) 基本原理

引入工作集机制是为了能有效降低缺页率，从而提高访存的时间效率。程序在运行时对页的访问是不均匀的，即往往在某段时间内的访问仅局限于较少的若干个页，而在另一段时间内，则又可能仅局限于访问另一些较少的页。若能预知程序在某段时间间隔内要访问哪些页，并能将它们提前调入内存，将会大大降低缺页率，从而减少置换次数，提高 CPU 的利

用率。工作集就是在某段时间间隔 Δ 里，进程实际要访问的页的集合。要使缺页少发生，必须使程序的工作集全部在内存中。

用 $W(t, \Delta)$ 表示当时间间隔为 Δ 时 t 时刻的工作集。

1) Δ 太大，工作集中的页太多，影响存储器利用率。

2) Δ 太小，工作集中的页太少，缺页率高。

3) $W(t, \Delta)$ 与 t 相关，不同的 t 对应不同的工作集。即程序运行的过程中，运行到不同的时刻，需要访问的页不同。

(2) Windows NT 的工作集机制

1) 进程被第一次创建时，为进程指定了一个最小工作集和一个最大工作集。最小工作集是进程正常运行需要调入内存的最小页数。当内存有足够的空间时，可以为进程分配足够的空间以装入它的最大工作集。

2) 当某进程在内存中的页数小于最大工作集时，若发生缺页，系统从空闲页框队列中取一页框分配给该进程。

3) 当某进程在内存中的页数等于最大工作集时，若发生缺页，系统从该进程的页中按 FIFO 的原则，换出一个该进程的页。

4) 当空闲页框队列中空闲页的数量减到一个最低值时，系统检查所有进程，对工作集大于最小工作集的进程，淘汰该进程的一些页，使该进程的工作集等于最小工作集。

3. 抖动产生的原因和预防方法

(1) 抖动

多道程序度太高，使运行进程的大部分时间都用于进行页的换入、换出，而几乎不能完成任何有效工作的状态称为抖动。引起系统抖动的主要原因是系统中的进程数量太多，每个进程能分配到的页框太少，以至于进程运行过程中频繁请求调页。

(2) 抖动的预防

可以采用以下方法预防抖动的发生。

1) 采取局部置换策略。当进程发现缺页后，仅在进程自己的内存空间范围内置换页，不允许从其他进程获得新的页框。

2) 在 CPU 调度程序中引入工作集算法。只有当每个进程在内存中都有足够大的驻留集时，才能再从外存中调入新的作业。

3) 挂起若干进程。为了预防抖动，挂起若干进程，腾出进程占用的空间。

第六节 分段存储管理

一、分段机制的引入

在分页存储管理方式中，逻辑地址是一维的，一个进程的逻辑地址形成一个地址空间。这种逻辑空间的使用方式对于程序员来说是很不方便的，这里希望能为进程提供多个地址空间，把逻辑上关联的部分放在一个地址空间中，逻辑上没有关联也没有共同特征的部分放在不同的地址空间中。这样不仅能方便用户编程，而且能很好地解决部分存储空间动态增长、信息共享和信息保护的问题。

把分别存放逻辑上相关的信息、相互独立的逻辑地址空间称为一个段，每个段由一个从 0 到最大线性地址的逻辑地址空间构成。各个段的长度可以是 0 到最大值之间的任何一个值，不同段的长度可以不相同，段的长度在进程运行期间可以改变。

在使用分段存储管理的系统中，程序员使用二维的逻辑地址，一个数用来表示段，另一个数用来表示段内偏移。段是一个逻辑实体，程序员可以通过使用二维地址来访问不同的逻辑段。一个段可能包括一个过程，或者一个数组、一个堆栈、一些数值变量，但是一般不会同时包含多种不同的内容。

简单来说，引入分段机制的优点是方便编程、分段共享、分段保护、动态链接，以及存储空间的动态增长。

二、分段系统的基本原理

1. 分段

在分段的存储管理方式中，进程的地址空间被划分成若干个段。每个段定义了一组逻辑信息，每个段的大小由相应的逻辑信息组的长度确定，段的大小不一样，每个段的逻辑地址从 0 开始，采用一段连续的地址空间。系统为每个段分配一个连续的物理内存区域，各个不同的段可以离散地放入物理内存不同的区域。系统为每个进程建立一张段表，段表的每一个表项记录的信息包括段号、段长和该段的基址，段表存放在内存中。

2. 分段的逻辑地址结构

分段机制的逻辑地址是二维的，由段号和段内地址组成。程序员采用二维地址使程序的可理解性更强，也方便编程。32 位系统中分段的地址形式如图 4-52 所示。

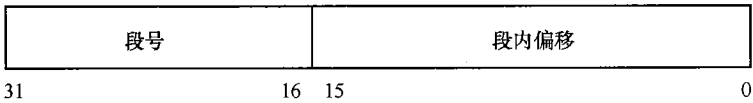


图 4-52 32 位系统中分段地址形式

3. 段表

段表是由操作系统维护的用于支持分段存储管理地址映射的数据结构。通常情况下，每个进程有一个段表，段表由段表项构成。每个段表项包含段号、段基址（段的起始地址）和段长（即段大小）3 个部分。一个进程可能包含若干个段，每一个段在段表中有一个段表项与之对应。如表 4-4 所示的段表，表示某进程分为 3 个段：0 号段的段长为 30 KB，段基址为 40 KB；1 号段的段长为 20 KB，段基址为 80 KB；2 号段的段长为 15 KB，段基址为 120 KB。

表 4-4 段表

段 号	段 长	段 基 址
0	30 KB	40 KB
1	20 KB	80 KB
2	15 KB	120 KB

在早期的 16 位计算机系统中，大多数情况下，段基址就是段在物理内存中的起始地址，段大小也称段界限，本书对分段原理的描述基于这个前提。根据段表可以知道一个段在物理内存中的位置，如图 4-53 所示。

4. 分段系统的地址变换

如图 4-54 所示，已知的逻辑地址由段号 s 和段内偏移 d 构成。CPU 上段表寄存器中存有当前在 CPU 上运行进程的段表起始地址。段号用作段表的索引，根据段表起始地址和段号可以获得存放段 s 的段表项的内存地址。

s 号段表项在内存中的起始地址 = 段表起始地址 + 段号 $s \times$ 段表项长度

段表项长度是指一个段表项所占用的字节数。

从相应段表项起始地址内存处可读取 s 段的段表项，获得 s 段的基地址和段界限。如果 $d \leq$ 段界限，在最简单的情况下，用 s 段基址加上段内偏移 d 就可得到已知逻辑单元的物理地址。

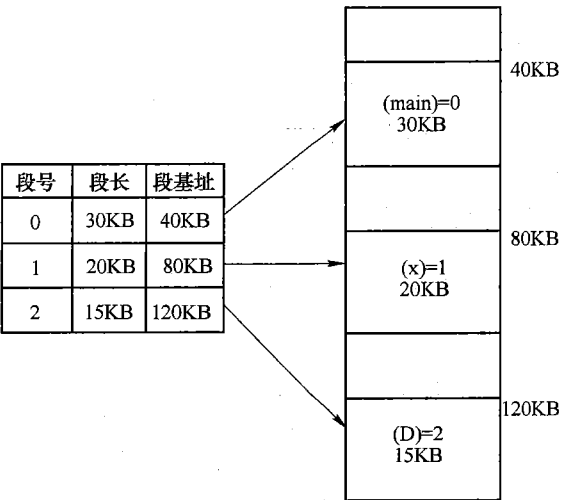


图 4-53 段表与段在内存中的位置

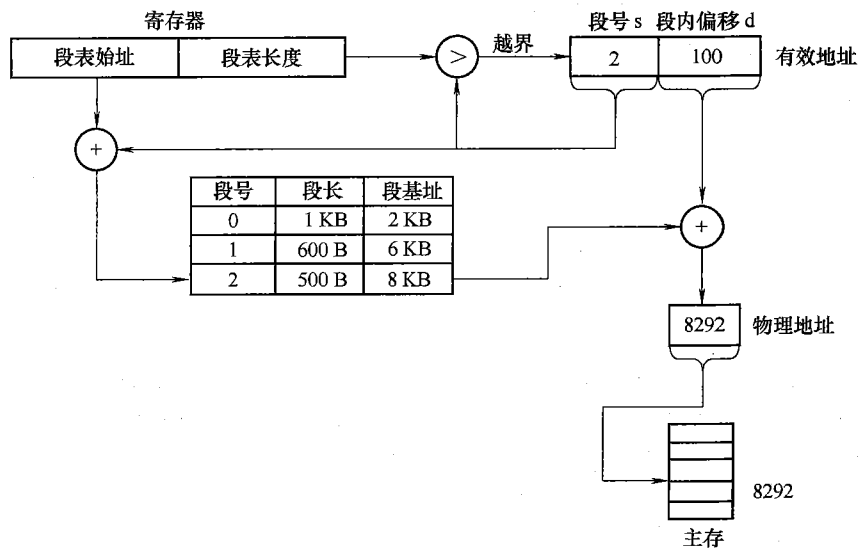


图 4-54 分段系统的地址变换

若已知逻辑单元的地址为 $s:d$ ，求相应物理地址的步骤如下。

- 1) 以段号 s 作索引，从段表中找到段号为 s 的段表项。
- 2) 从找到的段表项中读出 s 段的基地址和段大小。
- 3) 如果 $d \leq$ 段大小，则将段基址与段内偏移 d 相加，得到与逻辑单元 $s:d$ 相应的物理单元地址。

以图 4-54 为例, 已知逻辑单元的段号为 2, 段内偏移地址为 100 B (简单表示为 2:100)。根据段号 2 查找段表, 段号为 2 的段表项中存放了基地址 8 KB 和段大小 500 B (500 字节), 逻辑单元 2:100 对应的物理单元地址为 $8 \text{ KB} + 100$, 即 $8 \times 1024 + 100 = 8292$ 。

5. 分页和分段的主要区别

分页和分段都属于离散分配方式, 都要通过数据结构与硬件的配合来实现逻辑地址到物理地址的映射, 但两者存在着很大的不同, 主要区别如下。

1) 页是按物理单位划分的, 分页的引入是为了提高内存的利用率和支持虚拟存储。而段是按逻辑单位划分的, 一个段含有一组意义相对完整的信息。引入分段的目的是为了更方便程序员编程。

2) 页的大小是固定的。而段的大小不固定, 取决于用户编写的程序和编译器。

3) 分页的地址空间是一维的, 程序员给出的地址只是一个助记符, 已知的逻辑地址是一个数, 如 2568。分段的地址空间是二维的, 程序员在标识一个逻辑地址时需要给出两个数: 一个是段号, 一个是段内偏移。

6. 信息共享

采用分段机制比采用分页机制更容易实现信息的共享。下面举例说明, 例如有一个多用户系统, 同时有 10 个用户在执行同一个文本编辑程序。如果该文本编辑程序的代码占 160 KB 空间, 数据区占 40 KB 空间。通常每个用户执行的文本编辑程序代码是可重入的代码, 是被多个用户共享的同一个程序副本。在采用分页机制的系统中, 假定页大小为 4 KB, 那么文本编辑程序代码要占用 40 个页, 假设这 40 个页被存放在 21 ~ 60 号页框中。10 个用户进程有 10 个页表, 每个页表中都有 40 个页表项与文本编辑程序代码页对应。为共享文本编辑程序, 每个进程页表的这 40 个页表项都存放 21 ~ 60 号页框中的一个页框号。10 个进程要用总共 400 个页表项来实现对文本编辑程序代码的共享。但是, 如果采用分段机制, 只需要在每个进程的段表中为文本编辑程序的代码段增加一个段表项, 10 个用户进程只需要 10 个段表项就可以实现对文本编辑程序代码的共享。

三、段页式存储管理

为了使存储系统既具有分段系统便于编程、分段共享、易于保护及可动态链接等一系列优点, 又能像分页系统那样很好地解决内存的外部碎片问题, 以及可为各个分段离散地分配内存等问题, 引入了“段页式系统”。

1. 段页式存储管理的基本原理

在段页式存储管理系统中, 将用户进程的逻辑空间先划分成若干个段, 每个段再划分成若干个页。进程以页为单位在物理内存中离散存放, 每个段中被离散存放的页具有逻辑相关性。

为了实现段页式存储管理的地址映射, 操作系统为每个进程建立一个段表, 为进程的每个段建立一个页表。进程段表的每一个段表项存放某个段的页表起始地址和页表长度。

举例说明, 一个进程 process1 的逻辑地址空间划分为 5 个段, 段号分别为 0、1、2、3、4。段表、页表和进程在内存中的存放如图 4-55 所示。

2. 地址变换过程

在段页式存储管理系统中, 逻辑地址与分段系统的逻辑地址相同, 由段号 s 和段内偏移

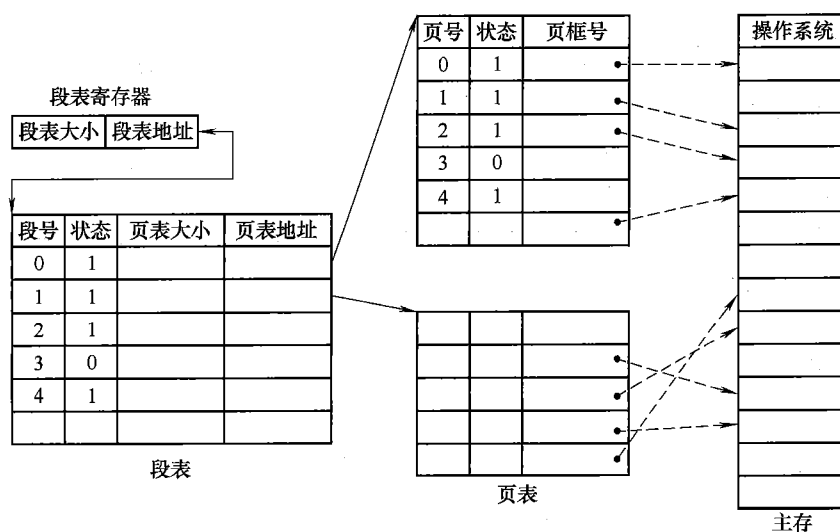


图 4-55 段页式存储

地址 d 构成, 地址变换过程如下。

- 1) 以段号 s 作索引, 找到段 s 的段表项, 得到该段页表的起始地址。
- 2) 通过分页机制从段内偏移 d 中分离出页号 P 和页内偏移 W 。
- 3) 以段内页号 P 作索引, 从段 s 的页表中搜索页号 P 对应的页表项。
- 4) 从页表项中得到页所在的页框号。
- 5) 由页框号与页内偏移 W 得到某逻辑地址对应的物理地址。

物理地址 = 页框号 $\times$ 页框大小 + 页内偏移 W

采用段页式内存管理的一个显然的好处是, 程序员可以使用分段的逻辑地址, 而实际上进程却以页为单位存放于物理内存中。

地址变换机制如图 4-56 所示。

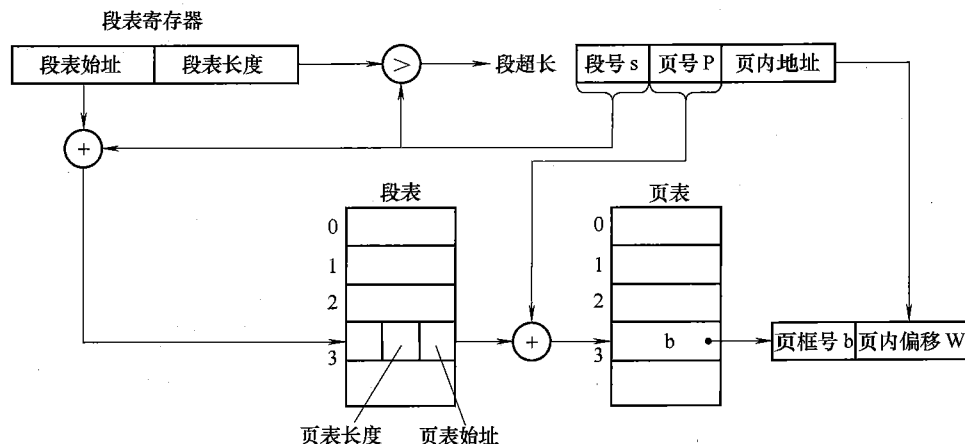


图 4-56 段页式系统的地址变换

第七节 Linux 的伙伴系统

因为在操作系统中，经常会遇到要为进程分配连续物理内存块的情况，典型的如在 Linux 中创建新进程时，需要为进程分配连续的 8 KB 空间，即两个连续的页框用于存放 `task_struct` 结构和进程的内核堆栈。因此，内核应该为分配一组连续的页框而建立一种稳定、高效的分配策略。

Linux 的伙伴系统算法把所有的空闲页框分组为 11 个块链表，每个块链表分别包含大小为 1、2、4、8、16、32、64、128、256、512 和 1024 个连续的页框。对 1024 个页框的最大请求对应着 4 MB 大小的连续页框。每个块的第一个页框的物理地址是该块大小的整数倍。例如，大小为 16 个页框的块，其起始地址是 16×2^{12} ($2^{12} = 4096\text{B}$ ，这是一个常规页的大小) 的倍数。

下面通过一个简单的例子来说明该算法的工作原理。

假设要请求一个 128 个页框的内存 (即 0.5 MB)。算法先在 128 个页框的链表中检查是否有一个空闲块。如果没有这样的块，算法会查找下一个更大的块。也就是说，在 256 个页框的链表中找一个空闲块。如果存在这样的块，内核就把 256 的页框分成两等份，一半用作满足进程请求，另一半插入到 128 个页框的链表中。如果在 256 个页框的块链表中也没找到空闲块，就继续找更大的块——512 个页框的块。如果这样的块存在，内核把 512 个页框的块中的 128 个页框用作满足进程请求，然后从剩余的 384 个页框中拿 256 个连续页框插入到 256 个页框的链表中，再把 128 个连续页框插入到 128 个页框的链表中。如果 512 个页框的链表还是空的，算法就放弃并发出错误信号。

以上过程的逆过程就是页框的释放过程，也是伙伴算法名字的由来。内核试图把大小为 b 的一对空闲伙伴块合并为一个大小为 $2b$ 的单独块。满足以下条件的两个块称为伙伴。

- 1) 两个块具有相同的大小，记作 b 。
- 2) 它们的物理地址是连续的，起始地址是 $2b$ 的整数倍。

第一块的第一个页框的物理地址是 $2 \times b \times 2^{12}$ 的倍数。该算法是迭代的，如果它成功合并所释放的块，它会试图合并 $2b$ 的块来形成更大的块。

思考题与练习题

一、单项选择题 (在每小题后的四个备选项中只有一个是符合题目要求的，请将其选出并将代码填写在题后的括号内)

1. 关于操作系统内存管理的功能，下列叙述错误的是【 】。
 - A. 回收内存
 - B. 为进程分配内存
 - C. 使程序尽可能用物理地址访问内存
 - D. 与硬件协同完成逻辑地址到物理地址的转换
2. 在基于分页的虚拟存储管理系统中，页表项增加了修改位字段，下列程序中，需要使用这个字段的是【 】。
 - A. 分配页框的程序
 - B. 选择换出页的程序

- C. 从外存调入页的程序 D. 访问页的程序
3. 多道程序环境下, 操作系统分配 CPU 资源的基本单位是【 】。
- A. 程序 B. 进程 C. 作业 D. 指令
4. 采用二级分页的存储管理系统中, 若分页逻辑地址用 32 位表示, 其中高 10 位表示页目录号 p1, 中间 10 位表示页号 p2, 低 12 位表示页内偏移, 则一个进程的逻辑地址空间大小最大为【 】。

A. 4 GB B. 2 GB C. 4 MB D. 4 KB

5. 在一个实际系统中, 进行页置换时, 通常选择换出的页是【 】。

A. 经常不用的页
B. 不会被修改的页
C. 最近既没有被修改也没有被访问过的页
D. 将来最久不会被访问的页面

二、填空题(请在题目空白处填写正确内容)

- 连续内存管理方式有 3 种, 即_____、_____和_____。
- CPU 的页表寄存器用于存放_____进程的页表起始地址。
- LRU 置换算法选择的换出页是_____的页。
- 快表中存放的内容是_____。
- 在分段存储管理系统中, 段表项中包含的基本内容有_____、_____和_____。
- 虚拟存储管理系统的特征有多次性、_____、_____和_____。

三、简答题

- 静态链接程序完成什么功能?
- 请说明分页存储管理的基本原理。
- 什么是抖动? 抖动产生的原因是什么?
- 请说明分段存储管理的原理。

四、综合应用

初始空闲链如图 4-57 所示, 分别表示第一个空闲区起始地址为 20 KB, 大小为 10 KB; 第二个空闲区起始地址为 50 KB, 大小为 100 KB; 第三个空闲区起始地址为 300 KB, 大小为 10 KB。

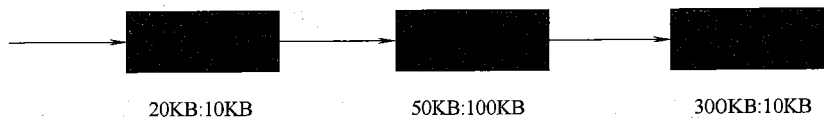


图 4-57 初始空闲链

画出在题目给定条件下系统释放下列空闲区后的空闲链, 如图 4-58 ~ 图 4-60 所示。

(1)

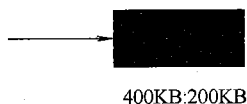


图 4-58 给定条件 1

(2)

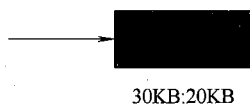


图 4-59 给定条件 2

(3)



图 4-60 给定条件 3

第五章 文件系统

文件系统管理是操作系统的重要功能之一，它为用户提供了在计算机系统中对数据信息进行长期、大量存储和访问的功能。文件的结构，以及文件的命名、访问、存储、保护和实现方法都是文件系统设计的主要内容。从总体上看，操作系统中处理文件的部分称为文件系统(File System)，文件系统包括了文件及管理文件的软件集合。

本章分两部分来讨论文件系统，一部分是文件系统的用户接口，另一部分是文件系统的实现，包括文件的按名访问和存储空间管理的实现。

第一节 文 件

本节讨论文件系统的用户接口，即用户可以“看见”和使用的文件系统部分，包括文件的命名、类型、属性和对文件的操作。

一、文件命名

文件命名向用户提供了简单、直观的文件访问方式，使用户在访问文件时不必了解信息存储的方法、位置及磁盘实际动作方式等细节，而只需给出要访问的文件名。在创建一个文件时，进程给出文件名。进程终止后，文件仍然存在，其他进程使用该文件名可以对它进行存取。

不同的文件系统为文件命名制定的规则略有不同，但所有操作系统都允许用1~8个字母组成的字符串作为文件名。例如，hello、bruce 和 oerw 都是合法文件名。一般情况下，文件名中也允许有数字和一些特殊字符，如1、error! 和 Tab. 6-1 都是合法的。许多文件系统支持长达255个字符的文件名。文件的命名规则和文件名的长度在文件系统设计时确定。

有的文件系统区分大小写字母，如UNIX的文件系统；有的不区分，如MS-DOS。因此，在UNIX中，hello、Hello 和 HELLO 是3个不同的文件名；而在MS-DOS中，它们是同一个文件名。

多数操作系统都支持文件名用圆点隔开分为两部分，如hello.c。圆点后面的部分称为文件扩展名。文件名info.txt用于表示这个文件的类型，给用户一个提示，而不是表示给计算机传送什么信息。但是，编译器、链接程序等利用扩展名区分哪些是C文件，哪些是汇编文件，哪些是其他文件。

在Windows系统中，扩展名被赋予了含义。用户可以在操作系统中注册扩展名，并且规定哪个程序与该扩展名关联，当用户双击某个文件名时，关联该扩展名的程序就会启动并访问该文件。例如，双击“孔子.rmvb”就会启动关联的电影播放软件，并播放该视频。

二、文件结构

本节介绍如图5-1所示的3种文件结构。

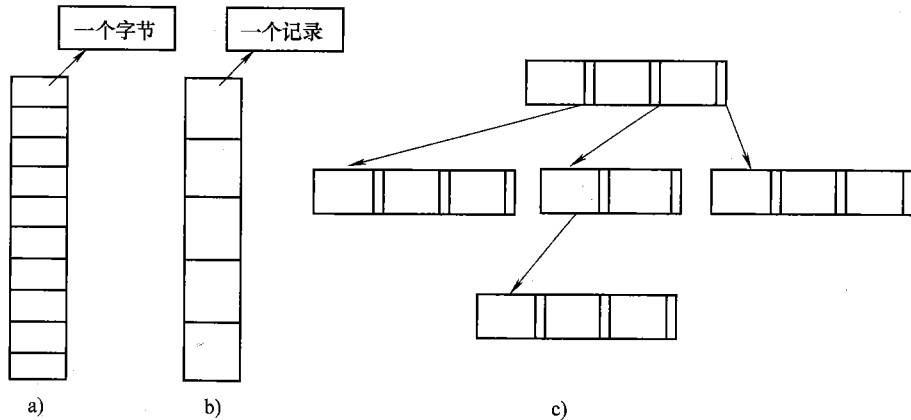


图 5-1 3 种文件结构

a) 无结构字节序列 b) 固定长度记录序列 c) 树形结构

1. 无结构字节序列

无结构字节序列文件也称为流式文件，操作系统不知道也不关心文件内容是什么，操作系统所见到的就是字节，其任何含义由使用该文件的程序自行理解。在 UNIX 和 Windows 系统中都采用这种方式。

把文件看成字节序列为操作系统提供了最大的灵活性。用户程序可以向文件中加入任何内容，并以方便的形式对文件进行命名。

2. 固定长度记录序列

在该模型中，构成文件的基本单位是具有固定长度的记录，每个记录都有其内部结构。把文件作为记录序列的中心思想是：读操作返回一个记录，而写操作重写或追加一个记录。

3. 树形结构

文件由一棵记录树构成，记录长度不定，在记录的固定位置包含一个关键字域，记录树按关键字域排序。在这种文件结构中，基本操作是获取具有特定关键字的记录（而不是获取下一条记录）。增加记录时，由操作系统决定记录在文件中的存放位置。这类文件结构与 UNIX 和 Windows 系统中采用的无结构字节序列有明显不同，它在一些处理商业数据的大型计算机中获得了广泛使用。

三、文件类型

在设计操作系统时要考虑系统支持哪些类型的文件，并为每种类型的文件定义合法的操作。文件的类型有正规文件、目录文件、字符设备文件和块设备文件等。正规文件包含用户信息，一般分为 ASCII 文件和二进制文件。目录文件是用于管理文件的系统文件。字符设备文件和输入/输出有关，用于串行 I/O 类设备，如终端、打印机和网络等。块设备文件用于磁盘类设备。本节主要介绍正规文件。

1. ASCII 文件(American Standard Code for Information Interchange)

ASCII 文件由多行正文组成，在某些系统中每行用回车符结束，某些则用换行符结束，而有些系统还同时采用回车符和换行符，如 MS-DOS。各行的长度不必相同。

ASCII 文件的明显优势是可以显示和打印，也可以用通常的文本编辑器进行编辑。另

外，如果程序以 ASCII 文件作为输入和输出，就很容易把一个程序的输出作为另一个程序的输入。

2. 二进制文件

二进制文件具有一定的内部结构，如可执行的 .exe 文件。用通常的文本编辑器不能直接显示或打印二进制文件，必须使用专门的二进制文件编辑器。不同的操作系统可以识别不同的二进制文件，比如把某一种结构(如 ELF 格式)的二进制文件作为系统中的可执行文件。

四、文件存取

用户通过对文件的存取来完成对文件的各种操作，文件的存取方式是由文件的性质和用户使用文件的情况确定的。常用的文件存取方式有两种：顺序存取和随机存取。

1. 顺序存取

早期的操作系统只有顺序存取这一种文件存取方式。进程可以从文件开始处读取文件中的所有字节或者记录，但不能跳过某些内容，也不能不按顺序存取。在存储介质是磁带而不是磁盘时，顺序存取文件是很方便的。

2. 随机存取

随机存取又称直接存取，即可以以任意顺序读取文件中的字节或记录。现代操作系统的文件一旦被创建，所有文件自动成为随机存取文件。定长记录的文件能很好地支持随机存取，而变长记录虽然可以随机存取，但实现起来复杂而且存取速度慢。

五、文件属性

为方便管理，除了文件名和文件数据外，文件系统还会保存其他与文件相关的信息，如文件的创建日期、文件大小和修改时间等，这些附加信息称为文件属性。文件属性在不同的操作系统中差别很大，表 5-1 列出了一些常用的文件属性。

表 5-1 一些常用的文件属性

属 性	含 义	属 性	含 义
保护	谁可以存取文件，以什么样的方式存取文件	临时标志	0 表示正常；1 表示进程退出时删除该文件
口令	存取文件需要的口令	加锁标志	0 表示未加锁；非零表示加锁
创建者	创建文件者的 ID	记录长度	一个记录中的字节数
所有者	当前所有者	键的位置	每个记录中的键的偏移量
只读标志	0 表示读/写；1 表示只读	键的长度	键字段的字节数
隐藏标志	0 表示正常；1 表示不在列表中显示	创建时间	文件创建的日期和时间
系统标志	0 表示普通文件；1 表示系统文件	最后一次存取时间	文件上一次存取的日期和时间
存档标志	0 表示已经备份；1 表示需要备份	最后一次修改时间	文件上一次修改的日期和时间
ASCII/二进制标志	0 表示 ASCII 文件；1 表示二进制文件	当前大小	文件的字节数
随机存取标志	0 表示只允许顺序存取；1 表示随机存取	最大长度	文件可能增长到的字节数

六、文件操作

使用文件的目的是存储信息并方便以后检索。不同的系统提供了不同的操作来存储和检索信息，以下是常用的文件操作。

1. CREATE

该操作完成创建文件的功能，并设置文件的一些属性。

2. DELETE

当不再需要某个文件时，删除该文件并释放磁盘空间。

3. OPEN

在使用文件之前，必须先打开文件。OPEN 调用的目的是将文件属性和文件的地址信息装入主存，便于在对文件的后续访问中能快速存取文件信息。

4. CLOSE

当存取结束后，不再需要文件属性和地址信息，这时应该关闭文件以释放内部表空间。很多系统限制进程打开文件的个数，以鼓励用户关闭不再使用的文件。

5. READ

从文件中读取数据。一般地，用户可以指定从哪个文件的第几个字节开始读取多少个字节的内容。此外，调用读文件的函数时，还需要给出存放被读取内容的内存缓冲区。

6. WRITE

往文件中写数据，写操作一般从写函数的参数指定的文件位置开始。如果当前位置是文件末尾，文件长度增加。如果当前位置在文件中间，则现有数据被覆盖，并且永远丢失。

7. APPEND

该操作是 WRITE 调用的限制形式，它只能在文件末尾添加数据。

8. SEEK

对于随机存取文件，要指定从何处开始取数据。通常的做法是用 SEEK 系统调用把当前位置指针指向文件中特定的位置。SEEK 调用结束后，就可以从该位置开始读写数据了。

9. GETATTRIBUTES

该操作用于获取文件属性。

10. SETATTRIBUTES

某些属性是可由用户设置的，文件创建以后，用户还可以通过系统调用 SETATTRIBUTES 来修改它们。

11. RENAME

该操作用于修改已有文件的名文件名。

第二节 目 录

文件系统通常提供目录或文件夹用于记录文件，很多系统中目录本身也是文件，目录是文件系统中实现按名访问文件的重要数据结构。本节讨论目录、目录的组成、目录的特性，以及可以对目录进行的操作。

一、层次目录系统

1. 目录文件的结构

目录文件有两种常见的结构：属性放在目录项中和放在 i 结点中。

目录文件包含许多目录项，每个目录项用于描述一个文件。在第一种结构中，每个目录项长度相同，每个文件对应一个目录项。其中包含文件名、文件属性和文件的地址。在第二种结构中，目录项中有一个文件名和 i 结点号。i 结点是一种数据结构，文件属性和文件的地址信息存放在 i 结点中。这两种结构都得到了广泛的应用。图 5-2 分别描述了以上两种结构。

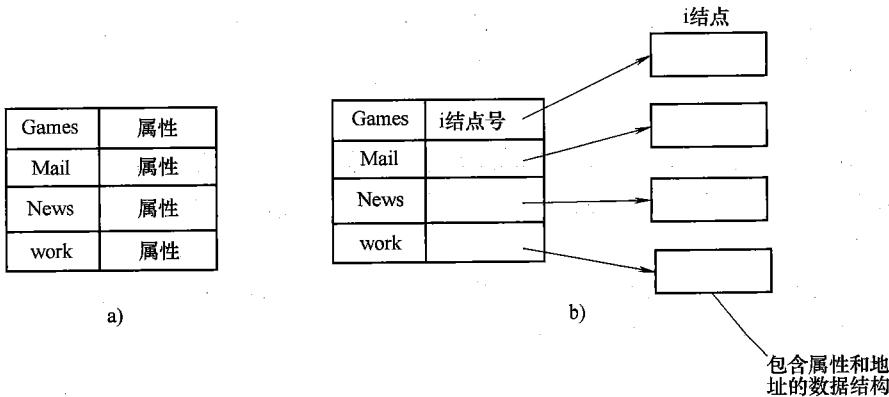


图 5-2 目录文件的结构

a) 简单目录，属性和地址放在目录中 b) 属性放在 i 结点中的目录

2. 目录结构

文件目录的组织和管理是文件管理的一个重要方面，包括单层目录、两级目录和树形目录。

(1) 单层目录

这种目录也被称为根目录。在整个系统中设置一张线性目录表，表中包括了所有文件的描述信息。早期的个人计算机中，这种系统很普遍。这种目录结构使得软件设计相对简单。

图 5-3 所示是一个单层目录系统的例子。该目录中有 4 个文件。

在多用户系统中，单层目录带来的一个显著问题是，不同用户可能会使用相同的文件名。例如，如果用户 A 创建了一个名为 hello.c 的文件，然后，用户 B 也创建了一个名为 hello.c 的文件，用户 B 的文件可能会覆盖用户 A 的文件。很显然，这种结构不适合在多用户系统中使用。

另外，由于使用单层目录时，要查找一个文件必须对单层目录表中的所有文件信息项进行搜索，因而搜索效率也较低。

(2) 两级目录

为了避免上述文件名冲突，一种改进方法是为每个用户提供一个私有目录。这样，一个用户选择文件名时就不会影响到其他用户。图 5-4 展示了一个两级目录系统。

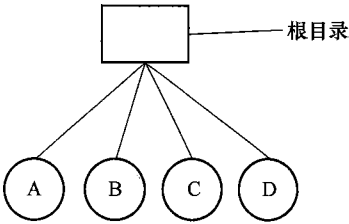


图 5-3 含有 4 个文件的单层目录系统

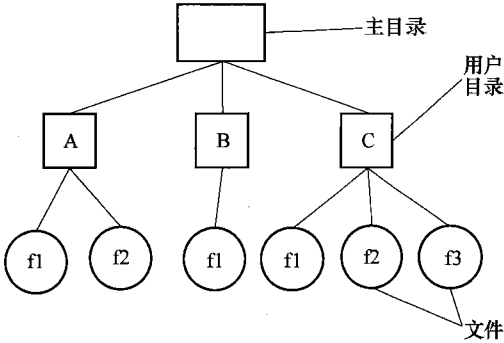


图 5-4 两级目录系统

在两级目录结构中，目录被分为两级，第一级称为主目录，给出了用户名和用户子目录所在的物理位置。第二级称为用户目录，给出了该用户所有文件的文件控制块。这一设计隐含的机制是，当一个用户试图打开文件时，系统知道是哪个用户，从而知道应该查询哪个目录。

使用两级目录的优点是解决了文件的重名问题和文件共享问题，查找时间降低。缺点是增加了系统的存储开销。

(3) 树形目录

把两级目录的层次关系加以推广，就形成了多级目录，又称树形目录，如图 5-5 所示。

在多级目录结构中，除了叶子结点对应的存储块中装有文件信息外，其他每一级目录中存放的都是下一级目录或文件的说明信息，由此形成层次关系。最高层为根目录，最底层为文件。在这种结构中，用户可以拥有多个所需的目录，自由地组织自己的文件。同时，用户可以创建任意数量子目录的功能，为用户组织其文件提供了一种强大的工具。

树形目录的优点是便于文件的分类，层次结构清晰，便于管理和保护，解决了重名问题，查找速度加快。缺点是查找一个文件按路径名逐层检查，由于每个文件都放在外存中，多次访问磁盘会影响速度，结构相对复杂。

二、路径名

用目录树组织文件系统时，需要有某种方法指明文件名。常用的方法有两种：绝对路径名和相对路径名。

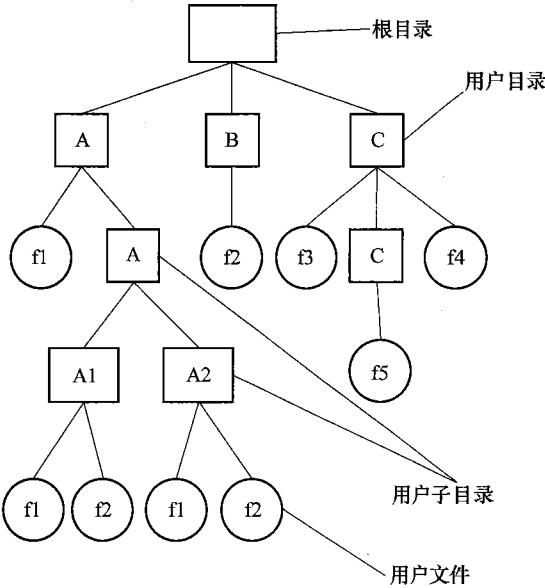


图 5-5 树形目录系统

1. 绝对路径名

绝对路径名由从根目录到文件的路径组成。例如，路径 `/program/practice/test` 表示根目录中有子目录 `program`，而 `program` 中又有子目录 `practice`，文件 `test` 就在子目录 `practice` 下。绝对路径名总是从根目录开始，并且是唯一的。在 Windows 系统中，路径各部分之间用“\”分隔；在 UNIX 系统中，分隔符为“/”。同样的路径名在两种系统中的书写形式如下。

Windows	<code>\ program \ practice \ test</code>
UNIX	<code>/program/practice/test</code>

不管采用哪种分隔符，只要路径名的第一个字符是分隔符，则这个路径就是绝对路径。

2. 相对路径名

当访问一个文件系统的目录包含很多级时，如果访问每个文件都要从根目录开始，直到叶子的文件名为止，包含所有经过的各级分目录在内的全路径名是相当麻烦的。在设计文件系统时，可以允许用户指定一个目录作为当前的工作目录，所有的不从根目录开始的路径名都是相对于工作目录的。例如，如果当前的工作目录是 `/program/practice`，则绝对路径名为 `/program/practice/test` 的文件可以直接用 `test` 来引用，用户使用相对路径名往往更方便，但是前提是用户访问的文件必须在当前工作目录下。一些程序需要存取某个特定的文件，而不管当前目录是什么，这时必须采用绝对路径名。不管当前的工作目录是什么，绝对路径名总能正常工作。

支持树形目录结构的大多数文件系统在每个目录中有两个特殊的目录项“.”和“..”。“.”指当前目录，“..”指当前目录的父目录。也就是说当在 UNIX 的终端或 Windows 的命令提示符中输入命令“`cd..`”时，将返回上一层目录，如果已经到根目录，再执行该命令，当前目录将一直留在根目录。

三、目录操作

本节以 UNIX 系统为例，说明对目录的操作。

1. CREATE

根据给定的目录文件名，创建目录。除了目录项“.”和“..”外，目录内容为空。“.”和“..”是系统自动放在目录中的。

2. DELETE

删除目录，根据指定的目录名删除一个目录文件。

3. OPENDIR

目录内容可以被读取。例如，为列出目录中的所有文件和子目录，程序必须先打开该目录，然后读取其中所有文件的文件名，同打开和读取文件一样，在读目录之前必须打开目录。

4. CLOSEDIR

读目录结束后，应关闭目录以释放内部表空间。

5. READDIR

以标准格式返回打开目录的下一级目录项。

6. RENAME

更换目录名。

第三节 文件系统的实现

前面讨论的文件系统，主要是从用户的角度探讨问题。用户关心的是文件怎样命名、可以进行哪些操作、目录树是什么样的，以及类似界面问题。而实现者感兴趣的是文件和目录是怎样存储的、文件的存储空间是怎样管理的，以及怎样使系统有效而可靠地工作等。下面，从设计者和实现者的角度讨论文件系统是如何实现的。

一、实现文件

本节以文件存放在磁盘中为例，说明实现文件存储的几种常用方式。文件系统通常是以 2ⁿ 个连续的扇区为单位对文件进行磁盘空间的分配，下面把分配给文件的连续扇区构成的磁盘块称为簇。

1. 连续分配

顾名思义，连续分配就是把每个文件作为一连串连续数据块存储在磁盘上。例如，系统规定的簇大小为 1 KB，则存储 5 KB 大小的文件，需要 5 个连续的簇。若簇大小为 2 KB，则存储 5 KB 大小的文件需要分配 3 个连续的簇，最后一个簇中，有 1 KB 的磁盘空间将不能使用，下一个文件要从一个新的簇开始，如图 5-6 所示。

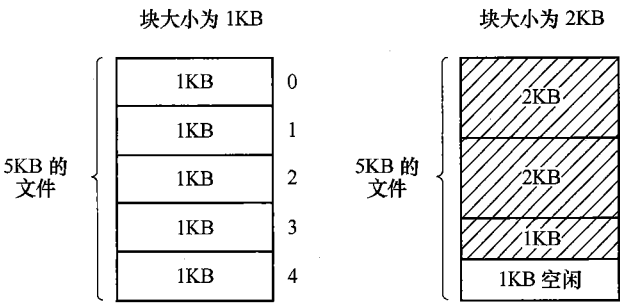


图 5-6 连续分配示例

连续分配方式有两大优点。首先，实现简单，记录每个文件用到的簇仅需存储两个数字即可：第一块的磁盘地址和文件的块数。给定第一块的簇号，就可以找到任何其他块的簇号。其次，读操作性能好，在单个操作中就能从磁盘上读取整个文件。只需要寻找一次第一个块，之后不再需要寻道和旋转延迟，所以数据以磁盘全带宽的速率输入。

但是，连续分配方式也有十分明显的缺点：随着时间的推移，磁盘会变得零碎。当删除文件时，文件所占的簇被释放，这些空闲的连续簇形成“空洞”。随着磁盘的使用，磁盘上会有许多空洞。当磁盘被充满而又需要创建新文件时，需要挑选大小合适的空洞存入文件，这时就有必要知道该文件的最终大小。但很多情况下，无法预知一个文件的最终大小，比如录入一个文档，文档的内容有多少，需要占用多大空间都是未知的，也就是说文件的大小可变，系统管理文件的存储会比较麻烦。

2. 使用磁盘链接表的分配

该方法为每个文件构造簇的链接表，每个簇开始的几个字节用于存放下一个簇的簇号，簇的其他部分存放数据，每个文件可以存放在不连续的簇中。在目录项中只需存放第一个数据块的磁盘地址，文件的其他块可以根据这个地址来查找。

如图 5-7 所示，某文件的数据信息需要占用 5 个簇，第 0 块数据存储在 4 号簇中，第 1 块数据存储在 7 号簇中，第 2 块数据存储在 2 号簇中，第 3 块数据存储在 10 号簇中，第 4

块数据存储在 12 号簇中。4 号簇的起始字节中存放了 1 号数据块所在簇的簇号 7，7 号簇的起始字节中存放了 2 号数据块所在簇的簇号 2，2 号簇的起始字节存放了 3 号数据块所在簇的簇号 10，10 号簇的起始字节存放了 4 号数据块所在簇的簇号 12。12 号簇的起始字节存放文件结束标志，表示 12 号簇是存放该文件的最后一个簇。对该文件的查找过程是先根据该文件的文件名搜索目录文件，找到该文件的目录项，从目录项中得到第 0 个数据块所在的簇号 4，从 4 号簇的起始字节得到第 1 号数据块所在的簇号 7，依此获得所有数据所在的簇号，访问文件的数据。

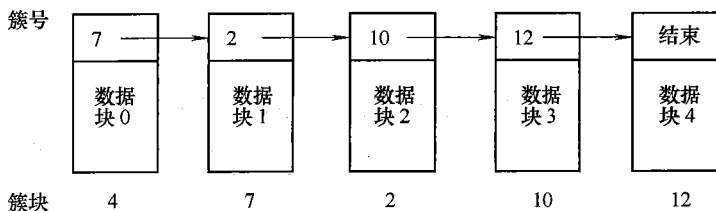


图 5-7 以磁盘块的链接表形式存储文件

这一方法的优点是可以充分利用每个簇，不会因为磁盘碎片（除了最后一块中的内部碎片）而浪费存储空间，管理也比较简单。缺点是随机存取相当缓慢。要获得文件的第 n 块，每一次都要从头开始读取前面的 $n-1$ 块。显然，进行如此多的读操作会占用很多时间。

3. 使用内存的链接表分配

该方法是将文件所在的磁盘的簇号存放在内存的表（文件分配表）中。访问文件时，只需从内存文件分配表中顺着某种链接关系查找簇的簇号。不管文件有多大，在目录项中只需记录文件的第一块数据所在簇的簇号，根据它查找到文件的所有块。

MS-DOS 就使用这种方法进行磁盘分配。图 5-8 表示了图 5-7 所示例子的文件分配表的内容。该文件依次使用了簇号为 4、7、2、10 和 12 的磁盘块。访问该文件时，先根据文件名检索目录文件，从目录文件中获得文件第 0 号数据块所在的簇号 4，然后利用图 5-8 中的文件分配表，从第 4 号表项中读到簇号 7，从 7 号表项中读到簇号 2，从 2 号表项中读到簇号 10，从 10 号表项中读到簇号 12。在 12 号表项中存该文件的结束标志，至此找到图 5-7 所示例子中的文件占用的全部簇的簇号。

这种方法的一个缺点是必须把整个表都存放在内存中。对于目前比较常见的 500 GB 的磁盘，如果簇大小为 1 KB，这张表需要有 500 M 个表项，每一项对应一个簇。若每项占 4 个字节，根据系统对空间或时间的优化方案，这张表要占用约 2 GB 的内存（500 M × 4 B）。很显然，这种方法不适合大容量的磁盘。

链接表
(如 FAT 表)

0	
1	
2	10
3	
4	7
5	
6	
7	2
8	
9	
10	12
11	
12	EOF

文件从这里开始

文件到这里结束

图 5-8 内存中的链接表

4. i-结点

该方法为每个文件赋予一个被称为 i 结点的数据结构，其中列出了文件属性和文件块的磁盘地址。图 5-9 所示是一个简单的例子。给定一个文件的 i 结点，就有可能找到文件的所有块。系统打开文件时，将文件的 i 结点从磁盘读入内存。当访问文件时，系统先根据文件名搜索文件所在的目录文件，从该文件对应的目录项中找到文件的 i 结点号，根据 i 结点号从磁盘中将 i 结点信息读入内存，文件在磁盘中的地址信息都存放在 i 结点中。

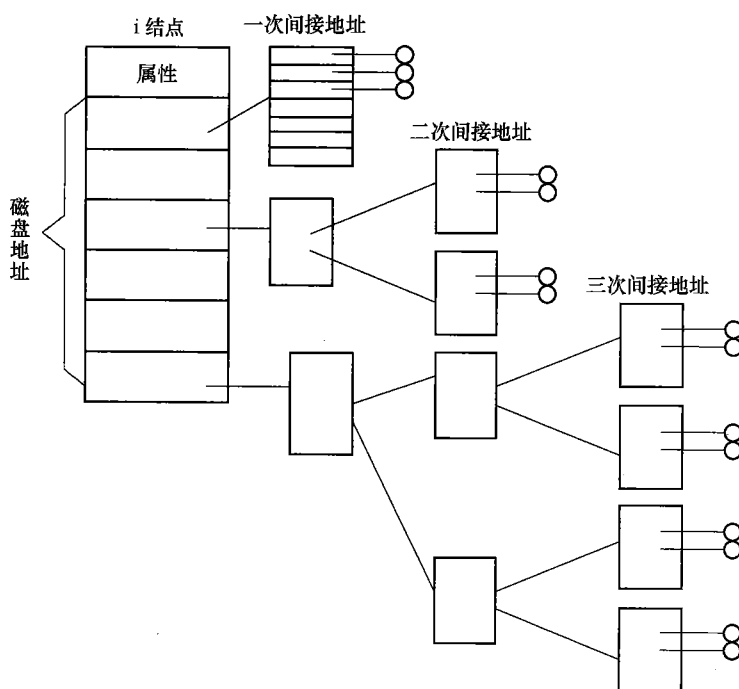


图 5-9 采用间接地址的 i 结点

如果每个 i 结点只能存储固定数量的磁盘地址，那么当一个文件比较大，所含簇的数目太多时，i 结点将无法记录所有的簇号。一个解决方案是采用间接地址，即使一个“磁盘地址”不存放数据块，而是存放簇号。对于一个大文件，i 结点内的其中一个地址是一次间接块的簇号，这个块包含了存放文件数据的簇的簇号。如果还不够的话，在 i 结点中还有二次间接块的簇号，其中存放了若干个一次间接块的簇号。文件再大的话，可以使用三次间接块，如图 5-9 所示。

Linux 的 Ext2 文件系统的 i 结点包括 15 个地址项，每个地址项存 32 位地址(4 个字节)，用其中 12 个地址项存直接地址，一个地址项存一次间接地址，一个地址项存二次间接地址，一个地址项存三次间接地址。当簇大小为 1 KB 时，系统能管理的单个文件的最大长度该如何计算呢？

首先，12 个地址项存放的是簇号，所以 12 个直接地址可表示的文件大小为 $12 \times 1 \text{ KB} = 12 \text{ KB}$ 。其次，每个簇大小为 1 KB，每个地址项占 4 个字节，所以，每个簇中可以存放 256 个簇号。这样，一次间接块可以表示的文件大小为 $256 \times 1 \text{ KB} = 256 \text{ KB}$ 。以此类推，二次间接块可以表示的文件大小为 $256 \times 256 \times 1 \text{ KB} = 64 \text{ MB}$ 。三次间接块可以表示的文件大小为

$256 \times 256 \times 256 \times 1 \text{ KB} = 16\text{GB}$ 。最后，一个文件的最大长度为 $12 \text{ KB} + 256 \text{ KB} + 64\text{MB} + 16 \text{ GB}$ 。

二、实现目录

系统在读文件前，必须先打开文件。打开文件时，操作系统利用用户给出的路径名找到相应的目录项，目录项中提供了查找文件簇所需要的信息。下面，以 CP/M、MS-DOS 和 UNIX 为例，说明 3 种不同的目录实现方式。

1. CP/M 中的目录

CP/M 是一个微机操作系统，它只有一层目录，因此只有一个目录文件。要查找文件名，就是在这个唯一的目录文件中找到文件对应的目录项，从目录项中获得文件存放的磁盘地址。图 5-10 所示为 CP/M 的目录项结构。其中，各字段的含义如下。

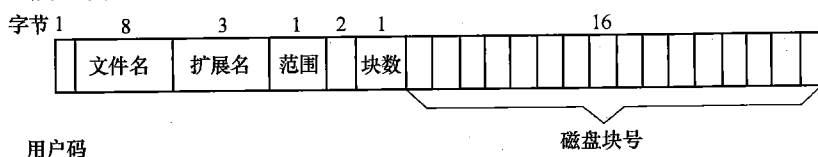


图 5-10 CP/M 的目录项结构

- 1) 用户码：记录了文件所有者。
- 2) 文件名：存放文件名。
- 3) 扩展名：标识文件类型。
- 4) 范围：由该域可知某目录项是文件的第几个目录项，因为一个很大的文件(大于 16 KB)，其簇的数量多，在一个目录项中记录不下。
- 5) 块数：文件实际使用的簇的数量。
- 6) 最后 16 个域记录了簇号。

需要注意的是，文件占用的最后一个簇可能没有写满，这样系统无法确切地知道文件的字节数，因为 CP/M 是以簇而不是以字节为单位来记录文件长度的。

2. MS-DOS 中的目录

MS-DOS 文件系统曾经是 IBM PC 系列所采用的文件系统，目前它已经不再是新的 PC 文件系统的标准了，但它和它的扩展(FAT32)一直被许多嵌入式系统所广泛使用。大部分的数码相机使用它，许多 MP3 播放器使用它，流行的苹果公司的 iPod 使用它作为默认的文件系统。现在使用 MS-DOS 文件系统的电子设备数量远远多于过去，并且远远多于使用更现代的 NTFS 文件系统的数量。下面介绍其中的一些细节。

图 5-11 所示是一个 MS-DOS 的目录项。它总共 32 个字节，其中包含了 8 个字节的文件名、1 个字节的文件属性和 2 个字节的文件第一个簇的簇号。根据第一个簇号，顺着索引链表可以找到所有的文件块。MS-DOS 中，目录可以包含其他子目录，从而形成树形目录。

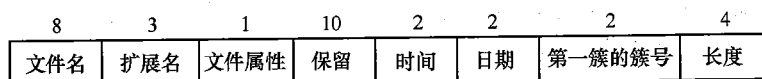


图 5-11 MS-DOS 的目录项

MS-DOS 用文件分配表即 FAT 作为索引表来存放文件数据所在簇的簇号。目录项包含了文件第一个数据块所在簇的簇号，将这个簇号用作 FAT 的索引，可依次找到文件的所有块。FAT 的结构见前面的图 5-8。

FAT 文件系统有 3 个版本：FAT-12，FAT-16 和 FAT-32，取决于用多少个二进制位存放簇号。

3. UNIX 中的目录

UNIX 中采用的目录结构非常简单，如图 5-12 所示，每个目录项只包含一个文件名及其 i 结点号。有关文件类型、长度、时间、所有者和簇号等信息都放在 i 结点中。当打开某个文件时，文件系统必须要获得文件名并且定位文件所在的第一个簇。下面说明文件系统寻找一个文件的过程，以查找 UNIX 中的 /usr/ast/mbox 为例。

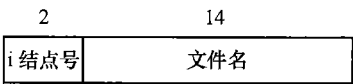


图 5-12 UNIX 中的目录结构

首先，文件系统找到根目录，在 UNIX 中根目录的 i 结点位于磁盘的固定位置。然后，系统读根目录文件并且在根目录文件中查找路径的第一部分 usr，获得文件 /usr 的 i 结点号。由 i 结点号来定位 i 结点是很直接的，因为每个 i 结点在磁盘上都有固定的位置。根据这个 i 结点号，系统定位 /usr 目录并查找下一部分 ast。一旦找到 ast 的目录项，便找到了 /usr/ast 的 i 结点。根据这个 i 结点，可以定位该目录并在其中查找 mbox。然后，这个文件的 i 结点被读入内存，并且在文件关闭之前会一直保留在内存中。查找的过程如图 5-13 所示。

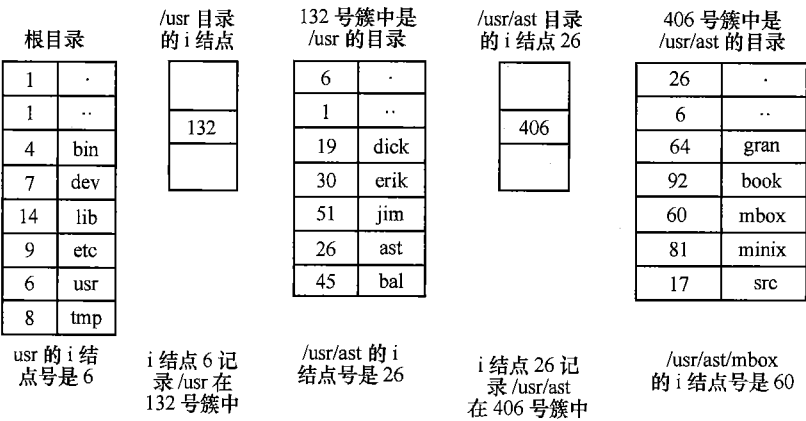


图 5-13 查找 /usr/ast/mbox 的过程

提示：相对路径的查找与绝对路径的查找方法相同，只不过是从当前工作目录开始查找而不是从根目录开始。

三、磁盘空间管理

磁盘空间管理是文件系统的重要功能，包括记录空闲磁盘信息、设计文件的存放方式，以及规定文件系统的簇大小等内容。

1. 簇大小

文件系统为文件分配磁盘空间是以簇为单位的，一旦用固定大小的簇来存储文件，就会出现一个问题：簇的大小应该是多少？

拥有大的簇尺寸意味着每个文件，甚至一个字节的文件，都要占用很大的空间，也就是说小的文件浪费了大量的磁盘空间。另一方面，小的尺寸意味着大多数文件会跨越多个簇，因此需要多次寻道与旋转延迟才能读出它们，从而降低了时间性能。因此，簇太大，容易造成空间的浪费；簇太小，则会使访问文件的时间延长。

一般簇大小是2的整数次幂个连续的扇区，如1个扇区，512个字节；连续两个扇区，大小为1 KB；连续4个扇区，大小为2 KB。根据系统管理的文件大小，可以选择合适的簇大小来格式化磁盘。

2. 记录空闲块

确定了簇大小后，下一个问题是怎样跟踪记录空闲簇。有以下两种方法被广泛采用。

(1) 空闲簇链接表

用一些空闲簇存放空闲簇的簇号。一个簇存放尽可能多的空闲簇的簇号，并专门留出最后几个字节存放指向下一个存放空闲簇的指针。对于1 KB大小的簇，可以存放256个32位的簇号(有一个存放指向下一个块的指针)。图5-14a显示了该方法的记录方式。

(2) 位图

用n位位图对应磁盘的n个簇，在位图中，空闲簇用1表示，已分配簇用0表示(或者反过来)，如图5-14b所示。很显然，位图方法所需空间少，因为每个簇只用一个二进制位标识，而在链接表方法中，每一个簇号都要用32位。

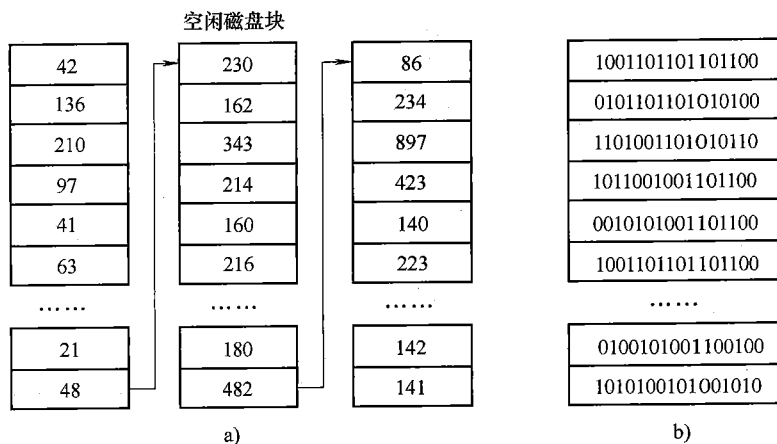


图 5-14 空闲块的记录方式

a) 空闲簇链接表 b) 位图

思考题与练习题

一、单项选择题(在每小题后的四个备选项中只有一个是符合题目要求的，请将其选出并将代码填写在题后的括号内)

- 关于文件系统，下列叙述正确的是【 】。
 - 文件的集合
 - 文件的目录
 - 实现文件管理的一组软件
 - 文件、管理文件的软件及数据结构的总体
- 位图可用于管理【 】。

- A. 文件目录
 - B. 磁盘空间
 - C. 文件的访问
 - D. 文件的保护
3. 从用户角度看, 引入文件系统的主要目的是【 】。
- A. 实现虚拟存储
 - B. 保存系统文档
 - C. 保存用户文档
 - D. 实现对文件的按名存取

二、填空题(请在题目空白处填写正确内容)

1. 常用的文件存取方式有两种: 顺序存取和_____。
2. 在使用文件之前, 必须先打开文件。打开文件的目的是将文件的_____和_____信息装入主存, 以便于在对文件的后续访问中能快速存取文件信息。
3. 目录是文件系统中实现_____的重要数据结构。

三、简答题

1. 举例说明文件系统是如何实现文件的“按名存取”的?
2. 文件系统如何管理空闲磁盘空间?
3. 什么是 i 结点? 在 i 结点中使用间接地址项有什么好处?

四、综合应用

Ext2 文件系统的一个 i 结点包括 15 个地址项, 每个地址项存 32 位地址(4 个字节), 其中 12 个地址项存直接地址; 一个地址项存一次间接地址; 一个地址项存二次间接地址, 一个地址项存三次间接地址。当簇大小为 4 KB 时, Ext2 能管理的文件的最大长度是多少?

第六章 I/O 设备管理

计算机系统中的 I/O 设备即输入/输出设备是用于计算机系统与人通信或与其他机器通信的所有设备，以及所有外存设备。I/O 设备管理是操作系统的重要功能之一，也简称为设备管理。

I/O 设备是计算机系统的重要组成部分，计算机 I/O 设备的多样性使设备的管理非常复杂。本章先从硬件角度介绍 I/O 设备的一般构成和控制方式，然后从操作系统设备管理软件的角度说明 I/O 设备管理软件的构成和软件各组成部分的功能，并说明部分功能实现的原理。

第一节 I/O 系统的组成

I/O 系统不仅包括各种 I/O 设备，还包括与设备相连的设备控制器，有些系统还配备了专门用于输入/输出控制的专用计算机，即通道。此外，I/O 系统要通过总线与 CPU、内存相连。本节介绍 I/O 系统的结构、I/O 设备的分类、设备控制器的功能和构成，以及通道的特点和功能。

一、I/O 系统的结构

I/O 系统的结构分为微机 I/O 系统和主机 I/O 系统两大类。

1. 微机 I/O 系统

总线型 I/O 系统结构如图 6-1 所示。CPU 与内存之间可以直接进行信息交换，但是不能与设备直接进行信息交换，必须经过设备控制器。

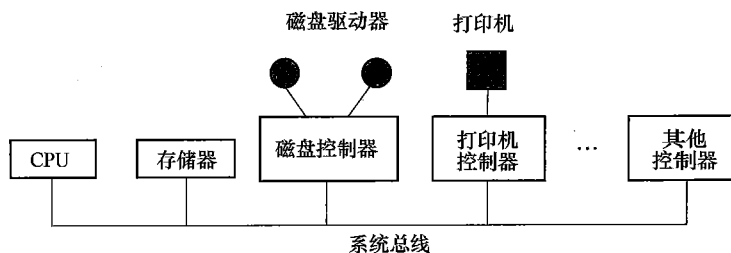


图 6-1 总线型 I/O 系统结构

2. 主机 I/O 系统

I/O 系统可能采用四级结构，包括主机、通道、控制器和设备。一个通道可以控制多个设备控制器，一个设备控制器也可以控制多个设备。图 6-2 所示为具有通道的 I/O 系统结构。

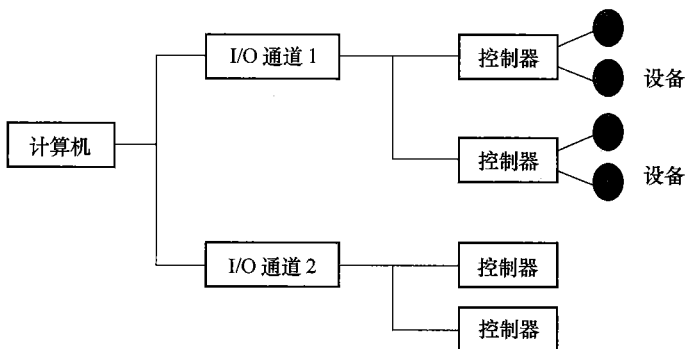


图 6-2 具有通道的 I/O 系统结构

二、I/O 设备的分类

不同功能的 I/O 设备种类繁多，可以根据设备的某种特点或功能分别对设备进行分类。是否需要对设备进行分类，如何分类，可根据实际需要的情况而定。

1. 按传输速率分类

- 1) 低速设备。如键盘和鼠标，传输速率为几个 ~ 几百个字节/秒。
- 2) 中速设备。如打印机，传输速率为数千个 ~ 数万个字节/秒。
- 3) 高速设备。如磁带机、磁盘机、光盘机，传输速率为几十万 ~ 几兆字节/秒。

2. 按信息交换的单位分类

1) 块设备。数据的存取以数据块为单位，如磁盘。块设备在块中保存信息，块的大小通常是固定的，并且一次只传送一块，通常可以通过块号访问数据。

2) 字符设备。传送字节流，没有使用块结构。终端、打印机、通信端口和鼠标等都是字符设备。

在操作系统实现驱动程序时，通常需要区分一个设备是块设备还是字符设备，操作系统对这两种设备的缓冲管理和驱动程序的实现方式是不同的。

3. 按设备的共享属性分类

1) 独占设备。是必须作为临界资源以互斥方式访问的设备。在一个进程没有使用完毕之前，其他任何进程不能访问该设备，直到设备被释放。例如，打印机是典型的独占设备。

2) 共享设备。是允许多个进程共同访问的设备，如硬磁盘是典型的共享设备。这里共同访问的含义是：一个进程访问该设备的全部任务还没有完成，就可以把设备分配给另一个进程使用。如进程 p1 需要读 5 个磁盘块的数据，p2 需要读 3 个磁盘块的数据。当进程 p1 读完一个磁盘块后，就可以把磁盘的访问权分配给 p2，让进程 p2 读一个或若干个磁盘块，再让 p1 继续读其余的磁盘块。如果必须让 p1 读完所有 5 个磁盘块后才能让 p2 开始访问磁盘，那磁盘就成了独占设备了。共享磁盘的含义并不是指在任意时刻不同的进程都可以同时读写磁盘。

3) 虚拟设备。是通过某种虚拟技术把一台物理设备变成若干逻辑设备，从用户的角度看，多个用户拥有各自的设备，可以随时向设备发出访问请求并得到系统应答。

三、设备控制器

I/O 设备分为机械和电子两部分，设备控制器对应电子部分，通常是可编程的。操作系统的程序员需要了解的是设备控制器的结构，编写设备驱动程序要针对具体的设备控制器，了解设备控制器中寄存器、端口的名称、地址和访问方式，以及使用方式。

1. 什么是设备控制器

1) 设备控制器是 CPU 与 I/O 设备之间的接口，接收 I/O 的命令并控制设备完成 I/O 工作。

2) 设备控制器是一个可编址设备，连接多个设备时可有多个设备地址。控制器可以直接做在主板上，也可以做成插卡插在主板上。现在有些设备的控制器嵌入在设备中，如激光打印机的控制器。

2. 设备控制器的功能

设备控制器的功能说明如下。

1) 接收和识别命令。

接收 CPU 的命令和参数存放在控制器的控制寄存器中，并对命令和地址译码。

2) 数据交换。通过数据寄存器进行数据交换。

①将驱动器中的比特流汇集在控制器的缓冲区中以形成字节块。

②实现 CPU 到控制器、控制器到 CPU 的双向数据传送。

③将控制器对设备的控制命令传送给设备控制器。

3) 设备状态的了解和报告。

设备控制器中有专门用来存放设备状态信息的寄存器或触发器，CPU 可以通过读取这些信息了解设备的当前状态。

4) 地址识别。

①设备控制器必须能识别它所控制的每个设备的地址。

②设备控制器中的寄存器本身应该有唯一的地址，以使 CPU 能向寄存器中读/写数据。

③将 CPU 要访问的外设地址送入控制器，由控制器的地址译码器译码后选中目标设备。

5) 数据缓冲。

在设备控制器中可以存储数据，作为 CPU 和 I/O 之间的缓冲。

6) 差错控制。

设备控制器需要具有差错检测功能，当通过数据校验发现数据传输出错时，可以向 CPU 报告，废弃错误数据，重新启动一次数据传输。

3. 设备控制器的组成

设备控制器的逻辑构成主要包括以下 3 部分。

1) 设备控制器与处理机的接口：数据线、控制线和地址线。

2) 设备控制器与设备的接口：设备与设备控制器接口中的 3 类信号为数据、状态和控制信号。

3) I/O 逻辑：I/O 逻辑主要由指令译码器和地址译码器两部分功能部件构成，将 CPU 的命令和地址分别译码，控制指定设备进行 I/O 操作。

设备控制器的逻辑结构如图 6-3 所示。

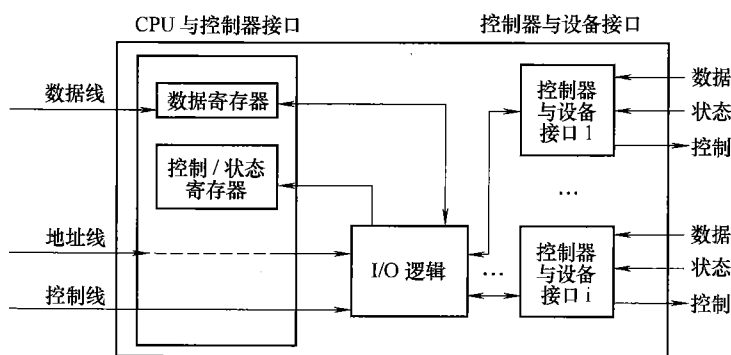


图 6-3 设备控制器的逻辑结构

四、I/O 通道

通道用于大型主机系统控制 I/O 设备，与控制设备结合，与微机和小型机的设备控制器具有对等的功能。即用来代替微机、小型机中的设备控制器，实现大型主机系统的 I/O 设备控制功能，提供操作系统与 I/O 设备间的接口。

I/O 通道是一种特殊的处理机，它具有执行 I/O 指令的能力，并通过执行通道程序来控制 I/O 操作。简单地说，通道是大型主机系统中专门用于 I/O 的专用计算机。引入通道能够使 CPU 从控制 I/O 的任务中解脱，使 CPU 与 I/O 并行工作，提高 CPU 的利用率和系统的吞吐量。

第二节 I/O 控制方式

输入/输出方式有早期的程序轮询控制方式。在中断机制被引入计算机系统后，输入/输出控制采用中断控制方式。为了提高块设备的输入/输出性能，可以利用 DMA (Direct Memory Access, 直接内存访问) 控制器对输入/输出进行 DMA 控制。通道控制方式则可以使输入/输出更大程度地独立于主机 CPU。在输入/输出控制方式的发展中始终追求的目标是尽量减少主机对输入/输出控制的干预，提高主机与输入/输出的并行程度，以提高整个系统的性能。本节主要介绍轮询、中断和 DMA 控制方式。

一、轮询

早期的计算机系统，因为没有中断机制，处理器对输入/输出的控制不得不采取程序轮询的方式。采用这种控制方式，主机试图发送 I/O 控制命令之前，先通过反复检测设备控制器状态寄存器的忙/闲标志位，若设备“忙”，主机继续检测该标志位，直到该位为“空闲”，主机发送 I/O 指令。在主机发送完 I/O 指令后，设备控制器把状态寄存器的忙/闲标志位再置成“忙”，主机再次进入轮询状态，以检测本次输入/输出是否结束。这种控制方式使 CPU 经常处于由于输入/输出而造成的循环测试状态，造成 CPU 的极大浪费，影响整个系统的吞吐量。

二、中断

现代计算机系统广泛采用中断控制方式完成对 I/O 的控制。采用中断控制方式的 I/O 工作模式是 CPU 执行进程中，发出输入/输出请求，若此时 I/O 设备忙，则进程阻塞等待。当处于“忙”状态的设备工作完毕，通过中断控制器发出中断请求信号，CPU 响应中断，执行对应该设备的中断处理程序，然后唤醒因等待该设备而被阻塞的进程。CPU 继续执行这个进程时，向设备控制器发送 I/O 指令，然后 CPU 被调度程序分配给某个进程，继续执行某个程序。自此，在设备控制器控制设备完成本次 I/O 的过程中，I/O 与 CPU 并行工作。当本次 I/O 结束后，设备控制器通过向 CPU 发送中断请求信号告知 CPU 本次数据传输结束。

中断控制的工作方式能使 CPU 与 I/O 设备在某些时间段上并行工作，提高 CPU 的利用率和系统的吞吐量。图 6-4 所示为 3 种 I/O 控制方式流程。

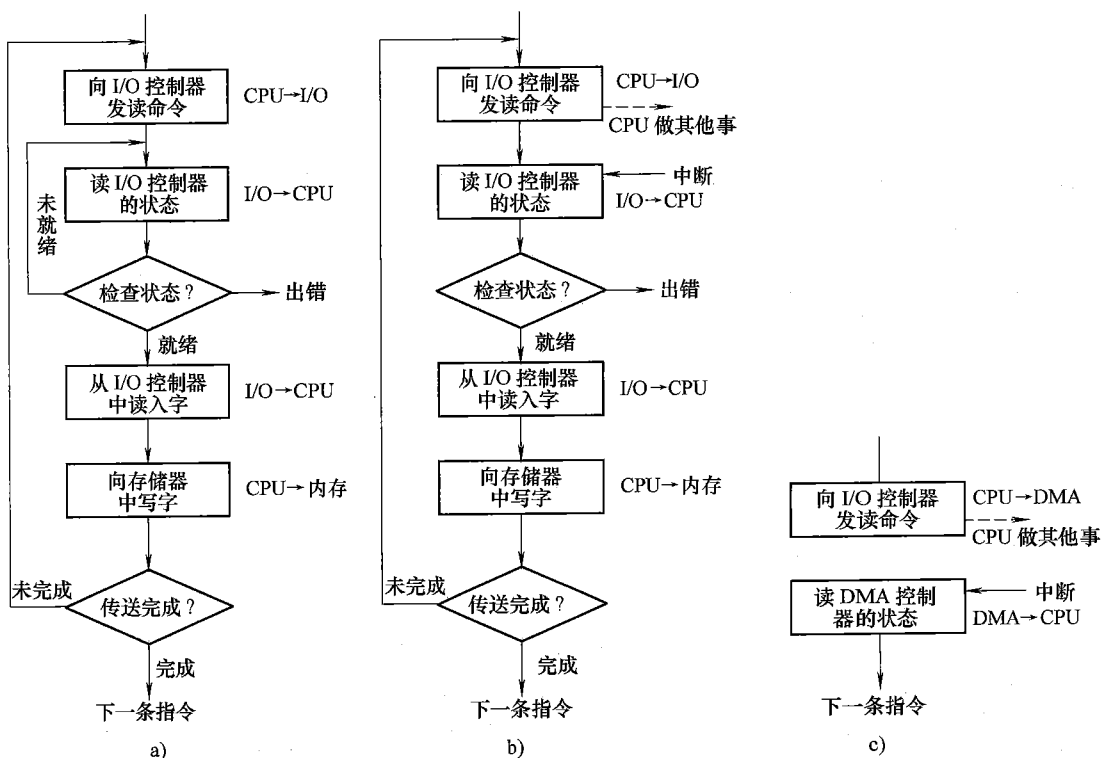


图 6-4 3 种 I/O 控制方式流程

a) 轮询方式 b) 中断控制方式 c) DMA 控制方式

三、DMA 控制方式

对于磁盘驱动器这类的设备，每次数据传输量较大，采用中断控制方式，传输一个数据块就需要进行多次中断处理。典型情况下，一个数据块大小为 512 B，采用中断控制方式，若磁盘控制器中数据寄存器为一个字节大小，则需要 CPU 执行一次中断处理。为了进一步提高 I/O 的速度和 CPU 与 I/O 的并行程度，可以采用 DMA 控制

方式。

DMA 控制需要特殊结构的设备控制器，DMA 控制器的逻辑组成包括 3 部分：主机与 DMA 的接口、DMA 与设备的接口，以及 I/O 控制逻辑。

为了实现主机与设备控制器之间成块数据的传送，在 DMA 控制器中设计了 4 类寄存器：命令/状态寄存器 CR、内存地址寄存器 MAR、数据寄存器 DR 和数据计数器 DC。DMA 控制器的逻辑结构如图 6-5 所示。

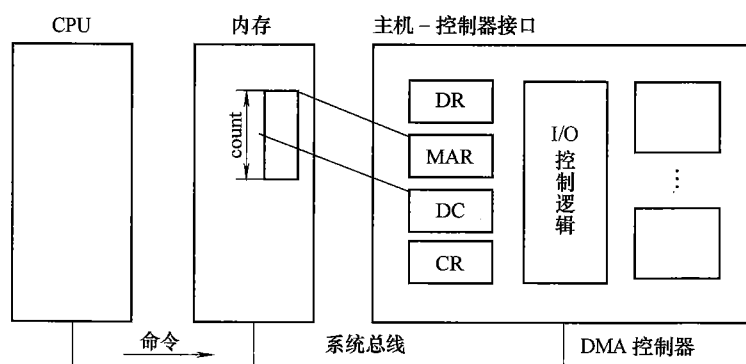


图 6-5 DMA 控制器的逻辑结构

1. 命令/状态寄存器 CR

用于接收从 CPU 发来的 I/O 命令或有关控制信息、设备状态。

2. 内存地址寄存器 MAR

存放内存地址，在输出数据时，存放输出数据在内存的起始地址，指示 DMA 应该从内存的什么地方读取输出数据。在输入数据时，存放输入数据将要被放入内存的起始地址，指示 DMA 应该把输入数据放到内存的什么地方。

3. 数据计数器 DC

指示 DMA，本次向 CPU 发中断信号前要读或写数据的次数。

4. 数据寄存器 DR

用于暂存 DMA 传输中要输入或输出的数据。

下面以从磁盘读数据为例，说明 DMA 控制方式的工作原理，以及 DMA 控制和中断控制的差别。

1) 没有使用 DMA 时磁盘读数据的过程。

①首先控制器一个比特一个比特地从设备完整地读出一块数据放入内部缓冲区中。

②确认该块数据的正确性。

③控制器发中断信号，CPU 执行中断处理程序，从控制器的设备寄存器中将数据读入内存。

2) 通过 DMA 从磁盘读数据的过程。

当 CPU 要从磁盘读入一个数据块时，便向磁盘控制器发送一条读命令。该命令被送到其中的命令寄存器 CR 中。同时，CPU 将本次读入数据将要放在内存中的起始地址送 DMA 的 MAR 寄存器，将本次要读的字节数送 DC 寄存器。然后，启动 DMA 控制器进行数据传送。在 DMA 控制输入的过程中，CPU 可以执行其他的进程。当本次读入的数据全部传送完

毕后，DMA 向 CPU 发送中断请求。

在 DMA 控制磁盘读入数据的过程中，每读入一个字(节)，便将该字(节)送到当前 MAR 指示的内存单元中，然后 MAR 的值递增，以指向下一个内存单元。DC 减 1，若 DC 递减后的值不为 0，说明本次数据传送没有结束，继续在 DMA 控制下传送下一个字节；若 DC 减 1 后变为 0，说明本次数据传输结束，由 DMA 向 CPU 发中断请求。图 6-6 所示为 DMA 工作方式流程。

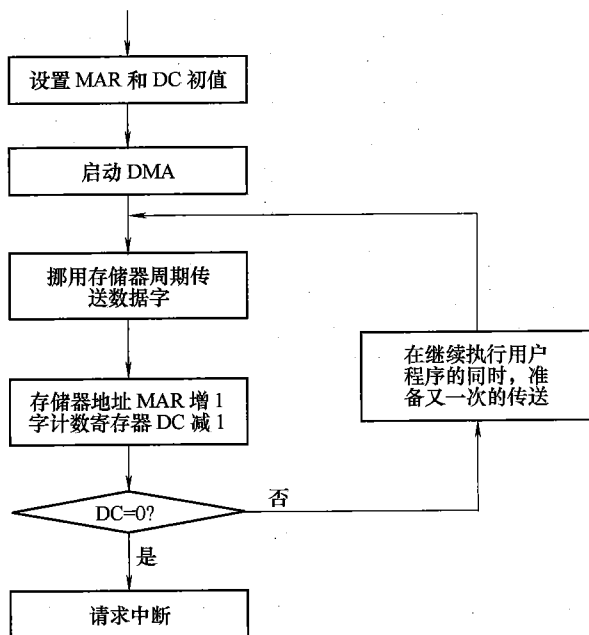


图 6-6 DMA 工作方式流程

第三节 缓冲管理

缓冲区是用来保存两个设备之间或设备与应用程序之间传输数据的内存区域。由于 CPU 的速度远高于 I/O 设备，为了尽可能使 CPU 与设备并行工作，提高系统的性能，通常需要操作系统在设备管理软件中提供缓冲区管理功能。

一、缓冲的引入

在数据到达速率与数据离去速率不同的地方，都可以引入缓冲区。引入缓冲区的主要原因可以归纳为以下两点。

1) 处理数据流的生产者与消费者之间的速度差异。例如，假如从调制解调器收到一个文件，并保存到硬盘上。调制解调器大约比硬盘慢数千倍，这种情况下，可以在内存中创建缓冲区以存放从调制解调器处收到的字节。当整个缓冲区填满时，就可以通过一次操作将缓冲区中的数据写入到磁盘中。由于写磁盘并不及时，而且调制解调器需要一个空间继续保存输入数据，因而需要两个缓冲区。当调制解调器填满第一个缓冲区后，就可以请求写磁盘。接着调制解调器开始填写第二个缓冲区，而这时第一个缓冲区中的数据正被写入磁盘。等到

调制解调器写满第二个缓冲区时，第一个缓冲区的数据也已写入磁盘。因此，调制解调器可以切换到第一个缓冲区，而第二个缓冲区中的数据可以开始写入磁盘。这种双缓冲将生产者与消费者进行分离，因而缓和了两者之间的时序要求。

2) 协调传输数据大小不一致的设备。这种不一致在计算机网络中特别常见，缓冲常常用来处理消息的分段和重组。在发送端，一个大消息分成若干小网络包。这些包通过网络传输，接收端将它们放在重组缓冲区内以生成完整的源数据镜像。

引入缓冲区除了可以缓和 CPU 与 I/O 设备之间速度不匹配的矛盾，还能降低对 CPU 中断频率的要求，放宽对中断响应时间的限制，提高 CPU 和 I/O 设备之间的并行性。

二、单缓冲

操作系统提供的最简单的缓冲类型是单缓冲区，如图 6-7 所示。当一个用户进程发出 I/O 请求时，操作系统为该操作分配一个位于主存的缓冲区。

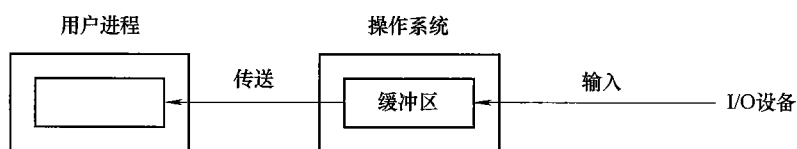


图 6-7 单缓冲工作示意图

对于面向块的设备，单缓冲区方案可以描述如下：输入数据被传送进入系统缓冲区。当传送完成时，进程将该块移到用户空间，并立即请求另一块系统缓冲区。用户进程可以在下一块数据正在读入系统缓冲区时，处理用户区中的那一块数据。相对于没有系统缓冲的情况，这种方法通常能提高速度。

类似的考虑也可以用于面向块的输出。当准备把数据发送到一台设备时，首先把它们从用户空间复制到系统缓冲区中，它们最终从系统缓冲区被传送给输出设备，此时 CPU 与输出设备可以并行工作。

对于面向流的 I/O，在每次传送一行的方式下，或者每次传送一个字节的方式下，可以使用单缓冲方案。键盘、打印机和传感器等都属于面向流的设备。

对于每次传送一行的 I/O，可以用缓冲区保存一行数据。在输入期间用户进程被阻塞，等待整行的到达。对于输出，用户进程可以把一行输出放置在缓冲区中，然后继续处理。只有当第一次输出操作的缓冲区内容清空之前，又需要发送第二行输出时，它才需要被阻塞。对于每次传送一个字节的 I/O，操作系统和用户进程之间的交互按照第三章进程同步讲述的生产者—消费者模型进行。

三、双缓冲

可以通过给操作系统指定两个系统缓冲区，如图 6-8 所示，对单缓冲方案进行改进。当一个进程往这一个缓冲区中传送数据（或从这个缓冲区中读取数据）时，操作系统正在清空（或填充）另一个缓冲区，这种技术称为双缓冲（Double Buffering），或缓冲交换（Buffering Swapping）。

双缓冲的性能比单缓冲的性能有所提高，但是这种提高是以增加复杂性为代价的。

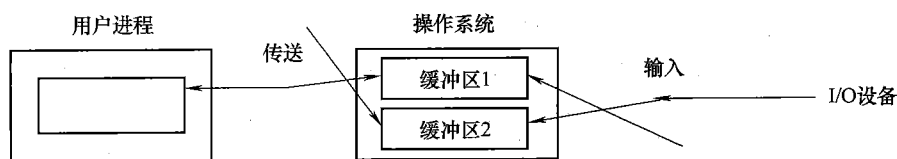


图 6-8 双缓冲工作示意图

四、循环缓冲

双缓冲方案可以平滑 I/O 设备和进程之间的数据流，如果某个特定进程的性能是关注的焦点，常常会希望相关 I/O 操作能够跟得上这个进程。如果这个进程突然快速执行了大量的 I/O，仅有双缓冲就不够了，在这种情况下，通常使用多于两个的缓冲区，如循环缓冲来解决这个问题。可以这样理解：在数据到达和数据离去的速度差别很大的情况下，需要增加缓冲区的数量。

1. 循环缓冲的组成

循环缓冲的组成如图 6-9 所示。

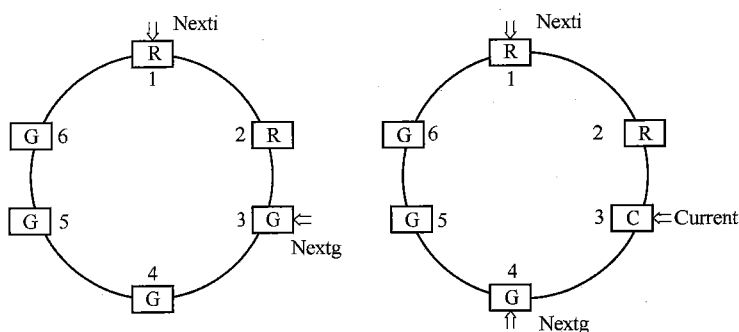


图 6-9 循环缓冲工作示意图

(1) 多个缓冲区

- 1) 空缓冲区 R：生产者进程下一个可用的空缓冲区。
- 2) 已装满数据的缓冲区 G：用于指示消费者进程下一个可用的装有产品的缓冲区。
- 3) 现行工作缓冲区 C：消费者进程正在使用的工作缓冲区。

(2) 多个指针

- 1) Nextg：用于指示消费者进程下一个可用的装有数据的缓冲区。
- 2) Nexti：用于指示生产者进程下一个可用的空缓冲区。
- 3) Current：用于指示进程正在使用的工作缓冲区。

2. 循环缓冲的使用

生产者进程和消费者进程可以利用 Getbuf 过程和 Releasebuf 过程来使用循环缓冲区。

(1) Getbuf 过程

消费者进程要使用缓冲区中的数据时，可调用 Getbuf 过程。该过程将 Nextg 指向的缓冲

区提供给进程使用，并把它改成由 Current 指针指向的现行工作缓冲区，同时使 Nextg 指向下一个可用的装有数据的缓冲区 G。而当生产者进程要使用空缓冲区来装数据时，也通过调用 Getbuf 过程，把 Nexti 指示的空缓冲区提供给生产者进程使用，然后使 Nexti 指向下一个空缓冲区 R。

(2) Releasebuf 过程

当进程使用完缓冲区之后，调用 Releasebuf 过程释放缓冲区。当消费者进程把 C 缓冲区中的数据提取完毕时，便调用 Releasebuf 过程释放 C 缓冲区，把该缓冲区改为空缓冲区 R。当生产者进程把缓冲区装满数据后，调用 Releasebuf 过程释放缓冲区，使装满数据的当前工作缓冲区成为 G 缓冲区。

3. 进程同步

由于使用缓冲可使生产者进程和消费者进程并行执行，相应地，指针 Nexti 和指针 Nextg 将不断地沿顺时针方向移动，在下列两种情况下需要保持两种进程同步的机制。

1) Nexti 指针追上 Nextg 指针，即生产者进程速度大于消费者进程速度，没有空缓冲区，全部缓冲区已满。此时，需要阻塞生产者进程，等待消费者进程为生产者进程释放空缓冲区 R。

2) Nextg 指针追上 Nexti 指针，消费者进程速度大于生产者进程速度，全部缓冲已空。此时，需要阻塞消费者进程，等待生产者进程为消费者进程释放装有数据的缓冲区 G。

五、缓冲池

公共缓冲池是被广泛应用的一种缓冲管理技术，公共缓冲池中设置多个可供若干进程共享的缓冲区，这种方式能提高缓冲区的利用率。

1. 缓冲池的组成

公共缓冲池既可用于输入，又可用于输出，其中至少包含 3 种类型的缓冲区、3 种缓冲队列和 4 种工作缓冲区。

1) 3 种类型的缓冲区。空缓冲区、装满输入数据的缓冲区和装满输出数据的缓冲区。

2) 3 种队列。

①空缓冲队列 emq：是由空缓冲区链接而成的队列。

②输入队列 inq：是由装满输入数据的缓冲区链接成的队列。

③输出队列 outq：是由装满输出数据的缓冲区链接成的队列。

3) 4 种工作缓冲区。

①收容输入数据的缓冲区。收容完输入数据后，缓冲区被插入输入队列中。

②提取输入数据的缓冲区。存在于输入队列中，进程需要输入数据时，先从输入队列中获取这种缓冲区。

③收容输出数据的缓冲区。收容完输出数据后，缓冲区被插入输出队列中。

④提取输出数据的缓冲区。存在于输出队列中，进程需要输出数据时，先从输出队列中获取这种缓冲区。

2. Getbuf 过程和 Putbuf 过程

对缓冲池的操作由 Getbuf 过程和 Putbuf 过程来完成。为了使进程能以互斥的方式访问缓冲池队列，可为每个队列设置一个相应的互斥信号量 MS(type)。为了保证进程同步使用

缓冲区, 可以为每个队列设置一个资源信号量 $RS(type)$ 。其中, $type$ 指示缓冲区队列类型, 可以是 emq 、 inq 或 $outq$ 。

(1) Getbuf 过程

Procedure Getbuf($type$)

Begin

```
Wait(RS( $type$ ));           //申请缓冲区
Wait(MS( $type$ ));           //申请缓冲队列的互斥访问权
B(number) = Takebuf( $type$ ); //提取缓冲区
Signal(MS( $type$ ));         //释放缓冲队列的互斥访问权
```

End

(2) Putbuf 过程

Procedure Putbuf($type$, $number$)

Begin

```
Wait(MS( $type$ ));           //申请缓冲队列的互斥访问权
Addbuf( $type$ ,  $number$ );     //向  $type$  指示的队列中插入  $number$  指示的缓冲区,  $number$  可以是
                               hin、sin、sout 或 hout, 见图 6-10
Signal(MS( $type$ ));         //释放缓冲队列的互斥访问权
Signal(RS( $type$ ));         //释放缓冲区
```

End

3. 缓冲区的工作方式

缓冲区可以工作在收容输入、提取输入、收容输出和提取输出 4 种工作方式下, 如图 6-10 所示。

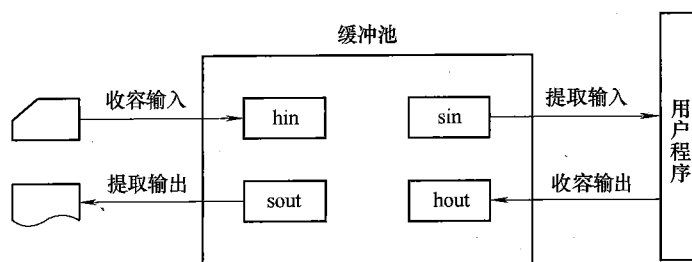


图 6-10 公共缓冲池中缓冲区的工作方式

(1) 收容输入

在进程需要收容输入数据时, 要先从空缓冲队列提取一个空缓冲区, 将输入数据写入缓冲后, 再把装入了输入数据的缓冲区插入到输入队列中去。操作步骤如下。

- 1) Getbuf(emq)。
- 2) 将输入数据写入缓冲区。
- 3) putbuf(inq , hin)。

(2) 提取输入

当进程需要输入数据时(如计算进程需要提取输入数据, 然后对数据进行计算处理), 先从输入队列提取输入缓冲区, 然后从中提取输入数据, 最后把缓冲区作为空缓冲区插入空

缓冲队列。操作步骤如下。

- 1) Getbuf(inq)。
- 2) 从缓冲区提取数据。
- 3) putbuf(emq, sin)。
- (3) 收容输出

在进程需要收容输出数据时,要先从空缓冲队列提取一个空缓冲区,将输出数据写入缓冲后,再把装入了输出数据的缓冲区插入到输出队列中去。操作步骤如下。

- 1) Getbuf(emq)。
- 2) 将输出数据写入缓冲区。
- 3) putbuf(outq, hout)。
- (4) 提取输出

当进程需要输出数据时(如打印进程需要提取输出数据送打印机),先从输出队列提取输出缓冲区,然后从中提取输出数据,最后把这个缓冲区插入空缓冲队列。操作步骤如下。

- 1) Getbuf(outq)。
- 2) 输出数据。
- 3) putbuf(emq, sout)。

第四节 设备分配

在多道程序环境下,系统中的设备不允许用户自行使用,而必须由系统分配。每当进程向系统提出 I/O 请求时,设备分配程序便按照一定的策略,把设备分配给用户。设备分配功能的完成,需要记录设备情况的数据结构和设备分配算法。

一、设备分配中的数据结构

支持设备分配的数据结构需要记录设备的状态(忙或空闲)、设备类型等基本信息。下面是一种支持设备分配的数据结构设计方案,该方案包括设备控制表、控制器控制表、通道控制表和系统设备表。这些数据结构记录了设备、控制器、通道的状态及控制器进行控制所需要的信息。

1. 设备控制表 DCT(Device Control Table)

系统为每个设备建立一张设备控制表,多台设备的设备控制表构成设备控制表集合。每张设备控制表包含以下信息。

- 1) 设备类型。
- 2) 设备标识符。
- 3) 设备状态:忙/闲。
- 4) 指向控制器表的指针。
- 5) 重复执行的次数或时间。

6) 设备队列的队首指针。设备队列也称设备请求队列,是因请求设备而被阻塞的进程的 PCB 构成的队列。设备队列的队首指针指向队首的 PCB。

2. 控制器控制表 COCT(Controller Control Table)

系统为每个控制器设置一张用于记录该控制器信息的控制器控制表，控制器控制表中通常包含以下几个字段。

- 1) 控制器标识符。
- 2) 控制器状态。
- 3) 与控制器相连接的通道表指针。
- 4) 控制器队列的队首指针。
- 5) 控制器队列的队尾指针。

3. 通道控制表 CHCT(Channel Control Table)

在一些主机系统中还有通道设备，系统会为每个通道设备设置一张通道控制表，通道控制表包括以下几个字段。

- 1) 通道标识符。
- 2) 通道状态。
- 3) 与通道连接的控制器表首址。
- 4) 通道队列的队首指针。
- 5) 通道队列的队尾指针。

4. 系统设备表 SDT(System Device Table)

系统设备表是系统范围的数据结构，其中记录了系统中全部设备的情况。每个设备占一个表目，其中包括设备类型、设备标识符、设备控制表及设备驱动程序的入口地址。

系统设备表、控制器控制表、设备控制表之间的关系及利用数据结构进行设备分配的流程如图 6-11 所示；其中的逻辑设备表 LUT 的用途及一般结构见后面“三、设备独立性”一节。

二、设备分配

为了使系统有条不紊地工作，系统在分配设备时应考虑以下 3 个因素。

- 1) 设备的固有属性。
- 2) 设备分配算法。
- 3) 设备分配时的安全性。

1. 设备的固有属性

在分配设备时，首先应考虑与设备分配有关的设备属性。设备的固有属性可分成 3 种：第一种是独占性，指这种设备在一段时间内只允许一个进程独占，即“临界资源”。第二种是共享性，指这种设备允许多个进程同时共享。第三种是可虚拟性，指设备本身虽是独占设备，但经过某种技术处理，可以把它改造成虚拟设备。对上述的独占、共享、可虚拟 3 种设备应采取不同的分配策略。

(1) 独占设备

对于独占设备，应采用独享分配策略，即将一个设备分配给某进程后，便由该进程独占，直至进程完成或释放该设备。然后，系统才能再将该设备分配给其他进程使用。这种分配策略的缺点是，设备得不到充分利用，而且还可能引起死锁。

(2) 共享设备

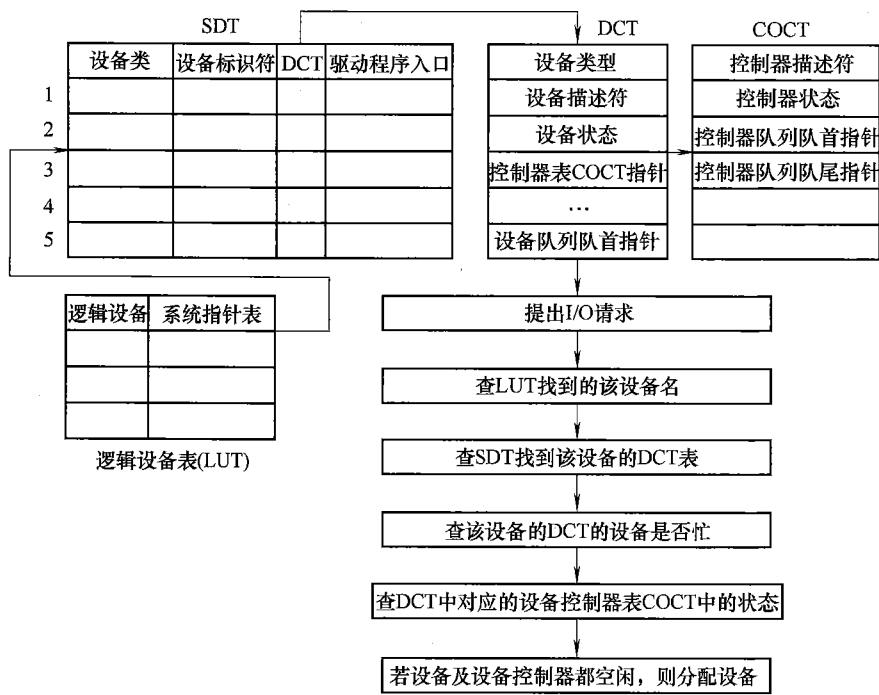


图 6-11 设备分配数据结构和分配流程的关系

对于共享设备，可同时分配给多个进程使用，此时需要注意对这些进程访问该设备的先后顺序进行合理的调度。

(3) 可虚拟设备

由于可虚拟设备是指一台物理设备在采用虚拟技术后可变成多台逻辑上的所谓虚拟设备，因而一台可虚拟设备是可共享的设备，可以将它同时分配给多个进程使用，并对进程访问该设备的先后顺序进行控制。

2. 设备分配算法

对设备分配的算法，与进程调度的算法有相似之处，但相对简单，通常只采用以下两种分配算法。

(1) 先来先服务

当有多个进程对同一设备提出 I/O 请求时，该算法是根据进程对某设备提出请求的先后顺序将这些进程排成一个设备请求队列，设备分配程序总是先把设备分配给队首进程。

(2) 基于优先权的分配算法

在进程调度中的这种策略，是优先权高的进程优先获得处理机。如果对这种高优先权进程所提出的 I/O 请求也赋予高优先权，显然有助于这种进程尽快完成。在利用该算法形成设备队列时，将优先权高的进程排在设备队列前面，而对于优先级相同的 I/O 请求，则按先来先服务原则排队。

3. 设备分配方式

从进程运行的安全性上考虑，设备分配有以下两种方式。

(1) 安全分配方式

在这种分配方式中，每当进程发出 I/O 请求后，便进入阻塞状态，直到其 I/O 操作完成时才被唤醒。在采用这种分配策略时，一旦进程已经获得某种设备(资源)后便阻塞，使该进程不可能再请求任何其他资源，而在它运行时又不能保持任何资源。因此，这种分配方式已经摒弃了造成死锁的 4 个必要条件之一的“请求和保持”条件，从而使设备分配是安全的。其缺点是进程进展缓慢，即对于同一个进程，CPU 与 I/O 设备是串行工作的。

(2) 不安全分配方式

在这种分配方式中，进程在发出 I/O 请求后仍在继续运行，需要时又发出其他的 I/O 请求。仅当进程所请求的设备已被另一个进程占用时，请求进程才进入阻塞状态。这种分配方式的优点是，一个进程可同时操作多个设备，使进程推进迅速。其缺点是分配不安全，因为它可能具备“请求和保持”条件，从而可能造成死锁。

三、设备独立性

1. 设备独立性的概念

为了提高操作系统的可适应性和可扩展性，在现代操作系统中都毫无例外地实现了设备独立性，也称为设备无关性。其基本含义是应用程序独立于具体使用的物理设备。为了实现设备独立性，引入了逻辑设备和物理设备这两个概念。在应用程序中，使用逻辑设备名称来请求使用某类设备，而系统在实际执行时，还必须使用物理设备名称。因此，系统必须具有将逻辑设备名称转换为物理设备名称的功能。

2. 实现设备独立性带来的好处

实现设备独立性带来的好处如下。

- 1) 应用程序与物理设备无关，系统增减或变更外围设备时不需要修改应用程序。
- 2) 易于处理输入/输出设备的故障。例如，某台打印机发生故障时，可用另一台不同型号、不同品牌的打印机替换，而不用修改应用程序。
- 3) 提高了系统的可靠性，增加了设备分配的灵活性。

3. 设备独立软件

设备独立软件完成的主要功能如下。

(1) 执行所有设备的公有操作

执行的操作包括：独占设备的分配与回收、将逻辑设备名映射为物理设备名、对设备进行保护、缓冲管理和差错控制。为了实现逻辑设备名到物理设备名的转化，可以利用称为逻辑设备表 LUT(Logical Unit Table)的数据结构。在该表的每个表目中都包含逻辑设备名、物理设备名。LUT 的设置可采取两种方式：为整个系统设置一张 LUT 或者为每个用户设置一张 LUT。

(2) 向用户层软件提供统一的接口

设备独立软件向用户层屏蔽访问硬件的细节，向应用软件和最终用户提供简单、统一的访问接口。例如，对各种设备的读操作在应用程序中都使用 read 函数调用，写操作都使用 write 函数调用。

例 6-1：在编程级，如果要在程序中在打印机/dev/ptl 上输出字符串“hello”，用下列语句即可。

```
fd = open("/dev/ptl");
```

```
write("hello");
close(fd);
```

四、独占设备的分配程序

对于具有 I/O 通道的系统，在进程提出 I/O 请求后，系统的设备分配程序可按下列步骤进行设备分配。

(1) 分配设备

根据用户请求的设备的物理名，查找系统设备表，从中找出该设备的设备控制表，检查设备控制表中的设备状态字。若设备忙，则将进程阻塞在该设备的阻塞队列中；若设备空闲，则根据设备分配算法将设备分配给进程。

(2) 分配控制器

若系统为进程分配了其请求的设备，就到该设备的控制表中找出与该设备连接的控制器的 COCT，即设备控制器控制表，检查其中的状态字段。若该控制器忙，则将请求 I/O 的进程阻塞在该设备控制器的阻塞队列中；若控制器空闲，则将它分配给进程。

(3) 分配通道

在有通道的系统中，还需要从相应的设备控制器控制表中找到与该控制器连接的通道控制表，检查表中的通道状态字段。若通道忙，则将进程阻塞在该通道的阻塞队列上；若通道空闲，则将该通道分配给进程。

当进程获得了设备、设备控制器或者获得了设备、设备控制器和通道时(在有通道的系统中)，系统的本次设备分配才算成功，系统可以启动进程的 I/O。

五、SPOOLing 技术

1. 什么是 SPOOLing

在多道程序环境下，利用一道程序来模拟脱机输入时的外围控制机的功能，把低速 I/O 设备上的数据传送到高速输出磁盘上，再利用另一道程序来模拟脱机输出时外围控制机的功能，把数据从磁盘传送到低速输出设备上。这种在联机情况下实现的同时外围操作称为 SPOOLing (Simultaneous Peripheral Operations On-Line)。

(1) SPOOLing 系统的组成

SPOOLing 系统的组成如图 6-12 所示。

1) 输入井和输出井。这是位于磁盘上的两个分别存放输入数据和输出数据的存储区域，作为大量输入或输出数据的缓存。

2) 输入缓冲区和输出缓冲区。输入缓冲区用来暂存由输入设备送来的输入数据，输出缓存区用来存放从输出井送来的输出数据，以后再传给输出设备。

3) 输入进程 SP_i 和输出进程 SP_o 。输入进程把输入设备送来的数据送入输入缓存，再把缓存中的数据送入输入井。当消费者进程需要输入数据时，再从输入井把输入数据读入

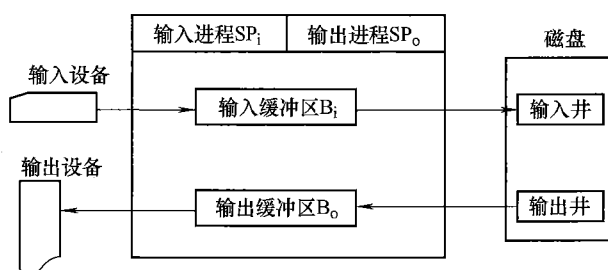


图 6-12 SPOOLing 系统的组成

内存。

输出进程把要输出的数据从内存送入输出井，当需要输出数据时，再从输出井把数据读到输出缓存(如打印缓存)，数据从输出缓存送往输出设备。

4) 请求 I/O 队列。请求输入或输出的进程提交的输入/输出任务组成的队列。

2. 利用 SPOOLing 技术实现共享打印机

当用户进程提出打印请求时，SPOOLing 系统先为用户做下列两件事。

1) 由输出进程在输出井中申请空闲盘块区，并将要打印的数据送入其中。

2) 输出进程再为用户申请并填写一张用户请求打印表，将该表挂到请求打印队列上。

当打印机空闲时，输出进程完成以下动作。

①从请求打印队列队首取一张请求打印表。

②将打印数据从输出井送到打印机缓冲区(输出缓冲区)。

③打印。

④打印完毕，若打印队列不为空，则转第①步。

SPOOLing 系统的特点如下。

1) 提高了 I/O 速度。由于使用了磁盘作为低速设备(如打印机、磁带等)的大容量缓存，提高了输入/输出的速度。

2) 将独占设备改造为共享设备。通过 SPOOLing 系统使独占设备变为了逻辑上的共享设备，系统可以同时接受多个用户对设备的访问请求。

3) 实现了虚拟设备功能。把一台物理上只能互斥使用的设备，变为了从用户眼里看到的共享设备。宏观上看，系统可以同时响应多个用户对设备的请求。微观上看，任意时刻设备只能为某一个用户进程服务，SPOOLing 系统实现了将独占设备变换成多个逻辑设备的功能。

第五节 I/O 软件原理

I/O 软件的总体目标是将软件组织成一种层次结构，低层软件用来屏蔽硬件的具体细节，高层软件则主要是为用户提供一个简洁、规范的界面。用户程序及操作系统中设备管理软件的构成和关系如图 6-13 所示，将设备管理软件组织成 4 个层次，即

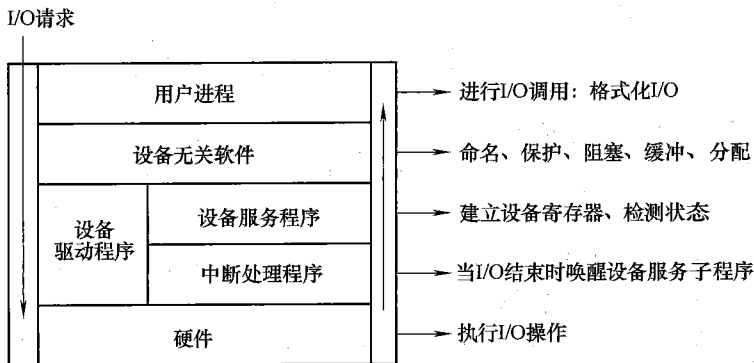


图 6-13 设备管理软件的层次结构

- 1) 用户层软件。
- 2) 与设备无关的软件层。
- 3) 设备驱动程序。
- 4) 中断处理程序(底层)。

设备管理软件与硬件关系最密切的是设备驱动程序,包括设备服务程序和中断处理程序。设备驱动程序上层是设备无关软件,通常完成设备命名、设备分配、设备独立性和缓冲管理等功能。最上层的用户进程向系统发送 I/O 请求,显示 I/O 操作的结果,提供用户与设备的接口。

一、设备管理软件的功能

本节综述设备管理软件的功能,后面的内容分别说明设备驱动程序、I/O 设备中断处理程序的功能和设备无关软件。操作系统设备管理软件(I/O 软件)应该实现以下功能。

1. 实现 I/O 设备的独立性

当用户使用的设备发生变化时,比如用激光打印机替代了喷墨打印机,应用程序的代码不需要修改。操作系统向应用软件层提供的这一支持方便了应用程序的开发和维护。

2. 错误处理

错误应该在尽可能接近硬件的地方处理,只有在低层软件处理不了的情况下才通知高层软件。例如,控制器进行一个读操作,它应该尽量处理并完成操作,如果因为某种故障控制器处理不了,则交给设备驱动程序,可能只需重读一次就可以解决问题。

3. 异步传输

多数物理 I/O 是异步传输,即 CPU 在启动传输操作后便可以转向其他工作,直到中断到达。

4. 缓冲管理

由于设备之间的速度差异,必须提供缓冲管理,为所有的块设备和字符设备提供缓冲管理功能,并向高层软件屏蔽由于设备差异带来的缓冲管理实现的具体细节。

5. 设备的分配和释放

对共享设备和互斥设备应该采取不同的方式为用户请求分配设备。设备使用完毕,要完成对设备的释放。

6. 实现 I/O 控制方式

针对不同的设备提供不同的 I/O 控制方式,如对打印机、键盘等字符设备实现中断控制。而对磁盘这样的块设备既可以采用中断控制方式,也可以采用 DMA 控制方式。但是,鉴于磁盘传输数据的特点,一般都采用 DMA 控制方式。

操作系统通过将 I/O 软件组织成如图 6-13 所示的 4 个层次,可以合理、高效地实现以上目标。

二、中断处理程序

I/O 中断处理程序的作用是将发出 I/O 请求而被阻塞的进程唤醒。用户进程在发出 I/O 请求后,由于等待 I/O 的完成而被阻塞。CPU 转去执行其他任务,当 I/O 任务完成,控制器向 CPU 发中断请求信号,CPU 转去执行中断处理程序,由中断处理程序唤醒被阻塞的设

备用户进程。

三、设备驱动程序

设备驱动程序是 I/O 进程与设备控制器之间的通信程序，其主要任务是接受上层软件发来的抽象的 I/O 请求，如 read 或 write 命令，把它们转换为具体要求后，发送给设备控制器，启动设备去执行。此外，它也将由设备控制器发来的信号传送给上层软件。设备驱动程序中包含所有与设备相关的代码。每个设备驱动程序只处理一种设备，或者一类紧密相关的设备。开发驱动程序需要知道某类设备控制器有多少个寄存器及它们的用途。驱动程序工作的简化流程如图 6-14 所示。

下面举例说明驱动程序的作用。例如，当有一个读第 n 块磁盘的请求时，磁盘驱动程序的工作如下。

- 1) 计算出所请求块的物理地址。
- 2) 检查驱动器电机是否正在运转。
- 3) 检查磁头臂是否定位在正确的柱面。
- 4) 确定需要哪些控制器命令及命令的执行顺序。
- 5) 向设备控制器的设备寄存器中写入命令。
- 6) I/O 完成后，向上层软件传送数据。

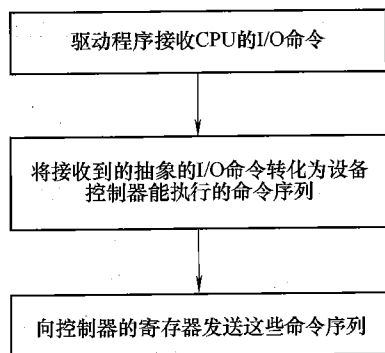


图 6-14 设备驱动程序的简化工作流程图

设备驱动程序属于操作系统的内核程序，但是一般由设备生产厂商开发，销售硬件设备时附送给用户，并不是由操作系统厂商提供。设备驱动程序要遵循操作系统提供的内核与设备驱动的接口标准。由于不同的操作系统提供的内核与驱动程序接口不一样，要使驱动程序在不同的操作系统环境中运行，对同一种设备需要开发针对不同操作系统的驱动程序。人们经常说某个驱动程序是 for Windows 或者 for Linux 的，就是出于这个原因。

四、与硬件无关的 I/O 软件

设备无关软件和设备驱动程序之间的精确界限在各个系统中不尽相同。对于一些以设备无关方式完成的功能，在实际中由于考虑到执行效率等因素，也可以考虑由驱动程序完成。

设备无关 I/O 软件的功能如下。

- 1) 设备命名：将设备名映射到相应的驱动程序。
- 2) 设备保护：为设备设置合理的访问权限。
- 3) 提供独立于设备的块大小。例如，不同磁盘的扇区大小可能不同，设备无关软件屏蔽了这一事实并向高层软件提供统一的数据块大小。例如，扇区大小为 512 B，若逻辑磁盘块大小为 1 KB，则软件发出读两个连续扇区的命令，将连续两个扇区作为一个大小为 1 KB 的逻辑块来处理。

- 4) 为块设备和字符设备提供必要的缓冲技术。

5) 块设备的存储分配。当创建了一个文件并向其输入数据时，该文件必须被分配新的磁盘块。为了完成分配工作，操作系统需要为每个磁盘都配置一张记录空闲盘块的表或位图，但定位一个空闲块的算法是独立于设备的，因此可以在高于驱动程序的层次处理。

- 6) 分配和释放独立设备。
- 7) 错误处理。

第六节 磁盘管理

磁盘存储器不仅容量大,存取速度快,而且可以实现随机存取,是存放大量程序和数据理想设备。在现代计算机系统中都配置了磁盘存储器,并以它为主存放文件。计算机系统中对文件的操作,基本上都涉及对磁盘的访问。磁盘 I/O 速度的高低和磁盘系统的可靠性,都将直接影响系统性能。磁盘管理的重要目标是提高磁盘空间利用率和磁盘访问速度。

一、磁盘结构

磁盘设备是一种复杂的机电设备,本节仅对磁盘的数据组织和格式、磁盘的类型和访问时间等方面进行简要介绍。

1. 数据的组织和格式

磁盘设备可包括一个或多个物理盘片,每个磁盘片分一个或两个存储面(Surface),如图 6-15a 所示。每个盘面被组织成若干个同心环,这种环称为磁道(Track),各磁道之间留有必要的间隙。为使处理简单,在每条磁道上可存储相同数目的二进制位。这样,磁盘密度即是每英寸中所存储的位数,显然是内层磁道的密度较外层磁道的密度高。每条磁道又被划分成若干个扇区(Sector)。图 6-15b 所示为一个磁道分成 8 个扇区,各扇区之间保留一定的间隙。

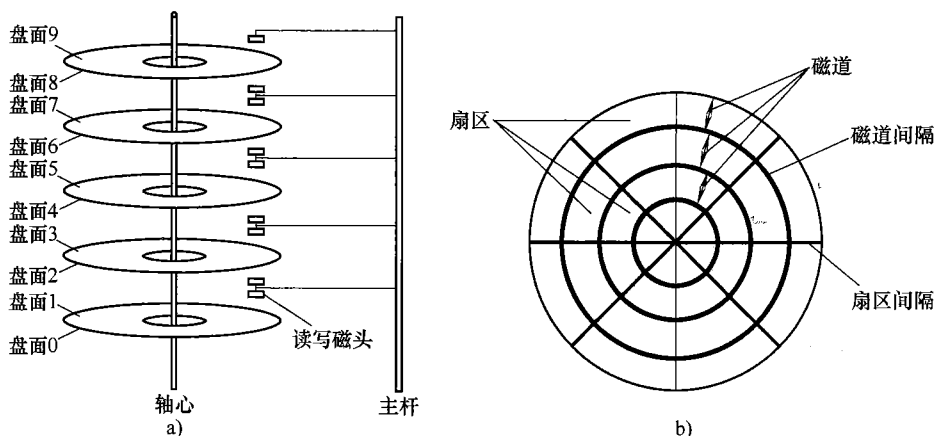


图 6-15 磁盘的结构和布局

a) 磁盘驱动器的结构 b) 磁盘的数据布局

一个物理记录存储在一个扇区上,磁盘上存储的物理记录数目是由扇区数、磁道数及盘面数所决定的。

为了提高磁盘的存储容量,充分利用磁盘外磁道的存储能力,现代磁盘不再把内外磁道划分为相同数目的扇区,而是利用外层磁道容量较内层磁道大的特点,将盘面划分成若干条

环带,使得同一环带内的所有磁道具有相同的扇区数。显然,外层环带的磁道拥有较内层环带的磁道更多的扇区。为了减少这种磁道和扇区在盘面分布的几何形式变化对驱动程序的影响,大多数现代磁盘都隐藏了这些细节。

为了在磁盘存储数据,必须先将磁盘低级格式化。图 6-16 显示出了一种温盘(温切斯特盘)中一条磁道格式化的情况。其中每条磁道含有 30 个固定大小的扇区,每个扇区容量为 600 个字节。其中 512 个字节存放数据,其余的用于存放控制信息。每个扇区包括两个字段。

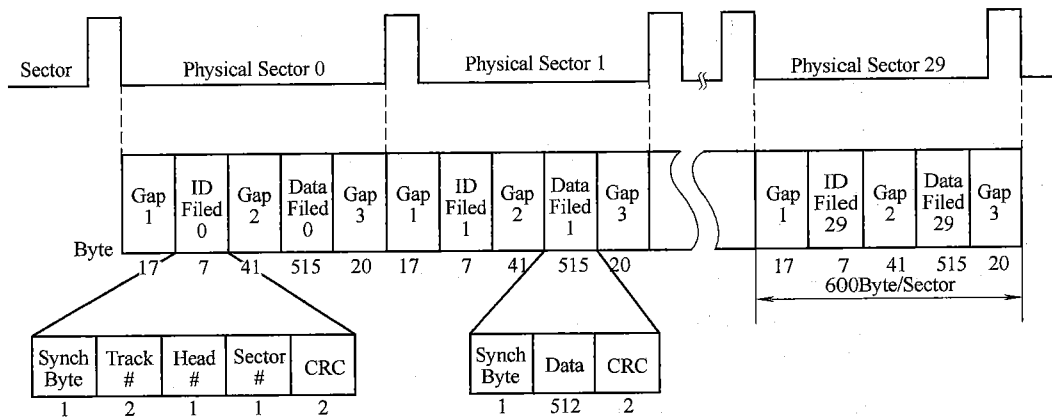


图 6-16 磁盘的格式化

1) 标识符字段,其中一个字节的 SYNCH 具有特定的位图像,作为该字段的定界符。利用磁道号、磁头号及扇区号三者来标识一个扇区。CRC 字段用于段校验。

2) 数据字段,其中可存放 512 个字节的数据。

磁盘格式化完成后,一般要对磁盘分区。在逻辑上,每个分区就是一个独立的逻辑磁盘。每个分区的起始扇区和大小都被记录在磁盘 0 扇区的主引导记录分区表所包含的分区表中。在这个分区表中必须有一个分区被标记成活动的,以保证能够从硬盘引导系统。

但是,在真正可以使用磁盘前,还需要对磁盘进行一次高级格式化,即设置一个引导块、根目录和一个空文件系统,同时在分区表中标记该分区所使用的文件系统。

2. 磁盘的类型

对磁盘可以从不同的角度进行分类。最常见的有:将磁盘分成硬盘和软盘、单片盘和多片盘、固定头磁盘和活动头(移动头)磁盘等。下面仅对固定头磁盘和移动头磁盘进行介绍。

(1) 固定头磁盘

这种磁盘在每条磁道上都有读/写磁头,所有的磁头都被装在一个刚性磁臂中。通过这些磁头可访问各磁道且进行并行读/写,有效地提高了磁盘的 I/O 速度。这种结构的磁盘主要用于大容量磁盘上。

(2) 移动头磁盘

这种磁盘每一个盘面仅配有一个磁头,也被装入磁臂中。为了能访问该盘面的所有磁

道,该磁头必须能移动并进行寻道。可见,移动磁头仅能以串行方式读/写,致使磁盘读写速度较慢。但由于其结构简单,故仍广泛应用于中小型磁盘设备中。在微型机上配置的硬盘采用移动磁头结构,故本节主要针对这类磁盘的 I/O 进行讨论。

3. 磁盘的访问时间

磁盘设备在工作时以恒定速率旋转。为了读或写,磁头必须能移动到所要求的磁道上,并等待所要求的扇区的开始位置旋转到磁头下,然后再开始读或写数据。故可把对磁盘的访问时间分成以下 3 部分。

(1) 寻道时间

这是指把磁臂(磁头)移动到指定磁道上所经历的时间。该时间是启动磁臂的时间与磁头移动 n 条磁道所花费的时间之和。

(2) 旋转延迟时间

这是指将指定扇区移动到磁头下面所经历的时间。不同的磁盘类型,旋转速度差别很大。

(3) 传输时间

这是指把数据从磁盘读出或向磁盘写入数据时所经历的时间。其大小与每次所读/写的字节数和旋转速度有关。

在磁盘访问时间中,寻道时间和旋转延迟时间基本上都与所读/写数据的多少无关,而且寻道时间和旋转延迟时间通常占据了访问时间中的大头。适当地集中数据在磁盘上存放的位置,可以减少磁臂移动距离,这将有利于提高传输速率。

二、磁盘调度

磁盘是可供多个进程共享的设备,当有多个进程都要求访问磁盘时,应采用一种最佳调度算法,以使各个进程对磁盘的平均访问时间最短。由于在访问磁盘的时间中,主要是寻道时间,因此磁盘调度的一个重要目标是使磁盘的平均寻道时间最少。目前常用的磁盘调度算法有先来先服务、最短寻道时间优先及扫描等算法,下面逐一进行介绍。

1. 先来先服务(First Come First Served, FCFS)

这是一种最简单的磁盘调度算法。它根据进程请求访问磁盘的先后顺序进行调度。此算法的优点是公平、简单,且每个进程的请求都能依次得到处理,不会出现某一进程的请求长期得不到满足的情况。但此算法由于未对寻道进行优化,致使平均寻道时间可能较长。表 6-1 给出了有 9 个进程先后提出磁盘 I/O 请求时,按 FCFS 算法进行调度的情况。这里将进程号(请求者)按他们发出的请求的先后顺序排队。这样,平均寻道距离为 55.3 条磁道,与后面即将讲到的几种调度算法相比,其平均寻道距离较大,故 FCFS 算法仅适用于请求磁盘 I/O 的进程数目较少的场合。

2. 最短寻道时间优先(Shortest Seek Time First, SSTF)

该算法选择这样的进程:其要求访问的磁道与当前磁头所在的磁道距离最近,以使每次的寻道时间最短。但这种算法不能保证平均寻道时间最短。表 6-2 给出了 SSTF 算法进行调度时,各进程被调度的顺序、每次磁头移动的距离,以及 9 次调度磁头平均移动的距离。比较表 6-1 和表 6-2 可以看出,SSTF 算法的每次磁头移动平均距离明显低于 FCFS 的距离,因而 SSTF 较之 FCFS 有更好的寻道性能,故曾一度被广泛采用。

表 6-1 FCFS 调度算法

(从 100 号磁道开始)	
被访问的下一个磁道号	移动距离(磁道数)
55	45
58	3
39	19
18	21
90	72
160	70
150	10
38	112
184	146
平均寻道长度: 55.3	

表 6-2 SSTF 调度算法

(从 100 号磁道开始)	
被访问的下一个磁道号	移动距离(磁道数)
90	10
58	32
55	3
39	16
38	1
18	20
150	132
160	10
184	24
平均寻道长度: 27.5	

3. 扫描(SCAN)算法

(1) 进程“饥饿”现象

SSTF 算法虽然能获得较好的寻道性能,但却可能导致某个进程发生“饥饿”(Starvation)现象。因为只要不断有新进程的请求到达,且其所要访问的磁道与磁头当前所在磁道的距离较近,这种新进程的 I/O 请求必然优先被满足,导致所要访问的磁道距离与磁头所在位置较远的磁盘任务总是不能得到调度。对 SSTF 算法略加修改后所形成的 SCAN 算法可防止进程出现“饥饿”现象。

(2) SCAN 算法

该算法不仅考虑到要访问的磁道与当前磁道的距离,更优先考虑磁头当前的移动方向。例如,当磁头正在自里向外移动时,SCAN 算法所考虑的下一个访问对象应是其要访问的磁道既在当前磁道之外,又是距离最近的。这样自里向外地访问,直至再无更外的磁道需要访问时,才将磁臂换为自外向里移动。这时,同样也是每次选择这样的进程来调度,即要访问的磁道在当前位置内距离最近者。这样,磁头逐步地自外向里移动,直至再无更里面的磁道要访问,从而避免出现“饥饿”现象。由于在这种算法中磁头移动的规律颇似电梯的运行,因而又常称之为电梯调度算法。表 6-3 给出了按 SCAN 算法对 9 个进程进行调度及磁头移动的情况。

4. 循环扫描(CSCAN)算法

SCAN 算法既能获得较好的寻道性能,又防止了“饥饿”现象,故被广泛用于大、中、小型机器和网络中的磁盘调度。但 SCAN 也存在这样的问题:当磁头刚从里向外移动而越过了某一磁道时,恰好又有一进程请求访问此磁道。这时,该进程必须等待,待磁头继续自里向外,然后再自外向里扫描完所有要访问的磁道后,才处理该进程的请求,致使该进程的请求被大大推迟。为了减少这种延迟,CSCAN 算法规定磁头是单向移动。例如,只是自里向外移动,当磁头移到最外的磁道后,磁头立即返回到最里的要访问的磁道,亦即将最小磁道号紧接着最大磁道号构成循环,进行循环扫描。表 6-4 给出了 CSCAN 算法对 9 个进程调度的顺序及每次磁头移动的距离。

表 6-3 SCAN 调度算法

从 100#磁道开始, 向磁道号加方向访问	
被访问的下一个磁道号	移动距离(磁道数)
150	50
160	10
184	24
90	94
58	32
55	3
39	16
38	1
18	20
平均寻道长度: 27.8	

表 6-4 CSCAN 调度算法

从 100#磁道开始, 向磁道号加方向访问	
被访问的下一个磁道号	移动距离(磁道数)
150	50
160	10
184	24
18	166
38	20
39	1
55	16
58	3
90	32
平均寻道长度: 35.8	

5. NStepSCAN 和 FSCAN 调度算法

(1) NStepSCAN 算法

在 SSTF、SCAN 及 CSCAN 几种调度算法中, 都可能会出现磁臂停留在某处不动的情况, 例如, 有一个或几个进程对某一磁道有较高的访问频率, 即这个(些)进程反复请求对某一磁道的 I/O 操作, 从而垄断了整个磁盘设备。把这一现象称为“磁臂粘着”(Armstickiness), 在高密度磁盘上容易出现此情况。NStepSCAN 算法是将磁盘请求队列分成若干个长度为 N 的子队列, 磁盘调度将按 FCFS 算法依次处理这些子队列。每处理一个队列时又是按 SCAN 算法, 对一个队列处理完后, 再处理其他队列。当正在处理某子队列时, 如果又出现新的磁盘 I/O 请求, 便将新请求进程放入其他队列, 这样就可避免出现磁臂粘着现象。当 N 值取得很大时, 会使 NStepSCAN 算法的性能接近于 SCAN 算法的性能。当 N = 1 时, NStepSCAN 算法便蜕化为 FCFS 算法。

(2) FSCAN 算法

FSCAN 算法实质上是 NStepSCAN 算法的简化, 即 FSCAN 只将磁盘请求队列分成两个子队列。一个是由当前所有请求磁盘 I/O 的进程形成的队列, 由磁盘调度按 SCAN 算法进行处理。在扫描期间, 将新出现的所有请求磁盘 I/O 的进程, 放入另一个等待处理的请求队列。这样, 所有的新请求都将被推迟到下一次扫描时处理。

三、提高磁盘 I/O 速度的方法

磁盘的 I/O 速度远低于内存的访问速度, 通常要低上 4~6 个数量级。因此, 磁盘的 I/O 已成为计算机系统的瓶颈。本节介绍几种已经广泛采用的提高磁盘 I/O 速度的方法。

1. 提前读

简单来说, 提前读就是系统根据现在用户请求读的内容, 把预计最近不久可能要读的内容与现在请求读的内容一起提前读入内存。用户读文件时, 经常是顺序读文件内容, 文件的存放是按内容顺序放入磁盘块的。操作系统知道文件内容是按照怎样的顺序存放在磁盘块中

的,所以可以在按照用户请求读当前磁盘块时,提前把下一个磁盘块的内容也读入缓冲区。比如文件 f1,内容依次存放在 10、20、60 号磁盘块中,如果当前用户请求读的内容在 10 号磁盘块中,系统在读入 10 号磁盘块的内容时,把 20 号磁盘块的内容也提前读入缓冲。这样,当下次要访问已经提前读入缓冲的磁盘块时,就不需要从磁盘中读,而是可以直接从内存的缓冲区中读。因此,大大减少了读数据的时间,提高了用户读磁盘数据的速度。

2. 延迟写

延迟写是在支持请求分页的虚拟存储管理中,对修改过的换出页,在把页标记为换出页时并不马上把页的内容写入磁盘,而是暂时保留在内存中,直到这些页所在的页框要被使用,导致页的内容将被覆盖前的“最后”时刻才启动磁盘操作,把修改过的一个或若干页写入磁盘,这种延迟写的策略减少了写磁盘的次数。

3. 优化物理块的分布

寻道时间和磁盘旋转延迟时间通常占据了磁盘 I/O 所耗时间中的主要部分,所以适当地集中数据在磁盘上存放的位置,可以减少磁臂移动距离,有利于提高传输速率。为了达到这一目的,可以采取优化文件物理块分布的方法。现在的文件系统都允许文件离散存放。理论上,一个文件的物理块可以分散在磁盘的任意位置。但是,如果将一个文件存放在过于分散的多个磁盘块上,会增加磁头的移动距离。例如,如果分配给一个文件的第一个磁盘块在最里面的磁道上,而第二个磁盘块在最外面的磁道上,当读完该文件的第一个磁盘块时,需要把磁头从最里面的磁道移动到最外面一个磁道。而如果把这个文件存放在同一个磁道或者相邻的两个磁道上,读这个文件的两个磁盘块磁头的移动距离会短得多,读磁盘的速度会快得多。

因此,现在的文件系统基本都会考虑对文件的位置进行优化,原则就是尽可能地把一个文件存放在同一个磁道或者相邻的磁道上。实现这个原则的技术一个是以连续的几个扇区即一个簇作为磁盘块的分配单位,另一个技术是把磁盘分成块组,一个块组中的不同簇都在相邻的磁道上,一个文件尽可能地放在同一个块组中,如 Linux 的 Ext2 文件系统。

4. 虚拟盘

虚拟盘是指利用内存空间去仿真磁盘,又称 RAM 盘。虚拟盘可以接受所有标准的磁盘操作,但这些操作的执行不是在磁盘上,而是在内存中。因此,对虚拟盘的访问比对磁盘的访问速度快。用户对虚拟盘的操作与对磁盘的操作完全相同,所有实现细节对用户都是透明的。虚拟盘通常用于存放临时性文件,如编译程序所产生的目标程序等。虚拟盘与磁盘高速缓存虽然都位于内存中,但是虚拟盘中的内容操作完全由用户控制,而高速缓存中的内容则是由操作系统控制的。

5. 磁盘高速缓存

磁盘高速缓存是指内存的一块存储空间,用来暂存从磁盘中读出的一系列盘块中的信息。因此,这里的高速缓存是一组逻辑上属于磁盘,而物理上是驻留在内存中的盘块。高速缓存在内存中可分成两种形式。第一种是在内存中开辟一个单独的存储空间来作为磁盘高速缓存,其大小是固定的,不会受应用程序多少的影响。第二种是把所有未利用的内存空间变为一个缓冲池,供请求分页系统和磁盘 I/O 时(作为磁盘高速缓存)共享。此时,高速缓存的大小不再是固定的。当磁盘频繁发生 I/O 时,该缓冲池可能包含更多的内存空间。而在应用程序运行得较多时,该缓冲池可能只剩下较少的内存空间。

思考题与练习题

一、单项选择题(在每小题后的四个备选项中只有一个是符合题目要求的,请将其选出并将代码填写在题后的括号内)

1. 关于 DMA 控制方式,下列说法正确的是【 】。
 - A. 采用 DMA 控制方式时, CPU 不用干预整个输入/输出过程
 - B. 采用 DMA 控制方式时,每传输完一个数据块, CPU 要执行中断处理
 - C. 采用 DMA 传输方式, CPU 只在输入/输出开始和结束时干预输入/输出
 - D. 采用 DMA 传输方式只能提高传输速度,不能提高 CPU 与输入/输出设备的并行程度
2. 关于 SPOOLing 系统的组成,下列说法正确的是【 】。
 - A. 输入井和输出井都在内存中
 - B. 输出井和输出缓存都在硬盘中
 - C. 输入井和输入缓存都在硬盘中
 - D. 输入井和输出井都在硬盘中
3. 设备控制器的 I/O 逻辑包含的主要部件是【 】。
 - A. 仅指令译码器
 - B. 指令译码器和地址译码器
 - C. 仅地址译码器
 - D. 仅控制寄存器

二、填空题(请在题目空白处填写正确内容)

1. I/O 设备按信息交换的单位分类,可分为字符设备和_____设备。
2. 在数据到达速率与数据离去速率不同的地方,都可以引入_____。
3. 设备独立性的基本含义是_____独立于具体使用的物理设备。

三、简答题

1. 说明设备控制器的结构和功能。
2. 说明 DMA 控制器的结构和控制方式。
3. 什么是设备驱动程序?

附录 部分习题参考答案

第一章 操作系统简介

一、单项选择题

1. D 2. D 3. B

二、填空题

1. 一
2. 等待人工操作的时间
3. 硬件、软件
4. 时间间隔
5. 共享

第二章 进程管理

一、单项选择题

1. A 2. C 3. C 4. B

二、填空题

1. 用户数据段、进程控制块
2. 就绪、阻塞
3. wait 操作、signal 操作
4. 临界资源

四、综合应用

为缓冲区设置互斥信号量 mutex，设置资源信号量 empty 和 full。3 个信号量的初始值分别为

```
mutex. value = 1
empty. value = 1
full. value = 0
pA
{ 从文件中读一条记录;
  wait(empty);
  wait(mutex);
  将数据写入缓冲区;
  signal(mutex);
  signal(full);
}
```

pB

```

{
    wait(full);
    wait(mutex);
    从缓冲区中读一条记录;
    signal(mutex);
    signal(empty);
    打印数据;
}

```

第三章 进程调度与死锁

一、单项选择题

1. A 2. C 3. D

二、填空题

1. 短、长
2. 饥饿
3. 银行家
4. 充分

四、综合应用

答: (1) need 矩阵的内容为

p0: 0 0 0 0

p1: 0 7 5 0

p2: 1 0 0 2

p3: 0 0 2 0

p4: 0 6 4 2

(2) 系统处于安全状态。

work = available = (1, 5, 2, 0)

finish[0-4] = false;

a. 因为 need0 < work 并且 finish[0] = false; 所以 finish[0] = true; work = work + allocation0 = (1, 5, 3, 2)

b. 因为 need2 < work 并且 finish[2] = false; 所以 finish[2] = true; work = work + allocation2 = (2, 8, 8, 6)

c. 因为 need3 < work 并且 finish[3] = false; 所以 finish[3] = true; work = work + allocation3 = (2, 14, 11, 8)

d. 因为 need4 < work 并且 finish[4] = false; 所以 finish[4] = true; work = work + allocation4 = (2, 14, 12, 12)

e. 因为 need1 < work 并且 finish[1] = false; 所以 finish[1] = true; work = work + allocation1 = (3, 14, 12, 12)

至此, finish[0-4] = true, 找到安全序列 <p0, p2, p3, p4, p1>, 所以系统处于安全

状态。

(3) p1 提出资源请求(0, 4, 2, 0)能够立刻被满足。

①request1 < need1 && request1 < available

所以资源试分配

available = available - request = (1, 1, 0, 0)

allocation1 = allocation1 + request = (1, 4, 2, 0)

need = need - request = (0, 3, 3, 0)

②进行安全性检测。

work = available = (1, 1, 0, 0)

finish[0-4] = false;

a. 因为 need0 < work 并且 finish[0] = false; 所以 finish[0] = true; work = work + allocation0 = (1, 1, 1, 2)

b. 因为 need2 < work 并且 finish[2] = false; 所以 finish[2] = true; work = work + allocation2 = (2, 4, 6, 6)

c. 因为 need3 < work 并且 finish[3] = false; 所以 finish[3] = true; work = work + allocation3 = (2, 10, 9, 8)

d. 因为 need4 < work 并且 finish[4] = false; 所以 finish[4] = true; work = work + allocation4 = (2, 10, 10, 12)

e. 因为 need1 < work 并且 finish[1] = false; 所以 finish[1] = true; work = work + allocation1 = (3, 14, 12, 12)

至此, finish[0-4] = true, 找到安全序列 < p0, p2, p3, p4, p1 >, 系统处于安全状态。

故请求能立刻被满足。

第四章 内存管理

一、单项选择题

1. C 2. B 3. B 4. A 5. C

二、填空题

1. 单一连续区分配、固定分区分配、动态分区分配
2. 当前
3. 最近最久未访问
4. 最近被访问过的页表项
5. 段号、段基址、段大小
6. 离散性、对换性、虚拟性

四、综合应用

回收3个分区后的空闲分区链如图 A-1a ~ 图 A-1c 所示。

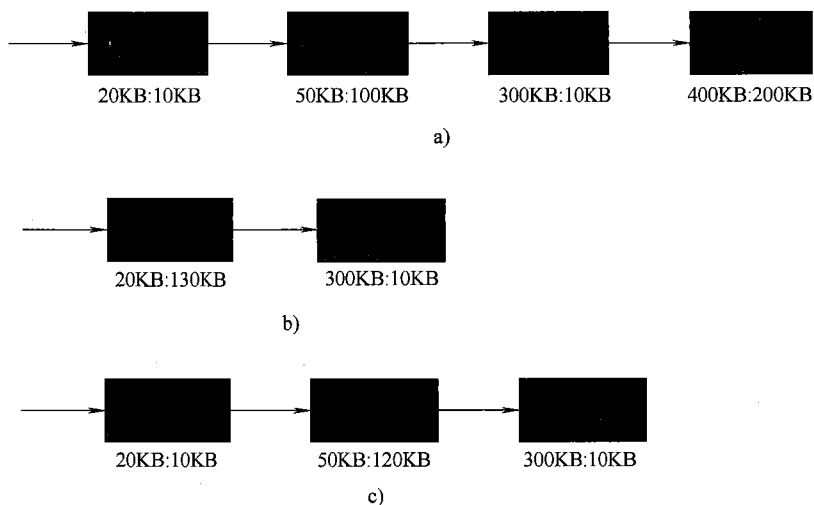


图 A-1 空闲分区链

第五章 文件系统

一、单项选择题

1. D 2. B 3. D

二、填空题

1. 随机存取
2. 属性、地址
3. 按名访问

四、综合应用

每个簇能存放的簇号数量： $4096\text{B}/4\text{B} = 1024$ (个)

12 个直接地址能访问的磁盘空间大小： $12 \times 4 \text{ KB} = 48 \text{ KB}$

一个一次间接地址能访问的磁盘空间大小： $1024 \times 4 \text{ KB} = 4 \text{ MB}$

一个二次间接地址能访问的磁盘空间大小： $1024 \times 1024 \times 4\text{KB} = 4\text{GB}$

一个三次间接地址能访问的磁盘空间大小： $1024 \times 1024 \times 1024 \times 4\text{KB} = 4\text{TB}$

EXT2 能管理的文件的最大长度是 $48\text{KB} + 4\text{MB} + 4\text{GB} + 4\text{TB}$

第六章 I/O 设备管理

一、单项选择题

1. C 2. D 3. B

二、填空题

1. 块
2. 缓存
3. 应用程序

参考文献

- [1] 汤晓丹, 梁红兵, 哲凤屏, 等. 计算机操作系统[M]. 3 版. 西安: 西安电子科技大学出版社, 2007.
- [2] Andrew S Tanenbaum. 现代操作系统[M]. 2 版. 陈向群, 马红兵, 译. 北京: 机械工业出版社, 2005.
- [3] Andrew S Tanenbaum. 操作系统: 设计与实现[M]. 王鹏, 尤晋元, 等译. 北京: 电子工业出版社, 1998.
- [4] 陈莉君. 深入分析 Linux 内核源代码[M]. 北京: 人民邮电出版社, 2002.
- [5] Daniel P Bovet&Marco Cesati. 深入理解 Linux 内核[M]. 3 版. 陈莉君, 张琼声, 张宏伟, 译. 北京: 中国电力出版社, 2007.
- [6] William Stallings. 操作系统——内核与设计原理[M]. 4 版. 魏迎梅, 王涌, 等译. 北京: 电子工业出版社, 2002.
- [7] 于渊. 自己动手写操作系统[M]. 北京: 电子工业出版社, 2006.
- [8] Randal E Bryant David R O'Hallaron. 深入理解计算机系统[M]. 龚奕利, 雷迎春, 译. 北京: 机械工业出版社, 2010.
- [9] 范志东, 张琼声. 自己动手构造编译系统[M]. 北京: 机械工业出版社, 2016.
- [10] 袁春风. 计算机系统基础[M]. 北京: 机械工业出版社, 2014.
- [11] 毛德操, 胡希明. Linux 内核源代码情景分析[M]. 杭州: 浙江大学出版社, 2001.

后 记

经全国高等教育自学考试指导委员会同意，由电子电工与信息类专业委员会负责计算机及应用专业(专科)教材的审定工作。

本教材由中国石油大学(华东)张琼声副教授负责编写。电子电工与信息类专业委员会在北京组织了本教材的审稿工作。参与本教材审稿的有山东大学杨兴强教授和上海大学武频副教授，谨向他们表示诚挚的谢意。

电子电工与信息类专业委员会最后审定通过了本教材。

全国高等教育自学考试指导委员会

电子电工与信息类专业委员会

2017 年 7 月

